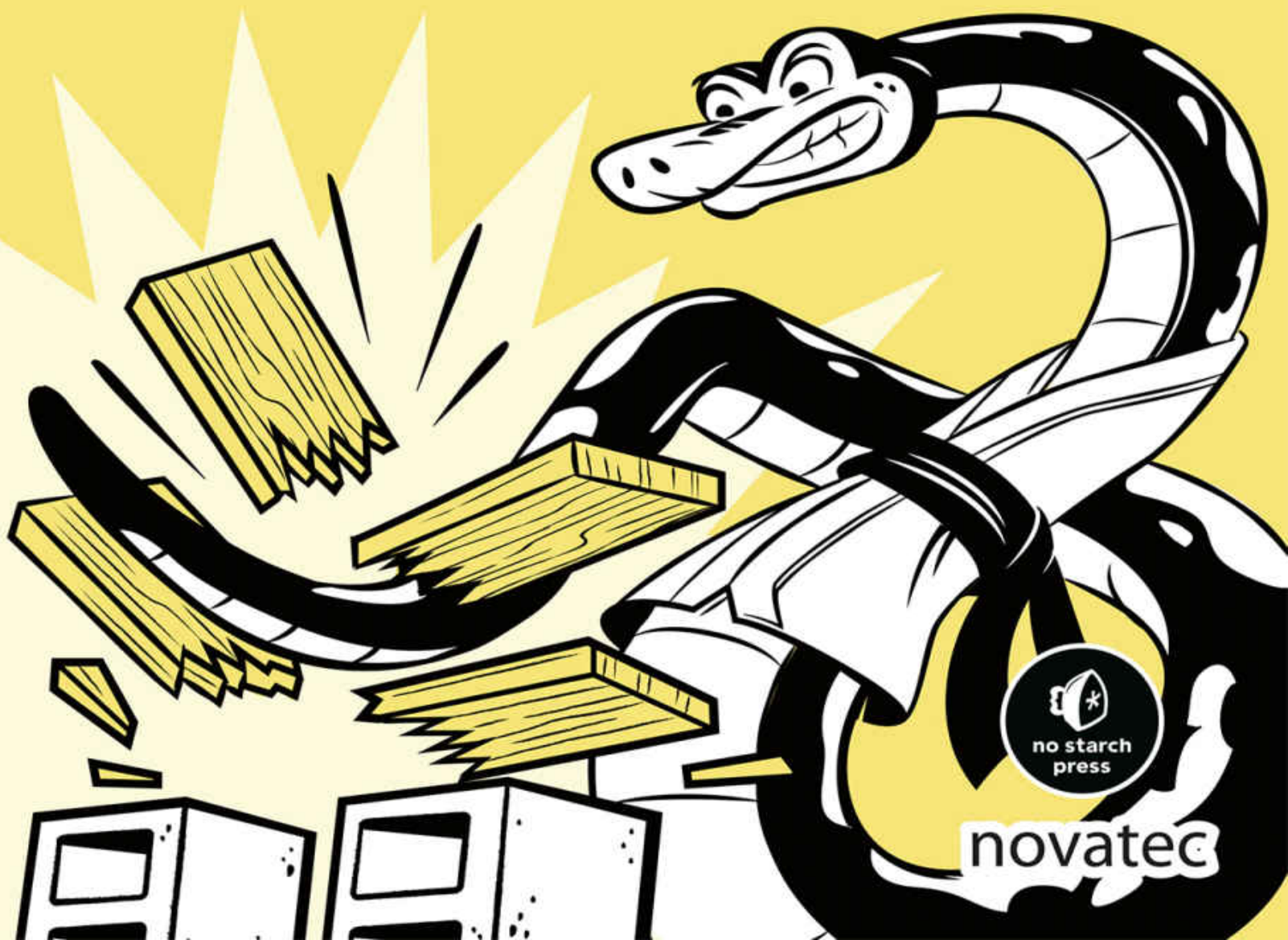


PYTHON LEVADO A SÉRIO

CONSELHOS DE UM FAIXA-PRETA SOBRE IMPLANTAÇÃO,
ESCALABILIDADE, TESTES E OUTROS ASSUNTOS

JULIEN DANJOU



Julien Danjou

Novatec

Copyright © 2019 by Julien Danjou. Title of English-language original: Serious Python: Black-Belt Advice on Deployment, Scalability, Testing, and More, ISBN 978-1-59327-878-6, published by No Starch Press. Portuguese-language edition copyright © 2020 by Novatec Editora Ltda. All rights reserved.

Copyright © 2019 por Julien Danjou. Título original em Inglês: Serious Python: Black-Belt Advice on Deployment, Scalability, Testing, and More, ISBN 978-1-59327-878-6, publicado pela No Starch Press. Edição em Português copyright © 2020 pela Novatec Editora Ltda. Todos os direitos reservados.

© Novatec Editora Ltda. 2020.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates GRA20200514

Tradução: Lúcia A. Kinoshita

Revisão gramatical: Tássia Carvalho

ISBN do ebook: 978-65-86057-18-8

ISBN do impresso: 978-65-86057-17-1

Histórico de impressões:

Maio/2020 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

E-mail: novatec@novatec.com.br

Site: novatec.com.br

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

SUMÁRIO

Sobre o autor

[Sobre o revisor técnico](#)

Agradecimentos

Introdução

[Quem deve ler este livro e por quê](#)

[Sobre este livro](#)

1 ■ Iniciando o seu projeto

[Versões de Python](#)

[Organizando o seu projeto](#)

[O que você deve fazer](#)

[O que você não deve fazer](#)

[Numeração de versões](#)

[Estilo de programação e verificações automáticas](#)

[Ferramentas para identificar erros de estilo](#)

[Ferramentas para identificar erros de código](#)

[Joshua Harlow fala sobre Python](#)

2 ■ Módulos, bibliotecas e frameworks

[Sistema de importação](#)

[Módulo sys](#)

[Paths de importação](#)

[Importadores personalizados](#)

[Meta path finders](#)

[Bibliotecas-padrões úteis](#)

[Bibliotecas externas](#)

[Lista de verificação de segurança para bibliotecas externas](#)

[Protegendo seu código com um wrapper de API](#)
[Instalação de pacotes: aproveitando melhor o pip](#)
[Usando e escolhendo frameworks](#)
[Doug Hellmann, desenvolvedor do núcleo de Python, fala sobre bibliotecas Python](#)

3 ■ Documentação e boas práticas para APIs

[Documentando com o Sphinx](#)
[Introdução ao Sphinx e ao reST](#)
[Módulos do Sphinx](#)
[Escrevendo uma extensão para o Sphinx](#)
[Administrando mudanças em suas APIs](#)
[Numeração das versões da API](#)
[Documentando as mudanças em sua API](#)
[Marcando funções obsoletas com o módulo warnings](#)
[Resumo](#)
[Christophe de Vienne fala sobre o desenvolvimento de APIs](#)

4 ■ Lidando com timestamps e fusos horários

[Problema da ausência de fusos horários](#)
[Criando objetos datetime default](#)
[Timestamps com informação sobre o fuso horário com o dateutil](#)
[Serializando objetos datetime com informação de fuso horário](#)
[Resolvendo horários ambíguos](#)
[Resumo](#)

5 ■ Distribuindo seu software

[Um pouco da história do setup.py](#)
[Empacotamento com setup.cfg](#)
[Padrão de distribuição do formato Wheel](#)
[Compartilhando seu trabalho com o mundo](#)
[Pontos de entrada](#)
[Visualizando os pontos de entrada](#)
[Usando scripts de console](#)

[Usando plugins e drivers](#)
[Resumo](#)
[Nick Coghlan fala sobre empacotamento](#)

6 ■ Testes de unidade

[Básico sobre testes](#)
[Alguns testes simples](#)
[Ignorando testes](#)
[Executando testes específicos](#)
[Executando testes em paralelo](#)
[Criando objetos usados nos testes com fixtures](#)
[Executando cenários de testes](#)
[Testes controlados usando simulação](#)
[Identificando um código não testado com o coverage](#)
[Ambientes virtuais](#)
[Configurando um ambiente virtual](#)
[Usando virtualenv com o tox](#)
[Recriando um ambiente](#)
[Usando diferentes versões de Python](#)
[Integrando outros testes](#)
[Política de testes](#)
[Robert Collins fala sobre testes](#)

7 ■ Métodos e decoradores

[Decoradores e quando usá-los](#)
[Criando decoradores](#)
[Escrevendo decoradores](#)
[Empilhando decoradores](#)
[Escrevendo decoradores de classe](#)
[Como os métodos funcionam em Python](#)
[Métodos estáticos](#)
[Métodos de classe](#)
[Métodos abstratos](#)
[Combinando métodos estáticos, métodos de classe e métodos abstratos](#)

[Colocando implementações em métodos abstratos](#)

[A verdade sobre o super](#)

[Resumo](#)

8 ■ Programação funcional

[Criando funções puras](#)

[Geradores](#)

[Criando um gerador](#)

[Devolvendo e passando valores com yield](#)

[Inspecionando geradores](#)

[List comprehensions](#)

[Funções funcionais em ação](#)

[Aplicando funções a itens usando map\(\)](#)

[Filtrando listas com filter\(\)](#)

[Obtendo índices com enumerate\(\)](#)

[Ordenando uma lista com sorted\(\)](#)

[Encontrando itens que satisfaçam condições com any\(\) e all\(\)](#)

[Combinando listas com zip\(\)](#)

[Um problema comum resolvido](#)

[Funções úteis de itertools](#)

[Resumo](#)

9 ■ Árvore Sintática Abstrata, HY e atributos do tipo LISP

[Observando a AST](#)

[Escrevendo um programa usando a AST](#)

[Objetos AST](#)

[Percorrendo uma AST](#)

[Estendendo o flake8 com verificações na AST](#)

[Escrevendo a classe](#)

[Ignorando códigos irrelevantes](#)

[Verificando se há um decorador apropriado](#)

[Procurando o self](#)

[Uma introdução rápida ao Hy](#)

[Resumo](#)

[Paul Tagliamonte fala sobre a AST e o Hy](#)

10 ■ Desempenho e otimizações

[Estruturas de dados](#)

[Entendendo o comportamento usando profiling](#)

[cProfile](#)

[Disassembling com o módulo dis](#)

[Definindo funções de modo eficiente](#)

[Listas ordenadas e o bisect](#)

[namedtuple e slots](#)

[Memoização](#)

[Python mais rápido com o PyPy](#)

[Evitando cópias com o protocolo de buffer](#)

[Resumo](#)

[Victor Stinner fala sobre otimização](#)

11 ■ Escalabilidade e arquitetura

[Multithreading em Python e suas limitações](#)

[Multiprocessamento versus multithreading](#)

[Arquitetura orientada a eventos](#)

[Outras opções e o asyncio](#)

[Arquitetura orientada a serviços](#)

[Comunicação entre processos com o ZeroMQ](#)

[Resumo](#)

12 ■ Gerenciando bancos de dados relacionais

[RDBMSs, ORMs e quando usá-los](#)

[Backends de bancos de dados](#)

[Streaming de dados com Flask e PostgreSQL](#)

[Escrevendo a aplicação de streaming de dados](#)

[Criando a aplicação](#)

[Dimitri Fontaine fala sobre bancos de dados](#)

13 ■ Escreva menos, programe mais

[Usando o six para suporte a Python 2 e 3](#)

[Strings e Unicode](#)

[Lidando com a mudança nos módulos Python](#)

[Módulo modernize](#)

[Usando Python como Lisp para criar um dispatcher simples](#)

[Criando métodos genéricos em Lisp](#)

[Métodos genéricos com Python](#)

[Gerenciadores de contexto](#)

[Menos boilerplate com attr](#)

[Resumo](#)

SOBRE O AUTOR

Julien Danjou é hacker de software livre há quase vinte anos, e desenvolve software com Python há doze. Atualmente trabalha como líder de equipe de projeto para a plataforma de nuvem distribuída OpenStack, que tem a maior base de código aberto Python em existência, com 2,5 milhões de linhas nessa linguagem. Antes de desenvolver nuvens, Julien criou um gerenciador de janelas incrível e contribuiu em vários softwares, por exemplo, Debian e GNU Emacs.

Sobre o revisor técnico

Mike Driscoll é programador Python há mais de uma década. Mike escreve sobre Python em seu blog *The Mouse vs. The Python* há vários anos, e é autor de diversos livros sobre Python, incluindo *Python 101*, *Python Interviews* e *ReportLab: PDF Processing with Python*. Você pode encontrar Mike no Twitter ou no GitHub utilizando o handle *@driscollis*.

AGRADECIMENTOS

Escrever este primeiro livro exigiu um esforço tremendo. Olhando para trás, eu não tinha nenhuma pista da loucura que seria essa jornada, mas também não tinha a mínima ideia do quão gratificante ela seria.

Dizem que, se você quer ir rápido, deve andar sozinho, mas, se quiser ir longe, deve andar junto. Esta é a quarta edição do livro que escrevi originalmente, e eu não teria chegado até aqui sem as pessoas que me ajudaram no caminho. Foi um esforço de equipe, e gostaria de agradecer a todos que participaram.

A maioria dos entrevistados concedeu seu tempo e confiou em mim sem pensar duas vezes, e devo muito do que ensinamos neste livro a eles: a Doug Hellmann, pelos ótimos conselhos sobre construção de bibliotecas, a Joshua Harlow, pelo bom humor e conhecimento sobre sistemas distribuídos, a Christophe de Vienne, pela experiência no desenvolvimento de frameworks, a Victor Stinner, pelo incrível conhecimento de CPython, a Dimitri Fontaine, pela sabedoria acerca dos bancos de dados, a Robert Collins, por entender de testes, a Nick Coghlan, pelo trabalho em aprimorar Python, e a Paul Tagliamonte, pelo incrível espírito de hacker.

Agradeço à equipe da No Starch por trabalhar comigo e levar este livro a um patamar totalmente novo – particularmente a Liz Chadwick, pelas habilidades de edição, a Laurel Chun, por me manter no caminho, e a Mike Driscoll, pelos insights técnicos.

Minha gratidão também vai para as comunidades de software livre que compartilharam conhecimentos e me ajudaram a crescer, particularmente à comunidade Python, que tem sido sempre receptiva e entusiástica.

INTRODUÇÃO

Se você está lendo este livro, há uma boa chance de que já trabalhe com Python há um tempo. Talvez tenha aprendido com alguns tutoriais, explorou minuciosamente alguns programas existentes ou tenha começado do zero. Qualquer que tenha sido o caso, você *hackeou* o seu próprio caminho para aprendê-lo. É exatamente assim que adquiri familiaridade com Python, até começar a trabalhar com projetos grandes de código aberto dez anos atrás.

É fácil pensar que você sabe e entende de Python depois de escrever seu primeiro programa. A linguagem é muito fácil de aprender. Contudo, são necessários anos para dominá-la e adquirir uma compreensão profunda acerca de suas vantagens e desvantagens.

Quando comecei a trabalhar com Python, desenvolvia minhas próprias bibliotecas e aplicações Python em uma escala de “projetos de garagem”. A situação mudou assim que comecei a trabalhar com centenas de desenvolvedores, em softwares dos quais milhares de usuários dependiam. Por exemplo, a plataforma OpenStack – um projeto para o qual contribuí – contém mais de nove milhões de linhas de código Python, as quais, coletivamente, precisam ser concisas, eficientes e escaláveis a fim de atender às necessidades de qualquer aplicação de computação em nuvem que seus usuários exijam. Quando se tem um projeto desse tamanho, tarefas como testes e documentação, definitivamente, exigem automação; caso contrário, não serão feitas.

Pensei que sabia muito sobre Python até começar a trabalhar em projetos nessa escala – uma escala que eu mal conseguia imaginar no início – porém, aprendi muito mais. Também tive a oportunidade de conhecer alguns dos melhores hackers de Python do mercado e

aprender com eles. Eles me ensinaram de tudo: de princípios gerais de arquitetura e design a diversas dicas e truques. Neste livro, espero compartilhar os pontos mais importantes que aprendi, para que você crie melhores programas Python – e o faça de modo mais eficiente, também!

A primeira versão deste livro, *The Hacker's Guide to Python*, foi publicada em 2014. *Python levado a sério* é a quarta edição, com conteúdo atualizado, além de outros tópicos totalmente novos. Espero que você o aprecie!

Quem deve ler este livro e por quê

Este livro é voltado aos programadores e desenvolvedores Python que queiram levar suas habilidades com Python para o próximo patamar.

Nesta obra, você verá métodos e conselhos que ajudarão a tirar o máximo proveito de Python e a criar programas à prova de futuro. Se você já está trabalhando em um projeto, poderá aplicar prontamente as técnicas discutidas e aprimorar o seu código atual. Se está iniciando seu primeiro projeto, poderá criar um modelo com as melhores práticas.

Apresentarei alguns detalhes sobre a natureza interna de Python para que você saiba melhor como escrever um código eficaz. Você terá mais insights sobre o funcionamento interno da linguagem, os quais ajudarão a entender seus problemas e deficiências.

O livro também apresenta soluções testadas na prática para problemas como testar, portar e escalar código Python, aplicações e bibliotecas. Isso ajudará você a evitar erros que outros já cometeram e a descobrir estratégias que ajudarão na manutenção de seu software no longo prazo.

Sobre este livro

Este livro não foi necessariamente projetado para ser lido do

começo ao fim. Sinta-se à vontade para avançar diretamente para as seções que sejam de seu interesse ou que sejam relevantes para o seu trabalho. Ao longo do livro, você verá diversos conselhos e dicas práticas. Apresentamos a seguir uma descrição rápida do que cada capítulo contém.

O **Capítulo 1** apresenta orientações sobre o que você deve considerar antes de iniciar um projeto, com conselhos sobre como estruturá-lo, como atribuir números de versões, como configurar uma verificação automática de erros e outros assuntos. No final, há uma entrevista com Joshua Harlow.

O **Capítulo 2** apresenta os módulos, as bibliotecas e os frameworks Python, e discute um pouco o modo como funcionam internamente. Você verá orientações sobre como usar o módulo `sys`, como tirar melhor proveito do gerenciador de pacotes `pip`, como escolher o melhor framework para você e como usar a biblioteca-padrão e as bibliotecas externas. Há também uma entrevista com Doug Hellmann.

O **Capítulo 3** dá conselhos sobre como documentar seus projetos e gerenciar suas APIs à medida que o seu projeto evoluir, mesmo após a publicação. Você verá orientações específicas sobre o uso do Sphinx para automatizar determinadas tarefas de documentação. Nesse capítulo, você verá uma entrevista com Christophe de Vienne.

O **Capítulo 4** aborda o velho problema dos fusos horários e a melhor maneira de lidar com eles em seus programas usando objetos `datetime` e `tzinfo`.

O **Capítulo 5** ajuda você a levar o seu software até os usuários, apresentando orientações sobre a distribuição. Conheceremos o empacotamento, os padrões de distribuição, as bibliotecas `distutils` e `setuptools`, e veremos como descobrir facilmente os recursos dinâmicos de um pacote usando pontos de entrada. Nick Coghlan é entrevistado nesse capítulo.

O **Capítulo 6** oferece conselhos sobre testes de unidade, com dicas

sobre as melhores práticas e tutoriais específicos sobre automação de testes de unidade com o `pytest`. Veremos também como usar ambientes virtuais para deixar seus testes mais isolados. A entrevista é com Robert Collins.

O **Capítulo 7** explora os métodos e os decoradores. Veremos como usar Python na programação funcional e daremos conselhos sobre como e quando utilizar decoradores, além de como criar decoradores *para* decoradores. Também exploraremos os métodos estáticos e abstratos e os métodos de classe, e como combinar esses três tipos para ter um programa mais robusto.

O **Capítulo 8** mostra outros truques de programação funcional que você pode implementar em Python. Esse capítulo discute os geradores, as list comprehensions, as funções funcionais e as ferramentas comuns para implementá-los, além de apresentar a conveniente biblioteca `functools`.

O **Capítulo 9** discute rapidamente o funcionamento interno da própria linguagem e apresenta a AST (Abstract Syntax Tree, ou Árvore Sintática Abstrata), que compõe a estrutura interna de Python. Também veremos como estender o `flake8` para trabalhar com a AST a fim de introduzir verificações automáticas mais sofisticadas em seus programas. O capítulo termina com uma entrevista com Paul Tagliamonte.

O **Capítulo 10** é um guia para otimizar o desempenho usando estruturas de dados apropriadas, definindo funções de modo eficiente e aplicando uma análise de desempenho dinâmica para identificar gargalos em seu código. Também falaremos rapidamente sobre a memoização e a redução de desperdícios em cópias de dados. Você verá uma entrevista com Victor Stinner.

O **Capítulo 11** lida com o complicado assunto do multithreading, incluindo como e quando usar em oposição ao multiprocessing, e se devemos usar uma arquitetura orientada a eventos ou orientada a serviços para criar programas escaláveis.

O **Capítulo 12** discute os bancos de dados relacionais. Veremos

como eles funcionam e como utilizar o PostgreSQL para gerenciar dados e fazê-los fluir de modo eficiente. Dimitri Fontaine é entrevistado nesse capítulo.

Por fim, o **Capítulo 13** oferece conselhos sólidos sobre diversos assuntos: deixar seu código compatível tanto com Python 2 como com Python 3, criar um código funcional do tipo Lisp, usar gerenciadores de contexto e diminuir a repetição com a biblioteca `attr`.

1

INICIANDO O SEU PROJETO

Neste primeiro capítulo, veremos alguns aspectos sobre o início de um projeto e sobre o que você deve pensar antes de começar, por exemplo, qual versão de Python deve usar, como estruturar seus módulos, como numerar as versões de software de modo eficaz e como garantir que usará as melhores práticas de programação com verificação automática de erros.

Versões de Python

Antes de iniciar um projeto, você terá de decidir qual(is) versão(ões) de Python será(ão) aceita(s). Essa não é uma decisão tão simples quanto parece.

Não é nenhum segredo que Python oferece suporte para várias versões ao mesmo tempo. Cada versão secundária (menor) do interpretador tem suporte para correção de bugs durante 18 meses, e para segurança durante 5 anos. Por exemplo, Python 3.7, lançado em 27 de junho de 2018, terá suporte até que Python 3.8 seja lançado, o que deve ocorrer por volta de outubro de 2019¹. Por volta de dezembro de 2019, uma última versão de Python 3.7 com correção de bugs será lançada², e espera-se que todos passem a usar Python 3.8. Cada nova versão de Python introduz novos recursos, enquanto outros recursos mais antigos passam a ser considerados obsoletos. A Figura 1.1 mostra essa linha do tempo.

Além disso, devemos levar em consideração o problema de Python 2 *versus* Python 3. As pessoas que trabalham com plataformas (muito) antigas talvez ainda precisem de suporte para Python 2 pelo

fato de Python 3 ainda não ter sido disponibilizado nessas plataformas, mas a regra geral é esquecer Python 2, se for possível.

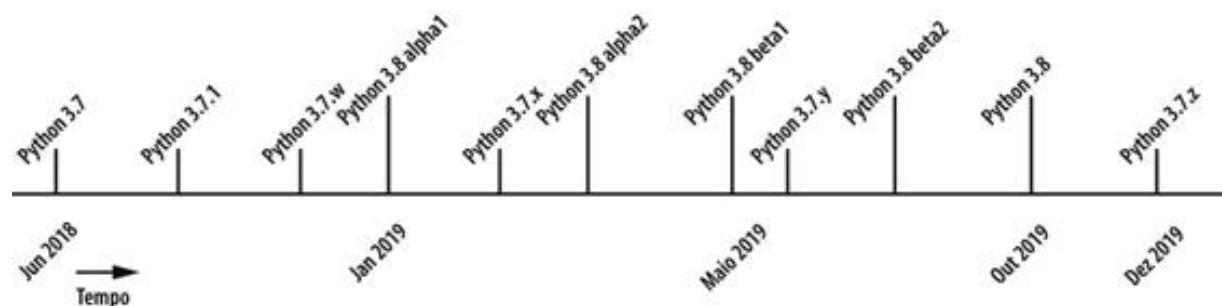


Figura 1.1 – Linha do tempo dos lançamentos das versões de Python.

Eis um modo rápido de descobrir a versão de que você precisará:

- As versões 2.6 e anteriores atualmente são consideradas obsoletas, portanto, não recomendo que você se preocupe em oferecer suporte a elas. Se realmente pretende oferecer suporte para essas versões mais antigas por qualquer que seja o motivo, saiba que terá muita dificuldade para garantir que seu programa aceite Python 3.x também. Apesar do que dissemos, talvez você ainda depare com Python 2.6 em alguns sistemas mais antigos – se isso acontecer, sinto muito!
- A versão 2.7 é e continuará sendo a última versão de Python 2.x. Todo sistema basicamente executa ou é capaz de executar Python 3 de uma maneira ou de outra atualmente, portanto, a menos que você esteja fazendo um trabalho de arqueologia, não deverá ser necessário se preocupar em oferecer suporte para Python 2.7 em programas novos. Python 2.7 deixará de ter suporte depois de 2020, portanto, a última coisa que você vai querer fazer é criar um software com base nele.
- A versão 3.7 é a versão mais recente do ramo de Python 3 atualmente³ – quando este livro foi escrito –, e é essa versão que você deve visar. No entanto, se seu sistema operacional vem com a versão 3.6 (a maioria dos sistemas operacionais, exceto Windows, vem com a versão 3.6 ou mais recente), certifique-se

de que sua aplicação funcionará também com essa versão.

Técnicas para escrever programas que aceitem tanto Python 2.7 como 3.x serão discutidas no Capítulo 13.

Por fim, saiba que este livro foi escrito com Python 3 em mente.

Organizando o seu projeto

Iniciar um novo projeto é sempre um pouco complicado. Você não tem certeza de como o seu projeto será estruturado, portanto, talvez não saiba como organizar seus arquivos. No entanto, assim que compreender apropriadamente as melhores práticas, saberá com qual estrutura básica deverá começar. Apresentaremos algumas dicas sobre o que você deve e o que não deve fazer para organizar o seu projeto.

O que você deve fazer

Considere inicialmente a estrutura de seu projeto, a qual deve ser bem simples. Utilize pacotes e hierarquia com sabedoria: navegar por uma hierarquia com vários níveis de profundidade pode ser um pesadelo, enquanto uma hierarquia com poucos níveis tende a se tornar inflada.

Evite cometer o erro comum de armazenar testes de unidade fora do diretório do pacote. Esses testes devem ser, sem dúvida, incluídos em um subpacote de seu software, de modo que não sejam instalados de forma automática como um módulo *tests* de nível mais alto pelo *setuptools* (ou por outra biblioteca de empacotamento) acidentalmente. Ao colocá-los em um subpacote, você garante que eles poderão ser instalados e futuramente utilizados por outros pacotes para que os usuários criem os próprios testes de unidade.

A Figura 1.2 mostra como deve ser uma hierarquia padrão de arquivos.

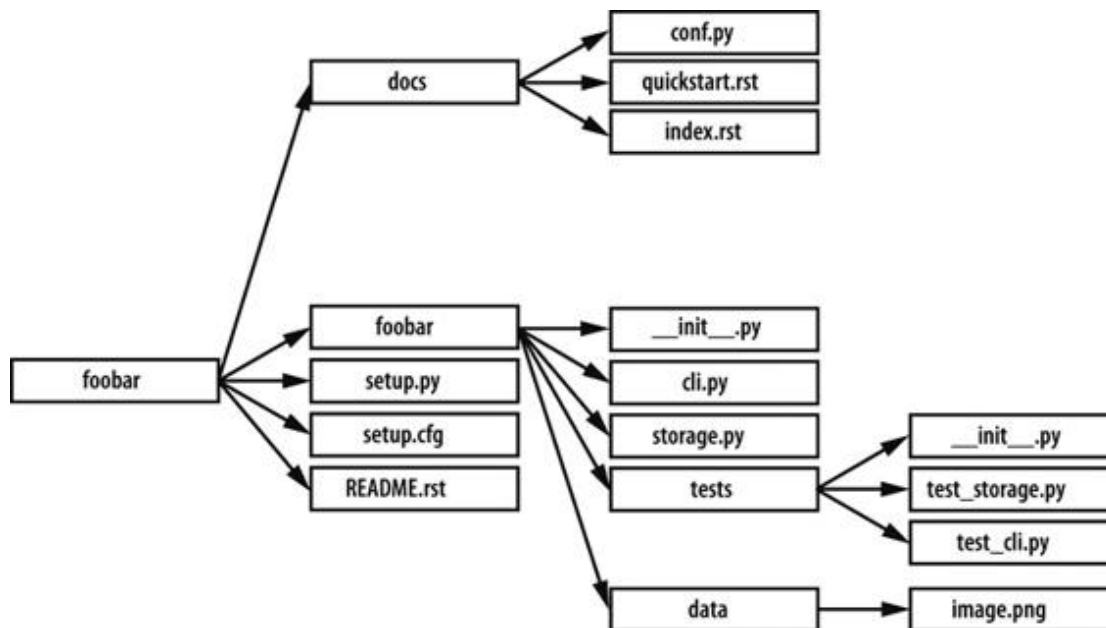


Figura 1.2 – Diretório padrão de pacotes.

O nome padrão de um script de instalação Python é `setup.py`. Ele é acompanhado por `setup.cfg`, que deve conter os detalhes de configuração do script de instalação. Ao ser executado, `setup.py` instalará o seu pacote com os utilitários de distribuição Python.

Você também pode fornecer informações importantes aos usuários em `README.rst` (ou em `README.txt`, ou qualquer arquivo que seja de sua preferência). Por fim, o diretório `docs` deve conter a documentação do pacote no formato *reStructuredText*, que será consumido pelo Sphinx (veja o Capítulo 3).

Com frequência, os pacotes devem fornecer dados extras para o software utilizar, por exemplo, imagens, shell scripts e outros. Infelizmente, não há um padrão universalmente aceito para o local em que esses arquivos devem ser armazenados, portanto, coloque-os simplesmente no lugar em que fizer mais sentido para o seu projeto, de acordo com suas funções. Por exemplo, templates de aplicações web poderiam estar em um diretório `templates` no diretório-raiz de seu pacote.

Os diretórios de nível mais alto a seguir também aparecem com frequência:

- *etc* para arquivos de configuração de exemplo;
- *tools* para shell scripts ou ferramentas relacionadas;
- *bin* para scripts binários escritos por você, e que serão instalados por *setup.py*.

O que você não deve fazer

Há um problema específico de design que muitas vezes vejo nas estruturas dos projetos, sobre o qual as pessoas não pensam muito: alguns desenvolvedores criam arquivos ou modelos com base no tipo de código que será armazenado. Por exemplo, eles podem criar arquivos *functions.py* ou *exceptions.py*. É uma *péssima* abordagem, e não ajudará nenhum desenvolvedor a navegar pelo código. Ao ler uma base de código, o desenvolvedor espera que uma área funcional de um programa esteja confinada em um determinado arquivo. A organização do código não se beneficiará com essa abordagem, a qual força os leitores a pular de arquivo em arquivo, sem que haja um bom motivo para isso.

Organize seu código com base em *funcionalidades*, e não com base em tipos.

Também é uma *péssima* ideia criar um diretório de módulos que contenha somente um arquivo *__init__.py*, pois implicará níveis desnecessariamente aninhados. Por exemplo, você não deve criar um diretório chamado *hooks* contendo um único arquivo de nome *hooks/__init__.py*, quando *hooks.py* bastaria. Se você criar um diretório, ele deverá conter vários outros arquivos Python pertencentes à categoria representada por esse diretório. Construir desnecessariamente uma hierarquia com vários níveis de profundidade gera confusão.

Você também deve tomar muito cuidado com o código que colocar no arquivo *__init__.py*. Esse arquivo é chamado e executado na primeira vez em que um módulo contido no diretório é carregado. Colocar códigos indevidos em seu *__init__.py* pode causar efeitos colaterais indesejados. Com efeito, arquivos *__init__.py* deverão estar

vazios na maior parte das vezes, a menos que você saiba o que está fazendo. Contudo, não tente remover totalmente os arquivos `__init__.py`, ou você, definitivamente, não será capaz de importar seu módulo Python: Python exige que um arquivo `__init__.py` esteja presente para que o diretório seja considerado um submódulo.

Numeração de versões

As versões de software devem ser atribuídas para que os usuários saibam qual é a versão mais recente. Em qualquer projeto, os usuários devem ser capazes de organizar a linha do tempo da evolução do código.

Há um número infinito de maneiras de organizar os números de versões. No entanto, a PEP 440 apresenta um formato de versão que todo pacote Python e, de modo ideal, toda aplicação, deve seguir, de modo que outros programas e pacotes possam identificar quais versões de seu pacote lhes são necessárias, de modo fácil e confiável.

A PEP 440 define a expressão regular no formato a seguir para a numeração de versões:

`N[.N]+[{a|b|c|rc}N][.postN][.devN]`

Essa expressão permite uma numeração padrão, por exemplo, 1.2 ou 1.2.3. Há mais alguns detalhes que devem ser observados:

- A versão 1.2 é equivalente à versão 1.2.0; a versão 1.3.4 é equivalente à versão 1.3.4.0, e assim por diante.
- Versões que correspondam a `N[.N]+` são consideradas versões *finais* .
- Versões baseadas em datas, como 2013.06.22, são consideradas inválidas. Ferramentas automáticas, criadas para detectar números de versões no formato da PEP 440, gerarão um erro (ou deveriam) caso detectem um número de versão maior ou igual a 1980.
- Componentes finais também podem usar o formato a seguir:

- $N[N].a_N$ (por exemplo, 1.2a1): representa uma versão alfa; é uma versão que pode estar instável ou há funcionalidades faltando.
- $N[N].b_N$ (por exemplo, 2.3.1b2): representa uma versão beta, isto é, uma versão que pode ter todas as funcionalidades, mas ainda contém bugs.
- $N[N].c_N$ ou $N[N].rc_N$ (por exemplo, 0.4rc1): representa uma candidata a lançamento (release). É uma versão que poderá ser lançada como definitiva, a menos que surjam bugs significativos. Os sufixos rc e c têm o mesmo significado, mas, se ambos forem usados, as versões rc são consideradas mais recentes do que as versões c .
- Os sufixos a seguir também podem ser usados:
 - O sufixo $.postN$ (por exemplo, 1.4.post2) representa uma versão posterior. Versões posteriores em geral são usadas em caso de erros menores no processo de publicação, por exemplo, erros nas notas de lançamento da versão. Você não deve utilizar o sufixo $.postN$ quando lançar uma versão com correção de bugs; em vez disso, incremente o número da versão secundária (minor).
 - O sufixo $.devN$ (por exemplo, 2.3.4.dev3) representa uma versão em desenvolvimento. Ela representa um pré-lançamento da versão especificada: por exemplo, 2.3.4.dev3 representa a terceira versão de desenvolvimento da versão 2.3.4, anterior a qualquer versão alfa, beta, candidata ou à versão definitiva. O uso desse sufixo não é incentivado porque é mais difícil de ser interpretado por pessoas.

Esse esquema deve bastar para os casos de uso mais comuns.

NOTA Talvez você já tenha ouvido falar da atribuição de *Versões Semânticas* (Semantic Versioning), que têm suas próprias diretrizes para a numeração de versões. Sua especificação se sobrepõe parcialmente à PEP 440, mas, infelizmente, elas não são totalmente compatíveis. Por exemplo, no sistema das Versões Semânticas, para a numeração de versões de pré-lançamento, a recomendação é utilizar um esquema como 1.0.0-alpha+001, que não é compatível com a PEP 440.

Muitas plataformas de *DVCS* (Distributed Version Control System, ou

Sistema Distribuído de Controle de Versões), por exemplo, o Git e o Mercurial, são capazes de gerar números de versões usando um hash de identificação (para o Git, consulte o `git describe`). Infelizmente, esse sistema não é compatível com o esquema definido pela PEP 440: para começar, hashes de identificação não são ordenáveis.

Estilo de programação e verificações automáticas

Estilo de programação é um assunto delicado, mas sobre o qual devemos falar antes de nos aprofundarmos na linguagem Python. De modo diferente de várias linguagens de programação, Python faz uso de *indentação* para definir blocos. Embora essa seja uma solução simples para a velha pergunta “Onde devo colocar minhas chaves?”, ela introduz outra questão: “Como devo indentar?”.

Essa foi uma das primeiras perguntas levantadas pela comunidade; desse modo, o pessoal de Python, com toda a sua vasta sabedoria, criou a *PEP 8: Style Guide for Python Code* (Guia de Estilo para Código Python, <https://www.python.org/dev/peps/pep-0008/>).

Esse documento define o estilo padrão para escrever código Python. A lista de diretrizes se reduz às seguintes:

- Utilize quatro espaços por nível de indentação.
- Limite todas as linhas a um máximo de 79 caracteres.
- Separe as definições de funções de nível mais alto e de classes com duas linhas em branco.
- Codifique os arquivos usando ASCII ou UTF-8.
- Faça uma importação de módulo por instrução `import` e por linha. Coloque as instruções de importação no início do arquivo, depois dos comentários e das docstrings, inicialmente agrupando as importações da biblioteca padrão, em seguida as bibliotecas de terceiros e, por fim, as bibliotecas locais.
- Não utilize espaços em branco irrelevantes entre parênteses, colchetes ou chaves, nem antes de vírgulas.

- Escreva os nomes das classes usando Camel Case (por exemplo, CamelCase), utilize o sufixo Error nas exceções (se for aplicável) e nomeie as funções com letras minúsculas, usando palavras e underscores (por exemplo, separado_por_underscores). Utilize um underscore na frente de atributos ou métodos `_private`.

Essas diretrizes não são realmente difíceis de seguir, e fazem bastante sentido. A maioria dos programadores Python não tem problemas para segui-las quando escrevem seus códigos.

No entanto, *errare humanum est*, e continua sendo difícil analisar todo o seu código a fim de garantir que ele obedeça às diretrizes da PEP 8. Felizmente, existe uma ferramenta `pep8` (que se encontra em <https://pypi.org/project/pep8/>), capaz de verificar automaticamente qualquer arquivo Python que você lhe enviar. Instale a `pep8` com o `pip`, e você poderá usá-la em um arquivo, assim:

```
$ pep8 hello.py
hello.py:4:1: E302 expected 2 blank lines, found 1
$ echo $?
1
```

Nesse caso, usei a `pep8` em meu arquivo `hello.py`, e a saída mostra as linhas e colunas que não estão em consonância com a PEP 8, exibindo cada problema com um código – no exemplo, são a linha 4 e a coluna 1. Violações das definições *OBRIGATÓRIAS* (MUST) da especificação são informadas como *erros*, e seus códigos de erro começam com *E*. Problemas menores são informados como *avisos* (warnings), e seus códigos de erro começam com *w*. O código de três dígitos após a primeira letra informa o tipo exato de erro ou de aviso.

O dígito das centenas informa a classe geral de um código de erro: por exemplo, erros que começam com E2 informam problemas com espaços em branco, erros que começam com E3 informam problemas com linhas em branco e avisos que começam com w6 sinalizam o uso de recursos obsoletos. Esses códigos estão todos listados na documentação da `pep8` em [readthedocs \(https://pep8.readthedocs.io/\)](https://pep8.readthedocs.io/).

Ferramentas para identificar erros de estilo

A comunidade ainda debate para saber se validar um código em relação à PEP 8, que não faz parte da Biblioteca-Padrão, é uma boa prática. Meu conselho é considerar a execução de uma ferramenta de validação da PEP 8 em seu código-fonte regularmente. Você pode fazer isso facilmente incluindo a ferramenta em seu sistema de integração contínua. Embora essa abordagem pareça um pouco radical, é uma boa maneira de garantir que você continuará a respeitar as diretrizes da PEP 8 no longo prazo. Discutiremos o modo de integrar a `pep8` com o `tox` para automatizar essas verificações na seção “Usando `virtualenv` com o `tox`” na página 118.

A maioria dos projetos de código aberto garante a conformidade com a PEP 8 por meio de verificações automáticas. Usar essas verificações automáticas desde o início do projeto pode deixar os desenvolvedores que estão começando a trabalhar nele frustrados, mas também garante que a base de código sempre terá o mesmo aspecto em todas as partes do projeto. Isso é muito importante para um projeto de qualquer tamanho, no qual haja vários desenvolvedores com diferentes opiniões sobre, por exemplo, a organização dos espaços em branco. Você sabe o que quero dizer.

Também é possível fazer com que seu código ignore determinados tipos de erros e avisos utilizando a opção `--ignore`, assim:

```
$ pep8 --ignore=E3 hello.py
$ echo $?
0
```

Com esse comando, qualquer erro de código E3 em meu arquivo `hello.py` será ignorado. A opção `--ignore` permite a você efetivamente ignorar partes da especificação da PEP 8 que você não queira seguir. Se estiver executando a `pep8` em uma base de código existente, a ferramenta também lhe permite ignorar determinados tipos de problemas, de modo que seja possível manter o foco na correção dos problemas de uma classe de cada vez.

NOTA Se você escreve código C para Python (por exemplo, módulos), o padrão da PEP 7 descreve o estilo de programação que deve ser seguido.

Ferramentas para identificar erros de código

Python também tem ferramentas para verificar erros de código em vez de erros de estilo. Eis alguns exemplos que merecem destaque:

- *Pyflakes* (<https://launchpad.net/pyflakes/>): extensível por meio de plugins.
- *Pylint* (<https://pypi.org/project/pylint/>): verifica se o código está em conformidade com a PEP 8, ao mesmo tempo que faz verificações de erros, por padrão; pode ser estendido por meio de plugins.

Todas essas ferramentas fazem uso de análises estáticas – isto é, fazem parse do código e o analisam, em vez de executá-lo de imediato.

Se você optar por utilizar o Pyflakes, observe que ele não verifica se há conformidade com a PEP 8 por conta própria, portanto, seria necessário utilizar a segunda ferramenta `pep8` para ter as duas tarefas executadas.

Para simplificar, Python tem um projeto chamado `flake8` (<https://pypi.org/project/flake8/>), que combina o `pyflakes` com a `pep8` em um único comando. Ele também acrescenta alguns recursos sofisticados: por exemplo, é capaz de ignorar linhas que contenham `# noqa`, e é extensível por meio de plugins.

Há vários plugins disponíveis para o `flake8`, que podem ser prontamente usados. Por exemplo, instalar `flake8-import-order` (com `pip install flake8-import-order`) fará com que o `flake8` seja estendido, de modo que verifique também se suas instruções `import` estão em ordem alfabética em seu código-fonte. Sim, alguns projetos querem isso.

Na maioria dos projetos de código aberto, o `flake8` é frequentemente usado para verificação de estilos de código. Alguns projetos grandes de código aberto até mesmo escrevem os próprios plugins para o `flake8`, acrescentando verificações de erros como: uso inusitado de `except`, problemas de portabilidade entre Python 2/3, estilo de

importação, formatação perigosa de strings, possíveis problemas de localização etc.

Se você está iniciando um novo projeto, recomendo enfaticamente que utilize uma dessas ferramentas para verificação automática da qualidade e do estilo de seu código. Se já tem uma base de código que não implementa uma verificação automática do código, uma boa abordagem seria executar a ferramenta de sua preferência com a maioria dos avisos desativados, e corrigir os problemas de uma classe de cada vez.

Embora nenhuma dessas ferramentas seja a opção *perfeita* para o seu projeto ou esteja totalmente de acordo com suas preferências, o `flake8` é uma boa maneira de melhorar a qualidade de seu código e torná-lo mais durável.

NOTA Muitos editores de texto, incluindo os famosos *GNU Emacs* e *vim*, têm plugins disponíveis (como o `Flycheck`), que podem executar ferramentas como a `pep8` ou o `flake8` diretamente em seu buffer de código, colocando em destaque qualquer parte do código que não esteja em consonância com a PEP 8, de modo interativo. É uma forma conveniente de corrigir a maioria dos erros de estilo enquanto você escreve o seu código.

Discutiremos como estender esse conjunto de ferramentas no Capítulo 9, com nosso próprio plugin para verificar se um método está declarado corretamente.

Joshua Harlow fala sobre Python

Joshua Harlow é desenvolvedor Python. Foi um dos líderes técnicos da equipe do OpenStack na Yahoo! entre 2012 e 2016 e, atualmente, trabalha na GoDaddy. Josh é autor de várias bibliotecas Python como *Taskflow*, *automaton* e *Zake*.

O que levou você a começar a usar Python?

Comecei a programar com Python 2.3 ou 2.4 aproximadamente em 2004, durante um estágio na IBM, perto de Poughkeepsie, estado de Nova York (a maioria dos meus parentes e familiares é do norte do estado de NY – um alô para todos eles!). Já me

esqueci do que eu estava fazendo exatamente por lá, mas envolvia wxPython e um pouco de código Python com o qual eles estavam trabalhando a fim de automatizar algum sistema.

Depois desse estágio, voltei para a escola, fui fazer mestrado no Rochester Institute of Technology e acabei trabalhando na Yahoo!.

Posteriormente, acabei fazendo parte da equipe de CTO, na qual eu e alguns colegas fomos incumbidos de determinar a plataforma de nuvem de código aberto que usaríamos. Chegamos ao OpenStack, o qual foi quase totalmente escrito em Python.

O que você adora e o que você detesta na linguagem Python?

Alguns dos aspectos que adoro (não é uma lista completa) são:

- Sua simplicidade: Python é realmente fácil para os iniciantes se envolverem, e para os desenvolvedores experientes se manterem envolvidos.
- Verificação de estilo: ler um código que você escreveu hoje no futuro é uma parte relevante do desenvolvimento de software, e ter uma consistência que seja garantida por ferramentas como flake8, pep8 e Pylint pode realmente ajudar.
- A capacidade de escolher estilos de programação e combiná-los conforme for mais apropriado.

Alguns dos aspectos que não gosto (não é uma lista completa) são:

- A transição, de certa forma complicada, de Python 2 para 3 (a versão 3.6 resolveu a maior parte dos problemas nesse caso).
- Lambdas são simplistas demais, e deveriam ser mais eficazes.
- A ausência de um instalador de pacotes decente: acho que o pip precisa de algum trabalho, por exemplo, o desenvolvimento de um verdadeiro solucionador de dependências.
- O GIL (Global Interpreter Lock, ou Trava Global do Interpretador) e a necessidade de tê-lo. Ele me deixa triste. . . (mais sobre o GIL no Capítulo 11).

- A falta de suporte nativo para multithreading: atualmente, é necessária a adição de um modelo `asyncio` explícito.
- A divisão da comunidade Python; ocorre basicamente em torno da separação entre CPython e PyPy (e outras variantes).

Você trabalha no `debtcollector`, um módulo Python para gerenciar avisos sobre recursos obsoletos. Como é o processo de iniciar uma nova biblioteca?

A simplicidade mencionada antes faz com que seja realmente fácil criar uma biblioteca e publicá-la para que outras pessoas possam usá-la. Como esse código foi proveniente de uma das outras bibliotecas com a qual trabalho (a `taskflow`), foi relativamente fácil transplantar e estender o código sem ter de me preocupar com um design ruim para a API. Fico muito feliz que outras pessoas (tanto na comunidade OpenStack como fora dela) a tenham achado necessária ou viram a sua utilidade, e espero que a biblioteca evolua de modo a acomodar outros estilos de padrões de recursos obsoletos que outras bibliotecas (e aplicações?) achem convenientes.

Em sua opinião, o que falta em Python?

Python poderia ter um melhor desempenho na compilação JIT (Just-in-Time). A maioria das linguagens mais recentes sendo criadas (como Rust, Node.js usando a engine JavaScript V8 do Chrome e outras) tem muitos dos recursos de Python, mas também são compiladas com JIT. Seria muito bom se o CPython padrão também pudesse ser compilado com JIT, de modo que Python pudesse concorrer com essas linguagens mais recentes no que diz respeito ao desempenho.

Python também precisa realmente de um conjunto mais robusto de padrões de concorrência; estou me referindo não só à biblioteca `asyncio` de baixo nível e a padrões de estilos de threading, mas a conceitos de alto nível, que ajudem a criar aplicações com um bom desempenho em uma escala maior. A biblioteca Python `goless`, na

verdade, adotou parte dos conceitos de Go, oferecendo um modelo de concorrência embutido (built-in). Acredito que esses padrões de nível mais alto deveriam estar disponíveis como padrões de primeira classe, disponíveis na Biblioteca-Padrão e mantidos para que os desenvolvedores possam usá-los quando acharem apropriado. Sem eles, não vejo como Python vai poder competir com outras linguagens que ofereçam esses recursos.

Até a próxima; continue programando e seja feliz!

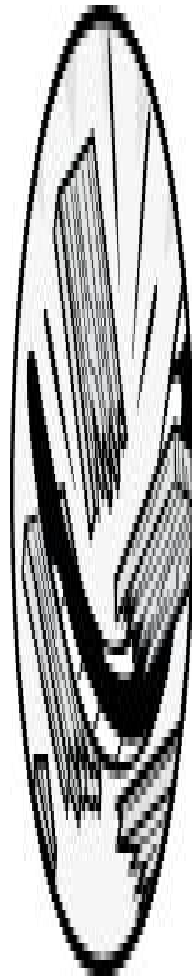
1 N.T.: O lançamento ocorreu em 14 de outubro de 2019, de acordo com <https://www.python.org/downloads/release/python-380/>.

2 N.T.: A versão mais recente, que é a versão 3.7.7, foi lançada em 10 de março de 2020, de acordo com <https://www.python.org/downloads/release/python-377/>.

3 N.T.: Conforme nota de rodapé anterior, a versão 3.8 foi lançada em outubro de 2019.

2

MÓDULOS, BIBLIOTECAS E FRAMEWORKS



Os módulos são uma parte essencial do que torna Python extensível. Sem eles, Python seria apenas uma linguagem criada em torno de um interpretador monolítico, e não iria florescer em um ecossistema gigantesco que permite aos desenvolvedores criar aplicações de forma simples e rápida, combinando extensões. Neste

capítulo, apresentarei alguns dos recursos que fazem com que os módulos Python sejam ótimos: dos módulos embutidos (built-in) que você precisa conhecer, até os frameworks gerenciados externamente.

Sistema de importação

Para usar módulos e bibliotecas em seus programas, você deve importá-los usando a palavra reservada `import`. Como exemplo, a Listagem 2.1 importa as importantes diretrizes do Zen de Python.

Listagem 2.1 – Zen de Python

```
>>> import this
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Complex is better than complicated.
```

```
Flat is better than nested.
```

```
Sparse is better than dense.
```

```
Readability counts.
```

```
Special cases aren't special enough to break the rules.
```

```
Although practicality beats purity.
```

```
Errors should never pass silently.
```

```
Unless explicitly silenced.
```

```
In the face of ambiguity, refuse the temptation to guess.
```

```
There should be one-- and preferably only one --obvious way to do it.
```

```
Although that way may not be obvious at first unless you're Dutch.
```

```
Now is better than never.
```

```
Although never is often better than *right* now.
```

```
If the implementation is hard to explain, it's a bad idea.
```

```
If the implementation is easy to explain, it may be a good idea.
```

```
Namespaces are one honking great idea -- let's do more of those!
```

O sistema de importação é bem complexo, e partirei do pressuposto de que você já conhece o básico, portanto, descreverei parte dos detalhes internos desse sistema, incluindo como funciona o módulo `sys`, como modificar ou adicionar paths de importação e como utilizar importadores personalizados.

Inicialmente, você deve saber que a palavra reservada `import`, na verdade, é um wrapper em torno de uma função chamada `__import__`. Eis um modo conhecido de importar um módulo:

```
>>> import itertools
>>> itertools
<module 'itertools' from '/usr/.../>
```

Esse comando é exatamente equivalente ao método a seguir:

```
>>> itertools = __import__("itertools")
>>> itertools
<module 'itertools' from '/usr/.../>
```

Também podemos imitar a palavra reservada `as` de `import`, conforme mostram estas duas maneiras equivalentes de fazer a importação:

```
>>> import itertools as it
>>> it
<module 'itertools' from '/usr/.../>
```

Eis o segundo exemplo:

```
>>> it = __import__("itertools")
>>> it
<module 'itertools' from '/usr/.../>
```

Embora `import` seja uma palavra reservada em Python, internamente, ela é uma função simples, acessível com o nome `__import__`. Conhecer a função `__import__` é muito conveniente, pois, em alguns casos (inusitados), talvez você queira importar um módulo cujo nome não seja previamente conhecido, da seguinte maneira:

```
>>> random = __import__("RANDOM".lower())
>>> random
<module 'random' from '/usr/.../>
```

Não se esqueça de que os módulos, uma vez importados, são basicamente objetos cujos atributos (classes, funções, variáveis e assim por diante) são objetos.

Módulo `sys`

O módulo `sys` provê acesso a variáveis e funções relacionadas ao próprio Python e ao sistema operacional no qual ele está executando. Esse módulo também contém muitas informações

sobre o sistema de importação de Python.

Em primeiro lugar, podemos obter a lista dos módulos importados no momento, utilizando a variável `sys.modules`. A variável `sys.modules` é um dicionário cuja chave é o nome do módulo que você quer inspecionar e o valor devolvido é o objeto módulo. Por exemplo, assim que o módulo `os` é importado, podemos consultá-lo digitando:

```
>>> import sys
>>> import os
>>> sys.modules['os']
<module 'os' from '/usr/lib/python2.7/os.pyc'>
```

A variável `sys.modules` é um dicionário Python padrão que contém todos os módulos carregados. Isso significa que chamar `sys.modules.keys()`, por exemplo, devolverá a lista completa dos nomes dos módulos carregados.

Também podemos consultar a lista de módulos embutidos (built-in) usando a variável `sys.builtin_module_names`. Os módulos embutidos compilados para o seu interpretador podem variar conforme as opções de compilação passadas para o sistema de construção de Python.

Paths de importação

Ao importar módulos, Python conta com uma lista de paths para saber em que locais o módulo deve ser procurado. Essa lista é armazenada na variável `sys.path`. Para verificar quais são os paths em que o seu interpretador procurará os módulos, basta digitar `sys.path`.

Você pode modificar essa lista, acrescentando ou removendo paths conforme for necessário, ou até mesmo alterando a variável de ambiente `PYTHONPATH` para acrescentar paths, sem escrever nenhum código Python. Acrescentar paths à variável `sys.path` pode ser conveniente se você quiser instalar módulos Python em locais que não são padrões, por exemplo, em um ambiente de testes. Em operações habituais, porém, não será necessário modificar a variável de path. As abordagens a seguir são quase equivalentes –

quase porque o path não será inserido no mesmo nível na lista; essa diferença talvez não vá importar, dependendo de seu caso de uso:

```
>>> import sys
>>> sys.path.append('/foo/bar')
```

Esse comando é (quase) igual a:

```
$ PYTHONPATH=/foo/bar python
>>> import sys
>>> '/foo/bar' in sys.path
True
```

É importante observar que uma iteração será feita na lista para encontrar o módulo solicitado, portanto, a ordem dos paths em `sys.path` é importante. Para agilizar a busca, é conveniente colocar antes o path com mais chances de conter o módulo que você está importando. Fazer isso também garante que, se houver dois módulos de mesmo nome, o primeiro módulo encontrado será selecionado.

A última propriedade é particularmente importante porque um erro comum é encobrir módulos Python embutidos com um módulo seu. Seu diretório atual é pesquisado antes do diretório da Biblioteca-Padrão de Python. Isso significa que, se você decidir nomear um de seus scripts como *random.py* e, em seguida, tentar usar `import random`, o arquivo de seu diretório atual será importado no lugar do módulo Python.

Importadores personalizados

Também é possível estender o sistema de importações usando importadores personalizados. Essa é uma técnica que o dialeto Lisp-Python Hy utiliza para ensinar Python a importar arquivos além dos arquivos *.py* ou *.pyc* padrões. (O Hy é uma implementação Lisp baseada em Python, que será discutida posteriormente, na seção “Uma introdução rápida ao Hy” na página 177.)

O *sistema de hook de importação*, como essa técnica é chamada, está definido na PEP 302. Ele permite estender o sistema de importação padrão; por sua vez, isso lhe permite modificar o modo como Python

importa módulos, possibilitando construir o seu próprio sistema de importações. Por exemplo, você poderia escrever uma extensão que importe módulos de um banco de dados pela rede ou que faça algumas verificações de sanidade antes de importar qualquer módulo.

Python oferece duas maneiras diferentes, porém relacionadas, de ampliar o sistema de importações: os meta path finders para usar com `sys.meta_path` e o path entry finders para usar com `sys.path_hooks`.

Meta path finders

O *meta path finder* é um objeto que permite carregar objetos personalizados, assim como arquivos `.py` padrões. Um objeto meta path finder deve expor um método `find_module(fullname, path=None)` que devolve um objeto loader. O objeto loader também deve ter um método `load_module(fullname)` responsável por carregar o módulo que está em um arquivo-fonte.

Para ilustrar, a Listagem 2.2 mostra como o Hy utiliza um meta path finder personalizado para permitir que Python importe arquivos-fontes terminados com `.hy` em vez de `.py`.

Listagem 2.2 – Um importador de módulos Hy

```
class MetaImporter(object):
    def find_on_path(self, fullname):
        fls = ["%s/___init__.hy", "%s.hy"]
        dirpath = "/" . join(fullname.split("."))

        for pth in sys.path:
            pth = os.path.abspath(pth)
            for fp in fls:
                composed_path = fp % ("%s/%s" % (pth, dirpath))
                if os.path.exists(composed_path):
                    return composed_path

    def find_module(self, fullname, path=None):
        path = self.find_on_path(fullname)
        if path:
            return MetaLoader(path)
```



```
sys.meta_path.append(MetaImporter())
```

Assim que Python tiver determinado que o path é válido e que aponta para um módulo, um objeto `MetaLoader` será devolvido, como mostra a Listagem 2.3.

Listagem 2.3 – Um objeto loader de módulos Hy

```
class MetaLoader(object):
    def __init__(self, path):
        self.path = path

    def is_package(self, fullname):
        dirpath = "/".join(fullname.split("."))
        for pth in sys.path:
            pth = os.path.abspath(pth)
            composed_path = "%s/%s/__init__.hy" % (pth, dirpath)
            if os.path.exists(composed_path):
                return True
        return False

    def load_module(self, fullname):
        if fullname in sys.modules:
            return sys.modules[fullname]

        if not self.path:
            return

        sys.modules[fullname] = None
        ❶ mod = import_file_to_module(fullname, self.path)

        ispkg = self.is_package(fullname)

        mod.__file__ = self.path
        mod.__loader__ = self
        mod.__name__ = fullname

        if ispkg:
            mod.__path__ = []
            mod.__package__ = fullname
        else:
            mod.__package__ = fullname.rpartition('.')[0]
```

```
sys.modules[fullname] = mod
return mod
```

Em ❶, `import_file_to_module` lê um arquivo-fonte `.hy`, compila esse arquivo para código Python e devolve um objeto módulo de Python.

Esse loader é bem simples: assim que o arquivo `.hy` é encontrado, ele é passado para o loader, que o compila se for necessário, registra, define alguns atributos e, em seguida, devolve-o para o interpretador Python.

O módulo `uprefix` é outro bom exemplo desse recurso em ação. As versões de Python 3.0 a 3.2 não aceitavam o prefixo `u` para representar strings Unicode, que existia em Python 2; o módulo `uprefix` garante a compatibilidade entre as versões 2 e 3 de Python, removendo o prefixo `u` das strings antes da compilação.

Bibliotecas-padrões úteis

Python inclui uma enorme biblioteca-padrão repleta de ferramentas e recursos para praticamente qualquer finalidade que você possa imaginar. Iniciantes em Python, que estavam acostumados a ter de escrever as próprias funções para tarefas básicas, com frequência ficam chocados ao descobrir que a própria linguagem já inclui muitas funcionalidades prontas para usar.

Sempre que se sentir tentado a escrever a própria função para lidar com uma tarefa simples, pare e procure na biblioteca-padrão antes. Na verdade, você deve passar os olhos por tudo ao menos uma vez, antes de começar a trabalhar com Python; desse modo, na próxima vez que precisar de uma função, você terá uma ideia e saberá se ela já está na biblioteca-padrão.

Falaremos de alguns desses módulos, como `functools` e `itertools`, em capítulos mais adiante, mas apresentarei a seguir alguns dos módulos padrões que, sem dúvida, você achará úteis:

- `atexit`: permite que você registre funções para o seu programa chamar ao sair.

- `argparse`: disponibiliza funções para fazer parse de argumentos da linha de comando.
- `bisect`: disponibiliza algoritmos de bissecção para ordenar listas (veja o Capítulo 10).
- `calendar`: disponibiliza uma série de funções relacionadas a datas.
- `codecs`: disponibiliza funções para codificação e decodificação de dados.
- `collections`: disponibiliza diversas estruturas de dados convenientes.
- `copy`: disponibiliza funções para copiar dados.
- `csv`: disponibiliza funções para ler e escrever arquivos CSV.
- `datetime`: disponibiliza classes para lidar com datas e horas.
- `fnmatch`: disponibiliza funções para correspondência de padrões para nomes de arquivo em estilo Unix.
- `concurrent`: disponibiliza processamento assíncrono (nativo em Python 3, disponível para Python 2 via PyPI).
- `glob`: disponibiliza funções para correspondência de padrões para paths em estilo Unix.
- `io`: disponibiliza funções para tratar streams de E/S. Em Python 3, contém também a `StringIO` (no módulo de mesmo nome em Python 2), que permite tratar strings como arquivos.
- `json`: disponibiliza funções para ler e escrever dados em formato JSON.
- `logging`: disponibiliza acesso à funcionalidade embutida de logging do próprio Python.
- `multiprocessing`: permite que você execute vários subprocessos a partir de sua aplicação, ao mesmo tempo que disponibiliza uma API que faz com que pareçam threads.
- `operator`: disponibiliza funções que implementam os operadores básicos de Python, que você poderá usar em vez de escrever as próprias expressões lambda (veja o Capítulo 10).
- `os`: disponibiliza acesso a funções básicas do sistema operacional.

- `random`: disponibiliza funções para gerar números pseudoaleatórios.
- `re`: disponibiliza a funcionalidade de expressões regulares.
- `sched`: disponibiliza um escalonador de eventos sem usar `multithreading`.
- `select`: disponibiliza acesso às funções `select()` e `poll()` para criar laços de eventos.
- `shutil`: disponibiliza acesso a funções de alto nível para arquivos.
- `signal`: disponibiliza funções para tratar sinais POSIX.
- `tempfile`: disponibiliza funções para criar arquivos e diretórios temporários.
- `threading`: oferece acesso de alto nível à funcionalidade de `threading`.
- `urllib` (e `urllib2` e `urlparse` em Python 2.x): disponibiliza funções para tratar e fazer parse de URLs.
- `uuid`: permite gerar UUIDs (Universally Unique Identifiers, ou Identificadores Únicos Universais).

Utilize essa lista como uma referência rápida para saber o que esses módulos de biblioteca convenientes fazem. Se você puder memorizar, ainda que seja uma parte dessa lista, melhor ainda. Quanto menos tempo você gastar procurando módulos da biblioteca, mais tempo terá para escrever o código que é realmente necessário.

A maior parte da biblioteca-padrão está escrita em Python, portanto não há nada que impeça você de analisar o código-fonte dos módulos e das funções. Se estiver na dúvida, olhe explicitamente o código e veja o que ele faz. Mesmo que a documentação descreva tudo que você precisa saber, sempre haverá uma chance de aprender algo útil.

Bibliotecas externas

Segundo a filosofia de “pilhas incluídas” de Python, assim que ele estiver instalado, você deverá ter tudo que é necessário para criar o que quiser. Isso serve para evitar o equivalente em programação a desembrulhar um presente incrível, somente para descobrir que quem quer que o tenha dado a você se esqueceu de comprar as pilhas.

Infelizmente, não há maneiras de os responsáveis por Python preverem *tudo* que você queira fazer. Mesmo que pudessem, a maioria das pessoas não iria querer lidar com um download de vários gigabytes, sobretudo se quiserem apenas escrever um script rápido para renomear arquivos. Desse modo, mesmo com suas diversas funcionalidades, a Biblioteca-Padrão de Python não inclui tudo. Felizmente, os membros da comunidade Python criaram bibliotecas externas.

A Biblioteca-Padrão de Python é um território seguro e bem definido: seus módulos são muito bem documentados, e há pessoas suficientes que os utilizam regularmente a ponto de você poder se sentir seguro de que ela não causará grandes falhas quando você experimentar usá-la – e, na pouco provável eventualidade de ela *realmente* falhar, você poderá ter certeza de que alguém vai corrigi-la rapidamente. As bibliotecas externas, por outro lado, são as partes do mapa em que se lê “pode haver dragões aqui”: a documentação talvez seja esparsa, as funcionalidades podem apresentar bugs e as atualizações podem ser esporádicas ou até mesmo inexistentes. Qualquer projeto sério provavelmente precisará de funcionalidades que somente as bibliotecas externas poderão oferecer, mas você deve estar ciente dos riscos envolvidos ao usá-las.

Eis uma história sobre os perigos das bibliotecas externas, diretamente das trincheiras. O OpenStack utiliza o SQLAlchemy, que é um conjunto de ferramentas de banco de dados para Python. Se você tem familiaridade com o SQL, saberá que os esquemas de banco de dados podem mudar com o tempo; assim, o OpenStack também fez uso do `sqlalchemy-migrate` para atender às necessidades de

migração de esquemas. E ele funcionou. . . até que deixou de funcionar. Os bugs começaram a se acumular, e nada estava sendo feito a esse respeito. Nessa época, o OpenStack também estava interessado em oferecer suporte para Python 3, mas não havia nenhum sinal de que o `sqlalchemy-migrate` estivesse se movendo em direção a Python 3. Naquela altura, estava claro que o `sqlalchemy-migrate` estava efetivamente morto para atender às nossas necessidades e que precisávamos mudar para outra ferramenta – nossas necessidades estavam além dos recursos da biblioteca externa. Atualmente – quando este livro foi escrito – os projetos do OpenStack estão migrando para usar o Alembic, uma nova ferramenta para migração de bancos de dados SQL com suporte para Python 3. Isso não está acontecendo sem que haja certo esforço, mas, felizmente, transcorre sem muito sofrimento.

Lista de verificação de segurança para bibliotecas externas

Tudo isso se reduz a uma pergunta importante: como podemos ter certeza de que não cairemos nessa armadilha das bibliotecas externas? Infelizmente não podemos: os programadores também são seres humanos e não há meios de saber, com certeza, se uma biblioteca que é zelosamente mantida hoje ainda estará em boa forma daqui a alguns meses. No entanto, o risco de usar esses tipos de biblioteca talvez possa compensar; é importante avaliar cuidadosamente a sua situação. Na OpenStack, usamos a seguinte lista de verificação ao decidir se vamos usar uma biblioteca externa, e incentivo você a fazer o mesmo:

- **Compatibilidade com Python 3** Mesmo que você não esteja visando a Python 3 no momento, há uma boa chance de que o fará em algum ponto no futuro, portanto, é uma boa ideia verificar se a biblioteca escolhida já é compatível com Python 3, e se ela está comprometida em permanecer dessa forma.
- **Desenvolvimento ativo** O GitHub e o Ohloh em geral fornecem

informações suficientes para determinar se uma dada biblioteca está em desenvolvimento ativo pelos seus mantenedores.

- **Manutenção ativa** Mesmo que uma biblioteca seja considerada concluída (isto é, completa, do ponto de vista das funcionalidades), os mantenedores devem garantir que ela permaneça livre de bugs. Verifique o sistema de monitoração do projeto para ver a rapidez com que os mantenedores respondem aos bugs.
- **Está incluída nas distribuições de sistemas operacionais** Se uma biblioteca está incluída nas principais distribuições de Linux, isso significa que outros projetos dependem dela – portanto, se algo der errado, você não será o único a reclamar. Também é uma boa ideia verificar isso caso você planeje lançar o seu software para o público: seu código será mais fácil de distribuir se suas dependências já estiverem instaladas na máquina do usuário final.
- **Compromisso com compatibilidade de APIs** Não há nada pior do que ver seu software repentinamente apresentar falhas porque uma biblioteca da qual ele depende mudou toda a API. Você pode verificar se a biblioteca escolhida já teve alguma ocorrência desse tipo no passado.
- **Licença** Você deve garantir que a licença seja compatível com o software que planeja escrever, e que ela permita fazer o que quer que você pretenda fazer com o seu código no que concerne à distribuição, modificação e execução.

Aplicar essa lista de verificação às dependências também é uma boa ideia, embora possa ser uma tarefa enorme. Como solução de compromisso, se você souber que a sua aplicação dependerá bastante de uma determinada biblioteca, aplique essa lista de verificação a cada uma das dependências dessa biblioteca.

Protegendo seu código com um wrapper de API

Não importa quais bibliotecas você acabe usando, trate-as como dispositivos úteis, com potencial de causar sérios danos. Por segurança, as bibliotecas devem ser tratadas como qualquer ferramenta física: devem ser mantidas em seu galpão de ferramentas, longe de seus objetos valiosos frágeis, porém disponíveis quando realmente forem necessárias.

Não importa quão útil possa ser uma biblioteca externa, tome cuidado para não deixar que ela coloque as garras em seu código-fonte. Caso contrário, se algo der errado e você tiver de trocar de biblioteca, talvez seja necessário reescrever porções enormes de seu programa. Uma ideia melhor é escrever a sua própria API – um wrapper que encapsule suas bibliotecas externas e as mantenha longe de seu código-fonte. Seu programa jamais precisará saber quais bibliotecas externas está usando, mas conhecerá somente as funcionalidades oferecidas pela sua API. Então, se precisar usar uma biblioteca diferente, tudo que terá de mudar é o seu wrapper. Desde que a nova biblioteca ofereça as mesmas funcionalidades, não será necessário modificar nenhuma das outras partes de sua base de código. Pode haver exceções, mas, provavelmente, não muitas; a maioria das bibliotecas é projetada para solucionar um conjunto muito específico de problemas e, desse modo, pode ser facilmente isolada.

Mais adiante no Capítulo 5, veremos também como é possível usar pontos de entrada para criar sistemas de drivers que permitirão tratar partes de seus projetos como módulos que podem ser trocados, conforme você desejar.

Instalação de pacotes: aproveitando melhor o pip

O projeto `pip` oferece um modo realmente simples de lidar com instalações de pacotes e de bibliotecas externas. Tem desenvolvimento ativo, boa manutenção e acompanha Python desde a versão 3.4. É capaz de instalar ou desinstalar pacotes do

Python Packaging Index (PyPI), um tarball ou um arquivo Wheel (será discutido no Capítulo 5).

Seu uso é simples:

```
$ pip install --user voluptuous
```

```
Downloading/unpacking voluptuous
```

```
Downloading voluptuous-0.8.3.tar.gz
```

```
Storing download in cache at
```

```
./cache/pip/https%3A%2F%2Fpypi.python.org%2Fpackages%2Fsource%2Fv%2Fvolu  
ptuous%2Fvoluptuous-0.8.3.tar.gz
```

```
Running setup.py egg_info for package voluptuous
```

```
Requirement already satisfied (use --upgrade to upgrade): distribute in  
/usr/lib/python2.7/dist-packages (from voluptuous)
```

```
Installing collected packages: voluptuous
```

```
Running setup.py install for voluptuous
```

```
Successfully installed voluptuous
```

```
Cleaning up...
```

Fazendo consultas no índice de distribuição PyPI – qualquer pessoa pode fazer o upload de um pacote aí, para que seja distribuído e instalado por outras pessoas –, o `pip install` é capaz de instalar qualquer pacote.

Você também pode especificar a opção `--user`, que faz com que o `pip` instale o pacote em seu diretório home. Isso evita poluir os diretórios de seu sistema operacional com pacotes disponíveis no âmbito do sistema.

O comando `pip freeze` pode ser usado para listar os pacotes já instalados, da seguinte maneira:

```
$ pip freeze
```

```
Babel==1.3
```

```
Jinja2==2.7.1
```

```
commando=0.3.4
```

```
--trecho omitido--
```

Também é possível desinstalar pacotes com o `pip`, utilizando o comando `uninstall`:

```
$ pip uninstall pika-pool
```

```
Uninstalling pika-pool-0.1.3:
```

```
/usr/local/lib/python2.7/site-packages/pika_pool-0.1.3.dist-info/DESCRIPTION.rst
/usr/local/lib/python2.7/site-packages/pika_pool-0.1.3.dist-info/INSTALLER
/usr/local/lib/python2.7/site-packages/pika_pool-0.1.3.dist-info/METADATA
```

--trecho omitido--

Proceed (y/n)? y

Successfully uninstalled pika-pool-0.1.3

Um recurso muito útil do pip é a sua capacidade de instalar um pacote sem copiar o seu arquivo. O caso de uso típico para esse recurso é aquele no qual você está trabalhando ativamente com um pacote e deseja evitar o processo demorado e enfadonho de reinstalá-lo sempre que precisar testar uma alteração. Isso pode ser feito com o uso da flag `-e <diretório>`:

```
$ pip install -e .
```

```
Obtaining file:///Users/jd/Source/daiquiri
```

```
Installing collected packages: daiquiri
```

```
Running setup.py develop for daiquiri
```

```
Successfully installed daiquiri
```

Nesse exemplo, o pip não copia os arquivos do diretório fonte local, mas coloca um arquivo especial, chamado `egg-link`, em seu path de distribuição. Por exemplo:

```
$ cat /usr/local/lib/python2.7/site-packages/daiquiri.egg-link
```

```
/Users/jd/Source/daiquiri
```

O arquivo `egg-link` contém o path a ser adicionado em `sys.path` para a busca de pacotes. O resultado pode ser facilmente conferido se executarmos o comando a seguir:

```
$ python -c "import sys; print('/Users/jd/Source/daiquiri' in sys.path)"
```

```
True
```

Outra ferramenta útil do pip é a opção `-e` do `pip install`, conveniente para a implantação de códigos que estão em repositórios de diversos sistemas de controle de versões: git, Mercurial, Subversion e até mesmo o Bazaar são aceitos. Por exemplo, você pode instalar qualquer biblioteca diretamente de um repositório git passando seu endereço na forma de um URL, após a opção `-e`:

```
$ pip install -e git+https://github.com/jd/daiquiri.git#egg=daiquiri
```

```
Obtaining daiquiri from git+https://github.com/jd/daiquiri.git#egg=daiquiri
```

```
Cloning https://github.com/jd/daiquiri.git to ./src/daiquiri
Installing collected packages: daiquiri
Running setup.py develop for daiquiri
Successfully installed daiquiri
```

Para que a instalação funcione corretamente, você deve fornecer o nome egg do pacote acrescentando `#egg=` no final do URL. Então, o `pip` simplesmente utiliza `git clone` para clonar o repositório em um `src/<nomeegg>` e cria um arquivo `egg-link` que aponta para esse mesmo diretório clonado.

Esse método é extremamente conveniente quando dependemos de versões de bibliotecas não lançadas, ou quando trabalhamos com um sistema de testes contínuos. No entanto, como não há nenhum sistema de atribuição de versões subjacente, a opção `-e` também pode ser muito problemática. Não há como saber com antecedência se o próximo commit nesse repositório remoto não causará uma falha geral.

Além do mais, todas as outras ferramentas de instalação estão em processo de serem consideradas obsoletas em favor do `pip`, portanto, você pode seguramente tratá-lo como a ferramenta individual que atenderá a todas as suas necessidades de gerenciamento de pacotes.

Usando e escolhendo frameworks

Python tem diversos frameworks disponíveis para vários tipos de aplicações Python: se você estiver escrevendo uma aplicação web, poderá usar Django, Pylons, TurboGears, Tornado, Zope ou Plone; se estiver procurando um framework orientado a eventos, poderá usar o Twisted ou o Circuits, e assim por diante.

A principal diferença entre frameworks e bibliotecas externas é que as aplicações utilizam frameworks fazendo o desenvolvimento com base neles: seu código estenderá o framework, e não o contrário. De modo diferente de uma biblioteca, que é basicamente um add-on que você pode incluir para dar um vigor extra ao seu código, um

framework será o *chassi* de seu código: tudo que você fizer terá esse chassi como base, de alguma maneira. Isso pode ser uma faca de dois gumes. Há muitas vantagens em usar frameworks, por exemplo, uma prototipação e um desenvolvimento rápidos, mas há também algumas desvantagens dignas de nota, por exemplo, o fato de se estar amarrado a eles. Você deve levar em conta essas considerações ao decidir se vai usar um framework.

As recomendações sobre o que você deve verificar para escolher o framework correto para a sua aplicação Python são, em sua maior parte, as mesmas descritas na seção “Lista de verificação de segurança para bibliotecas externas” na página 36 – o que faz sentido, pois os frameworks são distribuídos como pacotes de bibliotecas Python. Às vezes, os frameworks também podem incluir ferramentas para criar, executar e implantar aplicações, mas isso não muda os critérios que você deve aplicar. Já constatamos que substituir uma biblioteca externa depois que você já tiver escrito um código que faça uso dela é um problema; contudo, substituir um framework será mil vezes pior, em geral exigindo uma reescrita completa de seu programa, do zero.

Para dar um exemplo, o framework Twisted mencionado antes ainda não oferece suporte completo para Python 3: se você escreveu um programa usando o Twisted alguns anos atrás e quer atualizá-lo para que execute em Python 3, você tem um problema. Terá de reescrever todo o seu programa para que utilize um framework diferente, ou terá de esperar até que alguém, finalmente, resolva fazer um upgrade do Twisted, para que ele ofereça suporte completo para Python 3.

Alguns frameworks são mais leves que outros. Por exemplo, o Django tem a própria funcionalidade de ORM embutida; o Flask, por outro lado, não tem nada desse tipo. Quanto *menos* um framework tentar fazer por você, menos problemas você terá com ele no futuro. No entanto, cada funcionalidade que não estiver presente em um framework representará outro problema que você terá de resolver,

seja escrevendo o seu próprio código ou passando pela inconveniência de escolher outra biblioteca para cuidar desse problema. Cabe a você decidir o cenário com o qual prefere lidar, mas escolha com sabedoria: desistir de um framework quando a situação ficar ruim pode ser uma tarefa hercúlea, e, mesmo com todas as demais funcionalidades, não há nada em Python que poderá ajudar você nesse caso.

Doug Hellmann, desenvolvedor do núcleo de Python, fala sobre bibliotecas Python

Doug Hellmann é desenvolvedor sênior na DreamHost, além de colaborador e colega de trabalho no projeto OpenStack. Lançou o site *Python Module of the Week* (Módulo Python da Semana, <http://www.pymotw.com/>) e escreveu um ótimo livro chamado *The Python Standard Library by Example*. É também desenvolvedor do núcleo de Python. Fiz algumas perguntas a Doug sobre a Biblioteca-Padrão e sobre o design de bibliotecas e aplicações com base nela.

Ao começar a escrever uma aplicação Python do zero, qual é o seu primeiro passo?

Os passos para escrever uma aplicação do zero são parecidos com o hacking de uma aplicação existente, do ponto de vista abstrato, mas os detalhes mudam.

Quando modifico um código existente, começo descobrindo como ele funciona e onde minhas mudanças devem ser feitas. Posso utilizar algumas técnicas de depuração: adicionar logging ou instruções print, ou usar o `pdb` e executar a aplicação com dados de teste a fim de garantir que eu saiba o que ele está fazendo. Em geral, faço a alteração e a teste manualmente; em seguida, acrescento testes automatizados antes de contribuir com a correção.

Adoto a mesma abordagem exploratória quando crio uma aplicação – escrevo um pouco de código e o executo

manualmente; assim que tiver a funcionalidade básica pronta, escrevo testes para garantir que incluí todos os casos inusitados. Criar os testes também pode resultar em um pouco de refatoração para que fique mais fácil trabalhar com o código.

Isso foi, sem dúvida, o que aconteceu com o smiley (uma ferramenta para espionar seus programas Python e registrar suas atividades). Comecei fazendo experimentos com a API de trace de Python, usando alguns scripts descartáveis, antes de criar a verdadeira aplicação. Originalmente, planejei ter uma parte para instrumentar e coletar dados de outra aplicação em execução, e outra para coletar os dados enviados pela rede e salvá-los. Ao adicionar algumas funcionalidades de relatório, percebi que o processamento para reproduzir os dados coletados era quase idêntico ao processamento para coletá-los, antes de tudo. Refatorei algumas classes e consegui criar uma classe-base para a coleta de dados, o acesso ao banco de dados e o gerador de relatórios. Fazer essas classes estarem em conformidade com a mesma API permitiu que eu criasse facilmente uma versão da aplicação de coleta de dados que escrevesse diretamente no banco de dados, em vez de enviar as informações pela rede.

Ao fazer o design de uma aplicação, penso em como a interface de usuário vai funcionar, mas, no caso das bibliotecas, meu foco está no modo como um desenvolvedor utilizará a API. Também pode ser mais fácil escrever testes para os programas que usarão a nova biblioteca antes, e depois escrever o código da biblioteca. Em geral, crio uma série de programas de exemplo na forma de testes e, em seguida, construo a biblioteca para que funcione dessa maneira.

Também percebi que escrever a documentação para uma biblioteca antes de escrever qualquer código me ajuda a pensar nas funcionalidades e nos fluxos de trabalho sem me ater aos detalhes de implementação, e permite que eu registre as escolhas que fiz quanto ao design, de modo que o leitor saiba não só como

usar a biblioteca, mas quais eram as minhas expectativas quando a criei.

Qual é o processo de fazer um módulo ser incluído na Biblioteca-Padrão de Python?

O processo completo e as diretrizes para submeter um módulo para ser incluído na biblioteca-padrão podem ser encontrados no Manual do Desenvolvedor de Python em <https://docs.python.org/devguide/stdlibchanges.html>.

Antes que um módulo possa ser adicionado, o solicitante deve provar que ele é estável e útil de modo generalizado. O módulo deve oferecer algo que seja difícil de ser implementado corretamente por conta própria, ou deve ser muito útil, a ponto de muitos desenvolvedores terem criado as suas próprias variações. A API deve ser clara, e qualquer dependência do módulo deve estar somente na Biblioteca-Padrão.

O primeiro passo seria submeter a ideia de introduzir o módulo na biblioteca-padrão à comunidade por meio da lista *python-ideas*, para avaliar informalmente o nível de interesse. Supondo que a resposta seja positiva, o próximo passo é criar uma PEP (Python Enhancement Proposal, ou Proposta de Melhoria de Python), que deve incluir os motivos para o acréscimo de um módulo e os detalhes de implementação acerca de como ocorrerá a transição.

Como o gerenciamento de pacotes e as ferramentas de descoberta se tornaram muito confiáveis, particularmente o pip e o PyPI, talvez seja mais conveniente manter uma nova biblioteca fora da Biblioteca-Padrão de Python. Uma versão separada permite atualizações mais frequentes com novas funcionalidades e correções de bug, o que pode ser particularmente importante para bibliotecas voltadas a novas tecnologias ou APIs.

Quais são os três principais módulos da Biblioteca-Padrão que você gostaria que as pessoas conhecessem melhor?

Uma ferramenta realmente útil da Biblioteca-Padrão é o módulo

`abc`. Eu uso o módulo `abc` para definir as APIs de extensões dinamicamente carregadas como classes-base abstratas, a fim de ajudar os autores de extensões a saber quais métodos da API são necessários e quais são opcionais. As classes-base abstratas estão incluídas em outras linguagens de POO (Programação Orientada a Objetos), mas percebi que muitos programadores Python não sabem que nós também as temos.

O algoritmo de busca binária do módulo `bisect` é um bom exemplo de um recurso útil que, com frequência, é implementado incorretamente, tornando-o muito apropriado para estar na Biblioteca-Padrão. Particularmente, gosto do fato de ele ser capaz de fazer buscas em listas esparsas nas quais o valor pesquisado pode não estar incluído nos dados.

Há algumas estruturas de dados úteis no módulo `collections` que não são usadas com tanta frequência quanto poderiam. Gosto de usar `namedtuple` para criar estruturas de dados pequenas, semelhantes a classes, que precisem armazenar dados sem nenhuma lógica associada. É muito fácil converter uma `namedtuple` em uma classe comum caso uma lógica tenha de ser acrescentada mais tarde, pois a `namedtuple` aceita acessar atributos pelo nome. Outra estrutura de dados interessante do módulo é o `ChainMap`, que pode servir como uma boa pilha de namespaces. O `ChainMap` pode ser usado para criar contextos para renderizar templates ou gerenciar parâmetros de configuração de diferentes origens, com uma ordem de precedência claramente definida.

Muitos projetos, incluindo o OpenStack e bibliotecas externas, criam as próprias abstrações com base na Biblioteca-Padrão, por exemplo, para o tratamento de data/hora. Em sua opinião, os programadores deveriam se ater à Biblioteca-Padrão, criar suas próprias funções, mudar para alguma biblioteca externa ou deveriam começar a enviar patches para Python?

Todas as alternativas! Prefiro evitar reinventar a roda, portanto, defendo fortemente que as pessoas colaborem com correções e

melhorias em projetos que possam ser usados como dependências. Por outro lado, às vezes faz sentido criar outra abstração e manter esse código separadamente, seja dentro de uma aplicação ou na forma de uma nova biblioteca.

O módulo `timeutils`, utilizado em seu exemplo, é um wrapper razoavelmente fino em torno do módulo `datetime` de Python. A maioria das funções é pequena e simples, mas criar um módulo com as operações mais comuns garante que elas sejam tratadas de modo consistente em todos os projetos. Como muitas das funções são específicas de cada aplicação, pois exigem decisões sobre aspectos como strings de formatação de timestamp ou o que “agora” significa, elas não são boas candidatas para patches da biblioteca Python, nem para serem lançadas como uma biblioteca de propósito geral e adotadas por outros projetos.

Por outro lado, venho trabalhando para remover os serviços de API do OpenStack do framework WSGI (Web Server Gateway Interface, ou Interface de Porta de Entrada de Servidor Web), criado no início do projeto, passando-os para um framework de desenvolvimento web de terceiros. Há muitas opções para criar aplicações WSGI em Python, e, embora talvez precisemos aperfeiçoar uma delas para deixá-la totalmente adequada aos servidores de API do OpenStack, fazer uma contribuição dessas mudanças reutilizáveis no nível acima é preferível a manter um framework “privado”.

Qual seria o seu conselho aos desenvolvedores que hesitam no que concerne às principais versões de Python?

O número de bibliotecas de terceiros que oferece suporte para Python 3 alcançou um volume crítico. Nunca foi tão fácil criar bibliotecas e aplicações para Python 3, e, graças aos recursos de compatibilidade adicionados na versão 3.3, manter o suporte para Python 2.7 também está mais fácil. As principais distribuições de Linux estão trabalhando para serem lançadas com Python 3 instalado, por padrão. Qualquer pessoa que esteja iniciando um

novo projeto em Python deve voltar seriamente os olhos para Python 3, a menos que haja alguma dependência que não tenha sido portada. A essa altura, porém, as bibliotecas que não executam em Python 3 praticamente poderiam ser classificadas como “sem manutenção”.

Quais são as melhores maneiras de separar o código de uma aplicação em uma biblioteca no que diz respeito ao design, ao planejamento prévio, à migração etc.?

As aplicações são um conjunto de “códigos de cola” que mantêm unidas as bibliotecas, com vistas a um propósito específico. Fazer o design de sua aplicação com as funcionalidades para atender a esse propósito na forma de uma biblioteca antes e, em seguida, criar a aplicação garante que o código esteja devidamente organizado em unidades lógicas, o que, por sua vez, facilita os testes. Também significa que as funcionalidades de uma aplicação estarão acessíveis por meio da biblioteca e elas poderão ser recombinadas para criar outras aplicações. Se não adotar essa abordagem, você correrá o risco de as funcionalidades de sua aplicação estarem altamente acopladas à interface de usuário, e isso fará com que sejam mais difíceis de serem modificadas e reutilizadas.

Que conselhos você daria às pessoas que planejam fazer o design de suas próprias bibliotecas Python?

Sempre recomendo fazer o design das bibliotecas e das APIs de cima para baixo (top down), aplicando critérios de design como o SRP (Single Responsibility Principle, ou Princípio da Responsabilidade Única) em cada camada. Pense no que o cliente vai querer fazer com a biblioteca e crie uma API que ofereça esses recursos. Pense nos valores que podem ser armazenados em uma instância e utilizados pelos métodos *versus* o que deve ser sempre passado para cada método. Por fim, pense na implementação e se o código subjacente deveria estar organizado de modo diferente do código da API pública.

O SQLAlchemy é um excelente exemplo da aplicação dessas diretrizes. As camadas de declaração do ORM (Object Relational Mapping, ou Mapeamento Objeto-Relacional), do mapeamento de dados e da geração de expressões são camadas separadas. Um desenvolvedor pode decidir qual é o nível correto de abstração para acessar a API e usar a biblioteca de acordo com suas necessidades, em vez de estar sujeito às restrições impostas pelo design da biblioteca.

Quais são os erros mais comuns de programação que você vê quando lê códigos de desenvolvedores Python?

Uma área na qual os idioms de Python são significativamente diferentes dos idioms de outras linguagens são os laços e a iteração. Por exemplo, um dos antipadrões mais comuns que vejo é o uso de um laço `for` para filtrar uma lista, inicialmente concatenando os itens em uma nova lista e, em seguida, processando o resultado em um segundo laço (possivelmente depois de ter passado a lista como argumento para uma função). Quase sempre sugiro converter laços de filtragem como esses em expressões geradoras, que são mais eficientes e mais fáceis de entender. Também é comum ver listas serem combinadas, de modo que seus conteúdos possam, de alguma forma, ser processados em conjunto, em vez de fazer uso de `itertools.chain()`.

Há outros pontos mais sutis que, com frequência, sugiro nas revisões de código, por exemplo, usar um `dict()` como tabela de consulta no lugar de um longo bloco `if:then:else`, garantir que as funções sempre devolvam o mesmo tipo de objeto (por exemplo, uma lista vazia em vez de `None`), reduzir o número de argumentos exigidos por uma função, combinando valores relacionados em um objeto com uma tupla ou uma nova classe e definir classes a serem usadas em APIs públicas em vez de contar com dicionários.

Qual é a sua opinião sobre os frameworks?

Os frameworks são como qualquer outro tipo de ferramenta. Eles podem ajudar, mas você deve tomar cuidado ao escolher um, a fim

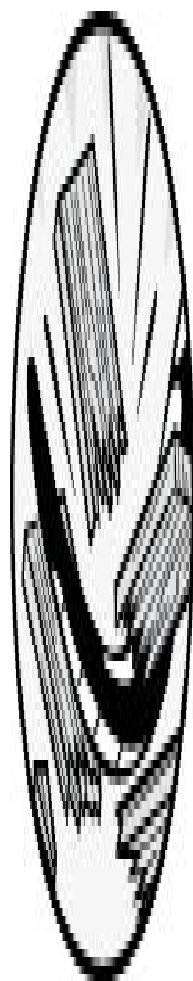
de garantir que ele seja apropriado para a tarefa que você tiver em mãos.

Extrair as partes comuns de sua aplicação e colocá-las em um framework ajuda a manter o foco dos seus esforços de desenvolvimento nos aspectos exclusivos de uma aplicação. Os frameworks também disponibilizam muito código de inicialização, por exemplo, para tarefas como executar em modo de desenvolvimento e escrever uma suíte de testes, os quais ajudam a deixar sua aplicação em um estado apropriado mais rapidamente. Eles também incentivam que haja consistência na implementação da aplicação, e isso significa que você acabará com um código mais fácil de entender e mais reutilizável.

Entretanto, também há algumas armadilhas em potencial. A decisão de usar um framework em particular em geral implica algo sobre o design da própria aplicação. Selecionar o framework errado pode fazer com que seja mais difícil implementar uma aplicação caso essas restrições de design não estejam naturalmente alinhadas com os requisitos da aplicação. Você poderá acabar tendo de lutar contra o framework caso tente usar padrões ou idioms que sejam diferentes daqueles que ele recomenda.

3

DOCUMENTAÇÃO E BOAS PRÁTICAS PARA APIS



Neste capítulo, discutiremos a documentação, especificamente, como automatizar os aspectos mais intrincados e enfadonhos de documentar o seu projeto usando o *Sphinx*. Embora você ainda vá ter de escrever a documentação por conta própria, o Sphinx simplificará a sua tarefa. Como é comum disponibilizar

recursos usando uma biblioteca Python, também veremos como gerenciar e documentar as mudanças em sua API pública. Considerando que a sua API terá de evoluir à medida que você fizer alterações em suas funcionalidades, é raro implementar tudo de forma perfeita desde o início, mas mostrarei algumas atitudes que você pode tomar para garantir que sua API seja a mais apropriada possível aos usuários.

Encerraremos este capítulo com uma entrevista com Christophe de Vienne, autor do framework Web Services Made Easy, na qual ele discute as melhores práticas para desenvolver e manter as APIs.

Documentando com o Sphinx

A documentação é uma das partes mais importantes da escrita de um software. Infelizmente, muitos projetos não oferecem uma documentação apropriada. Escrever a documentação é visto como uma tarefa complicada e desanimadora, mas não precisa ser assim: com as ferramentas disponíveis aos programadores Python, documentar o código pode ser tão fácil quanto escrevê-lo.

Um dos principais motivos de uma documentação ser esparsa ou inexistente é o fato de muitas pessoas partirem do pressuposto de que a única maneira de documentar um código é fazê-lo manualmente. Mesmo com várias pessoas em um projeto, um ou mais membros de sua equipe acabarão tendo de lidar simultaneamente com a tarefa de contribuir com código e manter a documentação – e se você perguntar a qualquer desenvolvedor qual tarefa essa pessoa prefere, pode ter certeza de que ela responderá que prefere escrever software a escrever *sobre* o software.

Às vezes, o processo de documentação é totalmente isolado do processo de desenvolvimento, o que significa que a documentação é feita por pessoas que não escreveram o código propriamente dito. Além do mais, qualquer documentação gerada dessa forma provavelmente estará desatualizada: é quase impossível que uma documentação manual se mantenha em dia com o ritmo do

desenvolvimento, não importa quem cuide dela.

O fato é que, quanto mais graus de separação houver entre o seu código e a documentação, mais difícil será manter essa última de modo apropriado. Então, por que os manter separados? Não só é possível colocar a sua documentação diretamente no próprio código, como também é simples converter essa documentação em arquivos HTML ou PDF, fáceis de ler.

O formato mais comum para documentação de Python é o *reStructuredText*, ou *reST* na forma abreviada. É uma linguagem de marcação leve (como o Markdown), fácil de ler e de escrever, tanto para os seres humanos quanto para os computadores. O Sphinx é a ferramenta mais comum usada para trabalhar com esse formato; ele é capaz de ler um conteúdo formatado em reST e gerar a documentação em diversos formatos diferentes.

Recomendo que a documentação de seu projeto sempre inclua o seguinte:

- O problema que seu projeto pretende solucionar, em uma ou duas sentenças.
- A licença com a qual seu projeto é distribuído. Se seu software tem código aberto, você também deve incluir essa informação em um cabeçalho em cada arquivo de código; somente porque você fez o upload de seu código na internet, não significa que as pessoas saibam o que elas poderão fazer com ele.
- Um pequeno exemplo de como o seu código funciona.
- Instruções de instalação.
- Links para comunidades de apoio, listas de discussão, IRC, fóruns e assim por diante.
- Um link para o seu sistema de monitoração de bugs.
- Um link para o seu código-fonte, de modo que os desenvolvedores possam fazer o download e começar a explorá-lo de imediato.

Inclua também um arquivo *README.rst* que explique o que seu projeto

faz. Esse README deve ser exibido em sua página de projeto no GitHub ou no PyPI; os dois sites sabem lidar com o formato reST.

NOTA Se você usa o GitHub, é possível adicionar também um arquivo `CONTRIBUTING.rst` que será exibido quando alguém submeter um pull request. Ele deverá conter uma lista de verificação a ser seguida pelos usuários antes de submeterem a requisição, incluindo itens como conferir se o seu código está de acordo com a PEP 8 e lembretes para executar testes de unidade. O Read the Docs (<http://readthedocs.org/>) permite a você gerar e publicar sua documentação online automaticamente. Cadastrar-se e configurar um projeto é simples. Em seguida, o Read the Docs buscará o seu arquivo de configuração do Sphinx, gerará a sua documentação e a disponibilizará para que seus usuários a acessem. É um ótimo companheiro para sites que hospedam código.

Introdução ao Sphinx e ao reST

Você pode obter o Sphinx acessando <http://www.sphinx-doc.org/>. Há instruções de instalação no site, mas o método mais simples é instalá-lo com `pip install sphinx`.

Assim que o Sphinx estiver instalado, execute `sphinx-quickstart` no diretório de nível mais alto de seu projeto. Esse comando criará a estrutura de diretórios que o Sphinx espera encontrar, junto com dois arquivos na pasta `doc/source`: `conf.py`, que contém os parâmetros de configuração do Sphinx (e é absolutamente imprescindível para o Sphinx funcionar), e `index.rst`, que serve como a página inicial de sua documentação. Assim que executar o comando quick-start, você será conduzido por uma série de passos para designar as convenções de nomenclatura, as convenções de versões e as opções para outras ferramentas e padrões úteis.

O arquivo `conf.py` contém algumas variáveis documentadas, por exemplo, o nome do projeto, o autor e o tema a serem usados na saída HTML. Sinta-se à vontade para modificar esse arquivo de acordo com a sua conveniência.

Assim que tiver criado a sua estrutura e definido seus defaults, você poderá gerar a sua documentação em HTML chamando `sphinx-build` com o diretório de entrada e o diretório de saída como argumentos, como mostra a Listagem 3.1. O comando `sphinx-build` lê o arquivo `conf.py`

no diretório de entrada e faz parse de todos os arquivos *.rst* desse diretório. Esses arquivos são renderizados em HTML no diretório de saída.

Listagem 3.1 – Gerando um documento HTML básico com o Sphinx

```
$ sphinx-build doc/source doc/build
import pkg_resources
Running Sphinx v1.2b1
loading pickled environment... done
No builder selected, using default: html
building [html]: targets for 1 source files that are out of date
updating environment: 0 added, 0 changed, 0 removed
looking for now-outdated files... none found
preparing documents... done
writing output... [100%] index
writing additional files... genindex search
copying static files... done
dumping search index... done
dumping object inventory... done
build succeeded.
```

Agora você pode abrir o *doc/build/index.html* em seu navegador favorito e ler a documentação.

NOTA Se você estiver usando *setuptools* ou *pbr* (veja o Capítulo 5) para empacotamento, o Sphinx os estenderá de modo a oferecer suporte para o comando `setup.py build_sphinx`, que executará `sphinx-build` automaticamente. A integração do Sphinx com o *pbr* tem alguns defaults mais razoáveis, por exemplo, gerar a documentação no subdiretório */doc*.

Sua documentação começa com o arquivo *index.rst*, mas não precisa parar por aí: o reST aceita diretivas `include` para incluir arquivos reST a partir de outros arquivos reST, portanto, não há nada que impeça você de separar a sua documentação em vários arquivos. Não se preocupe muito com a sintaxe e a semântica no começo; o reST oferece muitas possibilidades de formatação, mas você terá muito tempo para explorar o manual mais tarde. O manual de referência completo (<http://docutils.sourceforge.net/docs/ref/rst/restructuredtext.html>) explica como criar títulos, listas com itens, tabelas etc.

Módulos do Sphinx

O Sphinx é extremamente extensível: sua funcionalidade básica inclui apenas uma documentação manual, mas ele vem com uma série de módulos úteis que permitem uma documentação automática, além de outros recursos. Por exemplo, o `sphinx.ext.autodoc` extrai docstrings formatadas em reST de seus módulos e gera arquivos `.rst` para inclusão. Essa é uma das opções que o `sphinx-quickstart` perguntará se você quer ativar. Contudo, mesmo que você não tenha selecionado essa opção, é possível modificar seu arquivo `conf.py` e acrescentá-la como uma extensão, assim:

```
extensions = ['sphinx.ext.autodoc']
```

Observe que o `autodoc` *não* reconhecerá nem incluirá automaticamente os seus módulos. É necessário informar explicitamente quais módulos você quer que sejam documentados, acrescentando algo como o que está na Listagem 3.2 em um de seus arquivos `.rst`.

Listagem 3.2 – Informando os módulos que o `autodoc` deve documentar

```
.. automodule:: foobar
❶  :members:
❷  :undoc-members:
❸  :show-inheritance:
```

Na Listagem 3.2, fizemos três solicitações, e todas são opcionais: solicitamos que todos os membros documentados sejam exibidos ❶, que todos os membros não documentados sejam exibidos ❷ e que a herança seja mostrada ❸. Observe também o seguinte:

- Se nenhuma diretiva for incluída, o Sphinx não gerará nenhuma saída.
- Se você especificar somente `:members:`, os nós não documentados na árvore de seu módulo, classe ou método serão ignorados, mesmo que todos os seus membros estejam documentados. Por exemplo, se você documentar os métodos de uma classe, mas não a classe em si, `:members:` excluirá tanto a classe como os seus métodos. Para evitar que isso aconteça, você teria de escrever uma docstring para a classe ou especificar `:undoc-members:`

também.

- Seu módulo deve estar em um local a partir do qual Python possa importá-lo. Acrescentar `.`, `..`, e `/` ou `../` em `sys.path` pode ajudar.

A extensão `autodoc` lhe permite incluir a maior parte de sua documentação no código-fonte. Você pode até mesmo selecionar os módulos e os métodos que serão documentados – não é uma solução do tipo “tudo ou nada”. Ao manter sua documentação junto com o código-fonte, você pode facilmente garantir que ela permaneça atualizada.

Automatizando o índice com autosummary

Se você estiver escrevendo uma biblioteca Python, em geral vai querer formatar a documentação de sua API com um índice que contenha links para as páginas individuais de cada módulo.

O módulo `sphinx.ext.autosummary` foi criado especificamente para lidar com esse caso de uso comum. Inicialmente, você deve ativá-lo em seu `conf.py` acrescentando a seguinte linha:

```
extensions = ['sphinx.ext.autosummary']
```

Em seguida, pode acrescentar algo como o que vemos a seguir em um arquivo `.rst`, para gerar automaticamente um índice para os módulos especificados:

```
.. autosummary::
```

```
    mymodule
    mymodule.submodule
```

Arquivos de nomes `generated/mymodule.rst` e `generated/mymodule.submodule.rst` serão criados, contendo as diretivas para o autodoc descritas antes. Utilizando esse mesmo formato, você pode especificar as partes da API de seu módulo que você deseja incluir em sua documentação.

NOTA O comando `sphinx-apidoc` é capaz de criar esses arquivos automaticamente para você; consulte a documentação do Sphinx para saber mais.

Automatizando os testes com doctest

Outro recurso conveniente do Sphinx é a capacidade de executar `doctest` em seus exemplos automaticamente ao criar a documentação. O módulo Python `doctest` padrão busca trechos de código em sua documentação e testa se eles refletem exatamente o que seu código faz. Todo parágrafo iniciado com o prompt principal `>>>` é tratado como um trecho de código a ser testado. Por exemplo, se quiser documentar a função `print` padrão de Python, você poderia escrever o trecho de documentação a seguir, e o `doctest` verificaria o resultado:

```
To print something to the standard output, use the :py:func:`print` function:
>>> print("foobar")
    foobar
```

Ter exemplos como esse em sua documentação permite aos usuários compreender a sua API. No entanto, é fácil adiar e se esquecer de atualizar seus exemplos mais tarde, quando sua API evoluir. Felizmente o `doctest` ajuda a garantir que isso não aconteça. Se sua documentação inclui um tutorial passo a passo, o `doctest` ajudará você a mantê-lo atualizado durante o desenvolvimento, testando cada linha possível.

Você também pode usar o `doctest` para *DDD* (Documentation-Driven Development, ou Desenvolvimento Orientado a Documentação): escreva a sua documentação e os exemplos antes, e então escreva o código que corresponda à sua documentação. Tirar proveito desse recurso é simples e basta executar `sphinx-build` com o builder especial `doctest`, assim:

```
$ sphinx-build -b doctest doc/source doc/build
Running Sphinx v1.2b1
loading pickled environment... done
building [doctest]: targets for 1 source files that are out of date
updating environment: 0 added, 0 changed, 0 removed
looking for now-outdated files... none found
running tests...
```

Document: index

1 items passed all tests:

```
1 tests in default
1 tests in 1 items.
1 passed and 0 failed.
Test passed.
```

```
Doctest summary
=====
1 test
0 failures in tests
0 failures in setup code
0 failures in cleanup code
build succeeded.
```

Ao usar o builder `doctest`, o Sphinx lê os arquivos `.rst` usuais e executa os exemplos de código contidos nesses arquivos.

O Sphinx também oferece vários outros recursos, sejam para uso imediato ou por meio de módulos de extensão, incluindo:

- ligação entre projetos;
- temas HTML;
- diagramas e fórmulas;
- saída para formatos Texinfo e EPUB;
- ligação com documentação externa.

Talvez você não precise de todas essas funcionalidades de imediato, mas, se precisar delas no futuro, é bom conhecê-las com antecedência. Novamente, consulte toda a documentação do Sphinx para saber mais.

Escrevendo uma extensão para o Sphinx

Às vezes, soluções prontas simplesmente não são suficientes e você terá de criar ferramentas personalizadas para lidar com determinada situação.

Suponha que você esteja escrevendo uma API REST HTTP. O Sphinx documentará apenas o lado Python de sua API, e você será forçado a escrever a documentação de sua API REST manualmente, com todos os problemas que isso implica. O criador

do WSME (Web Services Made Easy) – entrevistado no final deste capítulo – criou uma solução: uma extensão para o Sphinx, chamada `sphinxcontrib-pecanwsme`, que analisa as docstrings e o código Python propriamente dito para gerar automaticamente a documentação da API REST.

NOTA Em outros frameworks HTTP, por exemplo, Flask, Bottle e Tornado, você pode usar o `sphinxcontrib.httpdomain`.

A questão principal é que sempre que você souber que pode extrair informações de seu código para gerar a documentação, deve fazê-lo e, além disso, deve automatizar o processo. Isso é melhor do que tentar manter uma documentação escrita manualmente, sobretudo se você puder tirar proveito de ferramentas de publicação automática como o Read the Docs.

Analisaremos a extensão `sphinxcontrib-pecanwsme` como um exemplo para escrever a sua própria extensão para o Sphinx. O primeiro passo é criar um módulo – de preferência, como um submódulo de `sphinxcontrib`, desde que o seu módulo seja suficientemente genérico – e escolher um nome para ele. O Sphinx exige que esse módulo tenha uma função predefinida chamada `setup(app)`, contendo os métodos que você usará para associar o seu código aos eventos e diretivas do Sphinx. A lista completa dos métodos está disponível na API de extensão do Sphinx em <http://www.sphinx-doc.org/en/master/extdev/appapi.html>.

Por exemplo, a extensão `sphinxcontrib-pecanwsme` inclui uma única diretiva chamada `rest-controller`, adicionada com a função `setup(app)`. Essa diretiva adicionada exige um nome de classe de controlador totalmente qualificado para o qual a documentação será gerada, conforme mostra a Listagem 3.3.

Listagem 3.3 – Código de `sphinxcontrib.pecanwsme.rest.setup` que adiciona a diretiva `rest-controller`.

```
def setup(app):  
    app.add_directive('rest-controller', RESTControllerDirective)
```

O método `add_directive` na Listagem 3.3 registra a diretiva `rest-controller` e

delega o seu tratamento para a classe `RestControllerDirective`. Essa classe expõe determinados atributos que informam como a diretiva tratará o conteúdo, se há argumentos, e assim por diante. A classe também implementa um método `run()` que extrai a documentação de seu código e devolve dados para o Sphinx depois de fazer o seu parse.

O repositório em <https://bitbucket.org/birkenfeld/sphinx-contrib/src/> tem vários módulos pequenos que podem ajudar você a desenvolver suas próprias extensões.

NOTA Apesar de o Sphinx ter sido escrito em Python e esteja voltado para essa linguagem por padrão, há extensões disponíveis que permitem que ele aceite também outras linguagens. Você pode usar o Sphinx para documentar totalmente o seu projeto, mesmo que ele utilize várias linguagens ao mesmo tempo.

Como outro exemplo, em um de meus projetos chamado Gnocchi – um banco de dados para armazenar e indexar dados de séries temporais em larga escala – utilizei uma extensão personalizada do Sphinx para gerar a documentação automaticamente. O Gnocchi disponibiliza uma API REST e, em geral, para documentar uma API como essa, haverá exemplos de como devem ser uma requisição à API e a sua resposta, criados manualmente nos projetos. Infelizmente, essa abordagem é suscetível a erros e não estará em sincronia com a realidade.

Utilizando o código de testes de unidade disponível para testar a API do Gnocchi, criamos uma extensão para o Sphinx para executar o Gnocchi e gerar um arquivo `.rst` contendo requisições e respostas HTTP, executadas com um servidor Gnocchi real. Desse modo, garantimos que a documentação esteja atualizada: as respostas do servidor não são criadas manualmente; se uma requisição escrita manualmente falhar, o processo de documentação falhará, e saberemos que é necessário corrigir a documentação.

Esse código é muito extenso para ser incluído no livro, mas você pode acessar o código-fonte do Gnocchi online e ver o módulo `gnocchi.gendoc` para ter uma ideia de como ele funciona.

Administrando mudanças em suas APIs

Um código bem documentado é um sinal aos outros desenvolvedores de que o código é apropriado para ser importado e utilizado na criação de outros componentes. Por exemplo, ao criar uma biblioteca e exportar uma API para outros desenvolvedores usarem, você deve oferecer a segurança de uma documentação sólida.

Esta seção discutirá as melhores práticas para APIs públicas. Elas serão expostas aos usuários de sua biblioteca ou aplicação e, embora você possa fazer o que quiser com as APIs internas, as APIs públicas devem ser tratadas com cuidado.

Para fazer uma distinção entre APIs públicas e privadas, Python tem como convenção prefixar o símbolo de uma API privada com um underscore: `foo` é pública, mas `_bar` é privada. Você deve usar essa convenção tanto para reconhecer se outra API é pública ou privada como para nomear suas próprias APIs. Em comparação com outras linguagens como Java, Python não impõe nenhuma restrição para acessar um código marcado como privado ou público. As convenções de nomenclatura servem apenas para facilitar a compreensão entre os programadores.

Numeração das versões da API

Se for gerado de forma apropriada, o número da versão pode fornecer muitas informações aos usuários de uma API. Python não tem nenhuma convenção ou sistema específico para numerar versões de APIs, mas podemos nos inspirar nas plataformas Unix, que utilizam um sistema complexo de gerenciamento para bibliotecas, com identificadores de versão bem detalhados.

Em geral, seu método de atribuição de versões deve refletir as mudanças na API que causarão impacto nos usuários. Por exemplo, quando a API tiver uma mudança importante, o número principal da versão (major) deve mudar de 1 para 2. Se somente algumas

chamadas novas de API forem acrescentadas, o número secundário poderia passar de 2.2 para 2.3. Se uma mudança envolver somente correções de bugs, a versão poderia passar de 2.2.0 para 2.2.1. Um bom exemplo de como usar numeração de versões está na biblioteca `requests` de Python (<https://pypi.python.org/pypi/requests/>). Essa biblioteca incrementa seus números de API com base no número de mudanças em cada nova versão e no impacto que as mudanças possam causar nos programas consumidores.

Os números das versões dão uma pista aos desenvolvedores, informando que eles devem observar as mudanças entre duas versões de uma biblioteca; contudo, por si só, isso não é suficiente para dar todas as orientações a um desenvolvedor: você deve fornecer uma documentação detalhada para descrever essas mudanças.

Documentando as mudanças em sua API

Sempre que você fizer mudanças em uma API, a primeira e mais importante tarefa a ser feita é documentá-las minuciosamente, de modo que um consumidor de seu código tenha uma visão geral rápida do que está mudando. Sua documentação deve incluir o seguinte:

- novos elementos da nova interface;
- elementos da interface antiga considerados obsoletos;
- instruções sobre como fazer a migração para a nova interface.

Você também deve garantir que não removerá a interface antiga de imediato. Recomendo manter a interface antiga até que haja problemas demais para fazer isso. Se você a marcou como obsoleta, os usuários saberão que não deverão usá-la.

A Listagem 3.4 mostra um exemplo de uma boa documentação de mudança de API em um código que representa um objeto carro, o qual é capaz de virar para qualquer direção. Por qualquer que seja o motivo, os desenvolvedores resolveram desistir do método `turn_left` e,

em seu lugar, disponibilizaram um método `turn` genérico que aceita a direção como argumento.

Listagem 3.4 – Exemplo de documentação de uma mudança de API em um objeto carro.

```
class Car(object):

    def turn_left(self):
        """Turn the car left.

        .. deprecated:: 1.1
            Use :func:`turn` instead with the direction argument set to left
        """
        self.turn(direction='left')

    def turn(self, direction):
        """Turn the car in some direction.

        :param direction: The direction to turn to.
        :type direction: str
        """
        # Escreva aqui o código da função turn
        pass
```

As aspas triplas no exemplo, `"""`, sinalizam o início e o fim das docstrings, que serão incluídas na documentação quando o usuário digitar `help(Car.turn_left)` no terminal, ou quando extrair a documentação com uma ferramenta externa, por exemplo, o Sphinx. O fato de o método `car.turn_left` ter se tornado obsoleto é sinalizado por `.. deprecated 1.1`, no qual 1.1 se refere à primeira versão que disponibiliza esse código como obsoleto.

Utilizar esse método para informar que um recurso está obsoleto e torná-lo visível por meio do Sphinx informa claramente aos usuários que a função não deve ser usada, além de lhes possibilitar um acesso direto à nova função, junto com uma explicação de como fazer a migração do código antigo.

A Figura 3.1 mostra a documentação do Sphinx com explicações sobre algumas funções obsoletas.

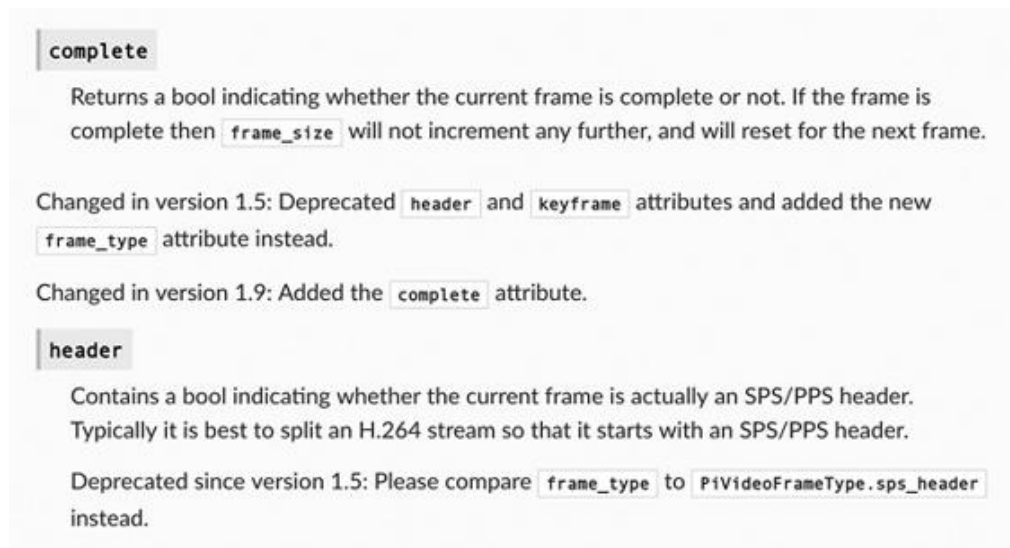


Figura 3.1 – Explicações acerca de algumas funções obsoletas.

A desvantagem dessa abordagem é que ela depende de os desenvolvedores lerem o seu changelog (log de alterações) ou a documentação quando fizerem um upgrade para uma versão mais recente de seu pacote Python. No entanto, há uma solução para isso: marque suas funções obsoletas com o módulo `warnings`.

Marcando funções obsoletas com o módulo `warnings`

Embora os módulos obsoletos devam estar bem marcados na documentação de modo que os usuários não tentem chamá-los, Python também disponibiliza o módulo `warnings`, que permite que seu código gere vários tipos de avisos (warnings) quando uma função obsoleta é chamada. Esses avisos, `DeprecationWarning` e `PendingDeprecationWarning`, podem ser usados para informar ao desenvolvedor que uma função que está sendo chamada é obsoleta, ou se tornará obsoleta, respectivamente.

NOTA Para aqueles que trabalham com C, essa é uma contrapartida conveniente para a extensão `__attribute__((deprecated))` do GCC.

Retomando o exemplo do objeto `carro` da Listagem 3.4, podemos usar esse método para avisar os usuários caso eles tentem chamar funções obsoletas, conforme vemos na Listagem 3.5.

Listagem 3.5 – Uma mudança documentada na API do objeto carro usando o módulo warnings

```
import warnings

class Car(object):
    def turn_left(self):
        """Turn the car left.

        ❶ .. deprecated:: 1.1
            Use :func:`turn` instead with the direction argument set to "left".
        """

        ❷ warnings.warn("turn_left is deprecated; use turn instead",
                        DeprecationWarning)
        self.turn(direction='left')

    def turn(self, direction):
        """Turn the car in some direction.

        :param direction: The direction to turn to.
        :type direction: str
        """

        # Escreva aqui o código propriamente dito
        pass
```

Nesse caso, a função `turn_left` foi definida como obsoleta ❶. Ao acrescentar a linha `warnings.warn`, podemos escrever nossa própria mensagem de erro ❷. Agora, se algum código chamar a função `turn_left`, um aviso semelhante a este será apresentado:

```
>>> Car().turn_left()
__main__:8: DeprecationWarning: turn_left is deprecated; use turn instead
```

Por padrão, Python 2.7 e versões mais recentes não exibem nenhum aviso gerado pelo módulo `warnings` porque os avisos são filtrados. Para ver esses avisos, você deve passar a opção `-w` para o executável de Python. A opção `-w` exibirá todos os avisos em `stderr`. Consulte a man page de Python para mais informações sobre os possíveis valores de `-w`.

Ao executar suítes de testes, os desenvolvedores podem executar Python com a opção `-W error`, que gerará um erro sempre que uma função obsoleta for chamada. Os desenvolvedores que usarem a

sua biblioteca poderão localizar rapidamente o ponto exato em que deverão corrigir o código. A Listagem 3.6 mostra como Python transforma avisos em exceções fatais quando Python é chamado com a opção `-W error`.

Listagem 3.6 – Executando Python com a opção `-W error` e obtendo um erro de recurso obsoleto

```
>>> import warnings
>>> warnings.warn("This is deprecated", DeprecationWarning)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
DeprecationWarning: This is deprecated
```

Os avisos em geral são ignorados durante a execução, e executar um sistema de produção com a opção `-W error` raramente seria uma boa ideia. Executar a suíte de testes de uma aplicação Python com a opção `-W error`, por outro lado, pode ser uma boa maneira de identificar logo os avisos e corrigi-los.

No entanto, escrever manualmente todos esses avisos, atualizar as docstrings, e assim por diante pode se tornar enfadonho; desse modo, a biblioteca `debtcollector` foi criada para ajudar a automatizar parte dessas tarefas. A biblioteca `debtcollector` disponibiliza alguns decoradores que podem ser usados com suas funções para garantir que os avisos corretos sejam gerados e a docstring seja devidamente atualizada. A Listagem 3.7 mostra como podemos, com um simples decorador, informar que uma função foi transferida para outro lugar.

Listagem 3.7 – Uma mudança de API automatizada com o `debtcollector`

```
from debtcollector import moves

class Car(object):
    @moves.moved_method('turn', version='1.1')
    def turn_left(self):
        """Turn the car left."""

        return self.turn(direction='left')
```

```
def turn(self, direction):
    """Turn the car in some direction.

    :param direction: The direction to turn to.
    :type direction: str
    """
    # Escreva aqui o código propriamente dito
    pass
```

Nesse exemplo, usamos o método `moves()` do `debtcollector`, cujo decorador `moved_method` faz com que `turn_left` gere um `DeprecationWarning` sempre que for chamado.

Resumo

O Sphinx é o padrão de mercado para documentar projetos Python. Ele aceita diversas sintaxes, e é fácil acrescentar uma nova sintaxe ou recursos caso seu projeto tenha necessidades específicas. O Sphinx também é capaz de automatizar tarefas como gerar índices ou extrair a documentação de seu código, facilitando mantê-la no longo prazo.

Documentar mudanças em sua API é essencial, particularmente se você tiver funcionalidades obsoletas, para que os usuários não sejam pegos desprevenidos. Os modos de documentar recursos obsoletos incluem a palavra reservada `deprecated` do Sphinx e o módulo `warnings`; além disso, a biblioteca `debtcollector` é capaz de automatizar a manutenção dessa documentação.

Christophe de Vienne fala sobre o desenvolvimento de APIs

Christophe é desenvolvedor Python e autor do framework WSME (Web Services Made Easy), o qual permite que os desenvolvedores definam web services de modo pythônico e oferece suporte para uma infinidade de APIs, além de ser possível conectá-lo a vários outros frameworks web.

Quais são os erros que os desenvolvedores tendem a cometer no design de uma API Python?

Há alguns erros comuns que costumo evitar no design de uma API Python seguindo estas regras:

- **Não complique demais.** Mantenha a simplicidade. APIs complicadas são difíceis de entender e de documentar. Embora a funcionalidade da biblioteca em si não *precise* ser igualmente simples, deixá-la simples para que os usuários não cometam erros facilmente é uma atitude inteligente. Por exemplo, uma biblioteca pode ser simples e intuitiva, mas pode executar tarefas complexas internamente. Por outro lado, uma biblioteca como a `urllib` é quase tão complicada quanto as tarefas que executa, dificultando o seu uso.
- **Deixe a magia visível.** Quando sua API executa tarefas que a documentação não explica, seus usuários finais vão querer explorar minuciosamente o seu código-fonte e ver o que acontece internamente. Não há problemas se você tiver um pouco de magia acontecendo internamente, mas seus usuários finais jamais deverão ver nada inesperado acontecendo; caso contrário, poderão ficar confusos ou contar com um comportamento que poderá mudar.
- **Não se esqueça dos casos de uso.** Quando você está muito concentrado em escrever um código, é fácil se esquecer de pensar no modo como a sua biblioteca será realmente usada. Pensar em bons casos de uso facilita fazer o design de uma API.
- **Escreva testes de unidade.** O *TDD* (Test-Driven Development, ou Desenvolvimento Orientado a Testes) é um modo muito eficaz de escrever bibliotecas, particularmente em Python, porque ele força o desenvolvedor a assumir o papel do usuário final desde o princípio, e o força a fazer o design com vistas à usabilidade. É a única abordagem que conheço que permite que um programador reescreva totalmente uma biblioteca, como último recurso.

Quais são os aspectos de Python que podem afetar o nível de

difficuldade em fazer o design da API de uma biblioteca?

Python não tem nenhum modo embutido (built-in) de definir quais seções da API são públicas e quais são privadas, o que pode ser tanto um problema quanto uma vantagem.

É um problema porque pode fazer com que o desenvolvedor não avalie totalmente quais partes de sua API são públicas e quais partes devem permanecer privadas. Contudo, com um pouco de disciplina, documentação e (se forem necessárias) ferramentas como o `zope.interface`, não seria um problema por muito tempo.

Isso será uma vantagem se for mais rápido e fácil refatorar as APIs, ao mesmo tempo que a compatibilidade com versões anteriores é mantida.

O que você considera ao pensar na evolução de sua API, no processo de torná-la obsoleta e em sua remoção?

Há diversos critérios que avalio ao tomar qualquer decisão acerca do desenvolvimento da API:

- **Qual será o nível de dificuldade dos usuários da biblioteca para adaptarem seus códigos?** Considerando que há pessoas que dependem de sua API, qualquer mudança que você fizer deve compensar o esforço necessário para adotá-la. Essa regra tem o intuito de evitar mudanças incompatíveis nas partes da API que sejam de uso comum. Apesar do que foi dito, uma das vantagens de Python é o fato de ser relativamente fácil refatorar um código para se adaptar a uma mudança de API.
- **Quão fácil será manter a minha API?** Simplificar a implementação, limpar a base de código, deixar a API mais fácil de usar, ter testes de unidade mais completos, deixar a API mais fácil de entender somente com uma observação rápida. . . tudo isso facilitará a sua vida como mantenedor do código.
- **Como posso manter minha API consistente ao aplicar uma mudança?** Se todas as funções em sua API seguirem um padrão semelhante (por exemplo, exigirem o mesmo parâmetro

na primeira posição), certifique-se de que as novas funções seguirão esse padrão também. Além disso, fazer muitas tarefas ao mesmo tempo é uma ótima maneira de acabar não fazendo nenhuma delas corretamente: mantenha sua API com foco naquilo que ela deve fazer.

- **Como os usuários se beneficiarão com a mudança?** Por fim, mas não menos importante, sempre considere o ponto de vista dos usuários.

Quais são os conselhos que você daria sobre documentação de APIs em Python?

Uma boa documentação facilita a adoção de sua biblioteca por aqueles que estiverem começando a trabalhar com ela. Negligenciar isso afastará muitos usuários em potencial – e não apenas aqueles que estão começando. O problema é que documentar é difícil e, portanto, é uma tarefa negligenciada o tempo todo!

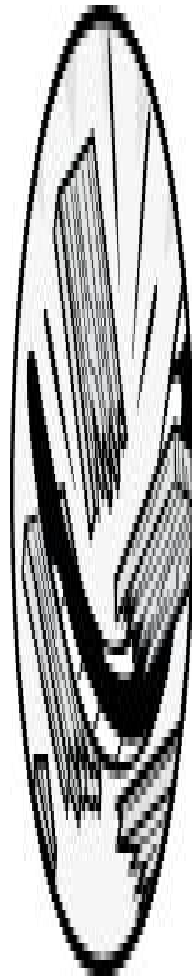
- **Comece a documentar logo e inclua a geração de sua documentação na integração contínua.** Com a ferramenta Read the Docs para gerar e hospedar a documentação, não há desculpas para não ter a documentação gerada e publicada (ao menos, para softwares de código aberto).
- **Utilize docstrings para documentar classes e funções em sua API.** Se você segue as diretrizes da PEP 257 (<https://www.python.org/dev/peps/pep-0257/>), os desenvolvedores não terão de ler seu código-fonte para entender o que a sua API faz. Gere uma documentação HTML a partir de suas docstrings – não se limite à referência para a API.
- **Dê exemplos práticos em todos os lugares.** Tenha pelo menos um “guia de introdução” que mostrará aos novos usuários como criar um exemplo funcional. A primeira página da documentação deve apresentar uma visão geral rápida do básico sobre a sua API e um caso de uso representativo.
- **Documente a evolução de sua API em detalhes, versão por**

versão. Logs de VCSs (Version Control Systems, ou Sistemas de Controle de Versões) são suficientes!

- **Deixe sua documentação acessível e, se possível, agradável para ler.** Seus usuários devem conseguir encontrá-la facilmente e obter as informações de que precisam, sem achar que estão sendo torturados. Publicar a sua documentação por meio do PyPI é um modo de fazer isso; publicar no Read the Docs também é uma boa ideia, pois os usuários esperarão encontrar a sua documentação ali.
- **Por fim, escolha um tema que seja igualmente eficaz e atraente.** Eu escolhi o tema “Cloud” do Sphinx para o WSME, mas há vários outros temas por aí para escolher. Você não precisa ser um expert em web para gerar uma documentação com uma boa aparência.

4

LIDANDO COM TIMESTAMPS E FUSOS HORÁRIOS



Fusos horários são complicados. A maioria das pessoas acha que lidar com fusos horários envolve somente somar ou subtrair algumas horas da referência universal de tempo, o UTC (Coordinated Universal Time), de -12 horas a +12 horas.

Entretanto, a realidade mostra o contrário: os fusos horários não são

lógicos nem previsíveis. Há fusos horários com intervalos de 15 minutos, países que mudam de horário duas vezes ao ano, que utilizam um horário personalizado durante o verão, conhecido como horário de verão, o qual começa em datas distintas, além de inúmeros casos especiais e inusitados. Tudo isso torna a história dos fusos horários interessante, mas também faz com que seja complicado lidar com eles. Todas essas particularidades deveriam fazer você parar e pensar quando tiver de lidar com fusos horários.

Este capítulo descreverá por que lidar com fusos horários é complicado e qual é a melhor maneira de lidar com eles em seus programas. Veremos como criar objetos timestamp, como fazer com que tenham conhecimento do fuso horário e por quê, e como lidar com casos inusitados com os quais você possa vir a deparar.

Problema da ausência de fusos horários

Um timestamp sem um fuso horário associado não fornece informações úteis, pois não é possível inferir o ponto no tempo ao qual a sua aplicação está realmente se referindo. Portanto, sem seus respectivos fusos horários, não podemos comparar dois timestamps; seria como comparar dias da semana sem as datas associadas – o fato de a segunda-feira estar antes ou depois da terça-feira depende da semana em que elas estão. Timestamps sem os fusos horários associados devem ser considerados irrelevantes.

Por esse motivo, sua aplicação jamais deve lidar com timestamps sem fusos horários. Ela deverá gerar um erro se nenhum fuso horário for fornecido, ou deverá deixar claro qual é o fuso horário default pressuposto – por exemplo, escolher o UTC como o fuso horário default é uma prática comum.

Também é preciso tomar cuidado ao fazer qualquer tipo de conversão de fusos horários antes de *armazenar* os timestamps. Suponha que um usuário crie um evento recorrente toda terça-feira às 10h em seu fuso horário local, por exemplo, CET (Central European Time, ou Horário da Europa Central). O CET está uma

hora à frente do UTC, portanto, se você converter esse timestamp para UTC para armazená-lo, o evento será armazenado como toda terça-feira às 9h. O fuso horário CET muda de UTC+01:00 para UTC+02:00 no verão, portanto, além disso, nos meses de verão, sua aplicação calculará que o evento se iniciará às 11h, horário CET, toda terça-feira. Você pode ver como esse programa rapidamente se tornará redundante!

Agora que você já conhece o problema geral de lidar com fusos horários, vamos explorar nossa linguagem favorita. Python tem um objeto timestamp chamado `datetime.datetime`, capaz de armazenar a data e a hora com precisão de microssegundos. O objeto `datetime.datetime` pode ter *conhecimento* do fuso horário, caso em que incluirá informações sobre esse fuso, ou pode *não ter conhecimento* e, nesse caso, não incluirá essa informação. Infelizmente, a API `datetime`, por padrão, devolve um objeto que não tem informação do fuso horário, como veremos logo adiante na Listagem 4.1. Vamos ver como criar um objeto timestamp default e, em seguida, como corrigi-lo para que utilize fusos horários.

Criando objetos `datetime` default

Para criar um objeto `datetime` com a data e a hora atuais como valores, a função `datetime.datetime.utcnow()` pode ser usada. Essa função devolve a data e a hora atuais no fuso horário UTC, conforme mostra a Listagem 4.1. Para criar esse mesmo objeto usando a data e a hora do fuso horário da região em que está o computador, você pode usar o método `datetime.datetime.now()`. A Listagem 4.1 obtém a data e a hora tanto no fuso horário UTC como no fuso de minha região.

Listagem 4.1 – Obtendo a data e a hora com `datetime`

```
>>> import datetime
>>> datetime.datetime.utcnow()
❶ datetime.datetime(2018, 6, 15, 13, 24, 48, 27631)
>>> datetime.datetime.utcnow().tzinfo is None
```

❷ True

```
>>> datetime.datetime.now()
```

❸ datetime.datetime(2018, 6, 15, 15, 24, 52, 276161)

Importamos a biblioteca `datetime` e definimos o objeto `datetime` para que utilize o fuso horário UTC. Essa instrução devolve um timestamp UTC cujos valores são o ano, o mês, o dia, as horas, os minutos, os segundos e os microssegundos ❶, respectivamente, na listagem. Podemos conferir se esse objeto inclui informações sobre fuso horário verificando o objeto `tzinfo` e, nesse caso, constatamos que ele não inclui ❷.

Em seguida, criamos o objeto `datetime` usando o método `datetime.datetime.now()` para obter a data e a hora atuais no fuso horário default para a região em que está o computador.

Esse timestamp também é devolvido sem informação do fuso horário, conforme podemos perceber pela ausência do campo `tzinfo` ❸ – se estivesse presente, a informação de fuso horário teria aparecido no final da saída, na forma de algo como `tzinfo=<UTC>`.

A API `datetime`, por padrão, sempre devolve objetos `datetime` sem informação sobre o fuso horário; como não há nenhuma maneira de você dizer qual é o fuso horário com base na saída, esses objetos são praticamente inúteis.

Armin Ronacher, criador do framework Flask, sugere que uma aplicação deve sempre supor que objetos `datetime` sem informação de fuso horário em Python utilizam UTC. No entanto, conforme acabamos de ver, isso não funciona para objetos devolvidos por `datetime.datetime.now()`. Ao criar objetos `datetime`, recomendo que você sempre garanta que esses objetos tenham informações sobre o fuso horário. Desse modo, será sempre possível comparar os objetos diretamente e verificar se são devolvidos de forma correta, com as informações necessárias. Vamos ver como criar timestamps com informações sobre fusos horários, usando objetos `tzinfo`.

BÔNUS: CRIANDO UM OBJETO DATETIME A PARTIR DE UMA DATA

Você também pode criar seu próprio objeto `datetime` com uma data específica,

passando os valores que você quer para os diferentes componentes da data, como mostra a Listagem 4.2.

Listagem 4.2 – Criando seu próprio objeto timestamp

```
>>> import datetime  
>>> datetime.datetime(2018, 6, 19, 19, 54, 49)
```

```
datetime.datetime(2018, 6, 19, 19, 54, 49)
```

Timestamps com informação sobre o fuso horário com o dateutil

Já existem muitos bancos de dados de fusos horários, mantidos por autoridades centrais como o IANA (Internet Assigned Numbers Authority, ou Autoridade para Atribuição de Números da Internet), e que acompanham todos os principais sistemas operacionais. Por esse motivo, em vez de criar nossas próprias classes de fuso horário e duplicá-las manualmente em cada projeto Python, os desenvolvedores Python contam com o projeto `dateutil` para obter classes `tzinfo`. O projeto `dateutil` inclui o módulo Python `tz`, que fornece diretamente as informações sobre fusos horários, sem exigir muito esforço: o módulo `tz` é capaz de acessar as informações sobre fusos horários do sistema operacional e disponibilizar o banco de dados de fusos horários de modo que esse seja diretamente acessível por Python.

Você pode instalar o `dateutil` usando o `pip`, com o comando `pip install python-dateutil`. A API de `dateutil` permite obter um objeto `tzinfo` com base no nome de um fuso horário, assim:

```
>>> from dateutil import tz
>>> tz.gettz("Europe/Paris")
tzfile('/usr/share/zoneinfo/Europe/Paris')
>>> tz.gettz("GMT+1")
tzstr('GMT+1')
```

O método `dateutil.tz.gettz()` devolve um objeto que implementa a interface `tzinfo`. Esse método aceita diversos formatos de string como argumento, por exemplo, o fuso horário com base em uma localidade (por exemplo, “Europe/Paris”) ou um fuso horário relativo ao GMT. Os objetos de fuso horário do `dateutil` podem ser usados diretamente como classes `tzinfo`, como mostra a Listagem 4.3.

Listagem 4.3 – Usando objetos de dateutil como classes tzinfo

```
>>> import datetime
```



```

>>> from dateutil import tz
>>> now = datetime.datetime.now()
>>> now
datetime.datetime(2018, 10, 16, 19, 40, 18, 279100)
>>> tz = tz.gettz("Europe/Paris")
>>> now.replace(tzinfo=tz)
datetime.datetime(2018, 10, 16, 19, 40, 18, 279100,
tzinfo=tzfile('/usr/share/zoneinfo/Europe/Paris'))

```

Desde que você saiba o nome do fuso horário desejado, é possível obter um objeto `tzinfo` que corresponda ao fuso horário visado. O módulo `dateutil` é capaz de acessar o fuso horário gerenciado pelo sistema operacional e, se essa informação, por algum motivo, estiver indisponível, ele recorrerá à sua própria lista de fusos horários. Se, algum dia, você tiver de acessar essa lista própria, poderá fazê-lo por meio do módulo `dateutil.zoneinfo`:

```

>>> from dateutil.zoneinfo import get_zonefile_instance
>>> zones = list(get_zonefile_instance().zones)
>>> sorted(zones)[:5]
['Africa/Abidjan', 'Africa/Accra', 'Africa/Addis_Ababa', 'Africa/Algiers', 'Africa/Asmara']
>>> len(zones)
592

```

Em alguns casos, seu programa não saberá em qual fuso horário está executando, portanto, será necessário determiná-lo por conta própria. A função `dateutil.tz.gettz()` devolverá o fuso horário local de seu computador se você não lhe passar nenhum argumento, como vemos na Listagem 4.4.

Listagem 4.4 – Obtendo seu fuso horário local

```

>>> from dateutil import tz
>>> import datetime
>>> now = datetime.datetime.now()
>>> localzone = tz.gettz()
>>> localzone
tzfile('/etc/localtime')
>>> localzone.tzname(datetime.datetime(2018, 10, 19))
'CEST'
>>> localzone.tzname(datetime.datetime(2018, 11, 19))
'CET'

```

Conforme podemos ver, passamos duas datas para `localzone.tzname(datetime.datetime())` separadamente, e `dateutil` foi capaz de nos informar que uma delas é CEST (Central European Summer Time, ou Horário de Verão da Europa Central) e que a outra é CET (Central European Time, ou Horário da Europa Central). Se você passar a sua data atual, seu próprio fuso horário atual será devolvido.

Podemos usar objetos da biblioteca `dateutil` em classes `tzinfo` sem ter de nos preocupar em implementá-los por conta própria na aplicação. Isso facilita converter objetos `datetime` sem informações sobre fuso horário em objetos `datetime` com essa informação.

IMPLEMENTANDO SUAS PRÓPRIAS CLASSES DE FUSO HORÁRIO

Há uma classe em Python que permite que você implemente classes de fuso horário por conta própria: a classe `datetime.tzinfo` é uma classe abstrata que fornece uma base para implementar classes que representem fusos horários. Se, algum dia, você quiser implementar uma classe que represente um fuso horário, deverá usá-la como a classe-pai e implementar três métodos distintos:

- `utcoffset(dt)`, que deve devolver um offset em relação ao UTC em minutos a leste do UTC para o fuso horário;
- `dst(dt)`, que deve devolver o ajuste de horário de verão em minutos a leste do UTC para o fuso horário;
- `tzname(dt)`, que deve devolver o nome do fuso horário na forma de string.

Esses três métodos incluirão um objeto `tzinfo`, que permite que você traduza qualquer `datetime` com informação de fuso horário para outro fuso horário.

No entanto, conforme mencionamos antes, como já existem bancos de dados de fusos horários, não seria conveniente implementar essas classes de fuso horário por conta própria.

Serializando objetos `datetime` com informação de fuso horário

Com frequência, você terá de transportar um objeto `datetime` de um ponto para outro, e esses diferentes pontos talvez não sejam nativos de Python. O caso típico atualmente seria com uma API REST HTTP que precise devolver objetos `datetime` serializados a um cliente. O método Python nativo chamado `isoformat` pode ser usado para

serializar objetos `datetime` para pontos que não são nativos de Python, como mostra a Listagem 4.5.

Listagem 4.5 – Serializando um objeto `datetime` com informação de fuso horário

```
>>> import datetime
>>> from dateutil import tz
❶ >>> def utcnow():
    return datetime.datetime.now(tz=tz.tzutc())
>>> utcnow()
❷ datetime.datetime(2018, 6, 15, 14, 45, 19, 182703, tzinfo=tzutc())
❸ >>> utcnow().isoformat()
'2018-06-15T14:45:21.982600+00:00'
```

Definimos uma nova função chamada `utcnow` e lhe dissemos explicitamente que devolvesse um objeto com o fuso horário UTC ❶. Como podemos ver, o objeto devolvido agora contém a informação sobre o fuso horário ❷. Em seguida, formatamos a string utilizando o formato ISO ❸, garantindo que o timestamp também contenha informações sobre o fuso horário (a parte `+00:00`).

Você pode notar que usei o método `isoformat()` para formatar a saída. Recomendo que você sempre formate as strings de entrada e de saída de `datetime` usando a ISO 8601, utilizando o método `datetime.datetime.isoformat()` para devolver timestamps formatados de modo legível, que incluam a informação de fuso horário.

Suas strings formatadas com ISO 8601 poderão então ser convertidas em objetos `datetime.datetime` nativos. O módulo `iso8601` disponibiliza apenas uma função, `parse_date`, que faz todo o trabalho pesado de parse da string, determinando os valores de timestamp e do fuso horário. O módulo `iso8601` não está disponível como um módulo embutido em Python, portanto, é necessário instalá-lo com `pip install iso8601`. A Listagem 4.6 mostra como fazer o parse de um timestamp usando a ISO 8601.

Listagem 4.6 – Usando o módulo `iso8601` para fazer parse de um timestamp no formato ISO 8601

```
>>> import iso8601
```

```

>>> import datetime
>>> from dateutil import tz
>>> now = datetime.datetime.utcnow()
>>> now.isoformat()
'2018-06-19T09:42:00.764337'
❶ >>> parsed = iso8601.parse_date(now.isoformat())
>>> parsed
datetime.datetime(2018, 6, 19, 9, 42, 0, 764337, tzinfo=<iso8601.Utc>)
>>> parsed == now.replace(tzinfo=tz.tzutc())
True

```

Na Listagem 4.6, o módulo `iso8601` foi usado para criar um objeto `datetime` a partir de uma string. Ao chamar `iso8601.parse_date` em uma string contendo um timestamp no formato ISO 8601 ❶, a biblioteca é capaz de devolver um objeto `datetime`. Como essa string não contém nenhuma informação de fuso horário, o módulo `iso8601` supõe que o fuso horário é UTC. Se um fuso horário contiver a informação correta, o módulo `iso8601` devolverá a informação correta.

Utilizar objetos `datetime` que tenham informação de fuso horário e usar uma representação em string no formato ISO 8601 é uma solução perfeita para a maioria dos problemas relacionados a fusos horários, garantindo que nenhum erro seja cometido, além de proporcionar um alto nível de interoperabilidade entre a sua aplicação e o mundo externo.

Resolvendo horários ambíguos

Há determinados casos em que a hora do dia pode ser ambígua; por exemplo, durante a transição para o horário de verão, quando o mesmo “horário do relógio” ocorre duas vezes em um dia. A biblioteca `dateutil` nos fornece o método `is_ambiguous` para distinguir timestamps desse tipo. Para mostrá-lo em ação, criaremos um timestamp ambíguo na Listagem 4.7.

Listagem 4.7 – Um timestamp confuso, que ocorre durante a transição do horário de verão

```

>>> import dateutil.tz
>>> localtime = dateutil.tz.gettz("Europe/Paris")

```

```
>>> confusing = datetime.datetime(2017, 10, 29, 2, 30)
>>> localtime.is_ambiguous(confusing)
True
```

Na noite do dia 30 de outubro de 2017, Paris passou do horário de verão para o horário de inverno. Houve uma mudança na cidade às 3h, quando o horário voltou para as 2h. Se tentássemos usar um timestamp às 2h30 nesse dia, não haveria maneiras de esse objeto saber ao certo se está antes ou depois da mudança do horário de verão.

No entanto, é possível especificar de que lado da mudança um timestamp se encontra, usando o atributo `fold`, acrescentado aos objetos `datetime` de Python 3.6 pela PEP 495 (Local Time Disambiguation, ou Eliminando a ambiguidade das horas locais – <https://www.python.org/dev/peps/pep-0495/>). Esse atributo informa de que lado da transição está o `datetime`, como mostra a Listagem 4.8.

Listagem 4.8 – Eliminando a ambiguidade do timestamp

```
>>> import dateutil.tz
>>> import datetime
>>> localtime = dateutil.tz.gettz("Europe/Paris")
>>> utc = dateutil.tz.tzutc()
>>> confusing = datetime.datetime(2017, 10, 29, 2, 30, tzinfo=localtz)
>>> confusing.replace(fold=0).astime zone(utc)
datetime.datetime(2017, 10, 29, 0, 30, tzinfo=tzutc())
>>> confusing.replace(fold=1).astime zone(utc)
datetime.datetime(2017, 10, 29, 1, 30, tzinfo=tzutc())
```

Você precisará usar esse recurso somente em casos muito raros, pois os timestamps ambíguos ocorrem apenas em uma pequena janela de tempo. Ater-se ao UTC é uma ótima solução alternativa para deixar a vida simples e evitar problemas com fusos horários. No entanto, é bom saber que o atributo `fold` existe, e que `dateutil` é capaz de ajudar em casos como esses.

Resumo

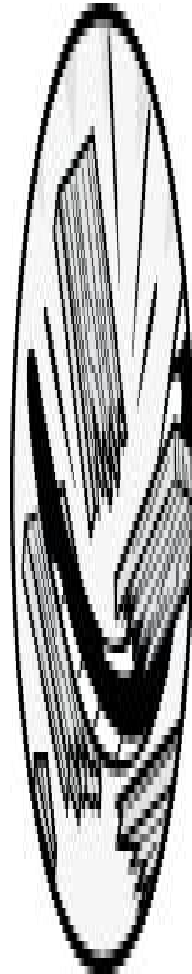
Neste capítulo, vimos a importância de incluir informações sobre

fusos horários nos timestamps. O módulo embutido `datetime` não é completo quanto a isso, mas o módulo `dateutil` é um ótimo complemento: ele nos permite obter objetos compatíveis com `tzinfo`, prontos para usar. O módulo `dateutil` também ajuda a resolver problemas sutis, como o problema da ambiguidade no caso do horário de verão.

O formato ISO 8601 padrão é uma excelente opção para serializar e desserializar timestamps, pois está prontamente disponível em Python e é compatível com qualquer outra linguagem de programação.

5

DISTRIBUINDO SEU SOFTWARE



É seguro dizer que, em algum momento, você vai querer distribuir o seu software. Por mais tentador que possa ser simplesmente compactar seu código e fazer o upload para a internet, Python disponibiliza ferramentas para que seus usuários finais consigam fazer seu software funcionar de modo mais fácil. Você já deve ter familiaridade com o uso de *setup.py* para instalar aplicações e bibliotecas Python, mas provavelmente não explorou o

seu modo de funcionamento interno de forma profunda nem como criar o próprio `setup.py`.

Neste capítulo, conheceremos a história do `setup.py`, como o arquivo funciona e como criar o próprio `setup.py` personalizado. Veremos também alguns dos recursos menos conhecidos da ferramenta de instalação de pacotes `pip` e como fazer com que seu software seja baixado com essa ferramenta. Por fim, veremos como usar os pontos de entrada (entry points) Python para que seja fácil encontrar funções entre os programas. Com essas habilidades, você poderá deixar seu software publicado acessível aos usuários finais.

Um pouco da história do `setup.py`

A biblioteca `distutils`, originalmente criada pelo desenvolvedor de software Greg Ward, faz parte da biblioteca-padrão de Python desde 1998. Ward procurou criar uma maneira fácil de os desenvolvedores automatizarem o processo de instalação para seus usuários finais. Os pacotes disponibilizam um arquivo `setup.py` como o script Python padrão para a instalação e eles podem utilizar `distutils` para se instalarem, conforme vemos na Listagem 5.1.

Listagem 5.1 – Criando um `setup.py` usando `distutils`

```
#!/usr/bin/python
from distutils.core import setup

setup(name="rebuild",
      description="Debian packages rebuild tool",
      author="Julien Danjou",
      author_email="acid@debian.org",
      url="http://julien.danjou.info/software/rebuild.html",
      packages=['rebuild'])
```

Com o arquivo `setup.py` como a raiz de um projeto, tudo que os usuários precisam fazer para construir ou instalar seu software é executar esse arquivo com o comando apropriado como argumento. Ainda que sua distribuição inclua módulos C além dos módulos Python nativos, o `distutils` é capaz de lidar com eles automaticamente.

O desenvolvimento do `distutils` foi abandonado em 2000; a partir de então, outros desenvolvedores continuaram do ponto em que ele havia parado. Um dos sucessores de destaque é a biblioteca de empacotamento conhecida como `setuptools`, que oferece atualizações mais frequentes e recursos avançados, como tratamento automático de dependências, o formato de distribuição `Egg` e o comando `easy_install`. Como o `distutils` ainda era o meio de empacotamento de software aceito e incluído na Biblioteca-Padrão de Python na época do desenvolvimento do `setuptools`, essa biblioteca oferecia certo grau de compatibilidade com ele. A Listagem 5.2 mostra como usaríamos o `setuptools` para criar o mesmo pacote de instalação da Listagem 5.1.

Listagem 5.2 – Criando um `setup.py` usando `setuptools`

```
#!/usr/bin/env python
import setuptools

setuptools.setup(
    name="rebuild",
    version="0.2",
    author="Julien Danjou",
    author_email="acid@debian.org",
    description="Debian packages rebuild tool",
    license="GPL",
    url="http://julien.danjou.info/software/rebuild",
    packages=['rebuild'],
    classifiers=[
        "Development Status :: 2 - Pre-Alpha",
        "Intended Audience :: Developers",
        "Intended Audience :: Information Technology",
        "License :: OSI Approved :: GNU General Public License (GPL)",
        "Operating System :: OS Independent",
        "Programming Language :: Python"
    ],
)
```

Mais tarde, o desenvolvimento do `setuptools` também desacelerou, mas não demorou muito para que outro grupo de desenvolvedores criasse um ramo e desenvolvesse uma nova biblioteca chamada `distribute`, que oferecia diversas vantagens em relação ao `setuptools`,

incluindo menos bugs e suporte para Python 3.

Todas as boas histórias, contudo, têm um final imprevisto: em março de 2013, as equipes responsáveis pelo `setuptools` e pelo `distribute` decidiram combinar suas bases de código sob a égide do projeto `setuptools` original. Desse modo, o `distribute` atualmente é considerado obsoleto, e o `setuptools` é, mais uma vez, o modo canônico de lidar com instalações Python sofisticadas.

Enquanto tudo isso acontecia, outro projeto, conhecido como `distutils2`, estava sendo desenvolvido com o intuito de substituir totalmente o `distutils` na Biblioteca-Padrão de Python. De modo diferente do `distutils` e do `setuptools`, ele armazenava metadados de pacotes em um arquivo texto simples, `setup.cfg`, que era mais fácil tanto para os desenvolvedores escreverem como para as ferramentas externas lerem. No entanto, o `distutils2` mantinha algumas das falhas do `distutils`, por exemplo, seu design obtuso baseado em comando, e não incluía suporte para pontos de entrada nem execução de scripts nativos no Windows – dois recursos oferecidos pelo `setuptools`. Por esses e outros motivos, os planos para incluir o `distutils2`, renomeado como `packaging`, na Biblioteca-Padrão de Python 3.3 caíram por terra, e o projeto foi abandonado em 2012.

Ainda há chances de o `packaging` ressurgir das cinzas por meio do `distlib`, um esforço nascente para substituir o `distutils`. Antes de seu lançamento, havia rumores de que o pacote `distlib` faria parte da Biblioteca-Padrão de Python 3.4, mas isso jamais ocorreu. Incluindo os melhores recursos de `packaging`, o `distlib` implementa o trabalho básico descrito nas PEPs relacionadas a empacotamentos.

Então, para recapitular:

- `distutils` faz parte da Biblioteca-Padrão de Python e é capaz de lidar com instalações simples de pacotes.
- `setuptools`, que é o padrão para instalações de pacotes sofisticados, inicialmente foi considerado obsoleto, porém, atualmente, está de volta em desenvolvimento ativo, e é o verdadeiro padrão de uso do mercado.

- **distribute** foi combinado de volta no `setuptools` na versão 0.7.
- **distutils2** (também conhecido como `packaging`) foi abandonado.
- **distlib** *poderá* vir a substituir o `distutils` no futuro.

Há outras bibliotecas de empacotamento por aí, mas essas são as cinco que você verá com mais frequência. Tome cuidado ao pesquisar sobre essas bibliotecas na internet: há muita documentação desatualizada em virtude da complicada história que apresentamos. A documentação oficial, no entanto, está atualizada.

Em suma, o `setuptools` é a biblioteca de distribuição que deve ser usada no momento, mas fique atento ao `distlib` no futuro.

Empacotamento com `setup.cfg`

É provável que você já tenha tentado escrever um `setup.py` para um pacote em algum momento, seja copiando um de outro projeto ou consultando a documentação e criando um por conta própria. Criar um `setup.py` não é uma tarefa intuitiva. Escolher a ferramenta correta a ser usada é só o primeiro desafio. Nesta seção, gostaria de apresentar umas das melhorias recentes feitas no `setuptools`: o suporte ao arquivo `setup.cfg`.

Eis o aspecto de um `setup.py` que utiliza um arquivo `setup.cfg`:

```
import setuptools
```

```
setuptools.setup()
```

Duas linhas de código – é simples assim. Os metadados propriamente ditos exigidos pela instalação são armazenados em `setup.cfg`, como mostra a Listagem 5.3.

Listagem 5.3 – Metadados em `setup.cfg`

```
[metadata]
name = foobar
author = Dave Null
author-email = foobar@example.org
license = MIT
long_description = file: README.rst
```

```
url = http://pypi.python.org/pypi/foobar
requires-python = >=2.6
classifiers =
    Development Status :: 4 - Beta
    Environment :: Console
    Intended Audience :: Developers
    Intended Audience :: Information Technology
    License :: OSI Approved :: Apache Software License
    Operating System :: OS Independent
    Programming Language :: Python
```

Como podemos ver, *setup.cfg* utiliza um formato fácil de escrever e de ser lido, inspirado diretamente no *distutils2*. Várias outras ferramentas, como o Sphinx ou o Wheel, também leem a configuração desse arquivo *setup.cfg* – só isso já é um bom argumento para começar a usá-lo.

Na Listagem 5.3, a descrição do projeto é lida do arquivo *README.rst*. Ter sempre um arquivo README é uma boa prática – de preferência no formato RST – de modo que os usuários possam rapidamente saber do que se trata o projeto. Apenas com esses arquivos *setup.py* e *setup.cfg* básicos, seu pacote estará pronto para ser publicado e usado por outros desenvolvedores e aplicações. A documentação de *setuptools* fornece mais detalhes, se houver necessidade – por exemplo, se houver alguns passos extras em seu processo de instalação ou se você quiser incluir arquivos adicionais.

Outra ferramenta útil de empacotamento é o *pbr*, que é a abreviatura de *Python Build Reasonableness*. O projeto teve início na OpenStack como uma extensão do *setuptools*, com o intuito de facilitar a instalação e a implantação de pacotes. A ferramenta de empacotamento *pbr*, usada com o *setuptools*, implementa recursos que não estão presentes em *setuptools*, incluindo:

- geração automática de documentação Sphinx;
- geração automática de arquivos *AUTHORS* e *ChangeLog* com base no histórico do *git*;
- criação automática de listas de arquivos para o *git*;
- gerenciamento de versões com base em tags do *git* usando atribuição de versão semântica.

Tudo isso com pouco – ou nenhum esforço – de sua parte. Para usar o `pbr`, basta ativá-lo, como mostra a Listagem 5.4.

Listagem 5.4 – setup.py usando pbr

```
import setuptools

setuptools.setup(setup_requires=['pbr'], pbr=True)
```

O parâmetro `setup_requires` informa que o `pbr` deve estar instalado antes de usar o `setuptools`. O argumento `pbr=True` garante que a extensão `pbr` para o `setuptools` seja carregada e chamada.

Uma vez ativado, o comando `python setup.py` tem os recursos do `pbr` acrescentados. Chamar `python setup.py --version`, por exemplo, devolverá o número da versão do projeto com base nas tags `git` existentes. Executar `python setup.py sdist` criará um tarball fonte com arquivos `ChangeLog` e `AUTHORS` gerados automaticamente.

Padrão de distribuição do formato Wheel

Durante a maior parte da existência de Python, não houve nenhum formato oficial de distribuição padrão. Embora, de modo geral, diferentes ferramentas de distribuição utilizem algum formato comum de arquivo – até mesmo o formato `Egg` introduzido pelo `setuptools` nada mais é do que um arquivo `zip` com uma extensão diferente – as estruturas de seus metadados e dos pacotes são incompatíveis entre si. Esse problema se agravou quando um padrão oficial de instalação foi finalmente definido na PEP 376, também incompatível com os formatos existentes.

Para resolver esses problemas, a PEP 427 foi escrita para definir um novo padrão de distribuição de pacotes Python, chamado `Wheel`. A implementação de referência desse formato está disponível na forma de uma ferramenta, também chamada `Wheel`.

O `Wheel` passou a ser aceito pelo `pip` a partir da versão 1.4. Se você está usando o `setuptools` e tem o pacote `Wheel` instalado, ele será automaticamente integrado como um comando do `setuptools`,

chamado `bdist_wheel`. Se você não tem o `Wheel` instalado, poderá instalá-lo com o comando `pip install wheel`. A Listagem 5.5 mostra parte da saída de uma chamada a `bdist_wheel`, reduzida para permitir que fosse exibida.

Listagem 5.5 – Chamando `setup.py bdist_wheel`

```
$ python setup.py bdist_wheel
running bdist_wheel
running build
running build_py
creating build/lib
creating build/lib/daiquiri
creating build/lib/daiquiri/tests
copying daiquiri/tests/__init__.py -> build/lib/daiquiri/tests
--trecho omitido--
running egg_info
writing requirements to daiquiri.egg-info/requirements.txt
writing daiquiri.egg-info/PKG-INFO
writing top-level names to daiquiri.egg-info/top_level.txt
writing dependency_links to daiquiri.egg-info/dependency_links.txt
writing pbr to daiquiri.egg-info/pbr.json
writing manifest file 'daiquiri.egg-info/SOURCES.txt'
installing to build/bdist.macosx-10.12-x86_64/wheel
running install
running install_lib
--trecho omitido--

running install_scripts
creating build/bdist.macosx-10.12-x86_64/wheel/daiquiri-1.3.0.dist-info/WHEEL
❶ creating '/Users/jd/Source/daiquiri/dist/daiquiri-1.3.0-py2.py3-none-any.whl'
and adding '.' to it
adding 'daiquiri/__init__.py'
adding 'daiquiri/formatter.py'
adding 'daiquiri/handlers.py'

--trecho omitido--
```

O comando `bdist_wheel` cria um arquivo `.whl` no diretório `dist` ❶. Assim como no formato `Egg`, um arquivo `Wheel` nada mais é do que um arquivo `zip` com uma extensão diferente. No entanto, arquivos `Wheel` não exigem instalação – você pode carregar e executar o seu

código simplesmente acrescentando uma barra seguida do nome de seu módulo:

```
$ python wheel-0.21.0-py2.py3-none-any.whl/wheel -h
```

```
usage: wheel [-h]
```

```
{keygen,sign,unsign,verify,unpack,install,install-scripts,convert,help}
```

```
--trecho omitido--
```

```
positional arguments:
```

```
--trecho omitido--
```

Talvez você se surpreenda ao saber que esse não foi um recurso introduzido pelo formato Wheel propriamente dito. Python também é capaz de executar arquivos zip comuns, como ocorre com os arquivos *.jar* de Java:

```
python foobar.zip
```

Esse comando é equivalente a:

```
PYTHONPATH=foobar.zip python -m __main__
```

Em outras palavras, o módulo `__main__` de seu programa será automaticamente importado de `__main__.py`. Você também pode importar `__main__` de um módulo que você especificar, concatenando uma barra seguida do nome do módulo, assim como no caso do Wheel:

```
python foobar.zip/mymod
```

Esse comando é equivalente a:

```
PYTHONPATH=foobar.zip python -m mymod.__main__
```

Uma das vantagens do Wheel é que suas convenções de nomenclatura permitem especificar se a sua distribuição visa a uma arquitetura e/ou implementação Python específica (CPython, PyPy, Jython, e assim por diante). Isso é particularmente conveniente caso você precise distribuir módulos escritos em C.

Por padrão, pacotes Wheel estão associados à versão principal de Python usada para criá-los. Quando chamado com `python2 setup.py bdist_wheel`, o padrão do nome de um arquivo Wheel será algo como *library-version-py2-none-any.whl*.

Se seu código for compatível com todas as versões principais de Python (isto é, Python 2 e Python 3), você poderá criar um Wheel universal:

```
python setup.py bdist_wheel --universal
```

O nome do arquivo resultante será diferente e conterá as duas versões principais de Python – algo como *library-version-py2.py3-none-any.whl*. Criar um Wheel universal evita que você acabe com dois Wheels distintos, quando somente um seria suficiente para as duas versões principais de Python.

Se não quiser passar a flag `--universal` sempre que criar um Wheel, bastará acrescentar o seguinte em seu arquivo *setup.cfg*:

```
[wheel]
universal=1
```

Se o Wheel que você criar contiver bibliotecas ou programas binários (por exemplo, uma extensão Python escrita em C), o Wheel binário talvez não seja tão portátil quanto você possa imaginar. Ele funcionará por padrão em algumas plataformas, como Darwin (macOS) ou Microsoft Windows, mas talvez não funcione em todas as distribuições Linux. A PEP 513 (<https://www.python.org/dev/peps/pep-0513>) tem esse problema de Linux como alvo e define uma nova tag de plataforma chamada `manylinux1`, além de um conjunto mínimo de bibliotecas o qual se garante que esteja disponível nessa plataforma.

O Wheel é um ótimo formato para distribuir bibliotecas e aplicações prontas para serem instaladas, portanto, você deve criá-las e fazer o seu upload no PyPI também.

Compartilhando seu trabalho com o mundo

Assim que você tiver um arquivo *setup.py* apropriado, será fácil criar um tarball fonte que seja distribuído. O comando `sdist setuptools` faz exatamente isso, conforme mostra a Listagem 5.6.

Listagem 5.6 – Usando `setup.py sdist` para gerar um tarball fonte


```
$ python setup.py sdist
```

```
running sdist
```

```
[pbr] Generating AUTHORS
```

```
running egg_info
```

```
writing requirements to ceilometer.egg-info/requirements.txt
```

```
writing ceilometer.egg-info/PKG-INFO
```

```
writing top-level names to ceilometer.egg-info/top_level.txt
```

```
writing dependency_links to ceilometer.egg-info/dependency_links.txt
```

```
writing entry points to ceilometer.egg-info/entry_points.txt
```

```
[pbr] Processing SOURCES.txt
```

```
[pbr] In git context, generating filelist from git
```

```
warning: no previously-included files matching '*.pyc' found anywhere in distribution
```

```
writing manifest file 'ceilometer.egg-info/SOURCES.txt'
```

```
running check
```

```
copying setup.cfg -> ceilometer-2014.1.a6-g772e1a7
```

```
Writing ceilometer-2014.1.a6-g772e1a7/setup.cfg
```

```
--trecho omitido--
```

```
Creating tar archive
```

```
removing 'ceilometer-2014.1.a6.g772e1a7' (and everything under it)
```

O comando `sdist` cria um tarball no diretório *dist* da árvore de códigos-fonte. O tarball contém todos os módulos Python que fazem parte dessa árvore. Conforme vimos na seção anterior, também podemos criar arquivos Wheel usando o comando `bdist_wheel`. Os arquivos Wheel são um pouco mais rápidos para serem instalados, pois já estão no formato correto para a instalação.

O último passo para deixar o código acessível é exportar o seu pacote para um local a partir do qual os usuários possam instalá-lo com o `pip`. Isso significa publicar o seu projeto no PyPI.

Se essa é a sua primeira exportação no PyPI, vale a pena testar o processo de publicação em uma sandbox protegida, em vez de usar o servidor de produção. Você pode usar o servidor de staging do PyPI para isso; ele replica todas as funcionalidades do índice principal, mas serve somente para testes.

O primeiro passo é registrar o seu projeto no servidor de testes. Comece abrindo o seu arquivo `~/.pypirc` e acrescente as linhas a

seguir:

```
[distutils]  
index-servers =  
    testpypi
```

```
[testpypi]  
username = <seu nome de usuário>  
password = <sua senha>  
repository = https://testpypi.python.org/pypi
```

Salve o arquivo; agora você poderá registrar o seu projeto no índice:

```
$ python setup.py register -r testpypi
```

```
running register  
running egg_info  
writing requirements to ceilometer.egg-info/requirements.txt  
writing ceilometer.egg-info/PKG-INFO  
writing top-level names to ceilometer.egg-info/top_level.txt  
writing dependency_links to ceilometer.egg-info/dependency_links.txt  
writing entry points to ceilometer.egg-info/entry_points.txt  
[pbr] Reusing existing SOURCES.txt  
running check  
Registering ceilometer to https://testpypi.python.org/pypi  
Server response (200): OK
```

Com isso, você se conectará com a instância do servidor de testes do PyPI e criará uma nova entrada. Não se esqueça de utilizar a opção `-r`; caso contrário, a instância real de produção do PyPI será usada!

Obviamente, se um processo de mesmo nome já estiver registrado, o processo falhará. Tente novamente usando outro nome; assim que conseguir registrar o seu programa e receber um OK como resposta, você poderá fazer o upload de um tarball de distribuição, conforme mostra a Listagem 5.7.

Listagem 5.7 – Fazendo o upload de seu tarball no PyPI

```
$ python setup.py sdist upload -r testpypi  
running sdist  
[pbr] Writing ChangeLog  
[pbr] Generating AUTHORS  
running egg_info  
writing requirements to ceilometer.egg-info/requirements.txt
```

```
writing ceilometer.egg-info/PKG-INFO
writing top-level names to ceilometer.egg-info/top_level.txt
writing dependency_links to ceilometer.egg-info/dependency_links.txt
writing entry points to ceilometer.egg-info/entry_points.txt
[pbr] Processing SOURCES.txt
[pbr] In git context, generating filelist from git
warning: no previously-included files matching '*.pyc' found anywhere in distribution
writing manifest file 'ceilometer.egg-info/SOURCES.txt'
running check
creating ceilometer-2014.1.a6.g772e1a7
```

--trecho omitido--

```
copying setup.cfg -> ceilometer-2014.1.a6.g772e1a7
Writing ceilometer-2014.1.a6.g772e1a7/setup.cfg
Creating tar archive
removing 'ceilometer-2014.1.a6.g772e1a7' (and everything under it)
running upload
Submitting dist/ceilometer-2014.1.a6.g772e1a7.tar.gz to https://testpypi.python.org/pypi
Server response (200): OK
```

Como alternativa, você poderia fazer o upload de um arquivo Wheel, como vemos na Listagem 5.8.

Listagem 5.8 – Fazendo o upload de um arquivo Wheel no PyPI

```
$ python setup.py bdist_wheel upload -r testpypi
running bdist_wheel
running build
running build_py
running egg_info
writing requirements to ceilometer.egg-info/requirements.txt
writing ceilometer.egg-info/PKG-INFO
writing top-level names to ceilometer.egg-info/top_level.txt
writing dependency_links to ceilometer.egg-info/dependency_links.txt
writing entry points to ceilometer.egg-info/entry_points.txt
[pbr] Reusing existing SOURCES.txt
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
```

--trecho omitido--

```
creating build/bdist.linux-x86_64/wheel/ceilometer-2014.1.a6.g772e1a7
.dist-info/WHEEL
running upload
Submitting /home/jd/Source/ceilometer/dist/ceilometer-2014.1.a6
.g772e1a7-py27-none-any.whl to https://testpypi.python.org/pypi
Server response (200): OK
```

Assim que essas operações estiverem concluídas, você e os demais usuários poderão pesquisar os pacotes enviados para o servidor de staging do PyPI e até mesmo instalá-los usando o `pip`, especificando o servidor de testes com a opção `-i`:

```
$ pip install -i https://testpypi.python.org/pypi ceilometer
```

Se tudo correr bem, você poderá fazer o upload de seu projeto no servidor principal do PyPI. Apenas não se esqueça de adicionar antes as suas credenciais e os detalhes do servidor em seu arquivo `~/.pypirc`, da seguinte maneira:

```
[distutils]
index-servers =
    pypi
    testpypi
```

```
[pypi]
username = <seu nome de usuário>
password = <sua senha>
```

```
[testpypi]
repository = https://testpypi.python.org/pypi
username = <seu nome de usuário>
password = <sua senha>
```

Se você executar `register` e `upload` com a flag `-r pypi` agora, o upload de seu pacote será feito no PyPI.

NOTA O PyPI é capaz de manter várias versões de seu software em seu índice, permitindo que você instale versões específicas e mais antigas, caso seja necessário. Basta passar o número da versão para o comando `pip install`; por exemplo, `pip install foobar==1.0.2`.

Esse processo é simples de usar e permite qualquer quantidade de uploads. Você pode lançar versões de seu software com a frequência que quiser, e seus usuários poderão instalá-lo e atualizá-lo com a frequência que lhes for necessária.

Pontos de entrada

Talvez você já tenha usado os pontos de entrada do `setuptools` sem saber nada sobre eles. Softwares distribuídos usando o `setuptools` incluem metadados importantes que descrevem recursos como as dependências necessárias e – mais relevantes para este tópico – uma lista de *pontos de entrada* (entry points). Os pontos de entrada são métodos por meio dos quais outros programas Python podem descobrir os recursos dinâmicos oferecidos por um pacote.

O exemplo a seguir mostra como disponibilizar um ponto de entrada chamado `rebuildd` no grupo de pontos de entrada `console_scripts`:

```
#!/usr/bin/python
from distutils.core import setup

setup(name="rebuildd",
      description="Debian packages rebuild tool",
      author="Julien Danjou",
      author_email="acid@debian.org",
      url="http://julien.danjou.info/software/rebuildd.html",
      entry_points={
          'console_scripts': [
              'rebuildd = rebuildd:main',
          ],
      },
      packages=['rebuildd'])
```

Qualquer pacote Python pode registrar pontos de entrada. Os pontos de entrada são organizados em grupos: cada grupo é composto de uma lista de pares com chave e valor. Esses pares utilizam o formato *path.para.o.módulo:nome_da_variável*. No exemplo anterior, a chave é `rebuildd` e o valor é `rebuildd:main`.

A lista de pontos de entrada pode ser manipulada com diversas ferramentas, que variam do `setuptools` ao `epi`, conforme mostrarei a seguir. Nas próximas seções, discutiremos como os pontos de entrada podem ser usados para proporcionar mais extensibilidade ao nosso software.

Visualizando os pontos de entrada

O modo mais fácil de visualizar os pontos de entrada disponíveis em um pacote é por meio de um pacote chamado `entry point inspector`. Você pode instalá-lo executando `pip install entry-point-inspector`. Depois de instalado, ele contém um comando `epi` que você pode executar de modo interativo em seu terminal, a fim de descobrir os pontos de entrada disponibilizados pelos pacotes instalados. A Listagem 5.9 mostra um exemplo da execução de `epi group list` em meu sistema.

Listagem 5.9 – Obtendo uma lista de grupos de pontos de entrada

```
$ epi group list
```

```
-----  
| Name           |  
-----  
| console_scripts      |  
| distutils.commands    |  
| distutils.setup_keywords |  
| egg_info.writers      |  
| epi.commands          |  
| flake8.extension      |  
| setuptools.file_finders |  
| setuptools.installation |  
-----
```

A saída de `epi group list` na Listagem 5.9 mostra os diferentes pacotes que disponibilizam pontos de entrada em um sistema. Cada item dessa tabela é o nome de um grupo de pontos de entrada. Observe que essa lista inclui `console_scripts`, sobre o qual discutiremos em breve. O comando `epi` pode ser usado com o comando `show` para exibir detalhes sobre um grupo específico de pontos de entrada, conforme vemos na Listagem 5.10.

Listagem 5.10 – Mostrando detalhes de um grupo de pontos de entrada

```
$ epi group show console_scripts
```

```
-----  
| Name   | Module | Member | Distribution | Error |  
-----  
| coverage | coverage | main | coverage 3.4 |      |
```

Podemos ver que, no grupo `console_scripts`, um ponto de entrada chamado `coverage` faz referência ao membro `main` do módulo `coverage`. Esse ponto de entrada em particular, disponibilizado pelo pacote `coverage 3.4`, informa qual função Python deve ser chamada quando o script de linha de comando `coverage` é executado. Nesse caso, a função `coverage.main` deve ser chamada.

A ferramenta `epi` é somente uma camada fina acima da biblioteca Python `pkg_resources` completa. Esse módulo nos permite descobrir pontos de entrada para qualquer biblioteca ou programa Python. Os pontos de entrada são importantes em diversas ocasiões, incluindo o uso de scripts de console e a descoberta dinâmica de código, como veremos nas próximas seções.

Usando scripts de console

Ao escrever uma aplicação Python, quase sempre você deve fornecer um programa possível de ser iniciado – um script Python que o usuário final possa executar –, o qual deverá ser instalado em um diretório, em algum lugar no path do sistema.

A maioria dos projetos tem um programa possível de ser iniciado, semelhante a:

```
#!/usr/bin/python
import sys
import mysoftware
```

```
mysoftware.SomeClass(sys.argv).run()
```

Esse tipo de script representa um cenário de melhor caso: muitos projetos têm um script muito mais extenso instalado no path do sistema. No entanto, scripts como esses podem levar a alguns problemas relevantes:

- Não há nenhuma maneira de o usuário saber onde está o interpretador Python ou qual versão ele utiliza.
- Esse script revela um código binário que não pode ser importado por softwares nem por testes de unidade.

- Não há uma maneira fácil de definir o local em que esse script será instalado.
- O modo de instalar esse script de forma portátil (por exemplo, tanto para Unix como para Windows) não é óbvio.

Para nos ajudar a contornar esses problemas, o `setuptools` disponibiliza o recurso `console_scripts`. Esse ponto de entrada pode ser usado para fazer o `setuptools` instalar um pequeno programa no path do sistema, o qual chamará uma função específica de um de seus módulos. Com o `setuptools`, podemos especificar uma chamada de função para iniciar o seu programa, configurando um par chave/valor no grupo de pontos de entrada `console_scripts`: a chave é o nome do script que será instalado e o valor é o path Python para a sua função (algo como `my_module.main`).

Vamos supor que haja um programa `foobar` composto de um cliente e um servidor. Cada parte é escrita em seu próprio módulo — `foobar.client` e `foobar.server`, respectivamente em `foobar/client.py`:

```
def main():  
    print("Client started")
```

e em `foobar/server.py`:

```
def main():  
    print("Server started")
```

É claro que esse programa não faz muita coisa — nosso cliente e o nosso servidor nem sequer conversam entre si. Em nosso exemplo, porém, eles só precisam exibir uma mensagem que nos permita saber que foram iniciados com sucesso.

Podemos agora criar o seguinte arquivo `setup.py` no diretório-raiz, com pontos de entrada definidos nesse arquivo.

```
from setuptools import setup  
  
setup(  
    name="foobar",  
    version="1",  
    description="Foo!",  
    author="Julien Danjou",  
    author_email="julien@danjou.info",
```



```

packages=["foobar"],
entry_points={
    "console_scripts": [
        ❶ "foobard = foobar.server:main",
        "foobar = foobar.client:main",
    ],
},
)

```

Definimos pontos de entrada usando o formato *módulo.submódulo:função*. Podemos ver que, no exemplo, definimos um ponto de entrada para cada um, para o client e para o server ❶.

Quando `python setup.py install` é executado, o `setuptools` criará um script com o aspecto apresentado na Listagem 5.11.

Listagem 5.11 – Um script de console gerado pelo setuptools

```

#!/usr/bin/python
# EASY-INSTALL-ENTRY-SCRIPT: 'foobar==1','console_scripts','foobar'
__requires__ = 'foobar==1'
import sys
from pkg_resources import load_entry_point

if __name__ == '__main__':
    sys.exit(
        load_entry_point('foobar==1', 'console_scripts', 'foobar')()
    )

```

Esse código varre os pontos de entrada do pacote `foobar` e obtém a chave `foobar` do grupo `console_scripts`, a qual é usada para localizar e executar a função correspondente. O valor de retorno de `load_entry_point` será então uma referência à função `foobar.client.main`, a qual será chamada sem argumentos e cujo valor de retorno será usado como o código de saída.

Observe que esse código utiliza `pkg_resources` para descobrir e carregar arquivos de pontos de entrada em seus programas Python.

NOTA Se você estiver usando *pbr* com o *setuptools*, o script gerado será mais simples (e, portanto, mais rápido) do que o script default criado pelo *setuptools*, pois ele chamará a função que você escreveu no ponto de entrada sem ter de pesquisar dinamicamente a lista de pontos de entrada no momento da execução.

Usar scripts de console é uma técnica que acaba com o fardo de

escrever scripts portáteis, ao mesmo tempo que garante que seu código permaneça em seu pacote Python e possa ser importado (e testado) por outros programas.

Usando plugins e drivers

Os pontos de entrada facilitam descobrir e carregar dinamicamente um código implantado por outros pacotes, mas essa não é a sua única utilidade. Qualquer aplicação pode propor e registrar pontos de entrada e grupos e, em seguida, usá-los conforme desejado.

Nesta seção, criaremos um daemon em estilo `cron` chamado `pycrond`, que permitirá que qualquer programa Python registre um comando a ser executado uma vez a intervalos de alguns segundos, registrando um ponto de entrada no grupo `pytimed`. O atributo informado nesse ponto de entrada deve ser um objeto que devolva `number_of_seconds`, `callable`.

Eis a nossa implementação do `pycrond` usando `pkg_resources` para descobrir os pontos de entrada, em um programa que chamei de *pytimed.py*:

```
import pkg_resources
import time

def main():
    seconds_passed = 0
    while True:
        for entry_point in pkg_resources.iter_entry_points('pytimed'):
            try:
                seconds, callable = entry_point.load>()
            except:
                # Ignora a falha
                pass
            else:
                if seconds_passed % seconds == 0:
                    callable()
        time.sleep(1)
        seconds_passed += 1
```

Esse programa é composto de um laço infinito que itera pelos

pontos de entrada do grupo `pytimed`. Cada ponto de entrada é carregado com o método `load()`. O programa então chama o método devolvido, o qual deve retornar o número de segundos que devemos aguardar para chamar o callable, assim como o callable propriamente dito.

O programa em `pytimed.py` tem uma implementação bem simplista e ingênua, mas é suficiente para o nosso exemplo. Agora podemos escrever outro programa Python, chamado `hello.py`, que exige que uma de suas funções seja chamada periodicamente:

```
def print_hello():  
    print("Hello, world!")
```

```
def say_hello():  
    return 2, print_hello
```

Assim que tivermos essa função definida, nós a registramos usando o ponto de entrada apropriado em `setup.py`.

```
from setuptools import setup  
  
setup(  
    name="hello",  
    version="1",  
    packages=["hello"],  
    entry_points={  
        "pytimed": [  
            "hello = hello:say_hello",  
        ],  
    },)
```

O script `setup.py` registra um ponto de entrada no grupo `pytimed` com a chave `hello` e um valor que aponta para a função `hello.say_hello`. Assim que esse pacote for instalado com esse `setup.py` — por exemplo, usando `pip install` —, o script `pytimed` será capaz de detectar o ponto de entrada recém-adicionado.

Na inicialização, `pytimed` pesquisará o grupo `pytimed` e encontrará a chave `hello`. Em seguida, ele chamará a função `hello.say_hello` obtendo dois valores: o número de segundos que devemos aguardar entre cada chamada e a função a ser chamada: dois segundos e `print_hello`,

nesse caso. Se o programa for executado, como mostra a Listagem 5.12, você verá “Hello, world!” ser exibido na tela a cada 2 segundos.

Listagem 5.12 – Executando pytimed

```
>>> import pytimed
>>> pytimed.main()
Hello, world!
Hello, world!
Hello, world!
```

As possibilidades que esse sistema oferece são imensas: você pode criar sistemas de drivers, sistemas de hook e extensões, de modo fácil e genérico. Implementar esse sistema manualmente em cada programa que você criar seria tedioso; felizmente, porém, há uma biblioteca Python capaz de cuidar das partes enfadonhas para nós.

A biblioteca `stevedore` oferece suporte para plugins dinâmicos, com base no mesmo mecanismo apresentado em nossos exemplos anteriores. O caso de uso desse exemplo já é simplista, mas podemos simplificá-lo mais ainda neste script, *pytimed_stevedore.py*:

```
from stevedore.extension import ExtensionManager
import time

def main():
    seconds_passed = 0
    extensions = ExtensionManager('pytimed', invoke_on_load=True)
    while True:
        for extension in extensions:
            try:
                seconds, callable = extension.obj
            except:
                # Ignora a falha
                pass
            else:
                if seconds_passed % seconds == 0:
                    callable()
        time.sleep(1)
        seconds_passed += 1
```

A classe `ExtensionManager` de `stevedore` fornece um modo simples de

carregar todas as extensões de um grupo de pontos de entrada. O nome é passado como o primeiro argumento. O argumento `invoke_on_load=True` garante que cada função do grupo seja chamada assim que for descoberta. Isso torna o resultado diretamente acessível por meio do atributo `obj` da extensão.

Se você consultar a documentação do `stevedore`, verá que `ExtensionManager` tem diversas subclasses capazes de lidar com diferentes situações, como carregar extensões específicas com base em seus nomes ou no resultado de uma função. Todos esses modelos são comumente usados e podem ser aplicados em seu programa a fim de implementar diretamente esses padrões.

Por exemplo, podemos carregar e executar somente uma extensão de nosso grupo de pontos de entrada. Tirar proveito da classe `stevedore.driver.DriverManager` nos permite fazer isso, como mostra a Listagem 5.13.

Listagem 5.13 – Usando o stevedore para executar uma única extensão a partir de um ponto de entrada

```
from stevedore.driver import DriverManager
import time

def main(name):
    seconds_passed = 0
    seconds, callable = DriverManager('pytimed', name, invoke_on_load=True).driver
    while True:
        if seconds_passed % seconds == 0:
            callable()
            time.sleep(1)
            seconds_passed += 1

main("hello")
```

Nesse caso, somente uma extensão é carregada e selecionada pelo nome. Isso nos permite criar rapidamente um *sistema de driver* no qual apenas uma extensão é carregada e usada por um programa.

Resumo

O ecossistema de empacotamento em Python tem uma história tumultuada; no entanto, a situação está se estabilizando agora. A biblioteca `setuptools` oferece uma solução completa para o empacotamento, não só para transportar o seu código em diferentes formatos e fazer o upload para o PyPI, mas também para lidar com a conexão com outros softwares e bibliotecas por meio de pontos de entrada.

Nick Coghlan fala sobre empacotamento

Nick é desenvolvedor do núcleo de Python e trabalha na Red Hat. Já escreveu diversas propostas PEP, incluindo a PEP 426 (Metadata for Python Software Packages 2.0, ou Metadados para Pacotes de Software Python 2.0), e atua como representante de nosso Benevolent Dictator for Life (Ditador Benevolente Vitalício), Guido van Rossum, autor de Python.

O número de soluções para empacotamento (distutils, setuptools, distutils2, distlib, bento, pbr, e assim por diante) de Python é bem grande. Em sua opinião, quais são os motivos para uma fragmentação e uma divergência como essa?

A resposta rápida é que a publicação, a distribuição e a integração de software é um problema complexo, com muito espaço para várias soluções personalizadas de acordo com os diferentes casos de uso. Em minhas conversas recentes sobre o assunto, tenho percebido que o problema está relacionado principalmente à idade, com diferentes ferramentas de empacotamento tendo surgido em diferentes eras da distribuição de software.

A PEP 426, que define um novo formato para metadados em pacotes Python, é razoavelmente recente e ainda não foi aprovada. Como você acha que ela enfrentará os problemas atuais de empacotamento?

A PEP 426 teve início originalmente como parte da definição do formato `Wheel`, mas Daniel Holth percebeu que o `Wheel` poderia

funcionar com o formato de metadados existente definido pelo `setuptools`. A PEP 426, portanto, é uma consolidação dos metadados existentes no `setuptools` com algumas das ideias do `distutils2` e de outros sistemas de empacotamento (como `RPM` e `npm`). Ela resolve alguns dos problemas frustrantes que estão em ferramentas atuais (por exemplo, separando claramente os diferentes tipos de dependências).

As principais vantagens serão uma API REST no PyPI que ofereça um acesso completo aos metadados, bem como a capacidade de gerar automaticamente pacotes com uma política de distribuição compatível com base em metadados em um nível superior (ou é o que se espera).

O formato do Wheel é, de certo modo, recente, e ainda não é amplamente usado, mas parece promissor. Por que ele não faz parte da Biblioteca-Padrão?

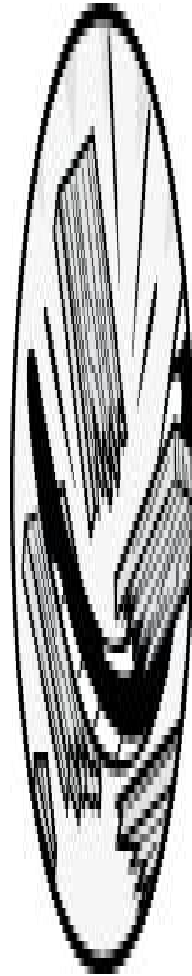
O fato é que a Biblioteca-Padrão não é realmente um lugar apropriado para padrões de empacotamento: ela evolui muito lentamente, e um acréscimo em uma versão mais recente da Biblioteca-Padrão não poderá ser usado com versões mais antigas de Python. Desse modo, no encontro de líderes sobre a linguagem Python este ano, ajustamos o processo da PEP para permitir que `distutils-sig` gerencie o ciclo completo de aprovação das PEPs relacionadas a empacotamento, e `python-dev` só será envolvido em propostas que dizem respeito a modificar CPython diretamente (por exemplo, na inicialização do `pip`).

Qual é o futuro dos pacotes Wheel?

Ainda temos alguns ajustes a serem feitos antes que o `Wheel` se torne apropriado para uso no Linux. No entanto, o `pip` está adotando o `Wheel` como uma alternativa ao formato `Egg`, permitindo um caching local de builds para uma criação rápida de ambientes virtuais e, além disso, o PyPI permite uploads de arquivos `Wheel` para Windows e macOS.

6

TESTES DE UNIDADE



Muitas pessoas acham que os testes de unidade são árdios e consomem tempo, e algumas pessoas e projetos não têm nenhuma política para testes. Este capítulo parte do pressuposto de que você vê sabedoria nos testes de unidade! Escrever um código que não seja testado é basicamente inútil, pois não há nenhuma maneira de provar, de forma conclusiva, que ele funciona. Se você precisa ser convencido, sugiro que comece a ler sobre as vantagens

do desenvolvimento orientado a testes (test-driven development).

Neste capítulo, conheceremos as ferramentas Python que podem ser usadas para criar uma suíte de testes abrangente, que farão com que os testes sejam mais simples e mais automatizados. Falaremos sobre como usar ferramentas para deixar seu software extremamente robusto e evitar regressões. Veremos como criar objetos de teste reutilizáveis, executar testes em paralelo, identificar códigos não testados e usar ambientes virtuais para garantir que seus testes estão limpos, assim como outros métodos e ideias relacionados às boas práticas.

Básico sobre testes

Escrever e executar testes de unidade em Python não é complicado. O processo não é intrusivo nem desestabilizador, e os testes de unidade ajudarão muito, a você e aos demais desenvolvedores, na manutenção de seu software. Nesta seção, discutiremos alguns dos aspectos absolutamente básicos sobre os testes, que facilitarão muito a sua vida.

Alguns testes simples

Inicialmente, você deve armazenar os testes em um submódulo `tests` da aplicação ou biblioteca à qual eles se aplicam. Fazer isso lhe permitirá disponibilizar os testes como parte de seu módulo, de modo que possam ser executados ou reutilizados por qualquer pessoa – mesmo depois que seu software estiver instalado – sem necessariamente utilizar o pacote fonte. Fazer dos testes um submódulo de seu módulo principal também evita que eles sejam instalados por engano em um módulo `tests` de nível mais alto.

Usar uma hierarquia em sua árvore de testes que imite a hierarquia da árvore de seu módulo facilitará a administração de seus testes. Isso significa que os testes que se aplicam ao código de `mylib/foobar.py` devem ser armazenados em `mylib/tests/test_foobar.py`. Uma nomenclatura consistente facilitará quando você estiver procurando os testes

relacionados a um arquivo em particular. A Listagem 6.1 mostra o teste de unidade mais simples que é possível escrever.

Listagem 6.1 – Um teste realmente simples em test_true.py

```
def test_true():  
    assert True
```

Esse teste simplesmente confirma se o comportamento do programa é o que você espera. Para executar esse teste, é necessário carregar o arquivo *test_true.py* e executar a função *test_true()* aí definida.

Contudo, escrever e executar um teste individual para cada um de seus arquivos de teste e funções seria complicado. Em projetos pequenos, de uso simples, o pacote *pytest* vem para nos salvar – uma vez instalado com o *pip*, esse pacote disponibiliza o comando *pytest*, que carrega todos os arquivos cujos nomes comecem com *test_* e, em seguida, executa todas as funções iniciadas com *test_* que estiverem nesses arquivos.

Se tivermos somente o arquivo *test_true.py* em nossa árvore de fontes, a execução de *pytest* nos fornecerá o resultado a seguir:

```
$ pytest -v test_true.py  
===== test session starts =====  
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2,  
    pluggy-0.6.0 -- /usr/local/opt/python/bin/python3.6  
cachedir: .cache  
rootdir: examples, inifile:  
collected 1 item  
  
test_true.py::test_true PASSED [100%]  
  
===== 1 passed in 0.01 seconds =====
```

A opção *-v* diz ao *pytest* para que seja verboso e exiba o nome de cada teste executado em uma linha separada. Se um teste falhar, a saída mudará de modo a informar a falha, acompanhada de todo o *traceback*.

Vamos acrescentar desta vez um teste que vai falhar, conforme mostra a Listagem 6.2.

Listagem 6.2 – Um teste que falha em `test_true.py`

```
def test_false():  
    assert False
```

Se executarmos o arquivo de teste novamente, eis o que acontecerá:

```
$ pytest -v test_true.py  
===== test session starts =====  
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0  
-- /usr/local/opt/python/bin/python3.6  
cachedir: .cache  
rootdir: examples, inifile:  
collected 2 items  
  
test_true.py::test_true PASSED [ 50%]  
test_true.py::test_false FAILED [100%]  
  
===== FAILURES =====  
_____ test_false _____  
  
    def test_false():  
>     assert False  
E       assert False  
  
test_true.py:5: AssertionError  
===== 1 failed, 1 passed in 0.07 seconds =====
```

Um teste falha assim que uma exceção `AssertionError` é gerada; nosso teste `assert` gerará um `AssertionError` quando seu argumento for avaliado com algo que seja falso (`False`, `None`, `0` etc.). Se alguma outra exceção for gerada, o teste também informará um erro.

Simples, não é mesmo? Embora seja simplista, muitos projetos pequenos utilizam essa abordagem, e ela funciona muito bem. Esses projetos não exigem nenhuma ferramenta ou biblioteca além do `pytest` e, desse modo, podem contar com testes simples do tipo `assert`.

À medida que você começar a escrever testes mais sofisticados, o `pytest` ajudará a saber o que há de errado nos testes que falharem. Suponha que tenhamos o seguinte teste:

```
def test_key():
    a = ['a', 'b']
    b = ['b']
    assert a == b
```

Quando o `pytest` for executado, o seguinte resultado será obtido:

```
$ pytest test_true.py
===== test session starts =====
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
rootdir: /Users/jd/Source/python-book/examples, inifile:
plugins: celery-4.1.0
collected 1 item
```

```
test_true.py F [100%]
```

```
===== FAILURES =====
_____ test_key _____
```

```
def test_key():
    a = ['a', 'b']
    b = ['b']
>    assert a == b
E     AssertionError: assert ['a', 'b'] == ['b']
E       At index 0 diff: 'a' != 'b'
E       Left contains more items, first extra item: 'b'
E       Use -v to get the full diff
```

```
test_true.py:10: AssertionError
===== 1 failed in 0.07 seconds =====
```

Esse resultado nos mostra que `a` e `b` são diferentes e que esse teste não passou. Também nos informa exatamente como eles são diferentes, facilitando corrigir o teste ou o código.

Ignorando testes

Se um teste não puder ser executado, você provavelmente vai querer que ele seja ignorado – por exemplo, talvez você queira executar um teste de forma condicional, com base na presença ou na ausência de uma determinada biblioteca. Para isso, a função `pytest.skip()` pode ser usada, a qual marcará o teste para que seja ignorado, passando para o próximo teste. O decorador `pytest.mark.skip`

ignora incondicionalmente a função de teste decorada, portanto, você deve usá-lo se um teste tiver de ser sempre ignorado. A Listagem 6.3 mostra como ignorar um teste usando esses métodos.

Listagem 6.3 – Ignorando testes

```
import pytest

try:
    import mylib
except ImportError:
    mylib = None

@pytest.mark.skip("Do not run this")
def test_fail():
    assert False

@pytest.mark.skipif(mylib is None, reason="mylib is not available")
def test_mylib():
    assert mylib.foobar() == 42

def test_skip_at_runtime():
    if True:
        pytest.skip("Finally I don't want to run it")
```

Quando executado, esse arquivo de testes exibirá o seguinte:

```
$ pytest -v examples/test_skip.py
===== test session starts =====
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
-- /usr/local/opt/python/bin/python3.6
cachedir: .cache
rootdir: examples, inifile:
collected 3 items

examples/test_skip.py::test_fail SKIPPED [ 33%]
examples/test_skip.py::test_mylib SKIPPED [ 66%]
examples/test_skip.py::test_skip_at_runtime SKIPPED [100%]

===== 3 skipped in 0.01 seconds =====
```

A saída da execução dos testes que estão na Listagem 6.3 mostra que, nesse caso, todos os testes foram ignorados. Essa informação garante que um teste que você espera que seja executado não seja

acidentalmente ignorado.

Executando testes específicos

Ao usar o `pytest`, muitas vezes você vai querer executar somente um subconjunto específico de seus testes. É possível selecionar os testes que você quer executar passando seu diretório ou os arquivos como argumento da linha de comando de `pytest`. Por exemplo, chamar `pytest test_one.py` executará somente o teste `test_one.py`. O `pytest` também aceita um diretório como argumento e, nesse caso, ele percorrerá recursivamente o diretório e executará qualquer arquivo que corresponda ao padrão `test_*.py`.

Também podemos adicionar um filtro com o argumento `-k` na linha de comando a fim de executar somente o teste que corresponder a um nome, como mostra a Listagem 6.4.

Listagem 6.4 – Filtrando testes, executando-os com base no nome

```
$ pytest -v examples/test_skip.py -k test_fail
===== test session starts =====
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
-- /usr/local/opt/python/bin/python3.6
cachedir: .cache
rootdir: examples, inifile:
collected 3 items

examples/test_skip.py::test_fail SKIPPED [100%]

=== 2 tests deselected ===
=== 1 skipped, 2 deselected in 0.04 seconds ===
```

Os nomes nem sempre são o melhor modo de filtrar os testes a serem executados. Em geral, um desenvolvedor agruparia os testes de acordo com as funcionalidades ou os tipos. O `pytest` tem um sistema de marcação dinâmico que permite marcar os testes com uma palavra reservada, a qual poderá ser usada como filtro. Para marcar os testes dessa forma, utilize a opção `-m`. Se criarmos dois testes assim:

```
import pytest
```

```
@pytest.mark.dicctest
def test_something():
    a = ['a', 'b']
    assert a == a

def test_something_else():
    assert False
```

podemos usar o argumento `-m` com o `pytest` para executar apenas um desses testes:

```
$ pytest -v test_mark.py -m dicctest
=== test session starts ===
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0 --
/usr/local/opt/python/bin/python3.6
cachedir: .cache
rootdir: examples, inifile:
collected 2 items

test_mark.py::test_something PASSED [100%]

=== 1 tests deselected ===
=== 1 passed, 1 deselected in 0.01 seconds ===
```

O marcador `-m` aceita consultas mais complexas, de modo que podemos também executar todos os testes que *não* estejam marcados:

```
$ pytest test_mark.py -m 'not dicctest'
=== test session starts ===
platform darwin -- Python 3.6.4, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
rootdir: examples, inifile:
collected 2 items

test_mark.py F [100%]

=== FAILURES ===
test_something_else

    def test_something_else():
>     assert False
E     assert False

test_mark.py:10: AssertionError
```

```
=== 1 tests deselected ===  
=== 1 failed, 1 deselected in 0.07 seconds ===
```

Nesse caso, o `pytest` executou todos os testes que não estavam marcados com `dicthtest` – o teste `test_something_else`, que falhou. O outro teste marcado, `test_something`, não foi executado e, portanto, foi listado como `deselected`.

O `pytest` aceita expressões complexas compostas das palavras reservadas `or`, `and` e `not`, permitindo uma filtragem mais sofisticada.

Executando testes em paralelo

Suítes de teste podem demorar bastante para executar. Não é incomum que uma suíte completa de testes de unidade demore dezenas de minutos para executar em projetos de software maiores. Por padrão, o `pytest` executa todos os testes serialmente, em uma ordem indefinida. Como a maioria dos computadores tem várias CPUs, em geral você poderá agilizar a execução se separar a lista de testes e executá-los em diversas CPUs.

Para cuidar dessa abordagem, o `pytest` oferece o plugin `pytest-xdist`, que pode ser instalado com o `pip`. Esse plugin estende a linha de comando do `pytest` com o argumento `--numprocesses` (abreviado como `-n`), que aceita o número de CPUs a serem usadas como argumento. Executar `pytest -n 4` executaria a sua suíte de testes em quatro processos paralelos, distribuindo a carga entre as CPUs disponíveis. Como o número de CPUs pode mudar de um computador para outro, o plugin também aceita a palavra reservada `auto` como valor. Nesse caso, ele sondará a máquina a fim de obter o número de CPUs disponíveis e iniciará essa quantidade de processos.

Criando objetos usados nos testes com fixtures

Nos testes de unidade, com frequência você terá de executar um conjunto de instruções comuns antes e depois de executar um teste, e essas instruções utilizarão determinados componentes. Por

exemplo, talvez você precise de um objeto que represente o estado da configuração de sua aplicação; é provável que você queira que esse objeto seja inicializado antes de cada teste, e então seja reiniciado com seus valores default quando o teste tiver terminado. De modo semelhante, se seu teste depender da criação temporária de um arquivo, esse arquivo deverá estar criado antes de o teste começar, e deverá ser removido assim que o teste for concluído. Esses componentes, conhecidos como *fixtures*, são configurados antes de um teste, e serão limpos depois que o teste estiver concluído.

Com o pytest, as fixtures são definidas como funções simples. A função de fixture deve devolver o(s) objeto(s) desejado(s), de modo que um teste que utilize essa fixture possa utilizar esse(s) objeto(s).

Eis uma fixture simples:

```
import pytest

@pytest.fixture
def database():
    return <conexão com algum banco de dados>

def test_insert(database):
    database.insert(123)
```

A fixture database será automaticamente utilizada por qualquer teste que tenha database em sua lista de argumentos. A função test_insert() receberá o resultado da função database() como seu primeiro argumento e usará esse resultado conforme desejar. Se usarmos uma fixture dessa maneira, não será necessário repetir o código de inicialização de banco de dados várias vezes.

Outro recurso comum em testes de código é o encerramento depois que um teste tiver usado uma fixture. Por exemplo, talvez seja necessário encerrar uma conexão com o banco de dados. Implementar a fixture como um gerador nos permite acrescentar a funcionalidade de encerramento, conforme vemos na Listagem 6.5.

Listagem 6.5 – Funcionalidade de encerramento

```

import pytest

@pytest.fixture
def database():
    db = <conexão com algum banco de dados>
    yield db
    db.close()

def test_insert(database):
    database.insert(123)

```

Como utilizamos a palavra reservada `yield` e fizemos `database` ser um gerador, o código após a instrução `yield` executará quando o teste tiver terminado. Esse código encerrará a conexão com o banco de dados no final do teste.

No entanto, encerrar uma conexão com o banco de dados a cada teste poderia impor um custo desnecessário à execução, pois os testes poderiam reutilizar essa mesma conexão. Nesse caso, podemos passar o argumento `scope` ao decorador da fixture, especificando o seu escopo:

```

import pytest

@pytest.fixture(scope="module")
def database():
    db = <conexão com algum banco de dados>
    yield db
    db.close()

def test_insert(database):
    database.insert(123)

```

Ao especificar o parâmetro `scope="module"`, a fixture será inicializada uma vez para todo o módulo, e a mesma conexão com o banco de dados será passada para todas as funções de teste que exigirem uma conexão com o banco de dados.

Por fim, você pode executar algum código comum antes e depois de seus testes, marcando as fixtures como *automaticamente usadas* com a palavra reservada `autouse`, em vez de especificá-las como um argumento para cada uma das funções de teste. Especificar o

argumento nomeado `autouse=True` na função `pytest.fixture()` garantirá que a fixture seja chamada antes de executar qualquer teste do módulo ou da classe no qual ele estiver definido, como vemos no exemplo a seguir:

```
import os

import pytest

@pytest.fixture(autouse=True)
def change_user_env():
    curuser = os.environ.get("USER")
    os.environ["USER"] = "foobar"
    yield
    os.environ["USER"] = curuser

def test_user():
    assert os.getenv("USER") == "foobar"
```

Esses recursos ativados automaticamente são convenientes, mas você não deve abusar das fixtures: elas serão executadas antes de cada um dos testes incluídos no escopo, portanto, podem deixar a execução dos testes significativamente mais lenta.

Executando cenários de testes

Ao executar testes de unidade, talvez você queira fazer o mesmo teste de tratamento de erros com vários objetos diferentes que disparem esse erro, ou queira executar uma suíte de testes completa com diferentes drivers.

Utilizamos intensamente essa última abordagem quando desenvolvemos o *Gnocchi*, que é um banco de dados de séries temporais. O *Gnocchi* disponibiliza uma classe abstrata que chamamos de *API de armazenagem*. Qualquer classe Python pode implementar essa classe abstrata e se registrar para que seja um driver. O software carrega o driver de armazenagem configurado quando solicitado e utiliza a API de armazenagem implementada para armazenar ou acessar dados. Nesse caso, precisamos de uma classe de testes de unidade que execute em cada driver –

executando, desse modo, com cada implementação dessa API de armazenagem – a fim de garantir que todos os drivers funcionem conforme esperado por quem faz as chamadas.

Um modo fácil de fazer isso é com o uso de *fixtures parametrizadas*; todos os testes que usam essas fixtures serão executados várias vezes, uma para cada um dos parâmetros definidos. A Listagem 6.6 mostra um exemplo do uso de fixtures parametrizadas para executar um único teste duas vezes, com diferentes parâmetros: uma vez para mysql e outra para postgresql.

Listagem 6.6 – Executando um teste usando fixtures parametrizadas

```
import pytest
import myapp

@pytest.fixture(params=["mysql", "postgresql"])
def database(request):
    d = myapp.driver(request.param)
    d.start()
    yield d
    d.stop()

def test_insert(database):
    database.insert("somedata")
```

Na Listagem 6.6, a fixture `driver` é parametrizada com dois valores distintos, isto é, com cada um dos nomes de um driver de banco de dados aceito pela aplicação. Quando `test_insert` é executado, isso é feito, na verdade, duas vezes: uma com uma conexão de banco de dados MySQL e outra com uma conexão de banco de dados PostgreSQL. Isso nos permite facilmente reutilizar o mesmo teste em diferentes cenários, sem acrescentar muitas linhas de código.

Testes controlados usando simulação

Objetos simulados (mocks) são objetos que imitam o comportamento de objetos reais da aplicação, mas de maneiras específicas e controladas. São particularmente convenientes para criar ambientes que descrevam exatamente as condições nas quais

você gostaria de testar o código. Podemos substituir todos os objetos, exceto um, por objetos simulados, a fim de isolar o comportamento de seu objeto principal, e criar um ambiente para testar o seu código.

Um caso de uso é a escrita de um cliente HTTP, pois é praticamente impossível (ou é, no mínimo, extremamente complicado) iniciar o servidor HTTP e testá-lo em todos os cenários para que devolva todos os valores possíveis. Os clientes HTTP são particularmente difíceis de testar em todos os cenários de falhas.

A biblioteca-padrão para criar objetos simulados em Python é a `mock`. A partir de Python 3.3, a `mock` foi incluída na Biblioteca-Padrão de Python como `unittest.mock`. Desse modo, podemos usar um trecho de código como o que vemos a seguir para manter a compatibilidade com versões anteriores a Python 3.3:

```
try:
    from unittest import mock
except ImportError:
    import mock
```

A biblioteca `mock` é bem fácil de usar. Qualquer atributo acessado em um objeto `mock.Mock` é criado dinamicamente durante a execução. Qualquer valor pode ser atribuído a um atributo desse tipo. A Listagem 6.7 mostra a `mock` sendo usada para criar um objeto simulado com um atributo simulado.

Listagem 6.7 – Acessando o atributo de `mock.Mock`

```
>>> from unittest import mock
>>> m = mock.Mock()
>>> m.some_attribute = "hello world"
>>> m.some_attribute
"hello world"
```

Também podemos criar dinamicamente um método em um objeto maleável, como vemos na Listagem 6.8, na qual criamos um método de simulação que sempre devolve 42 e aceita qualquer dado como argumento.

Listagem 6.8 – Criando métodos em um objeto `mock.Mock`

```

>>> from unittest import mock
>>> m = mock.Mock()
>>> m.some_method.return_value = 42
>>> m.some_method()
42
>>> m.some_method("with", "arguments")
42

```

Com poucas linhas, seu objeto `mock.Mock` agora tem um método `some_method()` que devolve 42. Ele aceita qualquer tipo de argumento e não há nenhuma verificação quanto aos valores – ainda.

Métodos criados dinamicamente também podem ter efeitos colaterais (intencionais). Em vez de serem métodos boilerplate que simplesmente devolvem um valor, eles podem ser definidos para executar um código útil.

A Listagem 6.9 cria um método de simulação cujo efeito colateral é exibir a string "hello world".

Listagem 6.9 – Criando métodos com efeitos colaterais em um objeto `mock.Mock`

```

>>> from unittest import mock
>>> m = mock.Mock()
>>> def print_hello():
...     print("hello world!")
...     return 43
...
❶ >>> m.some_method.side_effect = print_hello
>>> m.some_method()
hello world!
43
❷ >>> m.some_method.call_count
1

```

Atribuímos uma função completa ao atributo `some_method` ❶. Essa técnica nos permite implementar cenários mais complexos em um teste, pois podemos associar qualquer código necessário aos testes em um objeto simulado. Então, basta passar esse objeto simulado para qualquer função que espere usá-lo.

O atributo `call_count` ❷ é um modo simples de verificar o número de

vezes que um método foi chamado.

A biblioteca `mock` utiliza o padrão ação/asserção: isso significa que, uma vez que seu teste tenha executado, caberá a você verificar se as ações que você está simulando foram corretamente executadas. A Listagem 6.10 aplica o método `assert()` em nossos objetos simulados para fazer essas verificações.

Listagem 6.10 – Verificando as chamadas de métodos

```
>>> from unittest import mock
>>> m = mock.Mock()
❶ >>> m.some_method('foo', 'bar')
<Mock name='mock.some_method()' id='26144272'>
❷ >>> m.some_method.assert_called_once_with('foo', 'bar')
>>> m.some_method.assert_called_once_with('foo', ❸mock.ANY)
>>> m.some_method.assert_called_once_with('foo', 'baz')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python2.7/dist-packages/mock.py", line 846, in assert_called_once_with
    return self.assert_called_with(*args, **kwargs)
  File "/usr/lib/python2.7/dist-packages/mock.py", line 835, in assert_called_with
    raise AssertionError(msg)
AssertionError: Expected call: some_method('foo', 'baz')
Actual call: some_method('foo', 'bar')
```

Criamos um método com os argumentos `foo` e `bar` para representar nossos testes, chamando o método ❶. O modo usual de verificar chamadas a um objeto simulado é usar os métodos `assert_called()`, por exemplo, `assert_called_once_with()` ❷. Para esses métodos, devemos passar os valores que esperamos que quem fizer a chamada vai utilizar quando chamar o seu método de simulação. Se os valores passados não forem aqueles sendo usados, `mock` gerará um `AssertionError`. Se você não souber quais argumentos podem ser passados, poderá usar `mock.ANY` como valor ❸; qualquer argumento passado para o seu método de simulação lhe será correspondente.

A biblioteca `mock` também pode ser usada para corrigir alguma função, método ou objeto de um módulo externo. Na Listagem 6.11, substituímos a função `os.unlink()` por uma função simulada, que nós disponibilizamos.

Listagem 6.11 – Usando mock.patch

```
>>> from unittest import mock
>>> import os
>>> def fake_os_unlink(path):
...     raise IOError("Testing!")
...
>>> with mock.patch('os.unlink', fake_os_unlink):
...     os.unlink('foobar')
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
  File "<stdin>", line 2, in fake_os_unlink
IOError: Testing!
```

Quando usado como um gerenciador de contextos, `mock.patch()` substitui a função alvo pela função que fornecermos, de modo que o código executado no contexto utilizará o método corrigido. Com o método `mock.patch()`, é possível alterar qualquer parte de um código externo, fazendo com que ele se comporte de tal modo que permita testar todas as condições em sua aplicação, como vemos na Listagem 6.12.

Listagem 6.12 – Usando mock.patch() para testar um conjunto de comportamentos

```
from unittest import mock

import pytest
import requests

class WhereIsPythonError(Exception):
    pass

❶ def is_python_still_a_programming_language():
    try:
        r = requests.get("http://python.org")
    except IOError:
        pass
    else:
        if r.status_code == 200:
            return 'Python is a programming language' in r.content
        raise WhereIsPythonError("Something bad happened")
```



```

def get_fake_get(status_code, content):
    m = mock.Mock()
    m.status_code = status_code
    m.content = content

    def fake_get(url):
        return m

    return fake_get

def raise_get(url):
    raise IOError("Unable to fetch url %s" % url)

❷ @mock.patch('requests.get', get_fake_get(
    200, 'Python is a programming language for sure'))
def test_python_is():
    assert is_python_still_a_programming_language() is True

@mock.patch('requests.get', get_fake_get(
    200, 'Python is no more a programming language'))
def test_python_is_not():
    assert is_python_still_a_programming_language() is False

@mock.patch('requests.get', get_fake_get(404, 'Whatever'))
def test_bad_status_code():
    with pytest.raises(WhereIsPythonError):
        is_python_still_a_programming_language()

@mock.patch('requests.get', raise_get)
def test_ioerror():
    with pytest.raises(WhereIsPythonError):
        is_python_still_a_programming_language()

```

A Listagem 6.12 implementa uma suíte de testes que procura todas as instâncias da string “Python is a programming language” na página web <http://python.org/> ❶. Não há como testar os cenários negativos (aqueles em que essa sentença não está na página web) sem modificar a página propriamente dita – algo que não é possível fazer, obviamente. Nesse caso, usamos `mock` para trapacear e modificar o comportamento da requisição, de modo que ela devolva uma resposta simulada, com uma página simulada que não contém

essa string. Isso nos permite testar o cenário negativo, no qual <http://python.org/> não contém essa sentença, garantindo que o programa trate esse caso corretamente.

Esse exemplo utiliza a versão de decorador de `mock.patch()` ❷. Usar o decorador não modifica o comportamento da simulação, mas será mais simples se você precisar usar a simulação no contexto de uma função de teste completa.

Ao usar a simulação, podemos simular qualquer problema, por exemplo, um servidor web devolvendo um erro 404, um erro de E/S ou um problema de latência de rede. Podemos garantir que o código devolverá os valores corretos ou gerará a exceção correta em todos os casos, assegurando que nosso código sempre se comportará conforme esperado.

Identificando um código não testado com o coverage

A ferramenta `coverage` é um ótimo complemento para os testes de unidade, e ela identifica se há alguma parte de seu código que não tenha sido coberta pelos testes. O `coverage` utiliza ferramentas de análise de código e hooks de tracing para determinar quais linhas de seu código foram executadas; quando usada durante uma execução de testes de unidade, a ferramenta é capaz de mostrar quais partes de sua base de código foram verificadas e quais partes não foram. Escrever testes é conveniente, mas ter um modo de saber quais partes de seu código foram ignoradas durante o processo de testes é a cereja do bolo.

Instale o módulo Python `coverage` em seu sistema usando o `pip` a fim de poder ter acesso ao comando do programa `coverage` a partir de seu shell.

NOTA O comando também pode se chamar `python-coverage`, se você instalar o `coverage` com o software de instalação de seu sistema operacional. É o que acontece no Debian, por exemplo.

Usar o `coverage` em modo standalone é simples. Ele pode mostrar as

partes de seus programas que não são executadas e quais códigos poderiam ser “códigos mortos”, isto é, códigos que poderiam ser removidos sem modificar o fluxo de trabalho normal do programa. Todas as ferramentas de teste sobre as quais conversamos até agora neste capítulo estão integradas ao `coverage`.

Ao usar o `pytest`, basta instalar o plugin `pytest-cov` com `pip install pytest-pycov` e adicionar algumas flags de opções para gerar uma saída detalhada sobre a cobertura do código, conforme mostra a Listagem 6.13.

Listagem 6.13 – Usando `coverage` com o `pytest`

```
$ pytest --cov=gnocchiclient gnocchiclient/tests/unit
----- coverage: platform darwin, python 3.6.4-final-0 -----
Name                               Stmts Miss Branch BrPart Cover
-----
gnocchiclient/__init__.py          0    0    0    0 100%
gnocchiclient/auth.py              51   23    6    0  49%
gnocchiclient/benchmark.py        175  175   36    0   0%
--trecho omitido--
-----
TOTAL                             2040  1868  424    6   8%

=== passed in 5.00 seconds ===
```

A opção `--cov` permite que o relatório de cobertura seja apresentado no final da execução dos testes. Você deve passar o nome do pacote como argumento para o plugin a fim de filtrar devidamente o relatório de cobertura. A saída inclui as linhas de código que não foram executadas e, desse modo, não têm testes associados. Tudo que você precisa fazer agora é abrir seu editor de texto favorito e começar a escrever testes para esse código.

No entanto, o `coverage` vai além, permitindo que você gere relatórios organizados em HTML. Basta adicionar a flag `--cov-report=html`, e o diretório `htmlcov` a partir do qual você executou o comando será preenchido com páginas HTML. Cada página mostrará as partes de seu código-fonte que foram ou que não foram executadas.

Se você quiser ser o *tal*, poderá usar a opção `--cover-fail-`

`under=COVER_MIN_PERCENTAGE`, que fará a suíte de testes falhar se uma porcentagem mínima do código não for executada. Embora ter um bom percentual de cobertura seja um objetivo razoável, e ainda que a ferramenta seja útil para ter insights sobre o estado da cobertura de seus testes, definir um valor arbitrário para a porcentagem não nos fornece muitos insights. A Figura 6.1 mostra um exemplo de um relatório de cobertura, com a porcentagem na parte superior.

Por exemplo, um valor de 100% de cobertura do código é uma meta respeitável, mas não significa necessariamente que o código seja totalmente testado e que você possa ficar sossegado. Prova somente que todos os paths de seu código foram executados; não há nenhuma indicação de que todas as condições possíveis tenham sido testadas.

Você deve usar as informações de cobertura para consolidar sua suíte de testes e adicionar testes para qualquer código que não esteja sendo executado no momento. Isso facilitará a manutenção do projeto mais tarde e melhorará a qualidade geral de seu código.

Coverage for **ceilometer.publisher** : 75%

12 statements **9** run **3** missing **0** excluded

```
1 # -*- encoding: utf-8 -*-.
2 #
3 # Copyright © 2013 Intel Corp.
4 # Copyright © 2013 eNovance
5 #
6 # Author: Yunhong Jiang <yunhong.jiang@intel.com>
7 #         Julien Danjou <julien@danjou.info>
8 #
9 # Licensed under the Apache License, Version 2.0 (the "License"); you may
10 # not use this file except in compliance with the License. You may obtain
11 # a copy of the License at
12 #
13 #     http://www.apache.org/licenses/LICENSE-2.0
14 #
15 # Unless required by applicable law or agreed to in writing, software
16 # distributed under the License is distributed on an "AS IS" BASIS, WITHOUT
17 # WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the
18 # License for the specific language governing permissions and limitations
19 # under the License.
20
21 import abc
22 from stevedore import driver
23 from ceilometer.openstack.common import network_utils
24
25 def get_publisher(url, namespace='ceilometer.publisher'):
26     """Get publisher driver and load it.
27
28     :param URL: URL for the publisher
29     :param namespace: Namespace to use to look for drivers.
30     """
31     parse_result = network_utils.urlsplit(url)
32     loaded_driver = driver.DriverManager(namespace, parse_result.scheme)
33     return loaded_driver.driver(parse_result)
34
35 class PublisherBase(object):
36     """Base class for plugins that publish the sampler."""
37
38     __metaclass__ = abc.ABCMeta
39
40     def __init__(self, parsed_url):
41         pass
42
43     @abc.abstractmethod
44     def publish_samples(self, context, samples):
45         """Publish samples into final conduit."""
```

Figura 6.1 – Cobertura para ceilometer.publisher

Ambientes virtuais

Já mencionamos o perigo de seus testes talvez não capturarem a ausência de dependências. Qualquer aplicação de tamanho significativo depende inevitavelmente de bibliotecas externas que ofereçam recursos necessários à aplicação, mas há várias maneiras de as bibliotecas externas poderem apresentar problemas em seu sistema operacional. Eis algumas delas:

- seu sistema não tem a biblioteca necessária incluída;
- seu sistema não tem versão *correta* da biblioteca necessária

incluída;

- você precisa de duas versões diferentes da mesma biblioteca para duas aplicações distintas.

Esses problemas podem ocorrer na primeira implantação de sua aplicação ou mais tarde, quando ela já estiver executando. Fazer o upgrade de uma biblioteca Python instalada usando o seu gerenciador de sistemas pode causar falhas imediatas em sua aplicação, sem avisos, por motivos tão simples como uma mudança de API na biblioteca usada pela aplicação.

A solução está em cada aplicação utilizar um diretório de bibliotecas que contenha todas as suas dependências. Esse diretório será então usado para carregar os módulos Python necessários, em vez de carregar os módulos instalados no âmbito do sistema.

Um diretório como esse é conhecido como *ambiente virtual*.

Configurando um ambiente virtual

A ferramenta `virtualenv` cuida automaticamente dos ambientes virtuais para você. Até Python 3.2, você a encontrará no pacote `virtualenv`, que pode ser instalado com `pip install virtualenv`. Se você usa Python 3.3 ou versões mais recentes, ela estará disponível via Python com o nome `venv`.

Para usar o módulo, carregue-o como o programa principal, com um diretório de destino como argumento, assim:

```
$ python3 -m venv myvenv
$ ls foobar
bin    include  lib      pyvenv.cfg
```

Assim que for executado, o `venv` criará um diretório `lib/pythonX.Y` e o utilizará para instalar o `pip` no ambiente virtual, o qual será conveniente para instalar outros pacotes Python.

Então você poderá ativar o ambiente virtual fazendo “sourcing” do comando `activate`. Utilize o seguinte em sistemas Posix :

```
$ source myvenv/bin/activate
```

Em sistemas Windows, utilize o código a seguir:

```
> \myenv\Scripts\activate
```

Assim que fizer isso, seu prompt de shell deverá aparecer prefixado com o nome de seu ambiente virtual. Executar `python` fará com que a versão de Python que foi copiada para o ambiente virtual seja executada. Você pode verificar se ela está funcionando ao ler a variável `sys.path` e conferir se ela tem o diretório de seu ambiente virtual como o primeiro componente.

É possível interromper o ambiente virtual e sair dele a qualquer momento chamando o comando `deactivate`:

```
$ deactivate
```

É isso. Observe também que você não é obrigado a executar `activate` se quiser utilizar o Python instalado em seu ambiente virtual somente uma vez. Chamar o binário `python` também funcionará:

```
$ myenv/bin/python
```

Enquanto estivermos em nosso ambiente virtual ativado, não teremos acesso a nenhum dos módulos instalados e disponíveis no sistema principal. É para isso que usamos um ambiente virtual; contudo, isso significa que provavelmente teremos de instalar os pacotes de que precisaremos. Para isso, utilize o comando `pip` padrão para instalar cada pacote, e os pacotes serão instalados no local correto, deixando o seu sistema inalterado:

```
$ source myenv/bin/activate
(myenv) $ pip install six
Downloading/unpacking six
  Downloading six-1.4.1.tar.gz
  Running setup.py egg_info for package six
```

```
Installing collected packages: six
  Running setup.py install for six
```

```
Successfully installed six
Cleaning up...
```

Voilà! Podemos instalar todas as bibliotecas necessárias e, em seguida, executar nossa aplicação nesse ambiente virtual, sem

causar falhas em nosso sistema. É fácil ver como podemos colocar tudo isso em um script a fim de automatizar a instalação de um ambiente virtual com base em uma lista de dependências, conforme vemos na Listagem 6.14.

Listagem 6.14 – Criação automática do ambiente virtual

```
virtualenv myappvenv
source myappvenv/bin/activate
pip install -r requirements.txt
deactivate
```

Talvez seja conveniente ter acesso aos pacotes instalados no âmbito do sistema, de modo que o `virtualenv` lhe permite habilitá-los ao criar seu ambiente virtual, passando a flag `--system-site-packages` para o comando `virtualenv`.

Em `myvenv`, você encontrará um `pyvenv.cfg`, que é o arquivo de configuração para esse ambiente. Por padrão, ele não tem muitas opções de configuração. Você deve reconhecer `include-system-site-package`, cujo propósito é o mesmo de `system-site-packages` de `virtualenv`, o qual já descrevemos.

Como poderia supor, os ambientes virtuais são extremamente convenientes para execuções automáticas das suítes de teste de unidade. Seu uso é muito disseminado, a ponto de uma ferramenta em particular ter sido criada para lidar com isso.

Usando virtualenv com o tox

Um dos principais usos dos ambientes virtuais é fornecer um ambiente limpo para execução dos testes de unidade. Seria lamentável se você tivesse a impressão de que seus testes estavam funcionando, quando eles não estavam, por exemplo, respeitando a lista de dependências.

Uma forma de garantir que você está levando todas as dependências em consideração seria escrever um script para implantar um ambiente virtual, instalar o `setuptools` e, em seguida, instalar todas as dependências necessárias, tanto para a execução

de sua aplicação/biblioteca como para os testes de unidade. Felizmente, esse é um caso de uso muito comum, e uma aplicação dedicada a essa tarefa já foi criada: o `tox`.

A ferramenta de gerenciamento `tox` tem como objetivo automatizar e padronizar o modo como os testes são executados em Python. Para isso, ela oferece tudo que é necessário para executar uma suíte de testes completa em um ambiente virtual limpo, ao mesmo tempo que instala a sua aplicação para verificar se a instalação funciona.

Antes de usar o `tox`, você deve disponibilizar um arquivo de configuração chamado `tox.ini`, que deve estar no diretório-raiz de seu projeto, ao lado do arquivo `setup.py`.

```
$ touch tox.ini
```

Então você poderá executar o `tox` com sucesso:

```
% tox
GLOB sdist-make: /home/jd/project/setup.py
python create: /home/jd/project/.tox/python
python inst: /home/jd/project/.tox/dist/project-1.zip
_____ summary _____
python: commands succeeded
congratulations :)
```

Nesse exemplo, o `tox` cria um ambiente virtual em `.tox/python` usando a versão default de Python. O arquivo `setup.py` é usado para criar uma distribuição de seu pacote, o qual é então instalado nesse ambiente virtual. Nenhum comando será executado, pois não especificamos nenhum no arquivo de configuração. Ter somente isso não é particularmente útil.

Podemos modificar esse comportamento padrão acrescentando um comando a ser executado em nosso ambiente de testes. Edite `tox.ini` de modo que inclua o seguinte:

```
[testenv]
commands=pytest
```

Agora o `tox` executará o comando `pytest`. No entanto, como não temos o `pytest` instalado no ambiente virtual, esse comando provavelmente falhará. Devemos listar `pytest` como uma dependência a ser instalada:

```
[testenv]
deps=pytest
commands=pytest
```

Ao ser executado agora, o `tox` recriará o ambiente, instalará a nova dependência e executará o comando `pytest`, o qual executará todos os testes de unidade. Para acrescentar outras dependências, você pode listá-las na opção de configuração `deps`, como fizemos em nosso caso, ou pode utilizar a sintaxe `-file` para ler de um arquivo.

Recriando um ambiente

Às vezes, você terá de recriar um ambiente para, por exemplo, garantir que tudo funcionará conforme esperado quando um novo desenvolvedor clonar o repositório de códigos-fontes e executar o `tox` pela primeira vez. Para isso, o `tox` aceita uma opção `--recreate` que recriará o ambiente virtual do zero, com base nos parâmetros que você especificar.

Você deve definir os parâmetros para todos os ambientes virtuais gerenciados pelo `tox` na seção `[testenv]` de `tox.ini`. Além disso, conforme já mencionamos, o `tox` é capaz de gerenciar vários ambientes virtuais Python – na verdade, é possível executar nossos testes com uma versão de Python diferente da versão default, passando a flag `-e` para o `tox`, assim:

```
% tox -e py26
GLOB sdist-make: /home/jd/project/setup.py
py26 create: /home/jd/project/.tox/py26
py26 installdeps: nose
py26 inst: /home/jd/project/.tox/dist/rebuildd-1.zip
py26 runtests: commands[0] | pytest
--trecho omitido--
== test session starts ==
=== 5 passed in 4.87 seconds ===
```

Por padrão, o `tox` simula qualquer ambiente que corresponda a uma versão de Python existente: `py24`, `py25`, `py26`, `py27`, `py30`, `py31`, `py32`, `py33`, `py34`, `py35`, `py36`, `py37`, `jython` e `pypy`! Além do mais, você pode definir seus próprios ambientes. Basta acrescentar outra seção chamada

[testenv:_envname_]. Se quiser executar um comando específico apenas para um dos ambientes, poderá fazer isso facilmente listando o seguinte no arquivo *tox.ini*:

```
[testenv]
deps=pytest
commands=pytest

[testenv:py36-coverage]
deps={[testenv]deps}
    pytest-cov
commands=pytest --cov=myproject
```

Ao usar `pytest --cov=myproject` na seção `py36-coverage`, como mostrado no exemplo, você sobrescreverá os comandos para o ambiente `py36-coverage`; isso significa que, ao executar `tox -e py36-coverage`, o `pytest` será instalado como parte das dependências, mas o comando `pytest`, na verdade, será executado em seu lugar, com a opção de cobertura. Para que isso funcione, a extensão `pytest-cov` deve estar instalada: substituímos o valor de `deps` pelo `deps` de `testenv` e adicionamos a dependência `pytest-cov`. A interpolação de variáveis também é aceita pelo `tox`, portanto, você pode referenciar qualquer outro campo do arquivo *tox.ini* e usá-lo como uma variável; a sintaxe é `{[nome_amb]nome_da_variável}`. Isso nos permite evitar muitas repetições.

Usando diferentes versões de Python

Também podemos criar outro ambiente com uma versão de Python não aceita, usando o seguinte em *tox.ini*:

```
[testenv]
deps=pytest
commands=pytest

[testenv:py21]
basepython=python2.1
```

Ao executar isso, Python 2.1 será chamado (ou haverá uma tentativa de chamá-lo) para executar a suíte de testes – embora, como seja muito pouco provável que você tenha essa versão antiga de Python instalada em seu sistema, eu duvido que vá funcionar em

seu caso!

É provável que você queira oferecer suporte a várias versões de Python, caso em que seria conveniente fazer o `tox` executar todos os testes para todas as versões de Python que você queira aceitar por padrão. É possível fazer isso especificando a lista de ambientes que você quer usar quando o `tox` for executado sem argumentos:

```
[tox]
envlist=py35,py36,pypy
```

```
[testenv]
deps=pytest
commands=pytest
```

Quando o `tox` é iniciado sem outros argumentos, todos os quatro ambientes listados serão criados, carregados com as dependências e a aplicação e, então, executados com o comando `pytest`.

Integrando outros testes

Podemos também usar o `tox` para integração de testes como o `flake8`, discutido no Capítulo 1. O arquivo `tox.ini` a seguir fornece um ambiente PEP 8 que instalará e executará o `flake8`:

```
[tox]
envlist=py35,py36,pypy,pep8
```

```
[testenv]
deps=pytest
commands=pytest
```

```
[testenv:pep8]
deps=flake8
commands=flake8
```

Nesse caso, o ambiente `pep8` será executado com a versão default de Python, o que, provavelmente, não será um problema, embora você possa especificar a opção `basepython`, caso queira mudar isso.

Ao executar o `tox`, você perceberá que todos os ambientes são criados e executados sequencialmente. Isso pode deixar o processo muito lento, mas, como os ambientes virtuais são isolados, nada

impede que você execute os comandos `tox` em paralelo. É exatamente isso que faz o pacote `detox`, oferecendo um comando `detox` que executa todos os ambientes default de `envlist` em paralelo. Instale-o com `pip install!`

Política de testes

Incluir códigos de teste em seu projeto é uma excelente ideia, mas o modo como esse código é executado também é extremamente importante. Muitos projetos têm códigos de teste que falham ao ser executados, por um motivo ou outro. Esse assunto não está estritamente limitado a Python, mas considero muito importante enfatizar o seguinte: você deve ter uma política de tolerância zero para códigos não testados. Nenhum código deve ser incluído na base sem um conjunto apropriado de testes de unidade para ele.

O mínimo a que você deve visar é que cada um dos commits que você fizer passe em todos os testes. Automatizar esse processo será melhor ainda. Por exemplo, o OpenStack conta com um fluxo de trabalho específico baseado no *Gerrit* (um serviço web para revisão de código) e no *Zuul* (um serviço de integração e entrega contínuas). Cada commit enviado passa pelo sistema de revisão de código do Gerrit, e o Zuul é responsável por executar um conjunto de jobs de testes. O Zuul executa os testes de unidade e diversos testes funcionais de nível mais alto em cada projeto. Essa revisão de código, que é executada por alguns desenvolvedores, garante que todo código cujo commit seja feito tenha testes de unidade associados.

Se você utiliza o serviço popular de hospedagem GitHub, o *Travis CI* é uma ferramenta que permite executar testes depois de cada push ou merge, ou quando pull requests são submetidos. Embora seja lamentável que esses testes sejam feitos após um push, esse continua sendo um modo incrível de monitorar regressões. O Travis aceita prontamente todas as versões relevantes de Python e pode ser personalizado de modo significativo. Assim que tiver ativado o

Travis em seu projeto na interface web em <https://www.travis-ci.org/>, basta adicionar um arquivo `.travis.yml` que determinará como os testes serão executados. A Listagem 6.15 mostra um exemplo de um arquivo `.travis.yml`:

Listagem 6.15 – Exemplo de um arquivo `.travis.yml`

```
language: python
python:
  - "2.7"
  - "3.6"
# comando para instalar as dependências
install: "pip install -r requirements.txt --use-mirrors"
# comando para executar os testes
script: pytest
```

Com esse arquivo em seu repositório de código e o Travis ativado, esse vai iniciar um conjunto de jobs para testar o seu código com os testes de unidade associados. É fácil ver como é possível personalizar isso, simplesmente acrescentando dependências e testes. O Travis é um serviço pago, mas a boa notícia é que, para projetos de código aberto, ele é totalmente gratuito!

Também vale a pena conferir o pacote `tox-travis` (<https://pypi.python.org/pypi/tox-travis/>), pois ele melhorará a integração entre o `tox` e o Travis executando o alvo correto do `tox`, conforme o ambiente Travis em uso. A Listagem 6.16 mostra um exemplo de um arquivo `.travis.yml` que instalará o `tox-travis` antes de executar o `tox`.

Listagem 6.16 – Exemplo de um arquivo `.travis.yml` com o `tox-travis`

```
sudo: false
language: python
python:
  - "2.7"
  - "3.4"
install: pip install tox-travis
script: tox
```

Ao usar o `tox-travis`, podemos simplesmente chamar `tox` como o script no Travis, e ele chamará o `tox` com o ambiente que você especificar no arquivo `.travis.yml`, criando o ambiente virtual necessário,

instalando as dependências e executando os comandos especificados no *tox.ini*. Isso facilita usar o mesmo fluxo de trabalho, tanto em seu computador de desenvolvimento local como na plataforma Travis de integração contínua.

Hoje em dia, sempre que seu código estiver hospedado, é possível aplicar um teste automático em seu software a fim de garantir que seu projeto não fique estagnado com o acréscimo de bugs, mas continue progredindo.

Robert Collins fala sobre testes

Robert Collins é, entre outras coisas, o autor original do sistema distribuído de controle de versões *Bazaar*. Atualmente é um Distinguished Technologist na HP Cloud Services e trabalha no OpenStack. Robert também é autor de várias das ferramentas Python descritas neste livro, por exemplo as fixtures, testscenarios, testrepository, e até mesmo do python-subunit – talvez você já tenha usado um de seus programas sem saber!

Que tipo de política de testes você aconselharia que fosse usada? Há alguma situação em que seria aceitável não testar um código?

Eu acho que há uma relação de custo-benefício para os testes na engenharia: você deve considerar a probabilidade de uma falha chegar até o ambiente de produção sem que tenha sido detectada, o custo e a dimensão de uma falha não detectada e a coesão da equipe que faz o trabalho. Considere o OpenStack, que tem 1.600 colaboradores: é difícil trabalhar com uma política cheia de nuances, com tantas pessoas que possam ter suas próprias opiniões. Falando de modo geral, um projeto precisa de alguns testes automatizados para verificar se o código fará o que deve, e se fazer o que deve é aquilo que é necessário. Com frequência, isso exige testes funcionais que poderão estar em diferentes bases de código. Os testes de unidade são excelentes para agilizar e para identificar casos inusitados. Acho que é razoável variar o equilíbrio entre os estilos de testes, desde que haja testes.

Nos casos em que o custo dos testes é muito alto e os retornos são muito baixos, acho que tudo bem tomar uma decisão bem fundamentada sobre não testar, mas essa situação é relativamente rara: a maioria dos códigos pode ser testada com um custo razoavelmente baixo e, em geral, há muitas vantagens em identificar os erros com antecedência.

Ao escrever um código Python, quais são as melhores estratégias para facilitar o gerenciamento dos testes e melhorar a qualidade do código?

Separe as responsabilidades e não faça várias coisas em um só lugar; isso faz com que a reutilização seja natural e facilita muito usar dublês de testes (test doubles). Adote uma abordagem puramente funcional quando for possível; por exemplo, em um único método, calcule algo ou modifique algum estado, mas evite fazer as duas coisas. Desse modo, você poderá testar todos os comportamentos de cálculos sem ter de lidar com mudanças de estado, como escrever em um banco de dados ou conversar com um servidor HTTP. A vantagem se aplica também à situação inversa: você pode substituir a lógica de cálculo para testes a fim de provocar um comportamento associado aos casos inusitados, e utilizar simulações e dublês de teste para verificar se a propagação do estado desejado ocorre conforme desejado. A situação mais hedionda para testar são as pilhas com muitas camadas de profundidade, com dependências comportamentais complexas entre as camadas. Nesse caso, você vai querer que o código evolua de modo que o contrato entre as camadas seja simples, previsível e – o que é mais apropriado para os testes – substituível.

Qual é a melhor maneira de organizar os testes de unidade no código-fonte?

Tenha uma hierarquia clara, por exemplo, `$ROOT/$PACKAGE/tests`. Tenho a tendência de criar apenas uma hierarquia para uma árvore completa de códigos-fontes, por exemplo:

`$ROOT/$PACKAGE/$SUBPACKAGE/tests`.

Em *tests*, em geral, espelho a estrutura da parte restante da árvore de códigos-fontes: `$ROOT/$PACKAGE/foo.py` seria testado em `$ROOT/$PACKAGE/tests/test_foo.py`.

A parte restante da árvore não deve importar da árvore de testes, exceto, talvez, no caso de uma função `test_suite/load_tests` no `__init__` de nível mais alto. Isso permite que você desassocie facilmente os testes em instalações com poucos recursos.

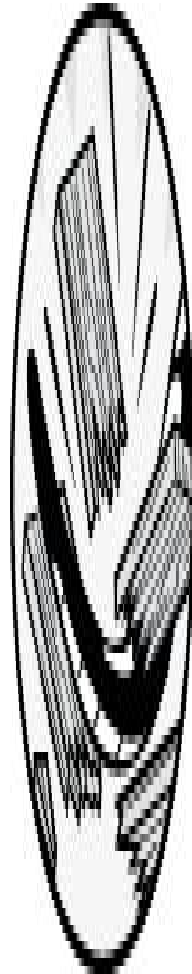
O que você vê como o futuro das bibliotecas e frameworks de testes de unidade em Python?

Os desafios significativos que vejo são:

- A expansão contínua de recursos paralelos em novos equipamentos, como telefones celulares com quatro CPUs. As APIs internas de testes de unidade atuais não estão otimizadas para cargas de trabalho em paralelo. Meu trabalho na classe `Java StreamResult` está diretamente voltado para resolver esse problema.
- Suporte mais complexo para o escalonamento – uma solução mais elegante para os problemas que as configurações com escopo de classe e de módulo tentam resolver.
- Encontrar algum modo de consolidar a enorme diversidade de frameworks que temos atualmente: para testes de integração, seria ótimo poder ter uma visão consolidada envolvendo vários projetos que tenham diferentes runners de teste em uso.

7

MÉTODOS E DECORADORES



Os decoradores de Python são um modo conveniente de modificar funções. Foram inicialmente introduzidos em Python 2.2 com os decoradores `classmethod()` e `staticmethod()`, porém foram revistos para se tornarem mais flexíveis e legíveis. Junto com esses dois decoradores originais, Python atualmente disponibiliza alguns decoradores prontos para uso imediato, além de oferecer suporte para a simples criação de decoradores personalizados. Contudo,

parece que a maioria dos desenvolvedores não entende como os decoradores funcionam internamente.

Este capítulo tem como propósito mudar isso – veremos o que é um decorador e como usá-lo, e como criar os próprios decoradores. Em seguida, veremos como usar decoradores para criar métodos estáticos, métodos de classe e métodos abstratos; conheceremos também com mais detalhes a função `super()`, que permite colocar um código implementável em um método abstrato.

Decoradores e quando usá-los

Um *decorador* é uma função que aceita outra função como argumento e a substitui por uma nova função modificada. O principal caso de uso para os decoradores é aquele em que fatoramos um código comum que deva ser chamado antes, depois ou em torno de várias funções. Se você já escreveu um código Emacs Lisp, talvez tenha usado o decorador `defadvice`, que permite definir um código chamado em torno de uma função. Se você já usou combinações de métodos em CLOS (Common Lisp Object System, ou Sistema Comum de Objetos Lisp), os decoradores Python seguem os mesmos conceitos. Veremos algumas definições de decoradores simples e, em seguida, analisaremos algumas situações comuns nas quais você utilizaria decoradores.

Criando decoradores

Há boas chances de que você já tenha usado decoradores para criar as próprias funções wrapper. O decorador mais insípido possível – e o exemplo mais simples – é a função `identity()`, que não faz nada além de devolver a função original. Eis a sua definição:

```
def identity(f):  
    return f
```

Então você usaria o seu decorador da seguinte maneira:

```
@identity  
def foo():
```

```
    return 'bar'
```

Forneça o nome do decorador precedido por um símbolo @ e, em seguida, insira a função na qual você queira usá-lo. Isso é o mesmo que escrever o código a seguir:

```
def foo():  
    return 'bar'  
foo = identity(foo)
```

Esse decorador é inútil, porém funciona. Vamos ver outro exemplo mais útil na Listagem 7.1.

Listagem 7.1 – Um decorador para organizar funções em um dicionário

```
_functions = {}  
def register(f):  
    global _functions  
    _functions[f.__name__] = f  
    return f  
@register  
def foo():  
    return 'bar'
```

Na Listagem 7.1, o decorador `register` armazena o nome da função decorada em um dicionário. O dicionário `_functions` poderá então ser usado e acessado com o nome da função a fim de obter uma função: `_functions['foo']` aponta para a função `foo()`.

Nas próximas seções, explicarei como escrever os próprios decoradores. Em seguida, discutirei como os decoradores embutidos (built-in) disponibilizados por Python funcionam, e explicarei como (e quando) usá-los.

Escrevendo decoradores

Conforme já mencionamos, os decoradores são frequentemente usados quando refatoramos um código repetido em torno das funções. Considere o seguinte conjunto de funções que devem verificar se o nome de usuário recebido como argumento é o admin ou não e, se o usuário não for o admin, uma exceção deverá ser

gerada:

```
class Store(object):
    def get_food(self, username, food):
        if username != 'admin':
            raise Exception("This user is not allowed to get food")
        return self.storage.get(food)

    def put_food(self, username, food):
        if username != 'admin':
            raise Exception("This user is not allowed to put food")
        self.storage.put(food)
```

Podemos ver que há um pouco de código repetido aí. O primeiro passo óbvio para deixar esse código mais eficiente é fatorar o código que verifica o status de admin:

```
❶ def check_is_admin(username):
    if username != 'admin':
        raise Exception("This user is not allowed to get or put food")

class Store(object):
    def get_food(self, username, food):
        check_is_admin(username)
        return self.storage.get(food)

    def put_food(self, username, food):
        check_is_admin(username)
        self.storage.put(food)
```

Passamos o código de verificação para uma função própria ❶. Agora nosso código parece estar um pouco mais organizado, mas podemos melhorá-lo se usarmos um decorador, como vemos na Listagem 7.2.

Listagem 7.2 – Acrescentando um decorador no código fatorado

```
def check_is_admin(f):
    ❶ def wrapper(*args, **kwargs):
        if kwargs.get('username') != 'admin':
            raise Exception("This user is not allowed to get or put food")
        return f(*args, **kwargs)
    return wrapper

class Store(object):
```

```
@check_is_admin
def get_food(self, username, food):
    return self.storage.get(food)
```

```
@check_is_admin
def put_food(self, username, food):
    self.storage.put(food)
```

Definimos o nosso decorador `check_is_admin` ❶ e, em seguida, nós o chamamos sempre que for necessário verificar os direitos de acesso. O decorador inspeciona os argumentos passados para a função usando a variável `kwargs` e obtém o argumento `username`, efetuando a verificação de nome do usuário antes de chamar a função propriamente dita. Usar decoradores dessa forma facilita gerenciar funcionalidades comuns. Para qualquer pessoa com bastante experiência em Python, esse truque provavelmente é antigo, mas talvez o que você não tenha percebido é que essa abordagem ingênua para implementar decoradores tem algumas desvantagens significativas.

Empilhando decoradores

Podemos também usar vários decoradores em uma única função ou método, como mostra a Listagem 7.3.

Listagem 7.3 – Usando mais de um decorador em uma única função

```
def check_user_is_not(username):
    def user_check_decorator(f):
        def wrapper(*args, **kwargs):
            if kwargs.get('username') == username:
                raise Exception("This user is not allowed to get food")
            return f(*args, **kwargs)
        return wrapper
    return user_check_decorator

class Store(object):
    @check_user_is_not("admin")
    @check_user_is_not("user123")
    def get_food(self, username, food):
        return self.storage.get(food)
```

Nesse caso, `check_user_is_not()` é uma função de factory para o nosso decorador `user_check_decorator()`. Ela cria um decorador de função que depende da variável `username`, e então devolve essa variável. A função `user_check_decorator()` servirá como um decorador de função para `get_food()`.

A função `get_food()` é decorada duas vezes com `check_user_is_not()`. A questão, nesse caso, é qual nome de usuário deve ser verificado antes: `admin` ou `user123`? A resposta está no código a seguir, no qual traduzi a Listagem 7.3 em um código equivalente, sem usar um decorador.

```
class Store(object):
    def get_food(self, username, food):
        return self.storage.get(food)

Store.get_food = check_user_is_not("user123")(Store.get_food)
Store.get_food = check_user_is_not("admin")(Store.get_food)
```

A lista de decoradores é aplicada de cima para baixo, portanto, os decoradores mais próximos à palavra reservada `def` serão aplicados antes e executados por último. No exemplo anterior, o programa verificará `admin` antes, e depois `user123`.

Escrevendo decoradores de classe

Também é possível implementar decoradores de classe, embora sejam usados com menos frequência por aí. Os *decoradores de classe* funcionam do mesmo modo que os decoradores de função, mas atuam em classes, em vez de atuar em funções. A seguir, vemos um exemplo de um decorador de classe que define atributos para duas classes:

```
import uuid

def set_class_name_and_id(klass):
    klass.name = str(klass)
    klass.random_id = uuid.uuid4()
    return klass

@set_class_name_and_id
```

```
class SomeClass(object):  
    pass
```

Quando a classe é carregada e definida, os atributos `name` e `random_id` serão definidos, assim:

```
>>> SomeClass.name  
"<class '__main__.SomeClass'>"  
>>> SomeClass.random_id  
UUID('d244dc42-f0ca-451c-9670-732dc32417cd')
```

Assim como no caso dos decoradores de função, esses decoradores podem ser muito convenientes para fatorar um código comum que manipule classes.

Outro possível uso para decoradores de classe é encapsular uma função ou classe com classes. Por exemplo, os decoradores de classe muitas vezes são usados para encapsular uma função que armazena um estado. O exemplo a seguir encapsula a função `print()` para verificar quantas vezes ela foi chamada em uma sessão:

```
class CountCalls(object):  
    def __init__(self, f):  
        self.f = f  
        self.called = 0  
  
    def __call__(self, *args, **kwargs):  
        self.called += 1  
        return self.f(*args, **kwargs)
```

```
@CountCalls  
def print_hello():  
    print("hello")
```

Então podemos usá-la para verificar quantas vezes a função `print_hello()` foi chamada:

```
>>> print_hello.called  
0  
>>> print_hello()  
hello  
>>> print_hello.called  
1
```

Obtendo os atributos originais com o decorador

update_wrapper

Conforme já mencionamos, um decorador substitui a função original por uma nova função criada durante a execução. No entanto, essa nova função não tem vários dos atributos da função original, por exemplo, sua docstring e o seu nome. A Listagem 7.4 mostra como a função `foobar()` perde a sua docstring e o seu atributo de nome assim que é decorada com `is_admin`.

Listagem 7.4 – Uma função decorada perde a sua docstring e o atributo de nome

```
>>> def is_admin(f):
...     def wrapper(*args, **kwargs):
...         if kwargs.get('username') != 'admin':
...             raise Exception("This user is not allowed to get food")
...         return f(*args, **kwargs)
...     return wrapper
...
>>> def foobar(username="someone"):
...     """Do crazy stuff."""
...     pass
...
>>> foobar.func_doc
'Do crazy stuff.'
>>> foobar.__name__
'foobar'
>>> @is_admin
... def foobar(username="someone"):
...     """Do crazy stuff."""
...     pass
...
>>> foobar.__doc__
>>> foobar.__name__
'wrapper'
```

Não ter a docstring e o atributo de nome corretos em uma função pode ser problemático em diversas situações, por exemplo, ao gerar a documentação do código-fonte.

Felizmente, o módulo `functools` da Biblioteca-Padrão de Python resolve esse problema com a função `update_wrapper()`, que copia os

atributos da função original que haviam sido perdidos para o wrapper. A Listagem 7.5 mostra o código-fonte de `update_wrapper()`.

Listagem 7.5 – Código-fonte de `update_wrapper()`

```
WRAPPER_ASSIGNMENTS = ('__module__', '__name__', '__qualname__', '__doc__',
                        '__annotations__')
WRAPPER_UPDATES = ('__dict__',)
def update_wrapper(wrapper,
                  wrapped,
                  assigned = WRAPPER_ASSIGNMENTS,
                  updated = WRAPPER_UPDATES):
    for attr in assigned:
        try:
            value = getattr(wrapped, attr)
        except AttributeError:
            pass
        else:
            setattr(wrapper, attr, value)
    for attr in updated:
        getattr(wrapper, attr).update(getattr(wrapped, attr, {}))
    # Issue #17482: set __wrapped__ last so we don't inadvertently copy it
    # from the wrapped function when updating __dict__
    wrapper.__wrapped__ = wrapped
    # Return the wrapper so this can be used as a decorator via partial()
    return wrapper
```

Na Listagem 7.5, o código-fonte de `update_wrapper()` enfatiza os atributos que valem a pena salvar ao encapsular uma função com um decorador. Por padrão, o atributo `__name__`, o atributo `__doc__` e alguns outros atributos são copiados. Também podemos personalizar quais atributos de uma função serão copiados para a função decorada. Se usarmos `update_wrapper()` para reescrever o nosso exemplo da Listagem 7.4, a situação parecerá bem melhor:

```
>>> def foobar(username="someone"):
...     """Do crazy stuff."""
...     pass
...
>>> foobar = functools.update_wrapper(is_admin, foobar)
>>> foobar.__name__
'foobar'
>>> foobar.__doc__
```

'Do crazy stuff.'

Agora a função `foobar()` tem a docstring e o nome corretos, mesmo quando é decorada com `is_admin`.

wraps: um decorador para decoradores

Pode ser enfeitado usar `update_wrapper()` manualmente ao criar decoradores, de modo que `functools` oferece um decorador para decoradores, chamado `wraps`. A Listagem 7.6 mostra o decorador `wraps` sendo usado.

Listagem 7.6 – Atualizando o nosso decorador com o wraps de functools

```
import functools

def check_is_admin(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        if kwargs.get('username') != 'admin':
            raise Exception("This user is not allowed to get food")
        return f(*args, **kwargs)
    return wrapper

class Store(object):
    @check_is_admin
    def get_food(self, username, food):
        """Get food from storage."""
        return self.storage.get(food)
```

Com `functools.wraps`, a função decoradora `check_is_admin()` que devolve a função `wrapper()` cuida de copiar a docstring, o nome da função e outras informações da função `f` passada como argumento. Assim, a função decorada (`get_food()`, nesse caso) continua vendo a sua assinatura inalterada.

Extraindo informações relevantes com inspect

Em nossos exemplos até agora, partimos do pressuposto de que a função decorada sempre terá um `username` passado para ela como um argumento nomeado, mas pode ser que isso não aconteça. Talvez haja um conjunto de informações a partir do qual devemos

extrair o nome do usuário para verificação. Com isso em mente, criaremos uma versão mais inteligente de nosso decorador, que possa verificar os argumentos da função decorada e extrair o que for necessário.

Para isso, Python tem um módulo `inspect`, que nos permite obter a assinatura de uma função e atuar nela, conforme vemos na Listagem 7.7.

Listagem 7.7 – Usando ferramentas do módulo `inspect` para extrair informações

```
import functools
import inspect

def check_is_admin(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        func_args = inspect.getcallargs(f, *args, **kwargs)
        if func_args.get('username') != 'admin':
            raise Exception("This user is not allowed to get food")
        return f(*args, **kwargs)
    return wrapper

@check_is_admin
def get_food(username, type='chocolate'):
    return type + " nom nom nom!"
```

A função que faz o trabalho pesado nesse caso é `inspect.getcallargs()`, que devolve um dicionário contendo os nomes e os valores dos argumentos na forma de pares chave-valor. Em nosso exemplo, essa função devolve `{'username': 'admin', 'type': 'chocolate'}`. Isso significa que o nosso decorador não precisa verificar se o parâmetro `username` é um argumento posicional ou um argumento nomeado; tudo que o decorador tem de fazer é procurar `username` no dicionário.

Usando `functools.wraps` e o módulo `inspect`, você poderá escrever qualquer decorador personalizado de que possa vir a precisar. No entanto, não abuse do módulo `inspect`: embora ser capaz de supor o que a função aceitará como argumento pareça ser conveniente, esse recurso pode ser frágil, falhando facilmente se as assinaturas

das funções mudarem. Os decoradores são uma ótima maneira de implementar o mantra *Don't Repeat Yourself* (Não se Repita), tão apreciado pelos desenvolvedores.

Como os métodos funcionam em Python

Os métodos são muito simples de usar e de entender, e é provável que você já os tenha usado corretamente sem ter de explorá-los muito além do necessário. Contudo, para entender o que certos decoradores fazem, você deve saber como os métodos funcionam nos bastidores.

Um *método* é uma função armazenada como um atributo de classe. Vamos ver o que acontece quando tentamos acessar um atributo como esse diretamente:

```
>>> class Pizza(object):
...     def __init__(self, size):
...         self.size = size
...     def get_size(self):
...         return self.size
...
>>> Pizza.get_size
<function Pizza.get_size at 0x7fdbfd1a8b90>
```

Somos informados de que `get_size()` é uma função – mas, por quê? O motivo é que, nessa etapa, `get_size()` não está vinculado a nenhum objeto em particular. Desse modo, ele é tratado como uma função usual. Python gerará um erro se tentarmos chamá-la diretamente, da seguinte maneira:

```
>>> Pizza.get_size()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: get_size() missing 1 required positional argument: 'self'
```

Python reclama que não fornecemos o argumento `self` necessário. De fato, como a função não está associada a nenhum objeto, o argumento `self` não pode ser definido automaticamente. Entretanto, podemos usar a função `get_size()` não só passando uma instância arbitrária da classe para o método, se quisermos, mas também

passando *qualquer* objeto, desde que ele tenha as propriedades que o método espera encontrar. Eis um exemplo:

```
>>> Pizza.get_size(Pizza(42))
42
```

Essa chamada funciona, conforme prometido. Contudo, ela não é muito conveniente: teremos de referenciar a classe sempre que quisermos chamar um de seus métodos.

Python faz o trabalho extra para nós vinculando os métodos de uma classe com suas instâncias. Em outras palavras, podemos acessar `get_size()` a partir de qualquer instância de `Pizza` e, melhor ainda, Python passará automaticamente o próprio objeto ao parâmetro `self` do método, assim:

```
>>> Pizza(42).get_size
<bound method Pizza.get_size of <__main__.Pizza object at 0x7f3138827910>>
>>> Pizza(42).get_size()
42
```

Conforme esperado, não precisamos fornecer nenhum argumento para `get_size()`, pois ele é um método vinculado (bound method): seu argumento `self` é automaticamente definido com a nossa instância de `Pizza`. Eis um exemplo mais claro ainda:

```
>>> m = Pizza(42).get_size
>>> m()
42
```

Desde que você tenha uma referência ao método vinculado, não será sequer necessário manter uma referência para o seu objeto `Pizza`. Além do mais, se você tiver uma referência para um método, mas quiser descobrir a qual objeto ele está vinculado, bastará verificar a propriedade `__self__` do método, assim:

```
>>> m = Pizza(42).get_size
>>> m.__self__
<__main__.Pizza object at 0x7f3138827910>
>>> m == m.__self__.get_size
True
```

Obviamente ainda temos uma referência para o nosso objeto, e podemos encontrá-lo se quisermos.

Métodos estáticos

Os *métodos estáticos* pertencem a uma classe, em vez de pertencerem a uma instância da classe, portanto, eles não atuam realmente em instâncias da classe e nem as afetam. Em vez disso, um método estático atua nos parâmetros que recebe. Os métodos estáticos em geral são usados para criar funções utilitárias, pois elas não dependem do estado da classe nem de seus objetos.

Por exemplo, na Listagem 7.8, o método estático `mix_ingredients()` pertence à classe `Pizza`, mas poderia ser usado para combinar os ingredientes de qualquer outro prato.

Listagem 7.8 – Criando um método estático como parte de uma classe

```
class Pizza(object):
    @staticmethod
    def mix_ingredients(x, y):
        return x + y

    def cook(self):
        return self.mix_ingredients(self.cheese, self.vegetables)
```

Você poderia escrever `mix_ingredients()` como um método não estático se quisesse, mas ele receberia um argumento `self` que jamais seria usado. Usar o decorador `@staticmethod` nos proporciona diversas vantagens.

A primeira é a velocidade: Python não precisará instanciar um método vinculado para cada objeto `Pizza` que criarmos. Métodos vinculados são objetos também, e criá-los tem um custo de CPU e de memória – ainda que seja baixo. Usar um método estático nos permite evitar isso, da seguinte maneira:

```
>>> Pizza().cook is Pizza().cook
False
>>> Pizza().mix_ingredients is Pizza.mix_ingredients
True
>>> Pizza().mix_ingredients is Pizza().mix_ingredients
True
```

Em segundo lugar, os métodos estáticos melhoram a legibilidade do código. Quando vemos `@staticmethod`, sabemos que o método não depende do estado do objeto.

A terceira vantagem é que os métodos estáticos podem ser sobrescritos nas subclasses. Se, em vez de um método estático, tivéssemos usado uma função `mix_ingredients()` definida no nível mais alto de nosso módulo, uma classe que herdasse de `Pizza` não seria capaz de modificar o modo de combinar os ingredientes de nossa pizza sem sobrescrever o próprio método `cook()`. Com métodos estáticos, as subclasses podem sobrescrever o método de acordo com seus próprios propósitos.

Infelizmente, Python nem sempre é capaz de detectar por conta própria se um método é estático ou não – chamo a isso de um defeito de design da linguagem. Uma possível abordagem é acrescentar uma verificação que detecte um padrão como esse e gere um aviso usando o `flake8`. Veremos como fazer isso na seção “Estendendo o `flake8` com verificações na AST” na página 172.

Métodos de classe

Os *métodos de classe* estão vinculados a uma classe, em vez de estarem vinculados às suas instâncias. Isso significa que esses métodos não podem acessar o estado do objeto, mas somente o estado e os métodos da classe. A Listagem 7.9 mostra como escrever um método de classe.

Listagem 7.9 – Vinculando um método de classe à sua classe

```
>>> class Pizza(object):
...     radius = 42
...     @classmethod
...     def get_radius(cls):
...         return cls.radius
...
>>> Pizza.get_radius
<bound method type.get_radius of <class '__main__.Pizza'>>
>>> Pizza().get_radius
```



```
<bound method type.get_radius of <class '__main__.Pizza'>>
>>> Pizza.get_radius is Pizza().get_radius
True
>>> Pizza.get_radius()
42
```

Como podemos ver, há diversas maneiras de acessar o método de classe `get_radius()`, mas, independentemente de como você decidir acessá-lo, o método estará sempre vinculado à classe à qual ele está associado. Além disso, seu primeiro argumento deve ser a própria classe. Lembre-se: as classes também são objetos!

Os métodos de classe são usados principalmente para criar *métodos de factory*, que instanciam objetos usando uma assinatura diferente de `__init__`:

```
class Pizza(object):
    def __init__(self, ingredients):
        self.ingredients = ingredients

    @classmethod
    def from_fridge(cls, fridge):
        return cls(fridge.get_cheese() + fridge.get_vegetables())
```

Se tivéssemos usado `@staticmethod` no lugar de `@classmethod` nesse caso, teríamos de deixar o nome da classe `Pizza` fixo em nosso método, fazendo com que qualquer classe que herdasse de `Pizza` fosse incapaz de usar a nossa factory com finalidades próprias. Nesse exemplo, porém, fornecemos um método de factory `from_fridge()` para o qual podemos passar um objeto `Fridge`. Se chamarmos esse método com algo como `Pizza.from_fridge(myfridge)`, ele devolverá uma nova `Pizza` com ingredientes que estejam disponíveis em `myfridge`.

Sempre que você escrever um método que se importe somente com a classe do objeto, e não com o seu estado, ele deverá ser declarado como um método de classe.

Métodos abstratos

Um *método abstrato* é definido em uma classe-base abstrata que, por si

só, pode não prover nenhuma implementação. Se uma classe tem um método abstrato, ela não pode ser instanciada. Como consequência, uma *classe abstrata* (definida como uma classe que tenha pelo menos um método abstrato) deve ser usada como uma classe-pai de outra classe. Essa subclasse será responsável por implementar o método abstrato, possibilitando instanciar a classe-pai.

Podemos usar classes-base abstratas para deixar claro os relacionamentos entre outras classes conectadas, derivadas da classe-base, mas deixando a própria classe-base abstrata impossível de instanciar. Ao usar classes-base abstratas, você pode garantir que as classes derivadas da classe-base implementarão métodos específicos da classe-base, ou uma exceção será lançada. O exemplo a seguir mostra o modo mais simples de escrever um método abstrato em Python:

```
class Pizza(object):
    @staticmethod
    def get_radius():
        raise NotImplementedError
```

Com essa definição, qualquer classe que herde de `Pizza` deve implementar e sobrescrever o método `get_radius()`; caso contrário, chamar o método fará a exceção exibida no exemplo ser lançada. Isso é conveniente para garantir que cada subclasse de `Pizza` implemente a sua própria maneira de calcular e devolver seu raio.

Esse modo de implementar métodos abstratos tem uma desvantagem: se você escrever uma classe que herde de `Pizza`, mas esquecer de implementar `get_radius()`, o erro será gerado somente se você tentar usar esse método durante a execução. Eis um exemplo:

```
>>> Pizza()
<__main__.Pizza object at 0x7fb747353d90>
>>> Pizza().get_radius()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in get_radius
NotImplementedError
```

Como `Pizza` pode ser diretamente instanciada, não há maneiras de impedir que isso aconteça. Um modo de garantir que você verá logo um aviso caso se esqueça de implementar e sobrescrever o método, ou tente instanciar um objeto com métodos abstratos, é usar o módulo embutido `abc` (abstract base classes, ou classes-base abstratas) de Python, da seguinte maneira:

```
import abc

class BasePizza(object, metaclass=abc.ABCMeta):

    @abc.abstractmethod
    def get_radius(self):
        """Method that should do something."""
```

O módulo `abc` disponibiliza um conjunto de decoradores a serem usados nos métodos que serão definidos como abstratos, e uma metaclasses que permite fazer isso. Ao usar `abc` e seu metaclass especial, conforme mostramos no código anterior, instanciar uma `BasePizza` ou uma classe que herde dela mas não sobrescrever `get_radius()` fará um `TypeError` ser gerado:

```
>>> BasePizza()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class BasePizza with abstract methods get_radius
```

Tentamos instanciar a classe abstrata `BasePizza`, mas fomos prontamente informados de que isso não pode ser feito!

Embora o uso de métodos abstratos não garanta que o método vá ser implementado pelo usuário, esse decorador ajudará você a identificar o erro mais cedo. Isso será particularmente conveniente se você disponibiliza interfaces que devam ser implementadas por outros desenvolvedores; é uma boa dica para documentação.

Combinando métodos estáticos, métodos de classe e métodos abstratos

Cada um desses decoradores é útil por si só, mas pode haver

ocasiões em que você terá de usá-los em conjunto.

Por exemplo, você poderia definir uma função de factory como um método de classe, ao mesmo tempo que obriga que a implementação seja feita em uma subclasse. Nesse caso, você teria de ter um método de classe definido como um método tanto abstrato como de classe. Esta seção apresenta algumas dicas que ajudarão você nessa situação.

Em primeiro lugar, o protótipo de um método abstrato não está definido a ferro e fogo. Ao implementar o método, nada impede que você estenda a lista de argumentos conforme achar apropriado. A Listagem 7.10 mostra um exemplo de código no qual uma subclasse estende a assinatura do método abstrato de sua classe-pai.

Listagem 7.10 – Usando uma subclasse para estender a assinatura do método abstrato de sua classe-pai.

```
import abc

class BasePizza(object, metaclass=abc.ABCMeta):

    @abc.abstractmethod
    def get_ingredients(self):
        """Returns the ingredient list."""

class Calzone(BasePizza):
    def get_ingredients(self, with_egg=False):
        egg = Egg() if with_egg else None
        return self.ingredients + [egg]
```

Definimos a subclasse `Calzone` de modo que herde da classe `BasePizza`. Podemos definir os métodos da subclasse `Calzone` do modo que quisermos, desde que eles aceitem a interface que definimos em `BasePizza`. Isso inclui implementar os métodos, sejam como métodos de classe ou como métodos estáticos. O código a seguir define um método abstrato `get_ingredients()` na classe-base e um método estático `get_ingredients()` na subclasse `DietPizza`:

```
import abc

class BasePizza(object, metaclass=abc.ABCMeta):
```

```
@abc.abstractmethod
def get_ingredients(self):
    """Returns the ingredient list."""
```

```
class DietPizza(BasePizza):
    @staticmethod
    def get_ingredients():
        return None
```

Mesmo que nosso método estático `get_ingredients()` não devolva um resultado com base no estado do objeto, ele aceita a interface de nossa classe abstrata `BasePizza`, portanto, continua sendo válido.

Também é possível usar os decoradores `@staticmethod` e `@classmethod` junto com `@abstractmethod` a fim de sinalizar que um método é, por exemplo, tanto estático como abstrato, conforme mostra a Listagem 7.11.

Listagem 7.11 – Usando um decorador de método de classe com métodos abstratos

```
import abc

class BasePizza(object, metaclass=abc.ABCMeta):

    ingredients = ['cheese']

    @classmethod
    @abc.abstractmethod
    def get_ingredients(cls):
        """Returns the ingredient list."""
        return cls.ingredients
```

O método abstrato `get_ingredients()` deve ser implementado por uma subclasse, mas é também um método de classe, o que significa que o primeiro argumento que ele receberá será uma classe (e não um objeto).

Observe que, ao definir `get_ingredients()` como um método de classe em `BasePizza` dessa forma, você não obriga nenhuma subclasse a definir `get_ingredients()` como um método de classe – esse poderia ser um método comum. O mesmo se aplicaria caso tivéssemos definido

esse método como um método estático: não há nenhuma maneira de forçar as subclasses a implementar métodos abstratos como um tipo específico de método. Conforme vimos, é possível modificar a assinatura de um método abstrato ao implementá-lo em uma subclasse, do modo que você quiser.

Colocando implementações em métodos abstratos

Espere um pouco: na Listagem 7.12, temos uma implementação *em* um método abstrato. *Podemos* fazer isso? A resposta é sim. Python não tem nenhum problema com isso! Você pode colocar um código em seus métodos abstratos e chamá-los com `super()`, conforme vemos na Listagem 7.12.

Listagem 7.12 – Usando uma implementação em um método abstrato

```
import abc

class BasePizza(object, metaclass=abc.ABCMeta):

    default_ingredients = ['cheese']

    @classmethod
    @abc.abstractmethod
    def get_ingredients(cls):
        """Returns the default ingredient list."""
        return cls.default_ingredients

class DietPizza(BasePizza):
    def get_ingredients(self):
        return [Egg()] + super(DietPizza, self).get_ingredients()
```

Nesse exemplo, toda `Pizza` que você criar, que herde de `BasePizza`, deve sobrescrever o método `get_ingredients()`, mas toda `Pizza` também terá acesso ao aparato default da classe-base para obter a lista de ingredientes. Esse aparato é particularmente conveniente para especificar uma interface a ser implementada, ao mesmo tempo que oferece um código de base que possa ser útil a todas as classes

que herdem dessa classe.

A verdade sobre o super

Python sempre permitiu que seu desenvolvedores usassem herança tanto simples como múltipla para estender suas classes; entretanto, mesmo atualmente, muitos desenvolvedores parecem não entender como esses sistemas, e o método `super()` associado, funcionam. Para compreender totalmente o seu código, você deve compreender suas vantagens e desvantagens.

As heranças múltiplas são usadas em vários lugares, particularmente em códigos que envolvam um padrão mixin. Uma *mixin* é uma classe que herda de duas ou mais classes diferentes, combinando seus recursos.

NOTA Muitos dos prós e contras das heranças simples e múltipla, da composição, ou até mesmo do duck typing (tipagem pato) estão fora do escopo deste livro, portanto, não discutiremos tudo. Se você não tem familiaridade com essas noções, sugiro que leia sobre elas a fim de compor suas próprias opiniões.

Como você já deve saber a essa altura, as classes são objetos em Python. A construção usada para criar uma classe é composta de uma instrução especial com a qual você já deve ter bastante familiaridade: *class nomedaclassa(expressão de herança)*.

O código entre parênteses é uma expressão Python que devolve a lista dos objetos classe a serem usados como os pais da classe. Em geral, você os especificaria diretamente, mas também seria possível escrever algo como o que vemos a seguir a fim de especificar a lista dos objetos pais:

```
>>> def parent():
...     return object
...
>>> class A(parent()):
...     pass
...
>>> A.mro()
[<class '__main__.A'>, <type 'object'>]
```

Esse código funciona como esperado: declaramos a classe A com

object como sua classe-pai. O método de classe `mro()` devolve a *ordem de resolução dos métodos* (method resolution order) usada para resolver atributos – ela define como o próximo método a ser chamado é encontrado na árvore de herança entre classes. O sistema de MRO atual foi originalmente implementado em Python 2.3, e seu funcionamento interno está descrito nas notas de lançamento (release notes) de Python 2.3. Ele define como o sistema navega pela árvore de herança entre classes para encontrar o método a ser chamado.

Já vimos que o modo canônico de chamar um método de uma classe-pai é por meio da função `super()`, mas o que você provavelmente não sabe é que `super()`, na verdade, é um construtor, e um objeto `super` é instanciado sempre que você chama essa função. Ela aceita um ou dois argumentos: o primeiro argumento é uma classe e o segundo é um argumento opcional que pode ser uma subclasse ou uma instância do primeiro argumento.

O objeto devolvido pelo construtor funciona como um proxy para as classes-pai do primeiro argumento. Ele tem seu próprio método `__getattr__` que itera pelas classes da lista MRO e devolve o primeiro atributo correspondente encontrado. O método `__getattr__` é chamado quando um atributo do objeto `super()` é acessado, conforme mostra a Listagem 7.13.

Listagem 7.13 – A função `super()` é um construtor que instancia um objeto `super`

```
>>> class A(object):
...     bar = 42
...     def foo(self):
...         pass
...
>>> class B(object):
...     bar = 0
...
>>> class C(A, B):
...     xyz = 'abc'
...
```



```

>>> C.mro()
[<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <type 'object'>]
>>> super(C, C()).bar
42
>>> super(C, C()).foo
<bound method C.foo of <__main__.C object at 0x7f0299255a90>>
>>> super(B).__self__
>>> super(B, B()).__self__
<__main__.B object at 0x1096717f0>

```

Ao requisitar um atributo do objeto `super` de uma instância de `c`, o método `__getattr__` do objeto `super()` percorre a lista MRO e devolve o atributo da primeira classe que encontrar, a qual tenha o atributo `super`.

Na Listagem 7.13, chamamos `super()` com dois argumentos, o que significa que usamos um objeto `super` *vinculado* (bound). Se chamarmos `super()` com apenas um argumento, ele devolverá um objeto `super` *não vinculado* (unbound).

```

>>> super(C)
<super: <class 'C'>, NULL>

```

Como nenhuma instância foi especificada como segundo argumento, o objeto `super` não pode ser vinculado a nenhuma instância. Portanto, você não poderá usar esse objeto não vinculado para acessar atributos da classe. Se tentar fazer isso, você verá os seguintes erros:

```

>>> super(C).foo
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'super' object has no attribute 'foo'
>>> super(C).bar
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'super' object has no attribute 'bar'
>>> super(C).xyz
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'super' object has no attribute 'xyz'

```

À primeira vista, pode parecer que esse tipo de objeto `super` não

vinculado é inútil, mas, na verdade, o modo como a classe `super` implementa o protocolo de descritor `__get__` torna os objetos `super` não vinculados muito úteis como atributos de classe:

```
>>> class D(C):
...     sup = super(C)
...
>>> D().sup
<super: <class 'C'>, <D object>>
>>> D().sup.foo
<bound method D.foo of <__main__.D object at 0x7f0299255bd0>>
>>> D().sup.bar
42
```

O método `__get__` do objeto `super` não vinculado é chamado com a instância `super(C).__get__(D())` e o nome do atributo `'foo'` como argumentos, permitindo que `foo` seja encontrado e resolvido .

NOTA Mesmo que você jamais tenha ouvido falar do protocolo de descritor, é provável que já o tenha usado por meio do decorador `@property`, sem sabê-lo. O *protocolo de descritor* é o sistema de Python que permite que um objeto armazenado como um atributo devolva algo diferente de si mesmo. Esse protocolo não será discutido neste livro, mas você pode obter mais informações sobre ele na documentação do modelo de dados de Python.

Há inúmeras situações em que usar `super()` pode ser complicado, por exemplo, quando lidamos com diferentes assinaturas de métodos na cadeia de herança. Infelizmente, não há uma solução única para todas as ocasiões. A melhor prevenção é usar truques como fazer com que todos os seus métodos aceitem argumentos usando `*args`, `**kwargs`.

A partir de Python 3, `super()` passou a incluir uma pitada de magia: ele agora pode ser chamado de dentro de um método sem qualquer argumento. Se nenhum argumento for passado para `super()`, ele pesquisará automaticamente o stack frame em busca dos argumentos:

```
class B(A):
    def foo(self):
        super().foo()
```

A forma padrão de acessar atributos-pais nas subclasses é com

`super()`, e você deve usá-lo sempre. Ele permite chamadas colaborativas de métodos-pais sem que haja surpresas, como os métodos-pai não serem chamados ou serem chamados duas vezes quando heranças múltiplas forem usadas.

Resumo

De posse do que aprendemos neste capítulo, você será imbatível quanto a tudo que diz respeito à definição de métodos em Python. Os decoradores são essenciais quando se trata de fatoração de código, e o uso apropriado dos decoradores embutidos disponibilizados por Python pode melhorar bastante a clareza de seu código Python. As classes abstratas são particularmente úteis quando disponibilizamos uma API para outros desenvolvedores e serviços.

A herança de classes muitas vezes não é totalmente compreendida, e ter uma visão geral do aparato interno da linguagem é uma boa maneira de apreender por completo o seu modo de funcionamento. A partir de agora, não restará a você mais nenhum segredo sobre esse assunto!

8

PROGRAMAÇÃO FUNCIONAL



Muitos desenvolvedores Python não sabem até que ponto a programação funcional pode ser usada em Python, o que é lamentável: com poucas exceções, a programação funcional permite escrever um código mais conciso e mais eficiente. Além disso, o suporte de Python à programação funcional é bem amplo.

Este capítulo discutirá alguns dos aspectos da programação funcional em Python, incluindo a criação e o uso de geradores.

Conheceremos os pacotes funcionais e as funções mais úteis que estão à sua disposição, e como usá-los em conjunto para deixar o código mais eficiente.

NOTA Se você realmente quer levar a programação funcional a sério, eis o meu conselho: dê um tempo em Python e aprenda uma linguagem que seja extremamente voltada à programação funcional, por exemplo, Lisp. Sei que pode parecer estranho falar de Lisp em um livro sobre Python, mas usar Lisp por muitos anos me ensinou a “pensar de modo funcional”. Você talvez não desenvolva os processos de raciocínio necessários para fazer um uso completo da programação funcional se toda a sua experiência vier da programação imperativa e orientado a objetos. O Lisp, por si só, não é puramente funcional, mas tem mais foco na programação funcional em comparação com o que você verá em Python.

Criando funções puras

Ao escrever código em um estilo funcional, suas funções são projetadas para não terem efeitos colaterais: elas recebem uma entrada e geram uma saída sem manter estados nem modificar nada que não esteja refletido no valor de retorno. As funções que obedecem a esse ideal são conhecidas como *puramente funcionais*.

Vamos começar com um exemplo de uma função comum, não pura, que remove o último item de uma lista:

```
def remove_last_item(mylist):  
    """Removes the last item from a list."""  
    mylist.pop(-1) # Modifica mylist
```

O código a seguir é uma versão pura da mesma função:

```
def butlast(mylist):  
  
    return mylist[:-1] # Devolve uma cópia de mylist
```

Definimos uma função `butlast()` para que funcione como `butlast` em Lisp, pois ela devolve a lista sem o último elemento, *sem* modificar a lista original. Ela devolve uma cópia da lista contendo as modificações, permitindo que a lista original seja preservada.

As vantagens práticas da programação funcional incluem:

- **Modularidade** Escrever em um estilo funcional força você a prover um certo grau de isolamento para resolver seus

problemas individuais, criando seções de código mais fáceis de reutilizar em outros contextos. Como a função não depende de nenhuma variável externa ou estado, chamá-la a partir de uma porção de código diferente é simples.

- **Concisão** A programação funcional, com frequência, é menos verbosa do que outros paradigmas.
- **Concorrência** Funções puramente funcionais são seguras para thread (thread-safe) e podem executar de forma concorrente. Algumas linguagens funcionais fazem isso automaticamente, o que pode ajudar bastante caso seja necessário escalar sua aplicação algum dia, embora esse ainda não seja exatamente o caso de Python.
- **Testabilidade** Testar um programa funcional é extremamente fácil: basta ter um conjunto de entradas e um conjunto de saídas esperadas. Elas são *idempotentes*, isto é, chamar a mesma função repetidamente com os mesmos argumentos sempre devolverá o mesmo resultado.

Geradores

Um *gerador* é um objeto que se comporta como um iterador, pois ele gera e devolve um valor a cada chamada de seu método `next()`, até que um `StopIteration` seja gerado. Os geradores, introduzidos na PEP 255, oferecem um modo fácil de criar objetos que implementam o *protocolo iterador*. Embora escrever geradores em um estilo funcional não seja estritamente necessário, fazer isso faz com que eles sejam mais fáceis de escrever e de depurar, e é uma prática comum.

Para criar um gerador, basta escrever uma função Python comum contendo uma instrução `yield`. Python detectará o uso de `yield` e marcará a função como um gerador. Quando a execução alcançar a instrução `yield`, a função devolverá um valor, como em uma instrução `return`, porém com uma diferença relevante: o interpretador salvará uma referência à pilha, e ela será usada para retomar a execução da função quando a função `next()` for chamada novamente.

Quando as funções são executadas, o encadeamento de sua execução gera uma *pilha* (stack) – dizemos que as chamadas de função são empilhadas. Quando uma função retorna, ela é removida da pilha e o valor que ela devolve é passado para a função que fez a chamada. No caso de um gerador, a função não retorna realmente, mas executa um *yield*, isto é, cede. Desse modo, Python salva o estado da função como uma referência na pilha, retomando a execução do gerador no ponto em que foi salvo, quando a próxima iteração do gerador for necessária.

Criando um gerador

Conforme já mencionamos, um gerador é criado ao escrever uma função comum e incluir um *yield* no corpo da função. A Listagem 8.1 cria um gerador chamado `mygenerator()` que inclui três *yields*, o que significa que haverá uma iteração com as três próximas chamadas a `next()`.

Listagem 8.1 – Criando um gerador com três iterações

```
>> def mygenerator():
...     yield 1
...     yield 2
...     yield 'a'
...
>>> mygenerator()
<generator object mygenerator at 0x10d77fa50>
>>> g = mygenerator()
>>> next(g)
1
>>> next(g)
2
>>> next(g)
'a'
>>> next(g)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

Quando acabarem as instruções *yield*, `StopIteration` será gerado na

próxima chamada a `next()`.

Em Python, os geradores mantêm uma referência à pilha quando uma função executa um `yield`, e essa pilha é restaurada quando uma chamada a `next()` é executada novamente.

A abordagem ingênua ao iterar por qualquer dado sem o uso de geradores consiste em criar a lista completa antes; com frequência, isso consome muita memória, sem que haja necessidade.

Suponha que você queira encontrar o primeiro número entre 1 e 10.000.000 e que seja igual a 50.000. Parece fácil, não é mesmo? Vamos fazer disso um desafio. Executaremos Python com uma restrição de memória de 128MB e testaremos a abordagem ingênua de criar inicialmente a lista completa:

```
$ ulimit -v 131072
$ python3
>>> a = list(range(10000000))
```

Esse método ingênuo inicialmente tenta criar a lista; contudo, se executarmos o programa do modo como está agora, veremos o seguinte:

```
Traceback (most recent call last): File "<stdin>", line 1, in <module> MemoryError
```

Opa. O fato é que não podemos criar uma lista com 10 milhões de itens usando apenas 128MB de memória!

AVISO Em Python 3, `range()` devolve um gerador em uma iteração. Para obter um gerador em Python 2, você deve usar `xrange()`. Essa função não existe mais em Python 3, pois é redundante.

Vamos tentar usar um gerador em seu lugar, com a mesma restrição de 128MB:

```
$ ulimit -v 131072
$ python3
>>> for value in range(10000000):
...     if value == 50000:
...         print("Found it")
...         break
...
Found it
```

Dessa vez, nosso programa executou sem problemas. Quando há

uma iteração, a classe `range()` devolve um gerador que gera dinamicamente a nossa lista de inteiros. Melhor ainda, como estamos interessados somente no 50.000º número, em vez de criar a lista completa, o gerador teve de gerar apenas 50.000 números até parar.

Ao gerar valores durante a execução, os geradores permitem que você lide com grandes conjuntos de dados com um consumo de memória e ciclos de processamento mínimos. Sempre que houver necessidade de trabalhar com muitos valores, os geradores poderão ajudar você a lidar com eles de modo mais eficiente.

Devolvendo e passando valores com `yield`

Uma instrução `yield` também tem um recurso menos usado de modo geral: ela pode devolver um valor, do mesmo modo que uma chamada de função. Isso nos permite passar um valor para um gerador chamando o seu método `send()`. Como exemplo de uso de `send()`, escreveremos uma função chamada `shorten()` que aceita uma lista de strings e devolve uma lista composta dessas mesma strings, porém truncadas (Listagem 8.2).

Listagem 8.2 – Retornando e usando um valor com `send()`

```
def shorten(string_list):
    length = len(string_list[0])
    for s in string_list:
        length = yield s[:length]

mystringlist = ['loremipsum', 'dolorsit', 'ametfoobar']
shortstringlist = shorten(mystringlist)
result = []
try:
    s = next(shortstringlist)
    result.append(s)
    while True:
        number_of_vowels = len(filter(lambda letter: letter in 'aeiou', s))
        # Trunca a próxima string dependendo
        # do número de vogais da string anterior
        s = shortstringlist.send(number_of_vowels)
```

```
        result.append(s)
    except StopIteration:
        pass
```

Nesse exemplo, escrevemos uma função chamada `shorten()` que aceita uma lista de strings e devolve uma lista composta dessas mesmas strings, porém truncadas. O tamanho de cada string truncada é igual ao número de vogais da string anterior: *loremipsum* tem quatro vogais, portanto, o segundo valor devolvido pelo gerador serão as quatro primeiras letras de *dolorsit*; *dolo* tem apenas duas vogais, portanto, *ametfoobar* será truncada e terá apenas suas duas primeiras letras, isto é, *am*. O gerador então para e gera `StopIteration`. Desse modo, o nosso gerador devolve o seguinte:

```
['loremipsum', 'dolo', 'am']
```

Usar `yield` e `send()` dessa forma permite que os geradores Python funcionem como as *corrotinas* que vemos em Lua e em outras linguagens.

A PEP 289 introduziu as expressões geradoras, possibilitando criar geradores com uma só linha usando uma sintaxe parecida com a de list comprehension:

```
>>> (x.upper() for x in ['hello', 'world'])
<generator object <genexpr> at 0x7ffab3832fa0>
>>> gen = (x.upper() for x in ['hello', 'world'])
>>> list(gen)
['HELLO', 'WORLD']
```

Nesse exemplo, `gen` é um gerador, e é como se tivéssemos usado a instrução `yield`. O `yield`, nesse caso, está implícito.

Inspecionando geradores

Para determinar se uma função é considerada um gerador, utilize `inspect.isgeneratorfunction()`. Na Listagem 8.3, criamos e inspecionamos um gerador simples.

Listagem 8.3 - Verificando se uma função é um gerador

```
>>> import inspect
>>> def mygenerator():
```

```

...     yield 1
...
>>> inspect.isgeneratorfunction(mygenerator)
True
>>> inspect.isgeneratorfunction(sum)
False

```

Importe o pacote `inspect` para usar `isgeneratorfunction()`; então basta lhe passar o nome da função a ser inspecionada. Ler o código-fonte de `inspect.isgeneratorfunction()` nos proporciona alguns insights acerca de como Python marca funções como sendo geradoras (veja a Listagem 8.4).

Listagem 8.4 – Código-fonte de `inspect.isgeneratorfunction()`

```

def isgeneratorfunction(object):
    """Return true if the object is a user-defined generator function.

    Generator function objects provides same attributes as functions.

    See help(isfunction) for attributes listing."""

    return bool((isFunction(object) or ismethod(object)) and
                object.func_code.co_flags & CO_GENERATOR)

```

A função `isgeneratorfunction()` verifica se o objeto é uma função ou um método, e se seu código tem a flag `CO_GENERATOR` definida. Esse exemplo mostra como é fácil entender de que modo Python funciona internamente.

O pacote `inspect` disponibiliza a função `inspect.getgeneratorstate()`, que informa o estado atual do gerador. Nós a usaremos em `mygenerator()` a seguir, em diferentes pontos da execução:

```

>>> import inspect
>>> def mygenerator():
...     yield 1
...
>>> gen = mygenerator()
>>> gen
<generator object mygenerator at 0x7f94b44fec30>
>>> inspect.getgeneratorstate(gen)
❶ 'GEN_CREATED'
>>> next(gen)

```

```

1
>>> inspect.getgeneratorstate(gen)
❷ 'GEN_SUSPENDED'
>>> next(gen)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>> inspect.getgeneratorstate(gen)
❸ 'GEN_CLOSED'

```

Essa função nos permite determinar se o gerador está esperando para ser executado pela primeira vez (GEN_CREATED) ❶, esperando para ser retomado em virtude de uma chamada a `next()` (GEN_SUSPENDED) ❷ ou se terminou de executar (GEN_CLOSED) ❸. Esse recurso pode ser conveniente para depurar seus geradores.

List comprehensions

Uma list comprehension (abrangência de lista), ou *listcomp* na forma abreviada, permite definir o conteúdo de uma lista inline com a sua declaração. Para transformar uma lista em uma listcomp, você deve colocá-la entre colchetes como sempre, mas deve incluir também uma expressão que vai gerar os itens da lista, e um laço `for` para percorrê-los.

O exemplo a seguir cria uma lista sem o uso de uma list comprehension:

```

>>> x = []
>>> for i in (1, 2, 3):
...     x.append(i)
...
>>> x
[1, 2, 3]

```

O próximo exemplo utiliza uma list comprehension para criar a mesma lista em uma única linha:

```

>>> x = [i for i in (1, 2, 3)]
>>> x
[1, 2, 3]

```

Usar uma list comprehension tem duas vantagens: um código

escrito com listcomps em geral é mais compacto e, desse modo, será compilado com menos operações para Python executar. Em vez de criar uma lista e chamar `append` repetidamente, Python pode simplesmente criar a lista de itens e movê-los para uma nova lista em uma única operação.

Podemos usar várias instruções `for` juntas e utilizar instruções `if` para filtrar os itens. No exemplo a seguir, criamos uma lista de palavras e usamos uma list comprehension para fazer com que cada item comece com uma letra maiúscula, separamos os itens compostos de várias palavras em palavras simples e removemos o `or` irrelevante:

```
x = [word.capitalize()
      for line in ("hello world?", "world!", "or not")
      for word in line.split()
      if not word.startswith("or")]
>>> x
['Hello', 'World?', 'World!', 'Not']
```

Esse código tem dois laços `for`: o primeiro itera pelas linhas de texto, enquanto o segundo itera pelas palavras em cada uma dessas linhas. A última instrução `if` filtra as palavras que começam com `or` de modo a excluí-las da lista final.

Usar uma list comprehension em vez de laços `for` é uma boa maneira de definir listas rapidamente. Como ainda estamos falando de programação funcional, vale a pena observar que as listas criadas usando list comprehensions não devem fazer mudanças no estado do programa: não é esperado que você modifique nenhuma variável quando criar a lista. Em geral, isso deixa a lista mais concisa e mais fácil de ler se comparada às listas criadas sem listcomp.

Observe que há também uma sintaxe para criar dicionários ou conjuntos do mesmo modo, assim:

```
>>> {x:x.upper() for x in ['hello', 'world']}
{'world': 'WORLD', 'hello': 'HELLO'}
>>> {x.upper() for x in ['hello', 'world']}
set(['WORLD', 'HELLO'])
```

Funções funcionais em ação

Você poderia deparar repetidamente com o mesmo conjunto de problemas quando estiver manipulando dados com a programação funcional. Para ajudá-lo a lidar com essa situação de modo eficaz, Python inclui uma série de funções para programação funcional. Esta seção apresenta uma visão geral rápida de algumas dessas funções embutidas que permitem criar programas totalmente funcionais. Assim que tiver uma ideia acerca do que está à sua disposição, incentivo você a pesquisar mais e a experimentar usar essas funções nos locais em que elas possam ser aplicadas em seu próprio código.

Aplicando funções a itens usando `map()`

A função `map()` assume a forma `map(function, iterable)`, e aplica `function` a cada item de `iterable` de modo a devolver uma lista em Python 2 ou um objeto `map` iterável em Python 3, como mostra a Listagem 8.5.

Listagem 8.5 – Usando `map()` em Python 3

```
>>> map(lambda x: x + "bzz!", ["I think", "I'm good"])
<map object at 0x7fe7101abdd0>
>>> list(map(lambda x: x + "bzz!", ["I think", "I'm good"]))
['I thinkbzz!', 'I'm goodbzz!']
```

Poderíamos escrever um equivalente a `map()` usando uma `list comprehension`, assim:

```
>>> (x + "bzz!" for x in ["I think", "I'm good"])
<generator object <genexpr> at 0x7f9a0d697dc0>
>>> [x + "bzz!" for x in ["I think", "I'm good"]]
['I thinkbzz!', 'I'm goodbzz!']
```

Filtrando listas com `filter()`

A função `filter()` assume a forma `filter(function ou None, iterable)` e filtra os itens em `iterable` com base no resultado devolvido por `function`. Uma lista será devolvida em Python 2, ou um objeto `filter` iterável em Python 3:

```
>>> filter(lambda x: x.startswith("I "), ["I think", "I'm good"])
<filter object at 0x7f9a0d636dd0>
>>> list(filter(lambda x: x.startswith("I "), ["I think", "I'm good"]))
['I think']
```

Também poderíamos escrever um equivalente a `filter()` usando uma list comprehension, assim:

```
>>> (x for x in ["I think", "I'm good"] if x.startswith("I "))
<generator object <genexpr> at 0x7f9a0d697dc0>
>>> [x for x in ["I think", "I'm good"] if x.startswith("I ")]
['I think']
```

Obtendo índices com `enumerate()`

A função `enumerate()` assume a forma `enumerate(iterable[, start])` e devolve um objeto iterável que fornece uma sequência de tuplas, cada uma composta de um índice inteiro (começando com `start`, se for especificado) e o item correspondente em `iterable`. Essa função será útil quando você tiver de escrever um código que referencie índices de arrays. Por exemplo, em vez de escrever:

```
i = 0
while i < len(mylist):
    print("Item %d: %s" % (i, mylist[i]))
    i += 1
```

you poderia fazer o mesmo de modo mais eficiente com `enumerate()`, da seguinte maneira:

```
for i, item in enumerate(mylist):
    print("Item %d: %s" % (i, item))
```

Ordenando uma lista com `sorted()`

A função `sorted()` assume a forma `sorted(iterable, key=None, reverse=False)` e devolve uma versão ordenada de `iterable`. O argumento `key` permite especificar uma função que devolve o valor com base no qual será feita a ordenação, conforme vemos a seguir:

```
>>> sorted([("a", 2), ("c", 1), ("d", 4)])
[('a', 2), ('c', 1), ('d', 4)]
>>> sorted([("a", 2), ("c", 1), ("d", 4)], key=lambda x: x[1])
[('c', 1), ('a', 2), ('d', 4)]
```

Encontrando itens que satisfaçam condições com `any()` e `all()`

As funções `any(iterable)` e `all(iterable)` devolvem um booleano, dependendo dos valores devolvidos por `iterable`. Essas funções simples são equivalentes ao código Python completo a seguir:

```
def all(iterable):
    for x in iterable:
        if not x:
            return False
    return True
```

```
def any(iterable):
    for x in iterable:
        if x:
            return True
    return False
```

Essas funções são úteis para verificar se algum valor ou se todos os valores em um iterável satisfazem a uma dada condição. Por exemplo, o código a seguir verifica uma lista de acordo com duas condições:

```
mylist = [0, 1, 3, -1]
if all(map(lambda x: x > 0, mylist)):
    print("All items are greater than 0")
if any(map(lambda x: x > 0, mylist)):
    print("At least one item is greater than 0")
```

A diferença, nesse caso, é que `any()` devolve `True` se houver pelo menos um elemento que atenda à condição, enquanto `all()` devolverá `True` somente se todos os elementos atenderem à condição. A função `all()` também devolverá `True` para um iterável vazio, pois nenhum dos elementos será `False`.

Combinado listas com `zip()`

A função `zip()` assume a forma `zip(iter1 [,iter2 [...]])`. Ela aceita várias sequências e as combina em tuplas. É útil quando precisamos combinar uma lista de chaves e uma lista de valores em um

dicionário. Assim como no caso das demais funções descritas, `zip()` devolve uma lista em Python 2 e um iterável em Python 3. No exemplo a seguir, mapeamos uma lista de chaves a uma lista de valores a fim de criar um dicionário:

```
>>> keys = ["foobar", "barzz", "ba!"]
>>> map(len, keys)
<map object at 0x7fc1686100d0>
>>> zip(keys, map(len, keys))
<zip object at 0x7fc16860d440>
>>> list(zip(keys, map(len, keys)))
[('foobar', 6), ('barzz', 5), ('ba!', 3)]
>>> dict(zip(keys, map(len, keys)))
{'foobar': 6, 'barzz': 5, 'ba!': 3}
```

FUNÇÕES FUNCIONAIS EM PYTHON 2 E 3

A essa altura, talvez você já tenha notado que os tipos devolvidos diferem entre Python 2 e Python 3. A maioria das funções embutidas de Python que são puramente funcionais devolvem uma lista em vez de um iterável em Python 2, tornando-as menos eficientes quanto à memória em comparação com seus equivalentes em Python 3.x. Se você planeja escrever algum código usando essas funções, saiba que elas serão mais bem aproveitadas em Python 3. Se você estiver preso a Python 2, não se desespere: o módulo `itertools` da Biblioteca-Padrão disponibiliza uma versão baseada em iterador para muitas dessas funções (`itertools.izip()`, `itertools.imap()`, `itertools.ifilter()` e assim por diante).

Um problema comum resolvido

Ainda resta uma ferramenta importante a ser discutida. Com frequência, quando trabalhamos com listas, queremos encontrar o primeiro item que satisfaça a uma condição específica. Conheceremos várias maneiras de fazer isso e, em seguida, veremos o modo mais eficiente: o pacote `first`.

Encontrando o item com um código simples

Podemos encontrar o primeiro item que satisfaça a uma condição usando uma função como esta:

```
def first_positive_number(numbers):
    for n in numbers:
        if n > 0:
            return n
```

Poderíamos reescrever a função `first_positive_number()` em um estilo funcional, assim:

```
def first(predicate, items):
    for item in items:
        if predicate(item):
            return item
```

```
first(lambda x: x > 0, [-1, 0, 1, 2])
```

Ao usar uma abordagem funcional, na qual o predicado é passado como argumento, a função passa a ser facilmente reutilizável. Poderíamos inclusive escrevê-la de modo mais conciso, da seguinte maneira:

```
# Menos eficiente
list(filter(lambda x: x > 0, [-1, 0, 1, 2]))[0]
# Eficiente
next(filter(lambda x: x > 0, [-1, 0, 1, 2]))
```

Observe que esse código pode gerar um `IndexError` se nenhum item satisfizer à condição, fazendo `list(filter())` devolver uma lista vazia.

Em casos simples, podemos contar com `next()` para evitar que um `IndexError` ocorra, assim:

```
>>> a = range(10)
>>> next(x for x in a if x > 3)
4
```

A Listagem 8.6 gerará um `StopIteration` se uma condição não puder ser satisfeita. Isso também pode ser resolvido com o acréscimo de um segundo argumento para `next()`, da seguinte maneira:

Listagem 8.6 – Devolvendo um valor default quando a condição não é atendida

```
>>> a = range(10)
>>> next((x for x in a if x > 10), 'default')
'default'
```

Esse código devolverá um valor default em vez de um erro se uma condição não puder ser atendida. Felizmente para nós, Python disponibiliza um pacote para cuidar disso tudo.

Encontrando o item usando first()

Em vez de escrever a função da Listagem 8.6 em todos os seus programas, você pode incluir o pequeno pacote Python `first`. A Listagem 8.7 mostra como esse pacote permite que você encontre o primeiro elemento de um iterável que atenda a uma condição.

Listagem 8.7 – Encontrando o primeiro item de uma lista que satisfaça a uma condição

```
>>> from first import first
>>> first([0, False, None, [], (), 42])
42
>>> first([-1, 0, 1, 2])
-1
>>> first([-1, 0, 1, 2], key=lambda x: x > 0)
1
```

Podemos ver que a função `first()` devolve o primeiro item válido e não vazio de uma lista.

Usando lambda() com functools

Você deve ter percebido que usamos `lambda()` em boa parte dos exemplos deste capítulo até agora. A função `lambda()` foi acrescentada em Python para facilitar funções de programação funcional como `map()` e `filter()`, as quais, do contrário, teriam exigido a escrita de uma função totalmente nova sempre que você quisesse verificar uma condição diferente. A Listagem 8.8 é equivalente à Listagem 8.7, porém foi escrita sem o uso de `lambda()`.

Listagem 8.8 – Encontrando o primeiro item que atenda à condição, sem o uso de lambda()

```
import operator
from first import first

def greater_than_zero(number):
    return number > 0

first([-1, 0, 1, 2], key=greater_than_zero)
```

Esse código funciona de modo idêntico ao código da Listagem 8.7,

devolvendo o primeiro valor não vazio de uma lista, o qual atenda à condição, mas é bem mais desajeitado: se quiséssemos obter o primeiro número da sequência que fosse maior que, digamos, 42 itens, teríamos de definir uma função apropriada usando `def`, em vez de defini-la inline com a nossa chamada a `first()`.

No entanto, apesar de sua utilidade em nos ajudar a evitar situações como essa, `lambda` ainda tem seus problemas. O módulo `first` contém um argumento `key` que pode ser usado para especificar uma função que receberá cada item como um argumento e devolverá um booleano informando se ele satisfaz à condição. Entretanto, não podemos passar uma função `key`, pois ela exigiria mais de uma única linha de código: uma instrução `lambda` não pode ser escrita em mais de uma linha. Essa é uma limitação significativa de `lambda`.

Em vez disso, teríamos de voltar ao padrão desajeitado de escrever novas definições de funções para cada `key` de que precisássemos. Ou não?

O pacote `functools` vem para nos salvar com o seu método `partial()`, que nos oferece uma alternativa mais flexível ao `lambda`. O método `functools.partial()` nos permite criar uma função wrapper com uma diferença: em vez de modificar o comportamento de uma função, ela modifica os *argumentos* que recebe, assim:

```
from functools import partial
from first import first
```

```
❶ def greater_than(number, min=0):
    return number > min
```

```
❷ first([-1, 0, 1, 2], key=partial(greater_than, min=42))
```

Nesse exemplo, criamos uma nova função `greater_than()` que funciona exatamente como a função `greater_than_zero()` anterior da Listagem 8.8 por padrão, porém essa versão nos permite especificar o valor com o qual queremos comparar nossos números, enquanto antes esse valor estava fixo no código. No exemplo, passamos `functools.partial()` para a nossa função e o valor que queremos para `min` ❶, e

obtivemos de volta uma nova função com `min` definido com 42, exatamente como queríamos ❷. Em outras palavras, podemos escrever uma função e usar `functools.partial()` para personalizar o comportamento de nossas novas funções a fim de atender às nossas necessidades em qualquer dada situação.

Mesmo essa versão pode ser mais reduzida ainda. Tudo que estamos fazendo nesse exemplo é comparar dois números, e o fato é que o módulo `operator` tem funções embutidas para fazer exatamente isso:

```
import operator
from functools import partial
from first import first
```

```
first([-1, 0, 1, 2], key=partial(operator.le, 0))
```

Esse é um bom exemplo de `functools.partial()` trabalhando com argumentos posicionais. Nesse caso, a função `operator.le(a, b)`, que aceita dois números e devolve um booleano para nos informar se o primeiro número é menor ou igual ao segundo, é passada para `functools.partial()`. O valor 0 que passamos para `functools.partial()` é atribuído a `a`, e o argumento passado para a função devolvida por `functools.partial()` é atribuído a `b`. Portanto, isso funciona do mesmo modo que o código da Listagem 8.8, mas sem o uso de `lambda` e sem definir nenhuma outra função adicional.

NOTA O método *`functools.partial()`* em geral é conveniente para ser usado no lugar de *`lambda`*, e deve ser considerado como uma alternativa superior. A função *`lambda`* é, de certo modo, um tipo de anomalia na linguagem Python, e considerou-se descartá-la totalmente em Python 3 em virtude do tamanho do corpo da função limitado em uma única linha.

Funções úteis de `itertools`

Por fim, veremos algumas funções úteis do módulo `itertools` da Biblioteca-Padrão de Python, as quais você deveria conhecer. Muitos programadores acabam escrevendo suas próprias versões dessas funções simplesmente porque não sabem que Python as têm prontas. Todas essas funções foram projetadas para ajudar

você a manipular um `iterator` (é por isso que o módulo se chama *iter-tools*) e, portanto, são todas puramente funcionais. Listarei a seguir algumas delas e apresentarei uma breve descrição geral do que fazem, e incentivo você a conhecê-las melhor caso pareçam ser úteis.

- `accumulate(iterable[, func])` devolve uma série de somas acumuladas dos itens dos iteráveis.
- `chain(*iterables)` itera por vários iteráveis, um após o outro, sem criar uma lista intermediária com todos os itens.
- `combinations(iterable, r)` gera todas as combinações de tamanho *r* a partir do dado `iterable`.
- `compress(data, selectors)` aplica uma máscara booleana de *selectors* em *data* e devolve somente os valores de *data* para os quais o elemento correspondente de *selectors* seja `True`.
- `count(start, step)` gera uma sequência infinita de valores, começando com *start* e fazendo um incremento de *step* a cada chamada.
- `cycle(iterable)` percorre repetidamente os valores em *iterable*.
- `repeat(elem[, n])` repete um elemento *n* vezes.
- `dropwhile(predicate, iterable)` filtra elementos de um iterável desde o início, até *predicate* ser `False`.
- `groupby(iterable, keyfunc)` cria um iterador que agrupa itens de acordo com o resultado devolvido pela função `keyfunc()`.
- `permutations(iterable[, r])` devolve sucessivas permutações de tamanho *r* dos itens em *iterable*.
- `product(*iterables)` devolve um iterável do produto cartesiano de *iterables* sem usar um laço `for` aninhado.
- `takewhile(predicate, iterable)` devolve elementos de um iterável desde o início, até *predicate* ser `False`.

Essas funções são particularmente úteis se combinadas com o módulo `operator`. Quando usados em conjunto, `itertools` e `operator` conseguem lidar com a maioria das situações nas quais os programadores, em geral, contam com uma `lambda`. Eis um exemplo

de uso de `operator.itemgetter()` em vez de escrever `lambda x: x['foo']`:

```
>>> import itertools
>>> a = [{'foo': 'bar'}, {'foo': 'bar', 'x': 42}, {'foo': 'baz', 'y': 43}]
>>> import operator
>>> list(itertools.groupby(a, operator.itemgetter('foo')))
[('bar', <itertools._grouper object at 0xb000d0>), ('baz', <itertools._grouper object at
0xb00110>)]
>>> [(key, list(group)) for key, group in itertools.groupby(a,
operator.itemgetter('foo'))]
[('bar', [{'foo': 'bar'}, {'x': 42, 'foo': 'bar'}]), ('baz', [{'y': 43, 'foo': 'baz'}])]
```

Nesse caso, também poderíamos ter escrito `lambda x: x['foo']`, porém usar `operator` nos permite evitar totalmente o uso de `lambda`.

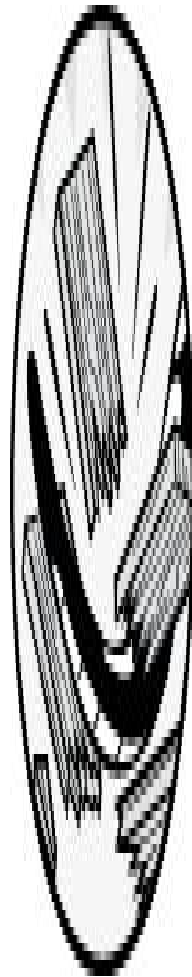
Resumo

Embora, com frequência, Python seja anunciado como orientado a objetos, ele pode ser usado de modo bastante funcional. Muitos de seus conceitos embutidos, como geradores e list comprehensions, são funcionalmente orientados, e não entram em conflito com uma abordagem orientada a objetos. Além disso, para o seu próprio bem, eles também limitam a dependência ao estado global de um programa.

Usar a programação funcional como um paradigma em Python pode ajudar você a deixar seu programa mais reutilizável e mais fácil de ser testado e depurado, oferecendo suporte para o mantra DRY (Don't Repeat Yourself, ou Não se Repita). Nesse espírito, os módulos Python padrões `itertools` e `operator` são ferramentas muito boas para melhorar a legibilidade de seu código funcional.

9

ÁRVORE SINTÁTICA ABSTRATA, HY E ATRIBUTOS DO TIPO LISP



A *AST* (Abstract Syntax Tree, ou Árvore Sintática Abstrata) é uma representação da estrutura do código-fonte de toda linguagem de programação. Qualquer linguagem, incluindo Python, tem uma *AST* específica; a *AST* de Python é construída como resultado do parsing de um arquivo-fonte Python. Como qualquer árvore, esta é

composta de nós ligados entre si. Um nó pode representar uma operação, uma instrução, uma expressão ou até mesmo um módulo. Cada nó pode conter referências a outros nós que compõem a árvore.

A AST de Python não é extremamente documentada e, desse modo, é difícil lidar com ela em um primeiro momento; contudo, compreender alguns dos aspectos mais profundos acerca de como Python é construído pode ajudar você a dominar o seu uso.

Este capítulo analisará a AST de alguns comandos Python simples para que você conheça bem a sua estrutura e saiba como ela é usada. Assim que você tiver familiaridade com a AST, criaremos um programa capaz de verificar se há métodos declarados incorretamente usando o `flake8` e a AST. Por fim, conheceremos o Hy: uma linguagem Python-Lisp híbrida, criada com base na AST de Python.

Observando a AST

O modo mais fácil de visualizar a AST de Python é fazer o parse de um código Python e exibir o dump da AST gerada. Para isso, o módulo `ast` de Python oferece tudo que é necessário, conforme mostra a Listagem 9.1.

Listagem 9.1 – Usando o módulo `ast` para fazer dump da AST gerada com o parse do código

```
>>> import ast
>>> ast.parse
<function parse at 0x7f062731d950>
>>> ast.parse("x = 42")
<_ast.Module object at 0x7f0628a5ad10>
>>> ast.dump(ast.parse("x = 42"))
"Module(body=[Assign(targets=[Name(id='x', ctx=Store())], value=Num(n=42))])"
```

A função `ast.parse()` faz parse de qualquer string que contenha código Python e devolve um objeto `_ast.Module`. Esse objeto é, na verdade, a raiz da árvore: é possível navegar por ela a fim de descobrir cada

um dos nós que compõe a árvore. Para visualizar a árvore, a função `ast.dump()` pode ser usada, a qual devolverá uma representação em string da árvore completa.

Na Listagem 9.1, o parse do código `x = 42` é feito com `ast.parse()` e o resultado é exibido com `ast.dump()`. Essa árvore sintática abstrata pode ser representada como mostra a Figura 9.1, que exhibe a estrutura do comando `assign` de Python.

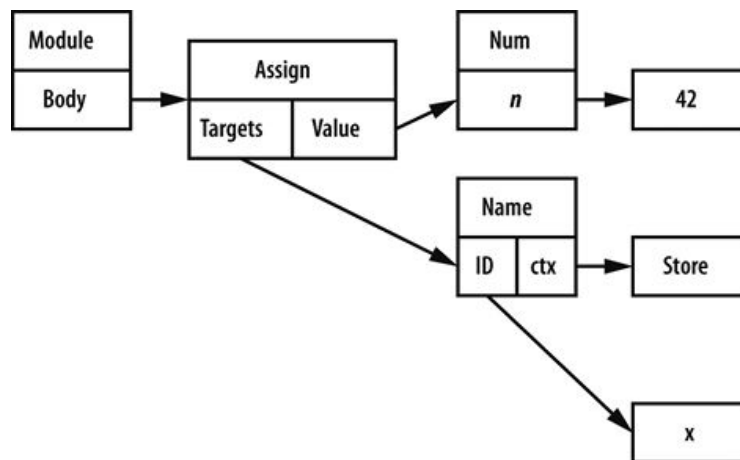


Figura 9.1 – A AST do comando assign de Python.

A AST sempre começa com um elemento raiz, o qual, em geral, é um objeto `_ast.Module`. Esse objeto módulo contém uma lista de instruções ou expressões a serem avaliadas em seu atributo *corpo* (body) e, em geral, representa o conteúdo de um arquivo.

Como você provavelmente deve adivinhar, o objeto `ast.Assign` exibido na Figura 9.1 representa uma *atribuição* (assignment), que é mapeada para o sinal `=` na sintaxe de Python. Um objeto `ast.Assign` tem uma lista de *alvos* (targets) e um *valor* (value) com o qual os alvos serão definidos. A lista de alvos, nesse caso, é composta de um objeto, `ast.Name`, que representa uma variável cujo ID é `x`. O valor é um número `n` com um valor igual a 42 (nesse caso). O atributo `ctx` armazena um *contexto* (context) – pode ser `ast.Store` OU `ast.Load` – conforme a variável seja usada para leitura ou para escrita. Nesse caso, a variável está recebendo um valor, portanto, um contexto `ast.Store` foi usado.

Poderíamos passar essa AST para Python para que pudesse ser compilada e avaliada, usando a função embutida `compile()`. Essa função aceita uma AST como argumento, o nome do arquivo-fonte e um modo (pode ser `'exec'`, `'eval'` ou `'single'`). O arquivo-fonte pode ter o nome que você quiser, do qual sua AST pareça ter se originado; é comum usar a string `<input>` como o nome do arquivo-fonte se os dados não vierem de um arquivo armazenado, como vemos na Listagem 9.2.

Listagem 9.2 – Usando a função `compile()` para compilar dados que não são provenientes de um arquivo armazenado

```
>>> compile(ast.parse("x = 42"), '<input>', 'exec')
<code object <module> at 0x111b3b0, file "<input>", line 1>
>>> eval(compile(ast.parse("x = 42"), '<input>', 'exec'))
>>> x
42
```

Os modos significam: executar (`exec`), avaliar (`eval`) e instrução única (`single`). O modo deve corresponder àquilo que for passado para `ast.parse()`, e o default é `exec`.

- O modo `exec` é o modo usual de Python, utilizado quando um `_ast.Module` é a raiz da árvore.
- O modo `eval` é um modo especial, que espera uma única `ast.Expression` como a árvore.
- Por fim, `single` é outro modo especial, que espera uma única instrução ou expressão. Se uma expressão for recebida, `sys.displayhook()` será chamado com o resultado, como ocorre quando o código é executado no shell interativo.

A raiz da AST é `ast.Interactive`, e seu atributo `body` será uma lista de nós. Poderíamos construir uma AST manualmente usando as classes disponibilizadas pelo módulo `ast`. Obviamente, esse é um longo caminho para escrever um código Python, e não é um método que eu recomendaria! Apesar disso, é divertido e ajuda a conhecer melhor a AST. Vamos ver como seria programar com a AST.

Escrevendo um programa usando a AST

Vamos escrever um bom e velho programa "Hello world!" em Python, criando manualmente uma árvore sintática abstrata.

Listagem 9.3 – Escrevendo hello world! usando a AST

```
❶ >>> hello_world = ast.Str(s='hello world!', lineno=1, col_offset=1)
❷ >>> print_name = ast.Name(id='print', ctx=ast.Load(), lineno=1, col_offset=1)
❸ >>> print_call = ast.Call(func=print_name, ctx=ast.Load(),
... args=[hello_world], keywords=[], lineno=1, col_offset=1)
❹ >>> module = ast.Module(body=[ast.Expr(print_call,
... lineno=1, col_offset=1)], lineno=1, col_offset=1)
❺ >>> code = compile(module, "", 'exec')
>>> eval(code)
hello world!
```

Na Listagem 9.3, construímos a árvore, uma folha de cada vez, e cada folha é um elemento (seja ele um valor ou uma instrução) do programa.

A primeira folha é uma string simples ❶: `ast.Str` representa uma string literal, a qual, nesse caso, contém o texto `hello world!`. A variável `print_name` ❷ contém um objeto `ast.Name`, que se refere a uma variável – nesse caso, é a variável `print`, que aponta para a função `print()`.

A variável `print_call` ❸ contém uma chamada de função. Ela referencia o nome da função a ser chamada, os argumentos usuais a serem passados para a chamada da função e os argumentos nomeados. Os argumentos a serem usados dependem das funções chamadas. Nesse caso, como é a função `print()`, passaremos a string que criamos e armazenamos em `hello_world`.

Por fim, criamos um objeto `_ast.Module` ❹ para que contenha todo esse código na forma de uma lista com uma expressão. Podemos compilar objetos `_ast.Module` usando a função `compile()` ❺, que faz o parse da árvore e gera um objeto `code` nativo. Esses objetos `code` são códigos Python compilados e podem finalmente ser executados por uma máquina virtual Python usando `eval`!

Esse processo todo é exatamente o que acontece quando

executamos Python em um arquivo `.py`: assim que o parse dos tokens de texto é feito, esses são convertidos em uma árvore de objetos `ast`, são compilados e avaliados.

NOTA Os argumentos *lineno* e *col_offset* representam o número da linha e o offset da coluna, respectivamente, do código-fonte usado para gerar a AST. Não faz muito sentido definir esses valores neste contexto, pois não estamos fazendo parse de um arquivo-fonte, mas encontrar a posição do código que gerou a AST pode ser conveniente. Por exemplo, Python utiliza essas informações ao gerar backtraces. Na verdade, Python se recusa a compilar um objeto AST que não forneça essas informações e, desse modo, passamos valores fictícios em seu lugar. Também poderíamos usar a função `ast.fix_missing_locations()` para definir os valores ausentes com aqueles definidos no nó pai.

Objetos AST

Podemos visualizar a lista completa dos objetos disponíveis na AST lendo a documentação do módulo `_ast` (observe o underscore).

Os objetos são organizados em duas categorias principais: instruções e expressões. As *instruções* (statements) incluem tipos como `assert`, atribuição (`=`), atribuição composta (`+=`, `/=` etc.), `global`, `def`, `if`, `return`, `for`, `class`, `pass`, `import`, `raise` e assim por diante. As instruções herdam de `ast.stmt`; elas influenciam o controle de fluxo de um programa e, em geral, são compostas de expressões.

As *expressões* (expressions) incluem tipos como `lambda`, `number`, `yield`, `name` (variável), `compare` e `call`. As expressões herdam de `ast.expr`; elas diferem das instruções porque, em geral, geram um valor e não causam impacto no fluxo do programa.

Há também algumas categorias menores, por exemplo, a classe `ast.operator`, que define operadores padrões como *soma* (`+`), *divisão* (`/`) e *deslocamento à direita* (`>>`), além do módulo `ast.cmpop`, que define operadores de comparação.

Esse exemplo simples deve dar a você uma ideia de como construir uma AST do zero. É fácil então imaginar como seria possível tirar proveito dessa AST para construir um compilador que fizesse parse de strings e gerasse código, permitindo que você implementasse a sua própria sintaxe de Python! É exatamente isso que levou ao

desenvolvimento do projeto Hy, que será discutido mais adiante neste capítulo.

Percorrendo uma AST

Para acompanhar o modo como uma árvore é criada ou para acessar nós específicos, você terá ocasionalmente de percorrer a árvore, navegando por ela e iterando pelos nós. Isso pode ser feito com a função `ast.walk()`. Como alternativa, o módulo `ast` também disponibiliza a classe `NodeTransformer`, a partir da qual você pode criar uma subclasse para percorrer uma AST e modificar nós específicos. Usar `NodeTransformer` facilita modificar o código de modo dinâmico, conforme vemos na Listagem 9.4.

Listagem 9.4 – Percorrendo uma árvore com NodeTransformer para alterar um nó

```
import ast

class ReplaceBinOp(ast.NodeTransformer):
    """Replace operation by addition in binary operation"""
    def visit_BinOp(self, node):
        return ast.BinOp(left=node.left,
                          op=ast.Add(),
                          right=node.right)

❶ tree = ast.parse("x = 1/3")
   ast.fix_missing_locations(tree)
   eval(compile(tree, "", 'exec'))
   print(ast.dump(tree))
❷ print(x)

❸ tree = ReplaceBinOp().visit(tree)
   ast.fix_missing_locations(tree)
   print(ast.dump(tree))
   eval(compile(tree, "", 'exec'))
❹ print(x)
```

O primeiro objeto `tree` criado ❶ é uma AST que representa a expressão $x = 1/3$. Assim que ele é compilado e avaliado, o resultado da exibição de `x` no final da função ❷ é 0.33333, que é o resultado

esperado de 1/3.

O segundo objeto `tree` ❸ é uma instância de `ReplaceBinOp`, que herda de `ast.NodeTransformer`. Ela implementa a sua própria versão do método `ast.NodeTransformer.visit()` e altera qualquer operação `ast.BinOp` para um `ast.BinOp` que executa `ast.Add`. De forma concreta, isso modifica qualquer operador binário (+, -, / e assim por diante) para o operador +. Quando essa segunda árvore é compilada e avaliada ❹, o resultado agora é 4, que é o resultado de 1 + 3, pois a / no primeiro objeto é substituída por +.

Podemos ver, a seguir, a execução do programa:

```
Module(body=[Assign(targets=[Name(id='x', ctx=Store())],
                                value=BinOp(left=Num(n=1), op=Div(), right=Num(n=3)))]),
0.3333333333333333
Module(body=[Assign(targets=[Name(id='x', ctx=Store())],
                                value=BinOp(left=Num(n=1), op=Add(), right=Num(n=3)))]),
4
```

NOTA Se for necessário avaliar uma string que deva devolver um tipo de dado simples, `ast.literal_eval` poderá ser usado. É uma alternativa mais segura a `eval`, e ela impede que a string de entrada execute um código.

Estendendo o flake8 com verificações na AST

No Capítulo 7, vimos que os métodos que não dependem do estado do objeto devem ser declarados como estáticos com o decorador `@staticmethod`. O problema é que muitos desenvolvedores simplesmente se esquecem de fazer isso. Pessoalmente, gasto bastante tempo revisando códigos e pedindo às pessoas que corrijam esse problema.

Já vimos como usar o `flake8` para fazer algumas verificações automáticas no código. Com efeito, o `flake8` é extensível e permite fazer outras verificações. Escreveremos uma extensão para o `flake8` que verifica omissões na declaração de métodos estáticos por meio da análise da AST.

A Listagem 9.5 mostra um exemplo de uma classe que omite a

declaração de estático e outra que a inclui corretamente. Escreva o programa a seguir e salve-o como `ast_ext.py`; em breve, nós o usaremos para escrever a nossa extensão.

Listagem 9.5 – Omitindo e incluindo `@staticmethod`

```
class Bad(object):
    # self não é usado; o método não precisa
    # ser vinculado (bound) e deveria ser declarado como static
    def foo(self, a, b, c):
        return a + b - c

class OK(object):
    # Está correto
    @staticmethod
    def foo(a, b, c):
        return a + b - c
```

Embora o método `Bad.foo` funcione bem, estritamente falando, é mais correto escrevê-lo como `OK.foo` (volte para o Capítulo 7 para ver mais detalhes sobre o motivo). Para verificar se todos os métodos em um arquivo Python estão corretamente declarados, devemos fazer o seguinte:

- Iterar por todos os nós de instruções da AST.
- Verificar se a instrução é uma definição de classe (`ast.ClassDef`).
- Iterar por todas as definições de função (`ast.FunctionDef`) dessa instrução de classe a fim de verificar se já estão declaradas com `@staticmethod`.
- Se o método não estiver declarado como estático, verificar se o primeiro argumento (`self`) é usado em algum lugar no método. Se `self` não for usado, o método poderá ser marcado como possivelmente escrito de forma indevida.

O nome de nosso projeto será `ast_ext`. Para registrar um novo plugin no `flake8`, devemos criar um projeto que inclua os arquivos `setup.py` e `setup.cfg` usuais. Em seguida, bastará apenas acrescentar um ponto de entrada no `setup.cfg` de nosso projeto `ast_ext`.

Listagem 9. 6– Permitindo plugins para o `flake8` em nosso capítulo


```
[entry_points]
flake8.extension =
    --trecho omitido--
    H904 = ast_ext:StaticmethodChecker
    H905 = ast_ext:StaticmethodChecker
```

Na Listagem 9.6, também registramos dois códigos de erro no `flake8`. Como você verá mais adiante, na verdade, aproveitaremos para acrescentar uma verificação extra em nosso código!

O próximo passo será implementar o plugin.

Escrevendo a classe

Como estamos escrevendo uma verificação do `flake8` para a AST, o plugin deverá ser uma classe que obedeça a uma determinada assinatura, como mostra a Listagem 9.7.

Listagem 9.7 – Classe para verificação da AST

```
class StaticmethodChecker(object):
    def __init__(self, tree, filename):
        self.tree = tree

    def run(self):
        pass
```

O template default é fácil de entender: ele armazena a árvore localmente para ser usada no método `run()`, que fará um *yield* dos problemas descobertos. O valor que será informado deve obedecer à assinatura esperada pela PEP 8: uma tupla no formato *(lineno, col_offset, error_string, code)*.

Ignorando códigos irrelevantes

Conforme mencionamos antes, o módulo `ast` disponibiliza a função `walk()`, que permite iterar facilmente por uma árvore. Usaremos essa função para percorrer a AST e determinar o que deve e o que não deve ser verificado.

Inicialmente, vamos escrever um laço que ignore as instruções que não sejam definições de classe. Acrescente esse código em seu

projeto `ast_ext`, conforme mostra a Listagem 9.8; o código que deve permanecer inalterado será exibido em cinza.

Listagem 9.8 – Ignorando instruções que não sejam definições de classe

```
class StaticmethodChecker(object):
    def __init__(self, tree, filename):
        self.tree = tree

    def run(self):
        for stmt in ast.walk(self.tree):
            # Ignora o que não é classe
            if not isinstance(stmt, ast.ClassDef):
                continue
```

O código da Listagem 9.8 ainda não está verificando nada, mas agora ele sabe como ignorar instruções que não sejam definições de classe. O próximo passo é fazer com que o nosso verificador ignore tudo que não seja uma definição de função.

Listagem 9.9 – Ignorando instruções que não sejam definições de função

```
for stmt in ast.walk(self.tree):
    # Ignora o que não é classe
    if not isinstance(stmt, ast.ClassDef):
        continue
    # Se for uma classe, itera pelos membros do corpo para encontrar os métodos
    for body_item in stmt.body:
        # Não é um método: ignora
        if not isinstance(body_item, ast.FunctionDef):
            continue
```

Na Listagem 9.9, ignoramos as instruções irrelevantes iterando pelos atributos da definição da classe.

Verificando se há um decorador apropriado

Estamos prontos para implementar a verificação do método, que está armazenado no atributo `body_item`. Em primeiro lugar, devemos conferir se o método sendo verificado já está declarado como estático. Se estiver, não haverá necessidade de fazer outras

verificações e poderemos sair.

Listagem 9.10 – Verificando se há um decorador de método estático

```
for stmt in ast.walk(self.tree):
    # Ignora o que não é classe
    if not isinstance(stmt, ast.ClassDef):
        continue
    # Se for uma classe, itera pelos membros do corpo para encontrar os métodos
    for body_item in stmt.body:
        # Não é um método: ignora
        if not isinstance(body_item, ast.FunctionDef):
            continue
        # Verifica se já tem um decorador
        for decorator in body_item.decorator_list:
            if (isinstance(decorator, ast.Name)
                and decorator.id == 'staticmethod'):
                # É uma função estática; está correta
                break
        else:
            # A função não é estática; por enquanto, não fazemos nada
            Pass
```

Observe que, na Listagem 9.10, utilizamos a forma especial de Python `for/else`, na qual `else` será avaliado a menos que usemos `break` para sair do laço `for`. A essa altura, somos capazes de detectar se um método está declarado como estático.

Procurando o `self`

O próximo passo é verificar se o método que *não* está declarado como estático utiliza o argumento `self`. Inicialmente, verifique se o método inclui algum argumento, conforme vemos na Listagem 9.11.

Listagem 9.11 – Verifica se há argumentos no método

```
--trecho omitido--
# Verifica se já tem um decorador
for decorator in body_item.decorator_list:
    if (isinstance(decorator, ast.Name)
        and decorator.id == 'staticmethod'):
        # É uma função estática; está correta
        break
```

```

else:
    try:
        first_arg = body_item.args.args[0]
    except IndexError:
        yield (
            body_item.lineno,
            body_item.col_offset,
            "H905: method misses first argument",
            "H905",
        )
    # Verifica o próximo método
    Continue

```

Finalmente adicionamos uma verificação! Essa instrução `try` na Listagem 9.11 toma o primeiro argumento da assinatura do método. Se o código não conseguir obter o primeiro argumento da assinatura porque esse não existe, já saberemos que há um problema: não podemos ter um método vinculado sem o argumento `self`. Se o plugin detectar esse caso, ele gerará o código de erro H905 que definimos antes, informando que é um método que não tem o primeiro argumento.

NOTA Um código PEP 8 segue um formato específico para os códigos de erro (uma letra seguida de um número), mas não há regras para definir o código escolhido. Você poderia ter escolhido qualquer outro código para esse erro, desde que ainda não tenha sido usado pela PEP 8 nem por outra extensão.

Agora você sabe por que registramos dois códigos de erro em *setup.cfg*: tínhamos uma boa oportunidade para matar dois coelhos com um só cajadada.

O próximo passo será verificar se o argumento `self` é usado no código do método.

Listagem 9.12 – Verifica se há o argumento `self` no método

--trecho omitido--

```

try:
    first_arg = body_item.args.args[0]
except IndexError:
    yield (
        body_item.lineno,
        body_item.col_offset,

```

```

        "H905: method misses first argument",
        "H905",
    )
    # Verifica o próximo método
    continue
for func_stmt in ast.walk(body_item):
    # Método de verificação deve ser diferente para Python 2 e Python 3
    if six.PY3:
        if (isinstance(func_stmt, ast.Name)
            and first_arg.arg == func_stmt.id):
            # O primeiro argumento é usado, está OK
            break
    else:
        if (func_stmt != first_arg
            and isinstance(func_stmt, ast.Name)
            and func_stmt.id == first_arg.id):
            # O primeiro argumento é usado, está OK
            break
    else:
        yield (
            body_item.lineno,
            body_item.col_offset,
            "H904: method should be declared static",
            "H904",
        )

```

Para verificar se o argumento `self` é usado no corpo do método, o plugin na Listagem 9.12 itera recursivamente, usando `ast.walk` no corpo e verificando se a variável chamada `self` é usada. Se a variável não for encontrada, o programa finalmente fará o `yield` do código de erro H904. Caso contrário, nada acontecerá e o código será considerado correto.

NOTA Como você talvez tenha percebido, o código percorre a definição do módulo AST várias vezes. Pode haver algum grau de otimização se navegarmos pela AST passando apenas uma vez por ela, mas não tenho certeza se vale a pena fazer isso, considerando o modo como a ferramenta é realmente usada. Deixarei isso como um exercício para você, meu caro leitor.

Conhecer a AST de Python não é estritamente necessário para usar Python, mas você terá insights importantes acerca de como a linguagem é construída e como ela funciona. Desse modo, compreenderá melhor de que modo o código que você escreve é

usado internamente.

Uma introdução rápida ao Hy

Agora que temos uma boa compreensão sobre como a AST de Python funciona, podemos começar a sonhar com a criação de outra sintaxe para Python. Poderíamos fazer parse dessa nova sintaxe, criar uma AST a partir daí e compilá-la a fim de gerar um código Python.

É exatamente o que o Hy faz. O *Hy* é um dialeto de Lisp que faz parse de uma linguagem do tipo Lisp e a converte para uma AST Python usual, deixando-a totalmente compatível com o ecossistema de Python. Poderíamos compará-la com o que Clojure representa para Java. O Hy, por si só, mereceria um livro, portanto ele será discutido apenas superficialmente. O Hy utiliza a sintaxe e alguns recursos da família de linguagens Lisp: é funcionalmente orientado, disponibiliza macros e é facilmente extensível.

Se você já conhece Lisp – e deveria –, a sintaxe do Hy parecerá familiar. Assim que tiver o Hy instalado (executando `pip install hy`), ao iniciar o interpretador `hy`, você verá um prompt REPL padrão a partir do qual poderá começar a interagir com o interpretador, conforme vemos na Listagem 9.13.

Listagem 9.13 – Interagindo com o interpretador do Hy

```
% hy
hy 0.9.10
=> (+ 1 2)
3
```

Para as pessoas que já conhecem bem a sintaxe de Lisp, os parênteses são usados para construir listas. Se uma lista não estiver entre aspas, ela será avaliada: o primeiro elemento deve ser uma função, e os demais itens da lista são passados como argumentos. Nesse exemplo, o código `(+ 1 2)` é equivalente a `1 + 2` em Python.

Em Hy, a maioria das construções, por exemplo, as definições de função, são mapeadas diretamente de Python.

Listagem 9.14 – Mapeando uma definição de função de Python

```
=> (defn hello [name]
... (print "Hello world!")
... (print (% "Nice to meet you %s" name)))
=> (hello "jd")
Hello world!
Nice to meet you jd
```

Conforme vemos na Listagem 9.14, internamente, o Hy faz parse do código fornecido, converte-o para uma AST Python e então a compila e avalia. Felizmente, é fácil fazer o parse de uma árvore Lisp: cada par de parênteses representa um nó da árvore, o que significa que a conversão é, na verdade, mais simples do que a conversão da sintaxe nativa de Python!

Uma definição de classe é aceita por meio da construção `defclass`, que é inspirada no CLOS (Common Lisp Object System).

Listagem 9.15 – Definindo uma classe com `defclass`

```
(defclass A [object]
  [[x 42]
   [y (fn [self value]
        (+ self.x value))]])
```

A Listagem 9.15 define uma classe chamada `A`, que herda de `object`, com um atributo de classe `x` cujo valor é 42; em seguida, um método `y` devolve o atributo `x` somado a um valor passado como argumento.

Um aspecto realmente incrível é que você pode importar *qualquer biblioteca Python* diretamente para o Hy e usá-la sem restrições. Utilize a função `import()` para importar um módulo, como mostra a Listagem 9.16, do mesmo modo que você comumente faria em Python.

Listagem 9.16 – Importando módulos Python comuns

```
=> (import uuid)
=> (uuid.uuid4)
UUID('f823a749-a65a-4a62-b853-2687c69d0e1e')
=> (str (uuid.uuid4))
'4efa60f2-23a4-4fc1-8134-00f5c271f809'
```

O Hy também tem construções e macros mais sofisticadas. Na

Listagem 9.17, pasme com o que a função `cond()` pode fazer por você no lugar do clássico `if/elif/else` verboso.

Listagem 9.17 – Usando `cond` no lugar de `if/elif/else`

```
(cond
  [(> somevar 50)
   (print "That variable is too big!")]
  [< somevar 10)
   (print "That variable is too small!")]
  [true
   (print "That variable is jusssst right!")])
```

A macro `cond` tem a seguinte assinatura: `(cond [expresssão_condicional expressão_de_retorno] ...)`. Cada expressão condicional é avaliada, começando pela primeira: assim que uma das expressões condicionais devolver um valor verdadeiro, a expressão de retorno será avaliada e devolvida. Se nenhuma expressão de retorno for fornecida, o valor da expressão condicional será devolvido. Portanto, `cond` é equivalente a uma construção `if/elif`, exceto pelo fato de poder devolver o valor da expressão condicional sem ter de avaliá-la duas vezes ou armazená-la em uma variável temporária!

O Hy permite que você mergulhe no mundo Lisp sem se afastar demais de sua zona de conforto, pois você continuará escrevendo código Python. A ferramenta `hy2py` pode até mesmo mostrar como seria seu código Hy se fosse traduzido para Python. Embora o Hy não seja amplamente utilizado, é uma ótima ferramenta para demonstrar o potencial da linguagem Python. Se estiver interessado em saber mais, sugiro que você consulte a documentação online e associe-se à comunidade.

Resumo

Assim como em qualquer outra linguagem de programação, o código-fonte de Python pode ser representado por uma árvore abstrata. Serão raras as ocasiões em que você usará diretamente a AST, mas, se entender como ela funciona, poderá ter uma perspectiva muito útil.

Paul Tagliamonte fala sobre a AST e o Hy

Paul criou o Hy em 2013 e, como amante de Lisp, associei-me a ele nessa aventura maravilhosa. Atualmente, Paul é desenvolvedor na Sunlight Foundation.

Como você aprendeu a usar a AST corretamente, e você tem algum conselho às pessoas que queiram fazer isso?

A AST é extremamente pouco documentada, portanto, a maior parte do conhecimento vem de ASTs geradas, para as quais uma engenharia reversa foi aplicada. Ao escrever scripts Python simples, podemos usar algo semelhante a `import ast; ast.dump(ast.parse("print foo"))` para gerar uma AST equivalente com o intuito de ajudar na tarefa. Com uma pequena dose de adivinhação e um pouco de persistência, não é inviável adquirir um conhecimento básico dessa forma.

Em algum momento, vou me dedicar à tarefa de documentar meu conhecimento sobre o módulo AST, mas acho que escrever código é a melhor maneira de conhecer a AST.

Como a AST de Python difere quanto às versões e aos usos?

A AST de Python não é privada, mas também não é uma interface pública. Não há nenhuma garantia de estabilidade de uma versão para outra – com efeito, há algumas diferenças enervantes entre Python 2 e Python 3, e inclusive entre diferentes versões de Python 3. Além disso, diferentes implementações podem interpretar a AST de modo distinto, ou podem até mesmo ter uma AST única. Não há nada que diga que Jython, PyPy ou CPython devam lidar com a AST de Python da mesma maneira.

Por exemplo, CPython é capaz de lidar com entradas AST levemente fora de ordem (com `lineno` e `col_offset`), enquanto PyPy lançará um erro de asserção. Embora às vezes cause chateações, em geral, a AST é razoável. Não é impossível criar uma AST que funcione em várias instâncias de Python. Com uma ou duas condicionais, será apenas levemente irritante criar uma AST que

funcione em CPython 2.6 até a versão 3.3 e com o PyPy, fazendo com que essa ferramenta seja bastante conveniente.

Qual foi o processo que você usou para criar o Hy?

Comecei com o Hy depois de uma conversa sobre como seria conveniente ter um Lisp que compilasse para Python em vez da JVM de Java (Clojure). Alguns dias depois, eu tinha a primeira versão de Hy. Essa versão lembrava um Lisp e até mesmo funcionava como um Lisp apropriado em alguns aspectos, mas era lenta. Quero dizer, era realmente lenta. Era cerca de uma ordem de grandeza mais lenta do que Python nativo, pois o próprio runtime de Lisp era implementado em Python.

Frustrado, quase desisti, mas então um colega de trabalho sugeriu usar a AST para implementar o runtime, em vez de implementar o runtime em Python. Essa sugestão foi o catalisador de todo o projeto. Passei todo o feriado de final de ano de 2012 trabalhando intensamente com o Hy. Depois de uma semana – ou algo assim – eu tinha algo que lembrava a base de código atual do Hy.

Logo depois de ter o Hy funcionando o suficiente para implementar uma aplicação Flask básica, dei uma palestra no Boston Python sobre o projeto, e a recepção foi extremamente calorosa – tão calorosa, na verdade, que eu comecei a ver o Hy como uma boa maneira de ensinar as pessoas o funcionamento interno de Python, por exemplo, como o REPL funciona, os hooks de importação da PEP 302 e a AST de Python. Era uma boa introdução para o conceito de código que escreve código.

Reescrevi algumas partes do compilador para corrigir alguns problemas filosóficos no processo, o que nos levou à iteração atual da base de código – a qual tem resistido muito bem!

Conhecer Hy também é uma boa maneira de começar a entender como ler Lisp. Os usuários podem adquirir familiaridade com expressões-s em um ambiente que conhecem, e até mesmo usar as bibliotecas que já utilizam, facilitando a transição para outros

Lisps, por exemplo, Common Lisp, Scheme ou Clojure.

Qual é o nível de interoperabilidade entre Python e Hy?

A interoperabilidade de Hy é incrível. É tão boa que o `pdb` é capaz de depurar o Hy de forma apropriada, sem que nenhuma alteração seja necessária. Já escrevi aplicações Flask, aplicações Django e módulos de todos os tipos com Hy. Python pode importar Python, Hy pode importar Hy, Hy pode importar Python e Python pode importar Hy. Isso é o que realmente torna o Hy único; outras variantes de Lisp, como o Clojure, são exclusivamente unidirecionais. O Clojure pode importar Java, porém Java teria uma dificuldade imensa para importar Clojure.

O Hy funciona traduzindo um código Hy (em expressões-s) para a AST de Python quase diretamente. Esse passo de compilação implica que o bytecode gerado é bastante razoável, o que significa que Python teria muita dificuldade inclusive de afirmar que o módulo não foi escrito em Python.

Construções típicas de Common Lisp, como `*earmuffs*` ou `using-dashes` são totalmente aceitas ao serem traduzidas em um equivalente Python (nesse caso, `*earmuffs*` se torna `EARMUFFS` e `using-dashes` se torna `using_dashes`), o que significa que Python não terá nenhuma dificuldade para usá-las.

Garantir que tenhamos realmente um bom grau de interoperabilidade é uma de nossas maiores prioridades, portanto, se você vir algum bug, informe!

Quais são as vantagens e as desvantagens de escolher o Hy?

Uma vantagem do Hy é que ele tem um sistema completo de macros, com o qual Python apresenta dificuldade. As macros são funções especiais que alteram o código durante o passo de compilação. Isso facilita criar outras linguagens específicas de domínios, que são compostas da linguagem base (nesse caso, Hy/Python), em conjunto com várias macros que permitem ter um código unicamente expressivo e sucinto.

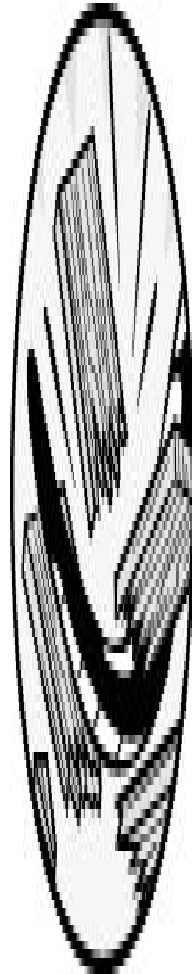
Quanto às desvantagens, o Hy, por ser um Lisp escrito com expressões-s, sofre com o estigma de ser difícil de aprender, de ler ou de manter. As pessoas podem sentir aversão em trabalhar com projetos que usem Hy por temerem a sua complexidade.

O Hy é o Lisp que todos amam odiar. O pessoal de Python talvez não aprecie a sua sintaxe, e os adeptos de Lisp podem evitá-lo porque o Hy utiliza objetos Python diretamente, o que significa que o comportamento dos objetos básicos às vezes pode ser surpreendente para os usuários que sejam profundos conhecedores de Lisp.

Espero que as pessoas olhem para além de sua sintaxe e considerem conhecer algumas partes de Python anteriormente inexploradas.

10

DESEMPENHO E OTIMIZAÇÕES



Raramente as otimizações são o primeiro tópico no qual você pensará ao fazer um desenvolvimento, mas sempre chegará o momento em que otimizar para ter um melhor desempenho será apropriado. Isso não quer dizer que você deva escrever um programa com a ideia de que ele será lento; contudo, pensar na otimização sem antes determinar quais são as ferramentas corretas a serem usadas e sem fazer um profiling apropriado é perda de

tempo. Como escreveu Donald Knuth, “Uma otimização prematura é a raiz de todos os males”.¹

Neste capítulo, explicarei como utilizar a abordagem correta para escrever um código rápido e mostrarei os pontos que você deve observar quando precisar de mais otimizações. Muitos desenvolvedores tentam adivinhar os locais em que Python poderia ser mais lento ou mais rápido. Em vez de ficar especulando, este capítulo ajudará você a gerar o perfil de sua aplicação para saber quais partes de seu programa estão causando lentidão e onde estão os gargalos.

Estruturas de dados

A maior parte dos problemas de programação pode ser resolvida de maneira simples e elegante com as estruturas de dados corretas – e Python oferece muitas estruturas de dados entre as quais podemos escolher. Saber tirar proveito dessas estruturas de dados existentes resulta em soluções mais limpas e estáveis, em comparação a implementar estruturas de dados personalizadas.

Por exemplo, todos usam `dict`, mas quantas vezes você já deve ter visto um código tentando acessar um dicionário capturando a exceção `KeyError`, conforme vemos a seguir:

```
def get_fruits(basket, fruit):
    try:
        return basket[fruit]
    except KeyError:
        return None
```

Ou verificando antes se a chave está presente:

```
def get_fruits(basket, fruit):
    if fruit in basket:
        return basket[fruit]
```

Se você usar o método `get()` já existente na classe `dict`, poderá evitar ter de capturar uma exceção ou verificar a presença da chave, antes de tudo:

```
def get_fruits(basket, fruit):
```

```
return basket.get(fruit)
```

O método `dict.get()` também pode devolver um valor default no lugar de `None`; basta chamá-lo com um segundo argumento:

```
def get_fruits(basket, fruit):  
    # Devolve a fruta, ou Banana se a fruta não puder ser encontrada.  
    return basket.get(fruit, Banana())
```

Podemos culpar muitos desenvolvedores por usarem estruturas de dados básicas de Python sem conhecer todos os métodos que elas oferecem. Isso também vale para conjuntos (sets): os métodos nas estruturas de dados para conjuntos podem solucionar vários problemas que, do contrário, teriam de ser resolvidos com a escrita de blocos `for/if` aninhados. Por exemplo, os desenvolvedores muitas vezes usam laços `for/if` para determinar se um item está em uma lista, assim:

```
def has_invalid_fields(fields):  
    for field in fields:  
        if field not in ['foo', 'bar']:  
            return True  
    return False
```

O laço itera em cada item da lista e verifica todos para saber se são `foo` ou `bar`. Contudo, podemos escrever isso de modo mais eficaz eliminando a necessidade de um laço:

```
def has_invalid_fields(fields):  
    return bool(set(fields) - set(['foo', 'bar']))
```

Isso muda o código de modo a converter os campos em um conjunto, obtendo o resto do conjunto subtraindo `set(['foo', 'bar'])`. Em seguida, o conjunto é convertido em um valor booleano que informa se restou algum item diferente de `foo` e de `bar`. Ao usar conjuntos, não há necessidade de iterar em nenhuma lista e verificar os itens um a um. Uma única operação em dois conjuntos, feita internamente por Python, será mais rápida.

Python também tem estruturas de dados mais sofisticadas, que podem reduzir bastante o fardo da manutenção do código. Por exemplo, observe a Listagem 10.1.

Listagem 10.1 – Acrescentando uma entrada em um dicionário de conjuntos

```
def add_animal_in_family(species, animal, family):
    if family not in species:
        species[family] = set()
    species[family].add(animal)

species = {}
add_animal_in_family(species, 'cat', 'felidea')
```

Esse código é perfeitamente válido, mas quantas vezes seus programas exigirão uma variação da Listagem 10.1? Dezenas? Centenas?

Python disponibiliza a estrutura `collections.defaultdict`, que resolve o problema de maneira elegante:

```
import collections

def add_animal_in_family(species, animal, family):
    species[family].add(animal)

species = collections.defaultdict(set)
add_animal_in_family(species, 'cat', 'felidea')
```

Sempre que você tentar acessar um item inexistente em seu `dict`, `defaultdict` usará a função passada como argumento para o seu construtor para criar outro valor, em vez de gerar um `KeyError`. Nesse caso, a função `set()` é usada para criar outro `set` sempre que for necessário.

O módulo `collections` oferece mais algumas estruturas de dados que podem ser usadas para resolver outros tipos de problemas. Por exemplo, suponha que você queira contar o número de itens distintos em um iterável. Vamos analisar o método `collections.Counter()`, que oferece métodos para resolver esse problema:

```
>>> import collections
>>> c = collections.Counter("Premature optimization is the root of all evil.")
>>> c
>>> c['P'] # Devolve o número de ocorrências da letra 'P'
1
>>> c['e'] # Devolve o número de ocorrências da letra 'e'
```


4

```
>>> c.most_common(2) # Devolve as duas letras mais comuns  
[(' ', 7), ('i', 5)]
```

O objeto `collections.Counter` funciona com qualquer iterável que tenha itens hashable, eliminando a necessidade de escrever suas próprias funções de contagem. Ele pode facilmente contar o número de letras em uma string e devolver os n itens mais comuns de um iterável. Talvez você tentasse implementar algo semelhante por conta própria caso não soubesse que o recurso já está disponível na Biblioteca-Padrão de Python.

Com a estrutura de dados correta, os métodos corretos e – obviamente – um algoritmo apropriado, seu programa deverá ter um bom desempenho. No entanto, se não estiver com um desempenho aceitável, a melhor maneira de obter pistas sobre os pontos em que ele poderia estar lento e que precisam de otimização é gerar o perfil de seu código.

Entendendo o comportamento usando profiling

O *profiling* (geração de perfil) é uma forma de análise dinâmica do programa, a qual permite compreender como um programa se comporta. Ele nos permite determinar os locais em que pode haver gargalos e a necessidade de otimização. O perfil de um programa assume a forma de um conjunto de dados estatísticos que descrevem a frequência com que as partes do programa executam, e por quanto tempo executam.

Python oferece algumas ferramentas para o profiling de seu programa. Uma delas, o `cProfile`, faz parte da Biblioteca-Padrão de Python, e não exige instalação. Veremos também o módulo `dis`, que é capaz de fazer o *disassembling* (desmontagem) de um código Python em partes menores, facilitando compreender o que acontece internamente.

cProfile

Python incluiu o cProfile a partir de Python 2.5, por padrão. Para usar o cProfile, chame-o com o seu programa usando a sintaxe `python -m cProfile <programa>`. Esse comando carregará e ativará o módulo cProfile e, em seguida, executará o programa habitual com a instrumentação ativada, conforme vemos na Listagem 10.2.

Listagem 10.2 – Saída padrão de cProfile usado com um script Python

```
$ python -m cProfile myscript.py
343 function calls (342 primitive calls) in 0.000 seconds

Ordered by: standard name

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1  0.000   0.000   0.000   0.000  :0(_getframe)
   1  0.000   0.000   0.000   0.000  :0(len)
 104  0.000   0.000   0.000   0.000  :0(setattr)
   1  0.000   0.000   0.000   0.000  :0(setprofile)
   1  0.000   0.000   0.000   0.000  :0(startswith)
 2/1  0.000   0.000   0.000   0.000  <string>:1(<module>)
   1  0.000   0.000   0.000   0.000  StringIO.py:30(<module>)
   1  0.000   0.000   0.000   0.000  StringIO.py:42(StringIO)
```

A Listagem 10.2 mostra a saída da execução de um script simples com cProfile. Ela informa o número de vezes que cada função do programa foi chamada e o tempo decorrido em sua execução. Também podemos utilizar a opção `-s` para ordenar de acordo com outros campos; por exemplo, `-s time` ordenaria os resultados com base no tempo interno.

É possível visualizar as informações geradas por cProfile usando uma ótima ferramenta chamada KCacheGrind. Essa ferramenta foi criada para lidar com programas escritos em C, mas felizmente podemos usá-la com dados Python, convertendo-os em uma árvore de chamadas.

O módulo cProfile tem uma opção `-o` que permite salvar os dados de profiling, e `pyprof2calltree` é capaz de converter os dados de um formato

para outro. Em primeiro lugar, instale o conversor com o seguinte comando:

```
$ pip install pyprof2calltree
```

Em seguida, execute o conversor conforme mostra a Listagem 10.3 para converter os dados (opção -i) e executar KCacheGrind com os dados convertidos (opção -k).

Listagem 10.3 – Executando o cProfile e iniciando o KCacheGrind

```
$ python -m cProfile -o myscript.cprof myscript.py
```

```
$ pyprof2calltree -k -i myscript.cprof
```

Assim que o KCacheGrind iniciar, ele exibirá informações como aquelas exibidas na Figura 10.1. Com esses resultados visuais, podemos usar o gráfico de chamadas para verificar a porcentagem de tempo gasta em cada função, permitindo determinar quais são as partes de seu programa que podem estar consumindo recursos demais.

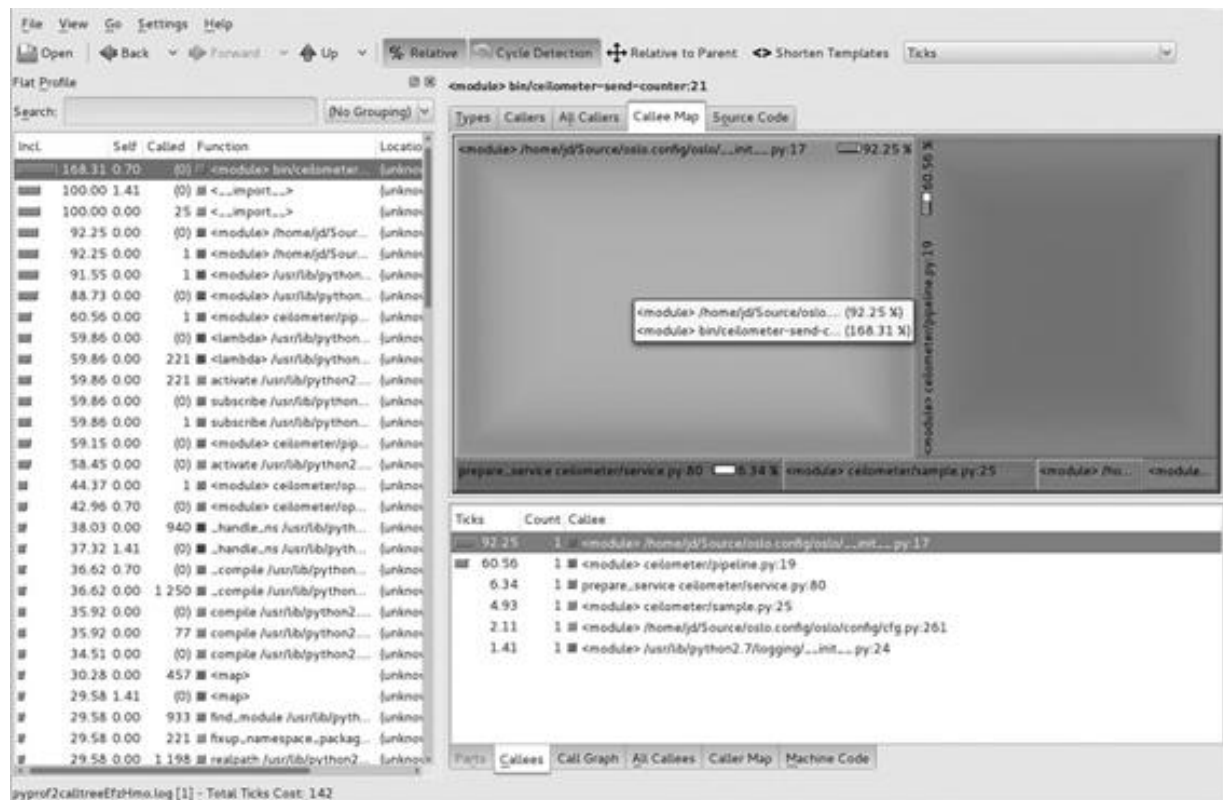


Figura 10.1 – Exemplo de saída do KCacheGrind.

O modo mais fácil de ler o KCacheGrind é começar com a tabela à esquerda da tela, a qual lista todas as funções e métodos executados pelo seu programa. Podemos ordená-las com base no tempo de execução e, em seguida, identificar aquela que consome mais tempo de CPU e clicar nela.

Os painéis à direita do KCacheGrind podem mostrar quais funções chamaram essa função e quantas vezes, assim como as outras funções que são chamadas por essa função. É fácil navegar pelo gráfico de chamadas de seu programa, o qual inclui também o tempo de execução de cada parte.

Isso lhe permitirá compreender melhor as partes de seu código que talvez precisem de otimização. Cabe a você determinar o modo de otimizar o seu código, e isso dependerá do que seu programa estiver tentando fazer!

Embora obter informações sobre como seu programa executa e visualizar esses dados seja bastante apropriado para ter uma visão macroscópica de seu programa, talvez você precise de uma visão mais microscópica de algumas partes do código a fim de inspecionar seus elementos de modo mais detalhado. Em casos como esses, acho melhor lançar mão do módulo `dis` para descobrir o que está acontecendo nos bastidores.

Disassembling com o módulo `dis`

O módulo `dis` é um disassembler de bytecode Python. Desmontar um código pode ser conveniente para entender o que acontece em cada linha, de modo que você possa otimizá-la de maneira apropriada. Por exemplo, a Listagem 10.4 mostra a função `dis.dis()`, que faz o disassembling de qualquer função que você lhe passar como parâmetro e exibe a lista das instruções executadas pela função em bytecode.

Listagem 10.4 – Disassembling de uma função

```
>>> def x():  
...     return 42
```

```
...
>>> import dis
>>> dis.dis(x)
2  0 LOAD_CONST    1 (42)
   3 RETURN_VALUE
```

Na Listagem 10.4, o disassembling da função `x` foi feito, e seus constituintes, compostos de instruções em bytecode, foram exibidos. Há somente duas operações nesse exemplo: carregar uma constante (`LOAD_CONST`), que é igual a 42, e devolver esse valor (`RETURN_VALUE`).

Para ver o `dis` em ação e como ele pode ser útil, definiremos duas funções que fazem a mesma tarefa – concatenam três letras – e faremos o disassembling para ver como elas a executam de maneiras distintas:

```
abc = ('a', 'b', 'c')

def concat_a_1():
    for letter in abc:
        abc[0] + letter

def concat_a_2():
    a = abc[0]
    for letter in abc:
        a + letter
```

As duas funções aparentemente realizam a mesma tarefa; no entanto, se fizermos o disassembling delas com `dis.dis`, como mostra a Listagem 10.5, veremos que o bytecode gerado é um pouco diferente.

Listagem 10.5 – Disassembling de funções que concatenam strings

```
>>> dis.dis(concat_a_1)
2      0 SETUP_LOOP      26 (to 29)
        3 LOAD_GLOBAL      0 (abc)
        6 GET_ITER
>>    7 FOR_ITER          18 (to 28)
        10 STORE_FAST       0 (letter)

3      13 LOAD_GLOBAL      0 (abc)
```

```

    16 LOAD_CONST      1 (0)
    19 BINARY_SUBSCR
    20 LOAD_FAST        0 (letter)
    23 BINARY_ADD
    24 POP_TOP
    25 JUMP_ABSOLUTE     7
>> 28 POP_BLOCK
>> 29 LOAD_CONST      0 (None)
    32 RETURN_VALUE
>>> dis.dis(concat_a_2)
2      0 LOAD_GLOBAL    0 (abc)
      3 LOAD_CONST      1 (0)
      6 BINARY_SUBSCR
      7 STORE_FAST      0 (a)

3      10 SETUP_LOOP      22 (to 35)
      13 LOAD_GLOBAL    0 (abc)
      16 GET_ITER
>> 17 FOR_ITER      14 (to 34)
      20 STORE_FAST      1 (letter)

4      23 LOAD_FAST      0 (a)
      26 LOAD_FAST      1 (letter)
      29 BINARY_ADD
      30 POP_TOP
      31 JUMP_ABSOLUTE  17
>> 34 POP_BLOCK
>> 35 LOAD_CONST      0 (None)
      38 RETURN_VALUE

```

Na segunda função da Listagem 10.5, armazenamos `abc[0]` em uma variável temporária antes de executar o laço. Isso deixa o bytecode executado dentro do laço um pouco menor que o bytecode da primeira função, pois evitamos ter de acessar `abc[0]` em cada iteração. Ao ser avaliada com `timeit`, a segunda versão é 10% mais rápida do que a primeira; ela demora um microssegundo inteiro a menos para executar! Obviamente não vale a pena otimizar esse microssegundo, a menos que você chame essa função bilhões de vezes; contudo, esse é o tipo de insight que o módulo `dis` pode proporcionar.

O fato de você contar com “truques” como armazenar o valor fora do laço dependerá da situação – em última instância, otimizar esse tipo de uso deverá ser uma tarefa do compilador. Por outro lado, é difícil para o compilador ter certeza de que uma otimização não terá efeitos colaterais negativos, pois Python é extremamente dinâmico. Na Listagem 10.5, usar `abc[0]` fará `abc.__getitem__` ser chamado, o que poderia ter efeitos colaterais se ele tivesse sido sobrescrito por meio de herança. Conforme a versão da função que você usar, o método `abc.__getitem__` será chamado uma ou várias vezes, e isso poderia fazer diferença. Portanto, tome cuidado ao escrever e otimizar o seu código!

Definindo funções de modo eficiente

Um erro comum que costumo ver em revisões de código são as definições de funções dentro de funções. Isso é ineficiente, pois a função é, então, redefinida repetidamente, sem que haja necessidade. Por exemplo, a Listagem 10.6 mostra a função `y()` sendo definida várias vezes.

Listagem 10.6 – Redefinição de função

```
>> import dis
>>> def x():
...     return 42
...
>>> dis.dis(x)
2  0 LOAD_CONST      1 (42)
  3 RETURN_VALUE
>>> def x():
...     def y():
...         return 42
...     return y()
...
>>> dis.dis(x)
2  0 LOAD_CONST      1 (<code object y at
x100ce7e30, file "<stdin>", line 2>)
  3 MAKE_FUNCTION    0
  6 STORE_FAST       0 (y)
```

```

4   9 LOAD_FAST      0 (y)
   12 CALL_FUNCTION   0
   15 RETURN_VALUE

```

A Listagem 10.6 mostra as chamadas a `MAKE_FUNCTION`, `STORE_FAST`, `LOAD_FAST` e `CALL_FUNCTION`, que exigem muito mais opcodes do que o necessário para devolver 42, conforme vimos na Listagem 10.4.

O único caso em que seria necessário definir uma função dentro de outra função é aquele em que criamos uma closure de função, e esse é um caso de uso perfeitamente identificado nos opcodes de Python com `LOAD_CLOSURE`, conforme vemos na Listagem 10.7.

Listagem 10.7 – Definindo uma closure

```

>>> def x():
...     a = 42
...     def y():
...         return a
...     return y()
...
>>> dis.dis(x)
2   0 LOAD_CONST      1 (42)
   3 STORE_DEREF       0 (a)

3   6 LOAD_CLOSURE     0 (a)
   9 BUILD_TUPLE       1
  12 LOAD_CONST        2 (<code object y at
x100d139b0, file "<stdin>", line 3>)
  15 MAKE_CLOSURE      0
  18 STORE_FAST        0 (y)

5  21 LOAD_FAST        0 (y)
  24 CALL_FUNCTION      0
  27 RETURN_VALUE

```

Embora seja provável que você não vá precisar utilizá-lo no cotidiano, o disassembling de código é uma ferramenta conveniente caso você queira analisar o que acontece internamente, de modo mais detalhado.

Listas ordenadas e o bisect

A seguir, veremos a otimização de listas. Se uma lista não estiver ordenada, o cenário de pior caso para encontrar a posição de um item específico na lista terá uma complexidade $O(n)$, o que significa que, no pior caso, você encontrará o seu item depois de iterar por todos os itens da lista.

A solução habitual para otimizar esse problema é usar uma lista *ordenada*. Listas ordenadas utilizam um algoritmo de bissecção para buscas a fim de proporcionar um tempo de busca de $O(\log n)$. A ideia é dividir recursivamente a lista na metade e determinar o lado – esquerdo ou direito – em que o item deverá estar e, portanto, em qual lado a busca deverá ser feita a seguir.

Python disponibiliza o módulo `bisect`, o qual contém um algoritmo de bissecção, como mostra a Listagem 10.8.

Listagem 10.8 – Usando bisect para encontrar uma agulha num palheiro

```
>>> farm = sorted(['haystack', 'needle', 'cow', 'pig'])
>>> bisect.bisect(farm, 'needle')
3
>>> bisect.bisect_left(farm, 'needle')
2
>>> bisect.bisect(farm, 'chicken')
0
>>> bisect.bisect_left(farm, 'chicken')
0
>>> bisect.bisect(farm, 'eggs')
1
>>> bisect.bisect_left(farm, 'eggs')
1
```

Conforme vemos na Listagem 10.8, a função `bisect.bisect()` devolve a posição na qual um elemento deverá ser inserido para manter a lista ordenada. Obviamente, isso só funcionará se a lista estiver devidamente ordenada, para começar. Uma ordenação inicial nos permite obter o índice *teórico* de um item: `bisect()` não informa se o item está na lista, mas onde o item deveria estar se estivesse aí. Obter o

item nesse índice responderá à pergunta sobre o item estar na lista. Se você quiser inserir de imediato o elemento na posição ordenada correta, o módulo `bisect` disponibiliza as funções `insort_left()` e `insort_right()`, como mostra a Listagem 10.9.

Listagem 10.9 – Inserindo um item em uma lista ordenada

```
>>> farm
['cow', 'haystack', 'needle', 'pig']
>>> bisect.insort(farm, 'eggs')
>>> farm
['cow', 'eggs', 'haystack', 'needle', 'pig']
>>> bisect.insort(farm, 'turkey')
>>> farm
['cow', 'eggs', 'haystack', 'needle', 'pig', 'turkey']
```

Ao usar o módulo `bisect`, poderíamos também criar uma classe especial `SortedList` que herda de `list` a fim de criar uma lista que esteja sempre ordenada, conforme vemos na Listagem 10.10.

Listagem 10.10 – Implementação de um objeto `SortedList`

```
import bisect
import unittest

class SortedList(list):
    def __init__(self, iterable):
        super(SortedList, self).__init__(sorted(iterable))

    def insort(self, item):
        bisect.insort(self, item)

    def extend(self, other):
        for item in other:
            self.insort(item)

    @staticmethod
    def append(o):
        raise RuntimeError("Cannot append to a sorted list")

    def index(self, value, start=None, stop=None):
        place = bisect.bisect_left(self[start:stop], value)
        if start:
```

```

        place += start
    end = stop or len(self)
    if place < end and self[place] == value:
        return place
    raise ValueError("%s is not in list" % value)

```

```

class TestSortedList(unittest.TestCase):
    def setUp(self):
        self.mylist = SortedList(
            ['a', 'c', 'd', 'x', 'f', 'g', 'w']
        )

    def test_sorted_init(self):
        self.assertEqual(sorted(['a', 'c', 'd', 'x', 'f', 'g', 'w']),
            self.mylist)

    def test_sorted_insort(self):
        self.mylist.insort('z')
        self.assertEqual(['a', 'c', 'd', 'f', 'g', 'w', 'x', 'z'],
            self.mylist)
        self.mylist.insort('b')
        self.assertEqual(['a', 'b', 'c', 'd', 'f', 'g', 'w', 'x', 'z'],
            self.mylist)

    def test_index(self):
        self.assertEqual(0, self.mylist.index('a'))
        self.assertEqual(1, self.mylist.index('c'))
        self.assertEqual(5, self.mylist.index('w'))
        self.assertEqual(0, self.mylist.index('a', stop=0))
        self.assertEqual(0, self.mylist.index('a', stop=2))
        self.assertEqual(0, self.mylist.index('a', stop=20))
        self.assertRaises(ValueError, self.mylist.index, 'w', stop=3)
        self.assertRaises(ValueError, self.mylist.index, 'a', start=3)
        self.assertRaises(ValueError, self.mylist.index, 'a', start=333)

    def test_extend(self):
        self.mylist.extend(['b', 'h', 'j', 'c'])
        self.assertEqual(
            ['a', 'b', 'c', 'c', 'd', 'f', 'g', 'h', 'j', 'w', 'x']
            self.mylist)

```

Usar uma classe `list` dessa forma é um pouco mais lento quando se trata de inserir o item porque o programa precisará encontrar a

posição correta para inseri-lo. No entanto, essa classe é mais rápida que a sua classe-pai no uso do método `index()`. Obviamente não devemos usar o método `list.append()` nessa classe: você não pode concatenar um item no final da lista, pois ela poderia acabar ficando desordenada!

Muitas bibliotecas Python implementam diversas versões da Listagem 10.10 para vários outros tipos de dados, por exemplo, estruturas binárias ou de árvores rubro-negras (red-black trees). Os pacotes Python `blis` e `bintree` contêm códigos que podem ser usados para essas finalidades, e são uma alternativa conveniente em relação a implementar e depurar uma versão criada por conta própria.

Na próxima seção, veremos como podemos tirar proveito do tipo de dado nativo tupla disponibilizado por Python para deixar seu código Python um pouco mais rápido.

namedtuple e slots

Muitas vezes na programação, você precisará criar objetos simples que tenham somente alguns atributos fixos. Uma implementação simples poderia ser algo semelhante ao que vemos nestas linhas:

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

Sem dúvida, esse código faz o trabalho. Contudo, há uma desvantagem nessa abordagem. Nesse exemplo, criamos uma classe que herda da classe objeto, portanto, ao usar essa classe `Point`, você instanciará objetos completos e alocará bastante memória.

Em Python, objetos comuns armazenam todos os seus atributos em um dicionário, e esse dicionário é armazenado no atributo `__dict__`, conforme vemos na Listagem 10.11.

Listagem 10.11 – Como os atributos são armazenados internamente

em um objeto Python

```
>>> p = Point(1, 2)
>>> p.__dict__
{'y': 2, 'x': 1}
>>> p.z = 42
>>> p.z
42
>>> p.__dict__
{'y': 2, 'x': 1, 'z': 42}
```

Em Python, a vantagem de usar um dict é que ele permite acrescentar a quantidade de atributos que você quiser em um objeto. A desvantagem é que utilizar um dicionário para armazenar esses atributos é custoso no que diz respeito à memória – é necessário armazenar o objeto, as chaves, as referências aos valores e tudo mais. Isso faz com que o dicionário seja lento para criar e manipular, além de exigir um custo alto de memória.

Como exemplo desse uso desnecessário de memória, considere a classe simples a seguir:

```
class Foobar(object):
    def __init__(self, x):
        self.x = x
```

Esse código cria um objeto Foobar simples, com um único atributo x. Vamos verificar o uso de memória dessa classe usando memory_profiler – um pacote Python interessante, que nos permite verificar o uso de memória de um programa linha a linha – e um pequeno script que cria 100.000 objetos, como vemos na Listagem 10.12.

Listagem 10.12 – Usando memory_profiler em um script que utiliza objetos

```
$ python -m memory_profiler object.py
Filename: object.py
```

Line #	Mem usage	Increment	Line Contents
5			@profile
6	9.879 MB	0.000 MB	def main():
7	50.289 MB	40.410 MB	f = [Foobar(42) for i in range(100000)]

A Listagem 10.12 mostra que criar 100.000 objetos da classe Foobar

consome 40MB de memória. Ainda que 400 bytes por objeto possam não parecer muito, quando criamos milhares de objetos, a soma total de memória será elevada.

Há um modo de usar objetos, ao mesmo tempo que evitamos esse comportamento default de dict: as classes em Python podem definir um atributo `__slots__` que conterá somente os atributos permitidos às instâncias dessa classe. Em vez de alocar um objeto dicionário completo para armazenar os atributos do objeto, podemos utilizar um objeto *lista* para armazená-los.

Se você analisar o código-fonte de CPython e observar o arquivo *Objects/typeobject.c*, será muito fácil entender o que Python faz quando `__slots__` é definido em uma classe. A Listagem 10.13 é uma versão resumida da função que cuida disso:

Listagem 10.13 – Uma parte do código de Objects/typeobject.c

```
static PyObject *
type_new(PyTypeObject *metatype, PyObject *args, PyObject *kwds)
{
    --trecho omitido--
    /* Check for a __slots__ sequence variable in dict, and count it */
    slots = _PyDict_GetItemId(dict, &PyId__slots__);
    nslots = 0;
    if (slots == NULL) {
        if (may_add_dict)
            add_dict++;
        if (may_add_weak)
            add_weak++;
    }
    else {
        /* Have slots */
        /* Make it into a tuple */
        if (PyUnicode_Check(slots))
            slots = PyTuple_Pack(1, slots);
        else
            slots = PySequence_Tuple(slots);
        /* Are slots allowed? */
        nslots = PyTuple_GET_SIZE(slots);
        if (nslots > 0 && base->tp_itemsize != 0) {
            PyErr_Format(PyExc_TypeError,
```

```

        "nonempty __slots__ "
        "not supported for subtype of '%s'",
        base->tp_name);
    goto error;
}
/* Copy slots into a list, mangle names and sort them.
   Sorted names are needed for __class__ assignment.
   Convert them back to tuple at the end.
*/
newslots = PyList_New(nslots - add_dict - add_weak);
if (newslots == NULL)
    goto error;
if (PyList_Sort(newslots) == -1) {
    Py_DECREF(newslots);
    goto error;
}
slots = PyList_AsTuple(newslots);
Py_DECREF(newslots);
if (slots == NULL)
    goto error;
}
/* Allocate the type object */
type = (PyTypeObject *)metatype->tp_alloc(metatype, nslots);
--trecho omitido--
/* Keep name and slots alive in the extended type object */
et = (PyHeapTypeObject *)type;
Py_INCREF(name);
et->ht_name = name;
et->ht_slots = slots;
slots = NULL;
--trecho omitido--
return (PyObject *)type;

```

Como podemos ver na Listagem 10.13, Python converte o conteúdo de `__slots__` em uma tupla e, em seguida, em uma lista, que ele cria e ordena antes de convertê-la de volta em uma tupla a ser usada e armazenada na classe. Dessa forma, Python é capaz de acessar os valores rapidamente, sem ter de alocar e usar um dicionário completo.

É muito fácil declarar e usar uma classe desse tipo. Tudo que você precisa fazer é definir o atributo `__slots__` com uma lista de atributos

que serão definidos na classe:

```
class Foobar(object):
    __slots__ = ('x',)

    def __init__(self, x):
        self.x = x
```

Podemos comparar o uso de memória das duas abordagens usando o pacote Python `memory_profiler`, como mostra a Listagem 10.14.

Listagem 10.14 – Executando `memory_profiler` no script que utiliza `__slots__`

```
% python -m memory_profiler slots.py
Filename: slots.py
```

Line #	Mem usage	Increment	Line Contents
7			@profile
8	9.879 MB	0.000 MB	def main():
9	21.609 MB	11.730 MB	f = [Foobar(42) for i in range(100000)]

A Listagem 10.14 mostra que, desta vez, menos de 12MB de memória foram necessários para criar 100.000 objetos – isto é, menos de 120 bytes por objeto. Portanto, ao usar o atributo `__slots__` das classes Python, podemos reduzir o uso de memória; desse modo, ao criar uma grande quantidade de objetos simples, o atributo `__slots__` será uma opção eficaz e eficiente. No entanto, essa técnica não deve ser usada para ter uma tipagem estática, deixando a lista de atributos fixa para todas as classes: fazer isso não estaria de acordo com o espírito dos programas Python.

A desvantagem, nesse caso, é que a lista de atributos agora está fixa. Nenhum atributo novo poderá ser acrescentado à classe `Foobar` durante a execução. Por causa da natureza fixa da lista de atributos, é muito fácil imaginar classes nas quais os atributos listados sempre teriam um valor e os campos estariam sempre ordenados de alguma maneira.

É exatamente o que acontece na classe `namedtuple` do módulo `collection`. Essa classe `namedtuple` nos permite criar dinamicamente uma classe que herdar da classe `tupla`, compartilhando características

como ser imutável e ter um número fixo de entradas.

Em vez de ter de referenciá-las pelo índice, uma `namedtuple` é capaz de acessar elementos da tupla referenciando um atributo nomeado. Isso faz com que seja mais fácil para as pessoas acessarem as tuplas, conforme mostra a Listagem 10.15.

Listagem 10.15 – Usando namedtuple para referenciar elementos da tupla

```
>>> import collections
>>> Foobar = collections.namedtuple('Foobar', ['x'])
>>> Foobar = collections.namedtuple('Foobar', ['x', 'y'])
>>> Foobar(42, 43)
Foobar(x=42, y=43)
>>> Foobar(42, 43).x
42
>>> Foobar(42, 43).x = 44
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
>>> Foobar(42, 43).z = 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Foobar' object has no attribute 'z'
>>> list(Foobar(42, 43))
[42, 43]
```

A Listagem 10.15 mostra como podemos criar uma classe simples com apenas uma linha de código e, em seguida, instanciá-la. Não podemos modificar nenhum atributo dos objetos dessa classe nem acrescentar atributos, tanto porque a classe herda de `namedtuple` como pelo fato de o valor de `__slots__` estar definido como uma tupla vazia, impedindo a criação do `__dict__`. Pelo fato de uma classe como essa herdar de tupla, é possível convertê-la facilmente em uma lista.

A Listagem 10.16 mostra o uso de memória da factory da classe `namedtuple`.

Listagem 10.16 – Usando namedtuple para executar memory_profiler em um script

```
% python -m memory_profiler namedtuple.py
```

Filename: namedtuple.py

Line #	Mem usage	Increment	Line Contents
4			@profile
5	9.895 MB	0.000 MB	def main():
6	23.184 MB	13.289 MB	f = [Foobar(42) for i in range(100000)]

Com aproximadamente 13MB para 100.000 objetos, usar `namedtuple` é um pouco menos eficiente do que utilizar um objeto com `__slots__`, mas a vantagem é que ela é compatível com a classe tupla. Desse modo, ela pode ser passada para várias funções Python nativas e para bibliotecas que esperam um iterável como argumento. Uma factory da classe `namedtuple` também desfruta as várias otimizações existentes para tuplas: por exemplo, tuplas com menos itens que `PyTuple_MAXSAVESIZE` (20, por padrão) usarão um alocador de memória mais rápido em CPython.

A classe `namedtuple` também oferece mais alguns métodos extras que, embora prefixados com underscore, na verdade, foram criados para serem públicos. O método `_asdict()` pode converter a `namedtuple` em uma instância de `dict`, o método `_make()` permite converter um objeto iterável existente para essa classe, e `_replace()` devolve uma nova instância do objeto com alguns campos substituídos.

As tuplas nomeadas são um ótimo substituto para objetos pequenos, compostos apenas de poucos atributos e que não exijam nenhum método personalizado – considere usá-las no lugar dos dicionários, por exemplo. Se seu tipo de dado exigir métodos, tiver uma lista fixa de atributos e puder ser instanciado milhares de vezes, criar uma classe personalizada com `__slots__` talvez seja uma boa ideia para economizar um pouco de memória.

Memoização

A *memoização* é uma técnica de otimização usada para agilizar as chamadas de função por meio do caching de seus resultados. O resultado de uma função poderá ser armazenado em cache somente se a função for *pura*, o que significa que ela não terá efeitos

colaterais e não depende de nenhum estado global. (Veja o Capítulo 8 para saber mais sobre funções puras.)

Uma função trivial que pode ser memoizada é a função `sin()`, exibida na Listagem 10.17.

Listagem 10.17 – Uma função `sin()` memoizada

```
>>> import math
>>> _SIN_MEMOIZED_VALUES = {}
>>> def memoized_sin(x):
...     if x not in _SIN_MEMOIZED_VALUES:
...         _SIN_MEMOIZED_VALUES[x] = math.sin(x)
...     return _SIN_MEMOIZED_VALUES[x]
>>> memoized_sin(1)
0.8414709848078965
>>> _SIN_MEMOIZED_VALUES
{1: 0.8414709848078965}
>>> memoized_sin(2)
0.9092974268256817
>>> memoized_sin(2)
0.9092974268256817
>>> _SIN_MEMOIZED_VALUES
{1: 0.8414709848078965, 2: 0.9092974268256817}
>>> memoized_sin(1)
0.8414709848078965
>>> _SIN_MEMOIZED_VALUES
{1: 0.8414709848078965, 2: 0.9092974268256817}
```

Na Listagem 10.17, na primeira vez em que `memoized_sin()` é chamada com um argumento que não esteja armazenado em `_SIN_MEMOIZED_VALUES`, o valor será calculado e armazenado nesse dicionário. Se chamarmos a função com o mesmo valor novamente, o resultado será obtido do dicionário, em vez de ser recalculado. Embora `sin()` faça cálculos muito rapidamente, algumas funções sofisticadas, que envolvam cálculos mais complicados, poderiam demorar mais, e é nesses casos que a memoização realmente se destaca.

Se você já leu a respeito dos decoradores (se não leu, veja a seção “Decoradores e quando usá-los” na página 126), talvez veja uma oportunidade perfeita para utilizá-los nesse caso – e você tem

razão. O PyPI lista algumas implementações de memoização por meio de decoradores, variando de casos bem simples até os mais complexos e completos.

A partir de Python 3.3, o módulo `functools` disponibiliza um decorador de cache *LRU* (Least Recently Used, ou Menos Recentemente Usados). Ele oferece a mesma funcionalidade da memoização, mas com a vantagem de limitar o número de entradas no cache, removendo as entradas menos recentemente usadas quando o cache alcançar seu tamanho máximo. O módulo também fornece estatísticas sobre acertos e erros no cache (se um dado estava ou não no cache acessado), entre outros dados. Em minha opinião, essas estatísticas são mandatórias ao implementar um cache como esse. A eficácia no uso da memoização, ou de qualquer técnica de caching, está na capacidade de avaliar seu uso e a sua utilidade.

A Listagem 10.18 mostra como usar o método `functools.lru_cache()` para implementar a memoização de uma função. Ao ser decorada, a função adquire um método `cache_info()`, que poderá ser chamado para obter estatísticas sobre o uso do cache.

Listagem 10.18 – Inspeccionando as estatísticas de cache

```
>>> import functools
>>> import math
>>> @functools.lru_cache(maxsize=2)
... def memoized_sin(x):
...     return math.sin(x)
...
>>> memoized_sin(2)
0.9092974268256817
>>> memoized_sin.cache_info()
CacheInfo(hits=0, misses=1, maxsize=2, currsize=1)
>>> memoized_sin(2)
0.9092974268256817
>>> memoized_sin.cache_info()
CacheInfo(hits=1, misses=1, maxsize=2, currsize=1)
>>> memoized_sin(3)
0.1411200080598672
>>> memoized_sin.cache_info()
CacheInfo(hits=1, misses=2, maxsize=2, currsize=2)
```

```
>>> memoized_sin(4)
-0.7568024953079282
>>> memoized_sin.cache_info()
CacheInfo(hits=1, misses=3, maxsize=2, currsize=2)
>>> memoized_sin(3)
0.1411200080598672
>>> memoized_sin.cache_info()
CacheInfo(hits=2, misses=3, maxsize=2, currsize=2)
>>> memoized_sin.cache_clear()
>>> memoized_sin.cache_info()
CacheInfo(hits=0, misses=0, maxsize=2, currsize=0)
```

A Listagem 10.18 mostra como seu cache está sendo usado e permite dizer se há otimizações a serem feitas. Por exemplo, se o número de erros for alto quando o cache não estiver cheio, o cache talvez seja inútil, pois os argumentos passados para a função nunca são idênticos. Isso ajudará a determinar o que deve e o que não deve ser memoizado!

Python mais rápido com o PyPy

O *PyPy* é uma implementação eficiente da linguagem Python e obedece a certos padrões: você deve ser capaz de executar qualquer programa Python com ele. De fato, a implementação canônica de Python, o CPython – assim chamado porque está escrito em C – pode ser muito lenta. A ideia por trás do PyPy foi escrever um interpretador Python com o próprio Python. Com o tempo, ele evoluiu e passou a ser escrito em RPython, que é um subconjunto restrito da linguagem Python.

O RPython impõe restrições à linguagem Python, de modo que o tipo de uma variável possa ser inferido em tempo de compilação. O código RPython é traduzido para código C, que é compilado para construir o interpretador. O RPython poderia, é claro, ser usado para implementar outras linguagens além de Python.

O interessante no PyPy, além do desafio técnico, é que, atualmente, ele está em uma fase na qual pode atuar como um substituto mais rápido para CPython. O PyPy tem um compilador *JIT* (*Just-In-Time*)

embutido; em outras palavras, ele permite que o código execute mais rápido, combinando a velocidade do código compilado com a flexibilidade da interpretação.

Quão rápido? Isso depende, mas, para um código algorítmico puro, é muito mais rápido. Para um código mais genérico, o PyPy afirma que consegue ser três vezes mais rápido que o CPython na maioria das vezes. Infelizmente, o PyPy também tem algumas das limitações de CPython, incluindo o *GIL* (Global Interpreter Lock, ou Trava Global do Interpretador), que permite que apenas uma thread execute em um determinado instante.

Embora não seja uma técnica estritamente de otimização, ter como meta o PyPy como uma das implementações de Python aceitas pode ser uma boa ideia. Para fazer do PyPy uma implementação aceita, você deve garantir que testará o seu software com o PyPy, assim como faria com CPython. No Capítulo 6, discutimos o *tox* (veja a seção “Usando o *virtualenv* com o *tox*” na página 118), que aceita a criação de ambientes virtuais que utilizem o PyPy, assim como o faz para qualquer versão de CPython, portanto configurar o suporte para PyPy deverá ser bem simples.

Testar o suporte para PyPy logo no início do projeto garante que não haverá muito trabalho a ser feito em uma etapa posterior caso você decida que quer executar seu software com o PyPy.

NOTA No projeto Hy discutido no Capítulo 9, adotamos essa estratégia com sucesso desde o início. O Hy sempre aceitou o PyPy e todas as demais versões de CPython sem muitos problemas. Por outro lado, o OpenStack não fez isso em seus projetos e, como resultado, atualmente apresenta impedimentos por causa de vários paths de código e dependências que não funcionam com o PyPy por diversos motivos; não foi exigido que esses projetos fossem totalmente testados desde as suas primeiras etapas.

O PyPy é compatível com Python 2.7 e com Python 3.5, e seu compilador JIT funciona com arquiteturas de 32 e 64 bits, x86 e ARM, além de diversos sistemas operacionais (Linux, Windows e Mac OS X). O PyPy muitas vezes acaba se defasando em relação ao CPython no que diz respeito às funcionalidades, mas o alcança com regularidade. A menos que o seu projeto dependa dos recursos

mais recentes de CPython, essa defasagem não será um problema.

Evitando cópias com o protocolo de buffer

Muitas vezes, os programas precisam lidar com quantidades enormes de dados na forma de grandes arrays de bytes. Lidar com um volume grande de entrada na forma de strings pode ser muito ineficiente se você começar a manipulá-las copiando, fatiando e modificando os dados.

Considere um pequeno programa que leia um arquivo grande de dados binários e copie parcialmente esses dados para outro arquivo. Para analisar o uso de memória desse programa, usaremos o `memory_profiler`, como fizemos antes. O script para copiar parcialmente o arquivo está sendo exibido na Listagem 10.19.

Listagem 10.19 – Copiando parcialmente um arquivo

```
@profile
def read_random():
    with open("/dev/urandom", "rb") as source:
        content = source.read(1024 * 10000)
        content_to_write = content[1024:]
        print("Content length: %d, content to write length %d" %
              (len(content), len(content_to_write)))
    with open("/dev/null", "wb") as target:
        target.write(content_to_write)

if __name__ == '__main__':
    read_random()
```

A execução do programa que está na Listagem 10.19 usando o `memory_profiler` gera a saída exibida na Listagem 10.20.

Listagem 10.20 – Profiling de memória da cópia parcial do arquivo

```
$ python -m memory_profiler memoryview/copy.py
Content length: 10240000, content to write length 10238976
Filename: memoryview/copy.py
```

```
Mem usage Increment Line Contents
@profile
```

```

9.883 MB 0.000 MB def read_random():
9.887 MB 0.004 MB     with open("/dev/urandom", "rb") as source:
19.656 MB 9.770 MB         content = source.read(1024 * 10000)❶
29.422 MB 9.766 MB         content_to_write = content[1024:]❷
29.422 MB 0.000 MB         print("Content length: %d, content to write length %d" %
29.434 MB 0.012 MB             (len(content), len(content_to_write)))
29.434 MB 0.000 MB         with open("/dev/null", "wb") as target:
29.434 MB 0.000 MB             target.write(content_to_write)

```

De acordo com a saída, o programa lê 10MB de `/dev/urandom` ❶. Python precisa alocar aproximadamente 10MB de memória para armazenar esses dados na forma de string. Em seguida, o bloco de dados completo é copiado, exceto o primeiro KB ❷.

O aspecto interessante na Listagem 10.20 é que o uso de memória do programa aumenta em cerca de 10MB quando a variável `content_to_write` é criada. Com efeito, o operador `slice` copia o conteúdo todo, exceto o primeiro KB, para um novo objeto string, alocando uma porção grande de 10MB.

Executar esse tipo de operação com arrays grandes de bytes será um desastre, pois porções enormes de memória serão alocadas e copiadas. Se você tem experiência em escrever código C, saberá que usar a função `memcpy()` tem um custo significativo no que concerne tanto ao uso de memória como ao desempenho em geral.

Contudo, como programador C, você também saberá que strings são arrays de caracteres, e que nada impedirá você de olhar somente para uma *parte* de um array sem copiá-lo. Isso pode ser feito com o uso de uma aritmética básica de ponteiros, supondo que a string como um todo esteja em uma área de memória contígua.

É possível fazer o mesmo em Python usando objetos que implementem o *protocolo de buffer*. O protocolo de buffer está definido na PEP 3118, na forma de uma API C que deve ser implementada em diversos tipos para que esses disponibilizem esse protocolo. A classe `string`, por exemplo, implementa esse protocolo.

Ao implementar esse protocolo em um objeto, você poderá utilizar o construtor da classe `memoryview` para criar um objeto `memoryview`, o qual

fará referência à memória do objeto original. Por exemplo, a Listagem 10.21 mostra como usar `memoryview` para acessar uma fatia de uma string sem fazer cópias:

Listagem 10.21 – Usando `memoryview` para evitar uma cópia dos dados

```
>>> s = b"abcdefgh"
>>> view = memoryview(s)
>>> view[1]
❶ 98 <1>
>>> limited = view[1:3]
>>> limited
<memory at 0x7fca18b8d460>
>>> bytes(view[1:3])
b'bc'
```

Em ❶, vemos o código ASCII para a letra *b*. Na Listagem 10.21, estamos usando o fato de o próprio operador `slice` do objeto `memoryview` devolver um objeto `memoryview`. Isso significa que ele *não* copia nenhum dado, mas simplesmente referencia uma fatia em particular dele, economizando a memória que seria utilizada em uma cópia. A Figura 10.2 ilustra o que acontece na Listagem 10.21.

Podemos reescrever o programa da Listagem 10.19, desta vez referenciando os dados que queremos escrever usando um objeto `memoryview` em vez de alocar uma nova string.

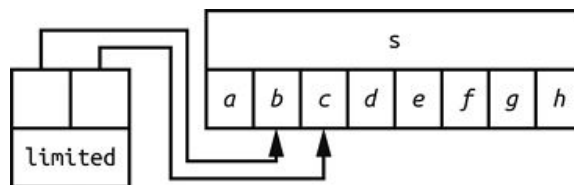


Figura 10.2 – Usando `slice` em objetos `memoryview`.

Listagem 10.22 – Copiando parcialmente um arquivo usando `memoryview`

```
@profile
def read_random():
    with open("/dev/urandom", "rb") as source:
        content = source.read(1024 * 10000)
        content_to_write = memoryview(content)[1024:]
```

```

print("Content length: %d, content to write length %d" %
      (len(content), len(content_to_write)))
with open("/dev/null", "wb") as target:
    target.write(content_to_write)

if __name__ == '__main__':
    read_random()

```

O programa na Listagem 10.22 utiliza metade da memória da primeira versão que está na Listagem 10.19. Podemos ver isso testando-o com `memory_profiler` novamente, assim:

```

$ python -m memory_profiler memoryview/copy-memoryview.py
Content length: 10240000, content to write length 10238976
Filename: memoryview/copy-memoryview.py

```

```

Mem usage Increment Line Contents
      @profile
9.887 MB 0.000 MB def read_random():
9.891 MB 0.004 MB ❶ with open("/dev/urandom", "rb") as source:
19.660 MB 9.770 MB ❷     content = source.read(1024 * 10000)
19.660 MB 0.000 MB         content_to_write = memoryview(content)[1024:]
19.660 MB 0.000 MB         print("Content length: %d, content to write length %d" %
19.672 MB 0.012 MB             (len(content), len(content_to_write)))
19.672 MB 0.000 MB         with open("/dev/null", "wb") as target:
19.672 MB 0.000 MB             target.write(content_to_write)

```

Esses resultados mostram que estamos lendo 10.000KB de `/dev/urandom` e não estamos fazendo muita coisa com eles ❶. Python precisa alocar 9,77MB de memória para armazenar esses dados como uma string ❷.

Referenciamos o bloco completo de dados, menos o primeiro KB, pois não escreveremos esse primeiro KB no arquivo de destino. Por não estarmos fazendo uma cópia, não utilizamos mais memória!

Esse tipo de truque é particularmente conveniente quando lidamos com sockets. Ao enviar dados por meio de um socket, é possível que esses dados sejam separados entre as chamadas, em vez de serem enviados em uma única chamada: os métodos `socket.send` devolvem o tamanho dos dados que puderam ser enviados pela rede, e esse tamanho poderá ser menor do que os dados que

deveriam ser enviados. A Listagem 10.23 mostra como essa situação é geralmente tratada.

Listagem 10.23 – Enviando dados por meio de um socket

```
import socket
s = socket.socket(...)
s.connect(...)
❶ data = b"a" * (1024 * 100000) <1>
while data:
    sent = s.send(data)
    ❷ data = data[sent:] <2>
```

Inicialmente, criamos um objeto `bytes` contendo a letra `a` mais de 100 milhões de vezes ❶. Em seguida, removemos os primeiros `sent` bytes ❷.

Ao usar um sistema implementado na Listagem 10.23, um programa copiará os dados várias vezes, até que o socket tenha enviado tudo. Podemos alterar o programa da Listagem 10.23 para que utilize `memoryview` a fim de ter a mesma funcionalidade sem que haja cópias e, desse modo, teremos um melhor desempenho, conforme mostra a Listagem 10.24.

Listagem 10.24 – Enviando dados por meio de um socket usando `memoryview`

```
import socket
s = socket.socket(...)
s.connect(...)
❶ data = b"a" * (1024 * 100000) <1>
mv = memoryview(data)
while mv:
    sent = s.send(mv)
    ❷ mv = mv[sent:] <2>
```

Em primeiro lugar, criamos um objeto `bytes` contendo a letra `a` mais de 100 milhões de vezes ❶. Em seguida, criamos um objeto `memoryview` que aponta para os dados restantes a serem enviados, em vez de copiá-los ❷. Esse programa não fará nenhuma cópia, portanto, não utilizará mais memória além dos 100MB inicialmente necessários para a variável `data`.

Vimos como os objetos `memoryview` podem ser usados para escrever dados de modo eficiente, e esse mesmo método pode ser utilizado para ler dados. A maioria das operações de E/S em Python sabe lidar com objetos que implementam o protocolo de buffer: elas conseguem ler desses objetos, além de escrever neles. Nesse caso, não precisaremos de objetos `memoryview`; podemos simplesmente pedir que uma função de E/S escreva em nosso objeto pré-allocado, conforme mostra a Listagem 10.25.

Listagem 10.25 – Escrevendo em um bytearray pré-allocado

```
>>> ba = bytearray(8)
>>> ba
bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00')
>>> with open("/dev/urandom", "rb") as source:
...     source.readinto(ba)
...
8
>>> ba
bytearray(b'm.z\x8d\x0f\xa1')
```

Na Listagem 10.25, ao usar o método `readinto()` do arquivo aberto, Python pode ler diretamente os dados do arquivo e escrevê-los em um `bytearray` pré-allocado. Com técnicas como essas, é fácil pré-alocar um buffer (como você faria em C para atenuar o número de chamadas a `malloc()`) e preenchê-lo conforme for mais conveniente para você. Ao usar `memoryview`, você poderá colocar os dados em qualquer ponto da área de memória, como mostra a Listagem 10.26.

Listagem 10.26 – Escrevendo em uma posição arbitrária de bytearray

```
>>> ba = bytearray(8)
❶ >>> ba_at_4 = memoryview(ba)[4:]
>>> with open("/dev/urandom", "rb") as source:
❷ ...     source.readinto(ba_at_4)
...
4
>>> ba
bytearray(b'\x00\x00\x00\x00\x0b\x19\xae\xb2')
```

Referenciamos o `bytearray` a partir do offset 4 até o final ❶. Em

seguida, escrevemos o conteúdo de `dev/urandom` a partir do offset 4 até o final de `bytearray`, lendo efetivamente apenas 4 bytes ❷.

O protocolo de buffer é extremamente importante para que haja pouco overhead de memória e para ter ótimos desempenhos. Como Python oculta todas as alocações de memória, os desenvolvedores tendem a se esquecer do que acontece internamente, cobrando um custo alto de seus programas no que diz respeito à velocidade!

Tanto os objetos do módulo `array` como as funções do módulo `struct` são capazes de lidar corretamente com o protocolo de buffer e, desse modo, têm um desempenho eficiente quando quiserem evitar cópias.

Resumo

Conforme vimos neste capítulo, há muitas maneiras de deixar um código Python mais rápido. Escolher a estrutura de dados correta e utilizar os métodos corretos para manipular os dados pode ter um impacto enorme no uso de CPU e da memória. É por isso que é importante saber o que acontece internamente em Python.

No entanto, a otimização jamais deve ser feita prematuramente, sem antes gerar um profiling adequado. É muito fácil desperdiçar tempo reescrevendo um código que mal seja utilizado, substituindo-o por uma variante mais rápida, porém ignorando pontos críticos importantes. Não deixe de verificar o panorama geral.

Victor Stinner fala sobre otimização

Victor é hacker de Python há muito tempo, colaborador do núcleo e autor de vários módulos Python. É autor da PEP 454 escrita em 2013, que propôs um novo módulo `tracemalloc` para trace da alocação de blocos de memória em Python, além de ter escrito um otimizador simples de AST chamado FAT. Também contribui regularmente para a melhoria do desempenho de CPython.

Qual seria uma boa estratégia inicial para otimizar um código

Python?

A estratégia é a mesma em Python e em outras linguagens. Em primeiro lugar, você precisa de um caso de uso bem definido a fim de ter um benchmark estável e reproduzível. Sem um benchmark confiável, experimentar diferentes otimizações pode ser um desperdício de tempo e resultar em uma otimização prematura. Otimizações inúteis podem piorar o código, deixá-lo menos legível ou até mesmo mais lento. Uma otimização útil deve agilizar o programa em no mínimo 5% para que valha a pena ser feita.

Se uma parte específica do código for identificada como “lenta”, um benchmark deverá ser preparado para esse código. Um benchmark em uma função pequena em geral é chamado de *microbenchmark*. O aumento de velocidade deve ser de pelo menos 20%, talvez 25%, para justificar uma otimização em um microbenchmark.

Talvez seja interessante executar um benchmark em computadores, sistemas operacionais ou compiladores diferentes. Por exemplo, o desempenho de `realloc()` pode variar entre Linux e Windows.

Quais são as ferramentas recomendadas para o profiling ou a otimização de códigos Python?

Python 3.3 tem uma função `time.perf_counter()` para medir o tempo decorrido em um benchmark. Ela tem a melhor resolução disponível.

Um teste deve ser executado mais de uma vez; três vezes é o mínimo, e cinco pode ser suficiente. Repetir um teste faz com que o cache de disco e de CPU enchem. Eu prefiro manter os tempos mínimos; outros desenvolvedores preferem a média geométrica.

No caso de microbenchmarks, o módulo `timeit` é fácil de usar e fornece resultados rapidamente, porém esses não são confiáveis com parâmetros default. Os testes devem ser repetidos manualmente para obter resultados estáveis.

A otimização pode consumir bastante tempo, portanto, é melhor manter o foco em funções que utilizem mais capacidade de CPU. Para identificar essas funções, Python tem os módulos `cProfile` e `profile` para registrar a quantidade de tempo que cada função consome.

Você conhece algum truque de Python capaz de melhorar o desempenho?

Você deve reutilizar a Biblioteca-Padrão o máximo possível – ela foi bem testada e, em geral, é eficiente. Tipos embutidos de Python são implementados em C e têm um bom desempenho. Utilize o contêiner correto para ter o melhor desempenho; Python oferece vários tipos de contêineres: `dict`, `list`, `deque`, `set` e assim por diante.

Há alguns hacks para otimizar Python, mas você deve evitá-los, pois deixam o código menos legível em troca de pouco ganho de velocidade.

O Zen de Python (PEP 20) afirma que “There should be one – and preferably only one – obvious way to do it” (Deve haver uma – e, de preferência, somente uma – maneira óbvia de fazer algo). Na prática, há diferentes maneiras de escrever um código Python, e os desempenhos não são iguais. Confie somente nos benchmarks em seu caso de uso.

Quais são as áreas de Python que apresentam os piores desempenhos, e com as quais devemos tomar cuidado?

Em geral, prefiro não me preocupar com o desempenho quando desenvolvo uma nova aplicação. Uma otimização prematura é a raiz de todos os males. Se identificar funções lentas, mude o algoritmo. Se o algoritmo e os tipos de contêineres foram bem selecionados, você poderia reescrever pequenas funções em C para ter o melhor desempenho possível.

Um gargalo em CPython é o Global Interpreter Lock (Trava Global do Interpretador), conhecido como GIL. Duas threads não podem

executar um bytecode Python ao mesmo tempo. Contudo, essa limitação só importará se duas threads estiverem executando um código Python puro. Se a maior parte do processamento estiver em chamadas de função, e essas funções liberarem o GIL, então ele não será o gargalo. Por exemplo, a maioria das funções de E/S libera o GIL.

O módulo de multiprocessing pode ser facilmente utilizado para contornar o GIL. Outra opção, mais complexa para implementar, é escrever um código assíncrono. Os projetos Twisted, Tornado e Tulip, que são bibliotecas voltadas para rede, fazem uso dessa técnica.

Quais são alguns dos erros de desempenho vistos frequentemente?

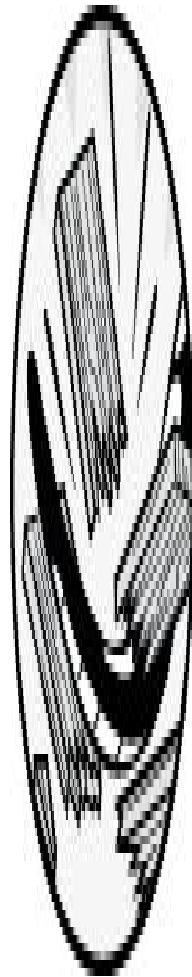
Quando não se compreende muito bem a linguagem Python, um código ineficiente poderá ser escrito. Por exemplo, já vi `copy.deepcopy()` sendo usado de forma indevida, quando não havia necessidade de nenhuma cópia.

Outro fator que prejudica bastante o desempenho é uma estrutura de dados ineficiente. Com menos de 100 itens, o tipo do contêiner não terá nenhum impacto no desempenho. Com mais itens, a complexidade de cada operação (`add`, `get`, `delete`) e seus efeitos devem ser conhecidos.

¹ Donald Knuth, “Structured Programming with go to Statements” (Programação estruturada com instruções go to), *ACM Computing Surveys* 6, n. 4 (1974): 261–301.

11

ESCALABILIDADE E ARQUITETURA



Cedo ou tarde, seu processo de desenvolvimento terá de levar em consideração a resiliência e a escalabilidade. A escalabilidade, a concorrência e o paralelismo de uma aplicação dependem, em boa medida, de sua arquitetura e do design iniciais. Conforme veremos neste capítulo, há alguns paradigmas – como o

multithreading – que não se aplicam corretamente a Python, enquanto outras técnicas, como a arquitetura orientada a serviços, funcionam melhor.

Discutir a escalabilidade de forma completa exigiria um livro inteiro e, de fato, o assunto já foi abordado em vários livros. Este capítulo discute os fundamentos básicos da escalabilidade, mesmo que você não esteja planejando criar aplicações com milhões de usuários.

Multithreading em Python e suas limitações

Por padrão, os processos Python executam somente em uma thread, chamada de *thread principal*. Essa thread executa o código em um único processador. O *multithreading* é uma técnica de programação que permite que um código execute de forma concorrente em um único processo Python, executando várias threads simultaneamente. É o principal método por meio do qual podemos introduzir a concorrência em Python. Se o computador estiver equipado com vários processadores, podemos até mesmo fazer uso do *paralelismo*, utilizando threads em paralelo em diversos processadores a fim de deixar a execução do código mais rápida.

O multithreading é mais comumente usado (embora nem sempre de forma apropriada) quando:

- É necessário executar tarefas em segundo plano ou orientadas a E/S sem interromper a execução de sua thread principal. Por exemplo, o laço principal de uma interface gráfica de usuário está ocupado à espera de um evento (por exemplo, um clique do usuário ou uma entrada do teclado), mas o código precisa executar outras tarefas.
- É necessário distribuir a carga de trabalho entre várias CPUs.

O primeiro cenário é um bom caso geral para o uso de multithreading. Embora implementar o multithreading nessa circunstância possa introduzir uma complexidade adicional, é possível controlar o gerenciamento do multithreading, e o

desempenho provavelmente não seria prejudicado, a menos que a carga de trabalho da CPU fosse intensa. O ganho de desempenho obtido com o uso da concorrência em cargas de trabalho com E/S intensas se tornará mais interessante se a E/S tiver latência alta: quanto maior a frequência com a qual você tiver de esperar para ler ou escrever, maiores serão as vantagens de executar outras tarefas nesse ínterim.

No segundo cenário, talvez você queira iniciar uma nova thread a cada nova requisição, em vez de tratá-las uma de cada vez. Esse parece ser um bom uso do multithreading. Contudo, se sua carga de trabalho for distribuída dessa forma, você vai deparar com o *GIL* (Global Interpreter Lock, ou Trava Global do Interpretador), um lock (trava) que deverá ser adquirido sempre que CPython tiver de executar um bytecode. O lock implica que somente uma thread poderá ter o controle do interpretador Python em um determinado instante. Essa regra foi originalmente introduzida para evitar condições de concorrência (race conditions), mas, infelizmente, significa que, se tentar escalar sua aplicação fazendo com que ela execute várias threads, você estará sempre limitado por esse lock global.

Portanto, embora usar threads possa parecer a solução ideal, a maioria das aplicações que executam requisições em várias threads tem dificuldade para atingir 150% de uso de CPU, ou o equivalente ao uso de 1,5 cores (núcleos de CPU). A maioria dos computadores tem de 4 a 8 cores, e os servidores oferecem de 24 a 48 cores, porém o GIL impede que Python utilize a CPU completa. Há algumas iniciativas em andamento para remoção do GIL, mas o esforço é extremamente complexo, pois exige comprometimentos quanto ao desempenho e à compatibilidade com versões anteriores.

Embora CPython seja a implementação mais comum para a linguagem Python, há outras que não têm um GIL. O *Jython*, por exemplo, é capaz de executar várias threads em paralelo de modo eficiente. Infelizmente, projetos como o Jython, por sua própria

natureza, têm uma defasagem em relação a CPython e, desse modo, não são alvos realmente convenientes; inovações surgem em CPython, e as demais implementações simplesmente seguem seus passos.

Vamos, portanto, rever nossos dois casos de uso com o que sabemos agora e achar uma melhor solução:

- Quando for necessário executar tarefas em segundo plano, você *pode* usar multithreading, mas a solução mais fácil será criar a sua aplicação com base em um laço de eventos. Há muitos módulos Python que possibilitam isso, e o módulo padrão atualmente é o `asyncio`. Existem também frameworks, como o Twisted, desenvolvidos com base no mesmo conceito. Os frameworks mais sofisticados permitirão que você tenha acesso aos eventos com base em sinais, timers e atividades de descritores de arquivos – falaremos disso mais adiante neste capítulo, na seção “Arquitetura orientada a eventos” na página 219.
- Quando for necessário distribuir a carga de trabalho, usar vários processos será o método mais eficiente. Veremos essa técnica na próxima seção.

Os desenvolvedores sempre devem pensar duas vezes antes de usar multithreading. Como exemplo, certa vez usei multithreading para despachar jobs no rebuild – um daemon para Debian que escrevi alguns anos atrás. Embora parecesse ser conveniente ter uma thread diferente para controlar cada job de construção em execução, eu caí rapidamente na armadilha do paralelismo de threading de Python. Se tivesse a chance de começar de novo, eu criaria algo com base no tratamento de eventos assíncronos ou no multiprocessamento, e não teria de me preocupar com o GIL.

O multithreading é complexo, e é difícil criar aplicações com várias threads. É necessário lidar com sincronização e locking de threads, o que significa que haverá muitas oportunidades para a introdução de bugs. Considerando o pequeno ganho geral, é melhor pensar duas vezes antes de investir muitos esforços nessa abordagem.

Multiprocessamento versus multithreading

Considerando que o GIL impede que o multithreading seja uma boa solução para a escalabilidade, você deve voltar os olhos para a solução alternativa oferecida pelo pacote de *multiprocessamento* de Python. O pacote expõe o mesmo tipo de interface que você teria ao usar o módulo de multithreading, exceto que ele inicia novos *processos* (com `os.fork()`), em vez de iniciar novas threads de sistema.

A Listagem 11.1 mostra um exemplo simples no qual um milhão de inteiros aleatórios são somados oito vezes, com essa atividade distribuída em oito threads ao mesmo tempo.

Listagem 11.1 – Usando multithreading para atividades concorrentes

```
import random
import threading
results = []
def compute():
    results.append(sum(
        [random.randint(1, 100) for i in range(1000000)]))
workers = [threading.Thread(target=compute) for x in range(8)]
for worker in workers:
    worker.start()
for worker in workers:
    worker.join()
print("Results: %s" % results)
```

Na Listagem 11.1, criamos oito threads utilizando a classe `threading.Thread` e as armazenamos no array `workers`. Essas threads executarão a função `compute()`. Elas utilizarão então o método `start()` para iniciar. O método `join()` retornará somente depois que a thread tiver acabado de executar. Nesse ponto, o resultado poderá ser exibido.

A execução desse programa devolve o seguinte:

```
$ time python worker.py
Results: [50517927, 50496846, 50494093, 50503078, 50512047, 50482863, 50543387,
50511493]
python worker.py 13.04s user 2.11s system 129% cpu 11.662 total
```

O programa foi executado em uma CPU com quatro cores ociosos,

o que significa que Python poderia ter usado até 400% da CPU, em potencial. No entanto, esses resultados mostram que ele foi claramente incapaz de fazer isso, mesmo com oito threads executando em paralelo. Em vez disso, o uso máximo de CPU foi de 129%, que corresponde apenas a 32% da capacidade do hardware (129/400).

Vamos agora reescrever essa implementação usando *multiprocessamento*. Em um caso simples como este, mudar para multiprocessamento é bem fácil, como mostra a Listagem 11.2.

Listagem 11.2 – Usando multiprocessamento para atividades concorrentes

```
import multiprocessing
import random

def compute(n):
    return sum(
        [random.randint(1, 100) for i in range(1000000)])

# Inicia 8 workers
pool = multiprocessing.Pool(processes=8)
print("Results: %s" % pool.map(compute, range(8)))
```

O módulo `multiprocessing` disponibiliza um objeto `Pool` que aceita o número de processos a serem iniciados como argumento. Seu método `map()` funciona do mesmo modo que o método `map()` nativo, exceto pelo fato de que um processo Python distinto será responsável pela execução da função `compute()`.

A execução do programa que está na Listagem 11.2 nas mesmas condições de execução do programa da Listagem 11.1 fornece o seguinte resultado:

```
$ time python workermp.py
Results: [50495989, 50566997, 50474532, 50531418, 50522470, 50488087,
0498016, 50537899]
python workermp.py 16.53s user 0.12s system 363% cpu 4.581 total
```

O multiprocessamento reduz o tempo de execução em 60%. Além do mais, conseguimos consumir até 363% da capacidade de CPU,

que é mais do que 90% (363/400) da capacidade de CPU do computador.

Sempre que você pensar que pode executar algum trabalho em paralelo, quase sempre será melhor contar com o multiprocessamento e fazer um fork de seus jobs a fim de distribuir a carga de trabalho entre os diversos cores da CPU. Essa não seria uma boa solução para tempos de execução muito baixos, pois o custo da chamada a `fork()` seria alto demais; contudo, para necessidades maiores de processamento, a solução funcionará bem.

Arquitetura orientada a eventos

Uma *programação orientada a eventos* é caracterizada pelo uso de eventos, por exemplo, uma entrada de usuário, para determinar como o controle flui por um programa, e é uma boa solução para organizar o fluxo do programa. Um programa orientado a eventos ouve vários eventos em uma fila e reage com base nesses eventos que chegam.

Suponha que você queira criar uma aplicação que preste atenção em uma conexão em um socket, e então processe a conexão recebida. Há, basicamente, três maneiras de abordar o problema:

- Faça fork de um novo processo sempre que uma nova conexão for estabelecida, contando com algo como o módulo `multiprocessing`.
- Inicie uma nova thread sempre que uma nova conexão for estabelecida, contando com algo como o módulo `threading`.
- Adicione essa nova conexão em seu laço de eventos e reaja ao evento que será gerado quando ele ocorrer.

Determinar como um computador moderno deve lidar com dezenas de milhares de conexões simultaneamente é conhecido como o *problema C10K*. Entre outros aspectos, as estratégias de resolução do C10K explicam que usar um laço de eventos para ouvir centenas de fontes de eventos escalará muito melhor do que, por exemplo, uma abordagem que utilize uma thread por conexão. Isso não quer dizer

que as duas técnicas não sejam compatíveis, mas significa que, em geral, você poderá substituir a abordagem de várias threads por um sistema orientado a eventos.

Uma arquitetura orientada a eventos utiliza um laço de eventos: o programa chama uma função que bloqueará a execução até que um evento seja recebido e esteja pronto para ser processado. A ideia é que seu programa possa se manter ocupado com outras tarefas enquanto espera que as entradas e as saídas estejam completas. Os eventos mais básicos são “dados prontos para leitura” e “dados prontos para escrita”.

Em Unix, as funções padrões para criar um laço de eventos como esse são as chamadas de sistema `select(2)` ou `poll(2)`. Essas funções esperam uma lista de descritores de arquivos para ouvir, e retornam assim que pelo menos um dos descritores estiver pronto para uma leitura ou uma escrita.

Em Python, podemos acessar essas chamadas de sistema com o módulo `select`. É fácil criar um sistema orientado a eventos com essas chamadas, embora fazer isso possa ser enfadonho. A Listagem 11.3 mostra um sistema orientado a eventos que executa nossa tarefa específica: ouvir um socket e processar qualquer conexão recebida.

Listagem 11.3 – Programa orientado a eventos que fica ouvindo à espera de conexões e as processa

```
import select
import socket

server = socket.socket(socket.AF_INET,
                       socket.SOCK_STREAM)
# Não bloqueia em operações de leitura/escrita
server.setblocking(0)

# Associa o socket à porta
server.bind(('localhost', 10000))
server.listen(8)

while True:
    # select() devolve 3 arrays contendo os objetos (sockets, arquivos...),
```



```
# prontos para serem lidos, escritos ou que geraram um erro
inputs,
outputs, excepts = select.select([server], [], [server])
if server in inputs:
    connection, client_address = server.accept()
    connection.send("hello!\n")
```

Na Listagem 11.3, um socket server é criado e definido como *não bloqueante*, o que significa que qualquer tentativa de operação de leitura ou de escrita nesse socket não bloqueará o programa. Se o programa tentar ler do socket quando não houver dados prontos para serem lidos, o método `recv()` do socket gerará um `OSError` informando que o socket não está pronto. Se não tivéssemos chamado `setblocking(0)`, o socket ficaria em modo bloqueante em vez de gerar um erro, e não é isso que queremos em nosso exemplo. Em seguida, o socket é associado a uma porta e fica ouvindo à espera de conexões, podendo acumular um máximo de oito conexões.

O laço principal é criado com o uso de `select()`, que recebe a lista dos descritores de arquivos que queremos ler (o socket, nesse caso), a lista dos descritores de arquivos nos quais queremos escrever (nenhum, nesse caso) e a lista dos descritores de arquivos dos quais queremos obter exceções (o socket, nesse caso). A função `select()` retorna assim que um dos descritores de arquivos selecionados estiver pronto para ler, estiver pronto para ser escrito ou tiver gerado uma exceção. Os valores devolvidos são listas de descritores de arquivos que correspondem às requisições. Desse modo, será fácil verificar se nosso socket está na lista de prontos para serem lidos e, em caso afirmativo, aceitar a conexão e enviar uma mensagem.

Outras opções e o asyncio

Como alternativa, há vários frameworks, como o Twisted ou o Tornado, que oferecem esse tipo de funcionalidade de forma mais

integrada; nesse aspecto, o Twisted é o verdadeiro padrão de mercado há anos. Bibliotecas C que exportam interfaces Python, como libevent, libev ou libuv, também oferecem laços de eventos muito eficientes.

Todas essas opções resolvem o mesmo problema. A desvantagem é que, embora haja uma diversidade enorme de opções, a maioria delas não apresenta interoperabilidade. Muitas também são *baseadas em callbacks*, o que significa que o fluxo do programa não será muito claro ao ler o código; você terá de pular para vários lugares diferentes para ler o programa.

Outra opção seriam as bibliotecas *gevent* ou *greenlet*, que evitam o uso de callbacks. No entanto, os detalhes de implementação incluem código CPython específico para x86 e uma modificação dinâmica das funções padrões durante a execução; isso significa que você não vai querer usar e manter um código que utilize essas bibliotecas no longo prazo.

Em 2012, Guido Van Rossum começou a trabalhar em uma solução de codinome *tulip*, documentada na PEP 3156 (<https://www.python.org/dev/peps/pep-3156>). O objetivo desse pacote era fornecer uma interface padrão para laços de eventos que fosse compatível com todos os frameworks e bibliotecas, e que apresentasse interoperabilidade.

O código do *tulip* foi então renomeado e incluído em Python 3.4 na forma do módulo *asyncio* e, atualmente, é o padrão de mercado na prática. Nem todas as bibliotecas são compatíveis com o *asyncio*, e a maioria dos bindings (vinculações) existentes precisa ser reescrita.

Em Python 3.6, o *asyncio* já estava muito bem integrado, a ponto de ter suas próprias palavras reservadas *await* e *async*, facilitando bastante o seu uso. A Listagem 11.4 mostra como a biblioteca *aiohttp*, que disponibiliza um binding HTTP assíncrono, pode ser usada com o *asyncio* para executar várias aquisições de páginas web de modo concorrente.

Listagem 11.4 – Adquirindo páginas web de modo concorrente com o aiohttp

```
import aiohttp
import asyncio

async def get(url):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return response

loop = asyncio.get_event_loop()

coroutines = [get("http://example.com") for _ in range(8)]

results = loop.run_until_complete(asyncio.gather(*coroutines))

print("Results: %s" % results)
```

Definimos a função `get()` como assíncrona, portanto, tecnicamente, ela é uma corrotina. Os dois passos da função `get()`, a conexão e a obtenção da página, são definidos como operações assíncronas, que cedem o controle a quem fez a chamada até que estejam concluídas. Isso possibilita ao `asyncio` escalonar outra corrotina em qualquer ponto. O módulo retomará a execução de uma corrotina quando a conexão for estabelecida ou a página estiver pronta para ser lida. As oito corrotinas são iniciadas e fornecidas ao laço de eventos ao mesmo tempo, e escaloná-las de modo eficiente é tarefa do `asyncio`.

O módulo `asyncio` é um ótimo framework para escrever códigos assíncronos e tirar proveito dos laços de eventos. Ele aceita arquivos, sockets e outros recursos, e há várias bibliotecas de terceiros disponíveis para dar suporte a diversos protocolos. Não hesite em usá-las!

Arquitetura orientada a serviços

Contornar as deficiências de Python no que concerne à escalabilidade pode parecer complicado. No entanto, Python é muito bom para implementar uma SOA (Service-Oriented Architecture, ou Arquitetura Orientada a Serviços): um estilo de design de software no qual diferentes componentes oferecem um conjunto de serviços por meio de um protocolo de comunicação. Por exemplo, o OpenStack faz uso de uma arquitetura SOA em todos os seus componentes. Os componentes utilizam HTTP REST para se comunicar com clientes externos (usuários finais) e um sistema abstrato de RPC (Remote Procedure Call, ou Chamada de Procedimento Remoto) desenvolvido com base no AMQP (Advanced Message Queuing Protocol, ou Protocolo Avançado de Fila de Mensagens).

Em suas situações de desenvolvimento, saber quais canais de comunicação devem ser usados entre esses blocos é, basicamente, uma questão de saber com quem você vai se comunicar.

Ao expor um serviço para o mundo externo, o canal preferido é o HTTP, particularmente para designs stateless (sem estados), como as arquiteturas em estilo REST (REpresentational State Transfer, ou Transferência de Representação de Estado). Esses tipos de arquitetura facilitam implantar, escalar, implantar e entender os serviços.

Contudo, ao expor e usar sua API internamente, o HTTP talvez não seja o melhor protocolo. Há vários outros protocolos de comunicação, e descrever, ainda que seja apenas um, de forma completa provavelmente seria assunto para um livro inteiro.

Em Python, há várias bibliotecas para criar um sistema RPC. O Kombu é interessante porque oferece um sistema de RPC com vários backends, sendo o protocolo AMQ o principal deles. Ele também aceita Redis, MongoDB, Beanstalk, Amazon SQS, CouchDB ou ZooKeeper.

No final das contas, você pode, indiretamente, ganhar muito no que concerne ao desempenho se usar uma arquitetura de baixo

acoplamento como essa. Se considerarmos que cada módulo disponibiliza e expõe uma API, poderemos executar vários daemons que também poderão expor essa API, permitindo ter vários processos – e, portanto, CPUs – para lidar com a carga de trabalho. Por exemplo, o *Apache httpd* criaria um *worker* usando um novo processo de sistema que cuidaria das novas conexões; poderíamos então despachar uma conexão para um *worker* diferente que estivesse executando no mesmo nó. Para isso, bastaria ter um sistema para despachar as tarefas aos nossos diversos *workers*, o que seria feito por essa API. Cada bloco será um processo Python distinto e, conforme vimos antes, essa abordagem é melhor do que a abordagem de multithreading para distribuir a carga de trabalho. Você poderá iniciar vários *workers* em cada nó. Mesmo que blocos stateless não sejam estritamente necessários, dê preferência a eles sempre que tiver a oportunidade.

Comunicação entre processos com o ZeroMQ

Conforme acabamos de discutir, um sistema de mensagens será sempre necessário na criação de sistemas distribuídos. Seus processos precisam se comunicar entre si a fim de trocar mensagens. O ZeroMQ é uma biblioteca de sockets capaz de atuar como um framework para concorrência. A Listagem 11.5 implementa o mesmo *worker* visto na Listagem 11.1, porém utiliza o ZeroMQ como uma forma de despachar as tarefas e fazer a comunicação entre os processos.

Listagem 11.5 – workers usando o ZeroMQ

```
import multiprocessing
import random
import zmq

def compute():
    return sum(
        [random.randint(1, 100) for i in range(1000000)])
```

```

def worker():
    context = zmq.Context()
    work_receiver = context.socket(zmq.PULL)
    work_receiver.connect("tcp://0.0.0.0:5555")
    result_sender = context.socket(zmq.PUSH)
    result_sender.connect("tcp://0.0.0.0:5556")
    poller = zmq.Poller()
    poller.register(work_receiver, zmq.POLLIN)

    while True:
        socks = dict(poller.poll())
        if socks.get(work_receiver) == zmq.POLLIN:
            obj = work_receiver.recv_pyobj()
            result_sender.send_pyobj(obj())

context = zmq.Context()
# Cria um canal para enviar uma tarefa a ser feita
❶ work_sender = context.socket(zmq.PUSH)
work_sender.bind("tcp://0.0.0.0:5555")
# Cria um canal para receber os resultados processados
❷ result_receiver = context.socket(zmq.PULL)
result_receiver.bind("tcp://0.0.0.0:5556")
# Inicia 8 workers
processes = []
for x in range(8):
    ❸ p = multiprocessing.Process(target=worker)
    p.start()
    processes.append(p)
# Envia 8 jobs
for x in range(8):
    work_sender.send_pyobj(compute)
# Lê 8 resultados

results = []
for x in range(8):
    ❹ results.append(result_receiver.recv_pyobj())
# Encerra todos os processos
for p in processes:
    p.terminate()
print("Results: %s" % results)

```

Criamos dois sockets: um para enviar a função (work_sender) ❶ e

outro para receber o job (`result_receiver`) ❷. Cada `worker` iniciado por `multiprocessing.Process` ❸ cria o próprio conjunto de sockets e os conecta ao processo mestre. O `worker` então executa qualquer que seja a função recebida e envia de volta o resultado. O processo mestre só precisa enviar oito jobs por meio do socket de envio e esperar que oito resultados sejam enviados de volta por meio do socket receptor ❹.

Conforme podemos ver, o ZeroMQ oferece um modo fácil de criar canais de comunicação. Decidi usar a camada de transporte TCP nesse caso a fim de ilustrar o fato de que poderíamos executar esse código em uma rede. Observe que o ZeroMQ também disponibiliza um canal para comunicação entre processos que funciona localmente (sem nenhuma camada de rede envolvida) utilizando sockets Unix. Obviamente, o protocolo de comunicação desse exemplo, criado com base no ZeroMQ, é bem simples para que fosse claro e conciso, mas não deve ser difícil imaginar a criação de uma camada de comunicação mais sofisticada com base nele. Também é fácil imaginar a criação de um sistema de comunicação totalmente distribuído entre aplicações, utilizando um sistema de mensagens via rede como o ZeroMQ ou o AMQP.

Observe que os protocolos como HTTP, ZeroMQ e AMQP não dependem da linguagem: você pode utilizar diferentes linguagens e plataformas para implementar cada parte de seu sistema. Embora todos concordem que Python seja uma boa linguagem, outras equipes poderão ter outras preferências, ou uma linguagem diferente talvez seja uma melhor solução para algumas partes do problema.

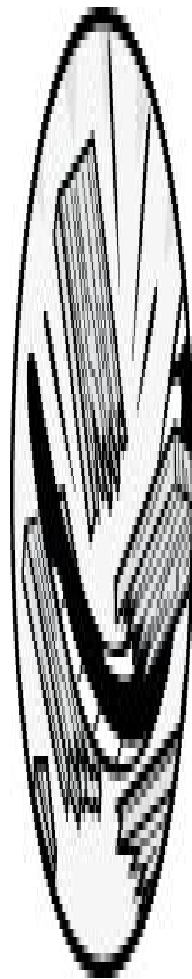
No final das contas, usar um sistema de transporte para desacoplar sua aplicação em várias partes é uma boa opção. Essa abordagem permite criar APIs tanto síncronas como assíncronas, que possam ser distribuídas em um ou vários milhares de computadores. Você não ficará preso a uma tecnologia ou a uma linguagem específica e, desse modo, poderá evoluir na direção correta.

Resumo

A regra geral em Python é usar threads somente para cargas de trabalho com E/S intensas, e passar para vários processos assim que houver uma carga de trabalho com uso intensivo de CPU. Cargas de trabalho distribuídas em uma escala maior – por exemplo, ao criar um sistema distribuído em uma rede – exigem bibliotecas externas e protocolos. Esses são aceitos por Python, mas são disponibilizados externamente.

12

GERENCIANDO BANCOS DE DADOS RELACIONAIS



As aplicações quase sempre terão de armazenar algum tipo de dado, e os desenvolvedores muitas vezes combinarão um RDBMS (Relational Database Management System, ou Sistema de Gerenciamento de Banco de Dados Relacional) com algum tipo de ferramenta ORM (Object Relational Mapping, ou Mapeamento

Objeto-Relacional). Os RDBMSs e os ORMs podem ser complicados, e não são um assunto predileto para muitos desenvolvedores, mas, cedo ou tarde, esse assunto deverá ser abordado.

RDBMSs, ORMs e quando usá-los

Um RDBMS é o banco de dados que armazena os dados relacionais de uma aplicação. Os desenvolvedores usarão uma linguagem como o SQL (Structured Query Language, ou Linguagem de Consulta Estruturada) para lidar com a álgebra relacional, e isso significa que uma linguagem como essa cuidará do gerenciamento dos dados e dos relacionamentos entre eles. Quando usados em conjunto, eles permitem que você armazene os dados e os consulte a fim de obter informações específicas do modo mais eficiente possível. Ter uma boa compreensão das estruturas dos bancos de dados relacionais – por exemplo, como utilizar uma normalização adequada ou quais são os diferentes tipos de serialização – evitará que você caia em muitas armadilhas. Obviamente esses assuntos mereceriam um livro próprio e não serão discutidos em sua totalidade neste capítulo; em vez disso, nosso foco estará no uso do banco de dados por meio de sua linguagem de programação usual, o SQL.

Os desenvolvedores talvez não queiram investir em aprender uma linguagem de programação totalmente nova para interagir com o RDBMS. Nesse caso, eles tendem a evitar completamente a escrita de queries SQL, contando com uma biblioteca que faça o trabalho para eles. As bibliotecas ORM são comumente encontradas nos ecossistemas das linguagens de programação, e Python não é uma exceção.

O propósito de um ORM é facilitar o acesso aos sistemas de bancos de dados, com a abstração do processo de criação de queries: ele gera o SQL, de modo que você não precise fazê-lo. Infelizmente, essa camada de abstração pode impedir que você execute tarefas

mais específicas e de mais baixo nível, as quais o ORM simplesmente não é capaz de fazer, por exemplo, escrever queries complexas.

Há também um conjunto específico de dificuldades quando usamos os ORMs em programas orientados a objetos, os quais são muito comuns, a ponto de serem coletivamente conhecidos como *diferença de impedância objeto-relacional* (object-relational impedance mismatch). Essa diferença de impedância ocorre porque os bancos de dados relacionais e os programas orientados a objetos têm representações diferentes dos dados, que não se mapeiam entre si de forma apropriada: mapear tabelas SQL para classes Python não trará resultados ideais, não importa o que você faça.

Conhecer SQL e os RDBMSs lhe permitirá escrever as próprias queries, sem ter de depender da camada de abstração para tudo.

No entanto, isso não quer dizer que você deva evitar totalmente os ORMs. As bibliotecas ORM podem ajudar na criação rápida de protótipos para o modelo de sua aplicação, e algumas bibliotecas até mesmo oferecem ferramentas úteis, por exemplo, para upgrades e downgrades de esquemas. É importante entender que utilizar um ORM não é um substituto para a aquisição de uma verdadeira compreensão dos RDBMSs: muitos desenvolvedores tentam resolver os problemas na linguagem de sua preferência, em vez de usar a API de seus modelos, e as soluções que criam são, na melhor das hipóteses, deselegantes.

NOTA Este capítulo parte do pressuposto de que você tem um conhecimento básico de SQL. Uma introdução às queries SQL e uma discussão sobre como as tabelas funcionam estão além do escopo deste livro. Se você não conhece SQL, recomendo que aprenda o básico antes de prosseguir. O livro *Practical SQL* de Anthony DeBarros (No Starch Press, 2018) é um bom ponto de partida.

Vamos ver um exemplo que mostra por que conhecer os RDBMSs pode ajudá-lo a escrever um código melhor. Suponha que você tenha uma tabela SQL para manter o registro de mensagens. Essa tabela tem uma coluna simples chamada `id`, que representa o ID de quem enviou uma mensagem e é a chave primária, e uma string

com o conteúdo da mensagem, da seguinte maneira:

```
CREATE TABLE message (  
  id serial PRIMARY KEY,  
  content text  
);
```

Queremos detectar quaisquer mensagens duplicadas que sejam recebidas e excluí-las do banco de dados. Para isso, um desenvolvedor típico escreveria um SQL usando um ORM, conforme mostra a Listagem 12.1.

Listagem 12.1 – Detectando e excluindo mensagens duplicadas com um ORM

```
if query.select(Message).filter(Message.id == some_id):  
    # Já temos a mensagem: ela está duplicada, ignora e informa  
    raise DuplicateMessage(message)  
else:  
    # Insere a mensagem  
    query.insert(message)
```

Esse código funcionará na maioria dos casos, mas apresenta algumas desvantagens significativas:

- A restrição de duplicação já está expressa no esquema SQL, portanto, há uma espécie de duplicação de código: usar PRIMARY KEY define implicitamente a unicidade do campo id.
- Se a mensagem ainda não estiver no banco de dados, esse código executará duas queries SQL: uma instrução SELECT e, em seguida, uma instrução INSERT. Executar uma query SQL pode demorar bastante e exige um acesso de ida e volta ao servidor SQL, introduzindo um atraso extra.
- O código não leva em consideração a possibilidade de alguém poder inserir uma mensagem duplicada depois de chamarmos `select_by_id()`, porém antes da chamada a `insert()`, o que faria o programa gerar uma exceção. Essa vulnerabilidade se chama *condição de concorrência* (race condition).

Há uma maneira muito melhor de escrever esse código, mas exige cooperação com o servidor RDBMS. Em vez de verificar se a

mensagem existe e então a inserir, podemos inseri-la de imediato e utilizar um bloco `try...except` para capturar um conflito de duplicação:

```
try:
    # Insere a mensagem
    message_table.insert(message)
except UniqueViolationError:
    # Está duplicada
    raise DuplicateMessage(message)
```

Nesse caso, inserir a mensagem diretamente na tabela funcionará sem problemas se a mensagem ainda não estiver presente. Se a mensagem já existir, o ORM gerará uma exceção informando a violação da restrição de unicidade. Esse método tem o mesmo efeito do código da Listagem 12.1, mas é mais eficiente e não haverá condições de concorrência. É um padrão muito simples, que não apresenta conflitos de espécie alguma com nenhum ORM. O problema é que os desenvolvedores tendem a tratar os bancos de dados SQL como áreas de armazenagem burras, em vez de vê-las como uma ferramenta que possa ser usada para garantir a devida integridade e consistência dos dados; conseqüentemente, eles poderão duplicar as restrições expressas em SQL no código de seu controlador, em vez de tê-las em seu modelo.

Tratar seu backend SQL como uma API do modelo é uma boa maneira de fazer um uso eficaz dele. Você pode manipular os dados armazenados em seu RDBMS com chamadas de funções simples, programadas em sua própria linguagem procedural.

Backends de bancos de dados

O ORM aceita vários backends de bancos de dados. Nenhuma biblioteca ORM oferece uma abstração completa para todos os recursos de um RDBMS, e simplificar o código para o RDBMS mais básico disponível tornaria impossível o uso de quaisquer funções sofisticadas de RDBMSs sem que houvesse uma quebra da camada de abstração. Até mesmo tarefas simples que não estão padronizadas em SQL, por exemplo, o tratamento de operações

com timestamps, são difíceis de lidar quando usamos um ORM. Isso será pior ainda se seu código for independente do RDBMS. É importante ter isso em mente ao escolher o RDBMS para a sua aplicação.

Isolar bibliotecas ORM (conforme descrito na seção “Bibliotecas externas” na página 35) ajudará a atenuar possíveis problemas. Essa abordagem lhe permite trocar facilmente a sua biblioteca ORM por outra caso haja necessidade, e otimizar seu uso do SQL identificando os locais com uso ineficiente de queries, o que permitirá que você deixe de lado a maior parte do boilerplate do ORM.

Por exemplo, você poderá usar seu ORM em um módulo de sua aplicação, como `myapp.storage`, para promover facilmente esse isolamento. Esse módulo deverá exportar funções e métodos que permitam manipular os dados apenas em um nível alto de abstração. O ORM deverá ser usado somente a partir desse módulo. A qualquer momento, você poderá incluir outro módulo que ofereça a mesma API e substituir `myapp.storage`.

A biblioteca ORM mais comum em Python (e que, sem dúvida, é o padrão de mercado na prática) é a `sqlalchemy`. Essa biblioteca aceita diversos backends e fornece abstrações para as operações mais comuns. Os upgrades de esquemas podem ser tratados por pacotes de terceiros como o `alembic` (<https://pypi.python.org/pypi/alembic/>).

Alguns frameworks, como o Django (<https://www.djangoproject.com>), disponibilizam as próprias bibliotecas ORM. Se você optar por usar um framework, uma atitude inteligente será utilizar a biblioteca embutida, pois, em geral, ela se integrará melhor ao framework, em comparação com uma biblioteca externa.

AVISO A arquitetura MVC (Module View Controller, ou Modelo Visão Controlador), que constitui a base de muitos frameworks, pode ser facilmente utilizada de forma indevida. Esses frameworks implementam (ou facilitam implementar) diretamente o ORM em seus modelos, mas sem um nível suficiente de abstração: qualquer código que você tiver em sua visão e nos controladores e que utilizem o modelo também usarão diretamente o ORM. Evite isso. Escreva um modelo de dados que *inclua* a biblioteca ORM, em vez de ser *constituído* por ela. Fazer isso facilitará os

testes e promoverá um isolamento, além de fazer com que trocar o ORM por outra tecnologia de armazenagem seja muita mais simples.

Streaming de dados com Flask e PostgreSQL

Nesta seção, mostrarei como é possível usar um dos recursos avançados do *PostgreSQL* para criar um sistema de streaming de eventos HTTP e ajudar a controlar sua armazenagem de dados.

Escrevendo a aplicação de streaming de dados

O propósito da microaplicação da Listagem 12.2 é armazenar mensagens em uma tabela SQL e fornecer acesso a essas mensagens por meio de uma API REST HTTP. Cada mensagem é composta de um número de canal, uma string para a origem e uma string com o conteúdo.

```
CREATE TABLE message (  
    id SERIAL PRIMARY KEY,  
    channel INTEGER NOT NULL,  
    source TEXT NOT NULL,  
    content TEXT NOT NULL  
);
```

Listagem 12.2 – Esquema de uma tabela SQL para armazenar mensagens

Também queremos fazer streaming dessas mensagens para o cliente a fim de que ele possa processá-las em tempo real. Para isso, utilizaremos as funcionalidades `LISTEN` e `NOTIFY` do *PostgreSQL*. Essas funcionalidades nos permitem ouvir as mensagens enviadas por uma função especificada por nós e que será executada pelo *PostgreSQL*:

```
❶ CREATE OR REPLACE FUNCTION notify_on_insert() RETURNS trigger AS $$  
❷ BEGIN  
    PERFORM pg_notify('channel_' || NEW.channel,  
        CAST(row_to_json(NEW) AS TEXT));
```

```
RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

Esse código cria uma função de trigger escrita em pl/pgsql, uma linguagem compreendida somente pelo PostgreSQL. Observe que também poderíamos ter escrito essa função em outras linguagens, por exemplo, no próprio Python, pois o PostgreSQL inclui o interpretador Python de modo a disponibilizar uma linguagem pl/python. A operação simples e única que executaremos em nosso caso não exige o uso de Python, portanto, ater-se ao pl/pgsql é uma decisão inteligente.

A função `notify_on_insert()` ❶ faz uma chamada a `pg_notify()` ❷, que é a função que realmente envia a notificação. O primeiro argumento é uma string que representa um *canal*, enquanto o segundo é uma string que contém o *payload* propriamente dito. Definimos o canal dinamicamente com base no valor da coluna referente ao canal, presente na linha. Nesse caso, o payload será a linha completa em formato JSON. Sim, o PostgreSQL sabe como converter uma linha em JSON de modo nativo!

Em seguida, queremos enviar uma mensagem de notificação a cada INSERT executado na tabela de mensagens, portanto, devemos disparar essa função quando esses eventos ocorrerem:

```
CREATE TRIGGER notify_on_message_insert AFTER INSERT ON message  
FOR EACH ROW EXECUTE PROCEDURE notify_on_insert();
```

A função agora estará vinculada e será executada a cada INSERT ocorrido com sucesso na tabela de mensagens.

Podemos conferir se ela está funcionando utilizando a operação LISTEN no psql:

```
$ psql  
psql (9.3rc1)  
SSL connection (cipher: DHE-RSA-AES256-SHA, bits: 256)  
Type "help" for help.
```

```
mydatabase=> LISTEN channel_1;  
LISTEN
```



```
mydatabase=> INSERT INTO message(channel, source, content)
mydatabase-> VALUES(1, 'jd', 'hello world');
INSERT 0 1
Asynchronous notification "channel_1" with payload
{"id":1,"channel":1,"source":"jd","content":"hello world"}"
received from server process with PID 26393.
```

Assim que a linha é inserida, a notificação é enviada, e podemos recebê-la por meio do cliente PostgreSQL. Agora, tudo que temos a fazer é criar a aplicação Python que fará o streaming desse evento, conforme vemos na Listagem 12.3.

Listagem 12.3 – Ouvindo e recebendo o stream de notificações

```
import psycopg2
import psycopg2.extensions
import select

conn = psycopg2.connect(database='mydatabase', user='myuser',
                        password='idkfa', host='localhost')

conn.set_isolation_level(
    psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT)

curs = conn.cursor()
curs.execute("LISTEN channel_1;")

while True:
    select.select([conn], [], [])
    conn.poll()
    while conn.notifies:
        notify = conn.notifies.pop()
        print("Got NOTIFY:", notify.pid, notify.channel, notify.payload)
```

A Listagem 12.3 faz a conexão com o PostgreSQL usando a biblioteca `psycopg2`. A biblioteca `psycopg2` é um módulo Python que implementa o protocolo de rede do PostgreSQL e nos permite conectar com um servidor PostgreSQL para enviar requisições SQL e receber os resultados. Poderíamos ter usado uma biblioteca que tivesse uma camada de abstração, como a `sqlalchemy`, mas bibliotecas com abstrações não oferecem acesso às funcionalidades de `LISTEN` e `NOTIFY` do PostgreSQL. É importante observar que ainda

é possível acessar a conexão subjacente com o banco de dados para executar o código ao utilizar uma biblioteca como a `sqlalchemy`, mas não faria sentido fazer isso neste exemplo, pois não precisamos de nenhuma das demais funcionalidades disponibilizadas pela biblioteca ORM.

O programa ouve `channel_1` e, assim que recebe uma notificação, ele a exibe na tela. Se executarmos o programa e inserirmos uma linha na tabela `message`, veremos a saída a seguir:

```
$ python listen.py
Got NOTIFY: 28797 channel_1
{"id":10,"channel":1,"source":"jd","content":"hello world"}
```

Assim que inserirmos a linha, o PostgreSQL executará o trigger e enviará uma notificação. Nosso programa a receberá e exibirá o payload da notificação; nesse caso, é a linha serializada para JSON. Agora podemos receber dados à medida que são inseridos no banco de dados, sem nenhuma requisição ou trabalho extras.

Criando a aplicação

A seguir, utilizaremos o *Flask* – um microframework HTTP simples – para criar nossa aplicação. Criaremos um servidor HTTP que faça streaming do fluxo de insert usando o protocolo de mensagens *Server-Sent Events* (Eventos Enviados pelo Servidor) definido no HTML5. Uma alternativa seria utilizar o *Transfer-Encoding: chunked* (Codificação de Transferência: em partes) definido pelo HTTP/1.1:

```
import flask
import psycopg2
import psycopg2.extensions
import select

app = flask.Flask(__name__)

def stream_messages(channel):
    conn = psycopg2.connect(database='mydatabase', user='mydatabase',
                           password='mydatabase', host='localhost')
    conn.set_isolation_level(
        psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT)
```

```

curs = conn.cursor()
curs.execute("LISTEN channel_%d;" % int(channel))

while True:
    select.select([conn], [], [])
    conn.poll()
    while conn.notifies:
        notify = conn.notifies.pop()
        yield "data: " + notify.payload + "\n\n"

@app.route("/message/<channel>", methods=['GET'])
def get_messages(channel):
    return flask.Response(stream_messages(channel),
                          mimetype='text/event-stream')

if __name__ == "__main__":
    app.run()

```

Essa aplicação é bastante simples: ela aceita streaming, mas não aceita nenhuma outra operação de obtenção de dados. Usamos o Flask para encaminhar a requisição HTTP GET */message/canal* ao nosso código de streaming. Assim que o código for chamado, a aplicação devolverá uma resposta com o mimetype `text/event-stream` e enviará de volta uma função geradora no lugar de uma string. O Flask chamará essa função e enviará o resultado sempre que o gerador fizer o `yield` de algo.

O gerador, `stream_messages()`, reutiliza o código que escrevemos antes para ouvir as notificações do PostgreSQL. Ele recebe o identificador do canal como argumento, ouve esse canal e, então, faz um `yield` do payload. Lembre-se de que usamos a função de codificação JSON do PostgreSQL na função de trigger, portanto já estamos recebendo dados JSON do PostgreSQL. Não é necessário fazer a transcodificação dos dados, pois não haverá problemas em enviar dados JSON para o cliente HTTP.

NOTA Para simplificar, esta aplicação de exemplo foi escrita em um único arquivo. Se fosse uma aplicação de verdade, eu passaria a implementação do tratamento da armazenagem para um módulo Python próprio.

Podemos agora executar o servidor:

```
$ python listen+http.py
```

```
* Running on http://127.0.0.1:5000/
```

Em outro terminal, podemos fazer a conexão e obter os eventos à medida que forem inseridos. Logo após a conexão, nenhum dado será recebido e a conexão permanecerá aberta:

```
$ curl -v http://127.0.0.1:5000/message/1
```

```
* About to connect() to 127.0.0.1 port 5000 (#0)
* Trying 127.0.0.1...
* Adding handle: conn: 0x1d46e90
* Adding handle: send: 0
* Adding handle: recv: 0
* Curl_addHandleToPipeline: length: 1
* - Conn 0 (0x1d46e90) send_pipe: 1, recv_pipe: 0
* Connected to 127.0.0.1 (127.0.0.1) port 5000 (#0)
> GET /message/1 HTTP/1.1
> User-Agent: curl/7.32.0
> Host: 127.0.0.1:5000
> Accept: */*
>
```

No entanto, assim que inserirmos algumas linhas na tabela `message`, começaremos a ver os dados chegando no terminal que está executando o `curl`. Em um terceiro terminal, inserimos uma mensagem no banco de dados:

```
mydatabase=> INSERT INTO message(channel, source, content)
mydatabase-> VALUES(1, 'jd', 'hello world');
INSERT 0 1
mydatabase=> INSERT INTO message(channel, source, content)
mydatabase-> VALUES(1, 'jd', 'it works');
INSERT 0 1
```

Eis a saída dos dados:

```
data: {"id":71,"channel":1,"source":"jd","content":"hello world"}
data: {"id":72,"channel":1,"source":"jd","content":"it works"}
```

Esses dados são exibidos no terminal que executa o `curl`. A conexão do `curl` com o servidor HTTP será mantida enquanto esperamos o próximo fluxo de mensagens. Implementamos um serviço de streaming sem fazer nenhum tipo de polling, criando um sistema

totalmente *baseado em pushes*, no qual as informações fluem naturalmente de um ponto para outro.

Uma implementação ingênua e, sem dúvida, mais portátil dessa aplicação seria executar uma instrução `SELECT` repetidamente em um laço para fazer polling de novos dados inseridos na tabela. Isso funcionaria para qualquer outro sistema de armazenagem que não tivesse suporte para um padrão de publicação-inscrição como este.

Dimitri Fontaine fala sobre bancos de dados

Dimitri é um Major Contributor (Colaborador Principal) proficiente para o PostgreSQL, trabalha na Citus Data e discute com outros gurus de bancos de dados na lista de discussão *pgsql-hackers*. Atuamos juntos em muitas aventuras de código aberto, e ele foi bastante gentil respondendo a algumas perguntas sobre o que você deve fazer ao lidar com bancos de dados.

Quais são os conselhos que você daria para os desenvolvedores que utilizam RDBMSs como seus backends de armazenagem?

Os RDBMSs foram inventados nos anos 1970 para solucionar alguns problemas comuns que atormentavam todos os desenvolvedores de aplicações naquela época, e os principais serviços implementados pelos RDBMSs não diziam respeito somente à armazenagem de dados.

De fato, os principais serviços oferecidos por um RDBMS são os seguintes:

- **Concorrência:** Acesse seus dados para leitura ou escrita com o máximo de threads de execução concorrentes que você quiser – o RDBMS está lá para cuidar disso de forma correta para você. Esta é a principal funcionalidade que você vai querer de um RDBMS.
- **Semântica da concorrência:** Os detalhes sobre o comportamento da concorrência ao usar um RDBMS são propostos por uma especificação de alto nível no que concerne à

atomicidade e ao isolamento, e estas talvez sejam as partes mais importantes do ACID (Atomicity, Consistency, Isolation, Durability, ou Atomicidade, Consistência, Isolamento, Durabilidade). A *atomicidade* é a propriedade segundo a qual, entre o instante em que você executa um BEGIN em uma transação e o instante em que ela é concluída (seja com um COMMIT ou com um ROLLBACK), nenhuma outra atividade concorrente no sistema terá permissão para saber o que você está fazendo – não importa o que seja. Ao usar um RDBMS adequado, inclua também a DDL (Data Definition Language, ou Linguagem de Definição de Dados), por exemplo, CREATE TABLE OU ALTER TABLE. O *isolamento* diz respeito àquilo que você tem permissão para observar sobre a atividade concorrente do sistema a partir de sua própria transação. O padrão SQL define quatro níveis de isolamento, conforme descritos na documentação do PostgreSQL (<http://www.postgresql.org/docs/9.2/static/transaction-iso.html>).

O RDBMS assume total responsabilidade pelos seus dados. Desse modo, ele permite que o desenvolvedor descreva as próprias regras de consistência e, em seguida, verificará se essas regras são válidas em momentos cruciais, por exemplo, no commit das transações ou nas fronteiras das instruções, dependendo da declaração de suas restrições quanto ao adiamento dessas verificações.

A primeira restrição que você pode impor aos seus dados é o formato esperado de entrada e saída, com a utilização do tipo de dado apropriado. Um RDBMS saberá trabalhar com tipos que vão muito além de texto, números e datas, e lidará de forma apropriada com datas que possam estar realmente em um calendário em uso atualmente.

Os tipos de dados, porém, não dizem respeito somente aos formatos de entrada e de saída. Eles também implementam comportamentos e algum nível de polimorfismo, pois qualquer pessoa espera que os testes básicos de igualdade sejam

específicos aos tipos de dado: não comparamos textos com números, datas com endereços IP, arrays com intervalos e assim por diante do mesmo modo.

Proteger seus dados também significa que a única opção para um RDBMS é efetivamente recusar os dados que não obedecem às suas regras de consistência, e a primeira delas é o tipo de dado escolhido por você. Se você acha que não há problemas em lidar com uma data como 0000-00-00, que jamais existiu no calendário, pense novamente.

A outra parte das garantias de consistência é expressa em termos de restrições, como em restrições CHECK, restrições NOT NULL e triggers de restrições, e uma delas é conhecida como chave estrangeira. Podemos pensar em tudo isso como uma extensão no nível de usuário para a definição e o comportamento dos tipos de dados, e a principal diferença está no fato de você poder optar por adiar a verificação dessas restrições com DEFER, e isso pode variar do final de cada instrução até o final da transação atual.

Todas as partes relacionais de um RDBMS dizem respeito à modelagem de seus dados e à garantia de que todas as tuplas que se encontram em uma relação compartilhem um conjunto comum de regras: a estrutura e as restrições. Ao garantir isso, garantiremos o uso de um esquema apropriado explícito para lidar com nossos dados.

Trabalhar com um esquema apropriado para seus dados é conhecido como *normalização*, e você pode visar a algumas formas normalizadas sutilmente diferentes em seu design. Às vezes, porém, você precisará de mais flexibilidade do que aquilo que será oferecido como resultado de seu processo de normalização. O senso comum diz que você deve inicialmente normalizar o esquema de seus dados, e somente então o modificar para readquirir um pouco de flexibilidade. Há uma boa chance de você não precisar realmente de mais flexibilidade.

Quando for realmente necessário ter mais flexibilidade, você

poderá usar o PostgreSQL para testar uma série de opções de desnormalização: tipos compostos, registros, arrays, H-Store, JSON ou XML, para citar apenas algumas.

Há, porém, uma desvantagem muito significativa na desnormalização, que é o fato de a linguagem de query sobre a qual falaremos a seguir ter sido projetada para lidar com dados normalizados. No caso do PostgreSQL, é claro, a linguagem de query foi estendida para aceitar o máximo possível de desnormalização quando usamos tipos compostos, arrays ou H-Store, e até mesmo JSON em versões mais recentes.

O RDBMS sabe muito sobre seus dados e pode ajudar você a implementar um modelo de segurança bastante detalhado, caso seja necessário. Os padrões de acesso são gerenciados no nível das relações e das colunas, e o PostgreSQL também implementa procedimentos armazenados SECURITY DEFINER, permitindo que você ofereça acesso a dados sigilosos de forma bastante controlada, de modo muito semelhante ao que ocorre quando usamos programas SUID (Saved User ID, ou ID de Usuários Salvos).

O RDBMS se oferece para acessar seus dados usando um SQL, que se tornou um padrão de mercado nos anos 1980 e, atualmente, é determinado por um comitê. No caso do PostgreSQL, muitas extensões estão sendo adicionadas, e cada nova versão principal lançada lhe permite acessar uma linguagem DSL muito rica. Todo o trabalho de planejamento e otimização de queries é feito para você pelo RDBMS, de modo que possa manter o foco em uma query declarativa, na qual descreverá somente o resultado desejado com base nos dados que você tiver.

É por isso também que deve prestar muita atenção nos produtos NoSQL nesse caso, pois a maioria desses produtos em alta, com efeito, estão removendo não só o SQL, mas também várias outras partes essenciais que você está acostumado a esperar que estejam disponíveis.

Quais são os conselhos que você daria para os desenvolvedores que utilizam RDBMSs como seus backends de armazenagem?

Meu conselho é este: lembre-se das diferenças entre um *backend de armazenagem* e um RDBMS. São serviços muito diferentes, e, se tudo que você precisa é de um backend de armazenagem, talvez deva considerar o uso de algo que não seja um RDBMS.

Na maioria das vezes, porém, você realmente precisará de um RDBMS completo. Nesse caso, a melhor opção à sua disposição é o PostgreSQL. Leia a sua documentação (<https://www.postgresql.org/docs/>); veja a lista dos tipos de dados, operadores, funções, recursos e extensões que ele disponibiliza. Veja alguns exemplos de uso em postagens de blog.

Considere então o PostgreSQL como uma ferramenta da qual você possa tirar proveito em seu desenvolvimento e possa incluir na arquitetura de sua aplicação. Partes dos serviços que você terá de implementar serão oferecidas de modo mais apropriado na camada de RDBMS, e o PostgreSQL é excelente para compor essa parte confiável de sua implementação como um todo.

Qual é a melhor maneira de usar ou de não usar um ORM?

O ORM funcionará melhor em aplicações *CRUD*: Create, Read, Update, and Delete (Criar, Ler, Atualizar e Apagar). A parte referente à leitura deve estar limitada a uma instrução `SELECT` bem simples, com vistas a uma única tabela, pois acessar mais colunas do que o necessário terá um impacto significativo no desempenho das consultas e nos recursos utilizados.

Qualquer coluna que você acessar no RDBMS e que acabe não sendo utilizada será um puro desperdício de recursos valiosos, e isso será extremamente prejudicial para a escalabilidade. Mesmo quando seu ORM for capaz de buscar somente os dados que você lhe solicitar, continuará sendo necessário, de alguma forma, gerenciar a lista exata das colunas que você vai querer em cada situação, sem usar um método abstrato simples que calcule automaticamente a lista de campos para você.

As queries para criar, atualizar e apagar são instruções INSERT, UPDATE e DELETE simples. Muitos RDBMSs possibilitam otimizações que não são aproveitadas pelos ORMs, por exemplo, a devolução de dados após um INSERT.

Além disso, no caso geral, uma relação é uma tabela ou o resultado de uma query qualquer. Ao usar um ORM, uma prática comum é criar um mapeamento relacional entre tabelas específicas e algumas classes de modelos, ou outros stubs auxiliares.

Se você considerar a semântica completa do SQL em suas generalidades, o mapeador relacional deveria ser realmente capaz de mapear qualquer query a uma classe. Então, supostamente, você teria de criar uma classe para cada query que quisesse executar.

A ideia, quando aplicada ao nosso caso, é que você confie que seu ORM fará um trabalho melhor do que o seu para escrever queries SQL eficientes, mesmo que você não lhe dê informações suficientes para que ele descubra o conjunto exato de dados nos quais você está interessado.

É verdade que, às vezes, o SQL pode se tornar bastante complexo, embora você vá estar muito longe da simplicidade se utilizar um gerador de API-para-SQL que não esteja em seu controle.

Contudo, há dois casos em que você pode ficar despreocupado e utilizar seu ORM, desde que esteja disposto a aceitar o seguinte compromisso: no futuro, talvez você precise remover o uso do ORM de sua base de código.

- **Time to market (tempo para atingir o mercado):** Quando você está realmente com pressa e deseja conquistar uma fatia do mercado o mais rápido possível, a única maneira de fazer isso será lançar uma primeira versão de sua aplicação e de sua ideia. Se sua equipe tiver mais proficiência para usar um ORM do que compor queries SQL, sem dúvida, você deve utilizá-lo. Contudo,

esteja ciente de que, assim que tiver sucesso com a sua aplicação, um dos primeiros problemas de escalabilidade que será preciso resolver estará relacionado com o ORM gerando queries realmente ruins. Além disso, seu uso do ORM poderá deixar você em um beco sem saída e fazer com que tenha tomado decisões ruins quanto ao design do código. No entanto, se chegar lá, é sinal de que foi suficientemente bem-sucedido, a ponto de poder gastar um pouco de dinheiro com uma refatoração, eliminando qualquer dependência com o ORM, certo?

- **Aplicação CRUD:** Esta é a situação ideal, na qual você modificará somente uma única tupla de cada vez, e não se importará realmente com o desempenho, como no caso da interface básica de administração da aplicação.

Quais são as vantagens de usar o PostgreSQL no lugar de outros bancos de dados quando estiver trabalhando com Python?

Eis os motivos principais pelos quais eu, como desenvolvedor, opto pelo PostgreSQL:

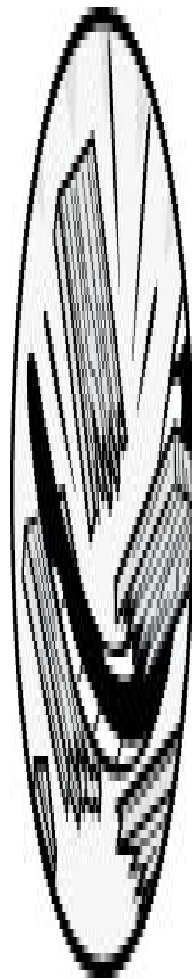
- **Suporte da comunidade:** A comunidade do PostgreSQL é vasta e receptiva aos novos usuários, e as pessoas, em geral, vão dispor de tempo para dar a melhor resposta possível. As listas de discussão ainda são a melhor forma de se comunicar com a comunidade.
- **Integridade e durabilidade dos dados:** Qualquer dado que você enviar para o PostgreSQL estará seguro quanto à sua definição e à capacidade de acessá-los novamente no futuro.
- **Tipos de dados, funções, operadores, arrays e intervalos:** O PostgreSQL tem um conjunto muito diversificado de tipos de dados, acompanhados por uma série de operadores e funções. É até mesmo possível fazer desnormalizações usando arrays ou tipos de dados JSON, e ainda ser capaz de escrever queries sofisticadas, inclusive com junções (joins), com esses dados.
- **planejador (planner) e o otimizador (optimizer):** Vale a pena

investir tempo para entender a complexidade e a eficácia desses recursos.

- **DDL transacional:** É possível fazer ROLLBACK de praticamente qualquer comando. Teste isso agora: basta abrir seu shell `psql` com um banco de dados que você tiver e digitar `BEGIN; DROP TABLE foo; ROLLBACK;`, substituindo `foo` pelo nome de uma tabela existente em sua instância local. Incrível, não?
- **PL/Python (e outras linguagens como C, SQL, Javascript ou Lua):** Você pode executar seu próprio código Python no servidor, exatamente onde estão os dados, de modo que não será necessário buscá-los por meio da rede somente para processá-los e, então, enviá-los de volta em uma query para executar o próximo nível de JOIN.
- **Indexação específica (GiST, GIN, SP-GiST, parcial e funcional):** Você pode criar funções Python para processar seus dados dentro do PostgreSQL e, então, indexar o resultado da chamada dessa função. Ao executar uma query com uma cláusula `WHERE` chamando essa função, ela será chamada somente uma vez com os dados da query e, em seguida, uma correspondência direta será feita com o conteúdo do índice.

13

ESCREVA MENOS, PROGRAME MAIS



Neste último capítulo, reuni alguns dos recursos mais sofisticados de Python que utilizo para escrever um código melhor. Esses recursos não estão limitados à Biblioteca-Padrão de Python. Discutiremos como deixar seu código compatível com Python 2 e Python 3, como criar um dispatcher de métodos em estilo Lisp,

como utilizar gerenciadores de contexto e como criar um boilerplate para classes com o módulo `attr`.

Usando o `six` para suporte a Python 2 e 3

Como provavelmente deve ser de seu conhecimento, Python 3 deixou de ser compatível com Python 2 e fez uma série de mudanças. No entanto, o básico da linguagem não mudou entre essas versões, possibilitando implementar códigos compatíveis para trás e para a frente por meio da criação de uma ponte entre Python 2 e Python 3.

Felizmente para nós, esse módulo já existe! Ele se chama `six` — porque $2 \times 3 = 6$.

O módulo `six` disponibiliza a variável útil `six.PY3`: um booleano que informa se você está executando Python 3 ou não. É uma variável essencial para qualquer base de código que tenha duas versões: uma para Python 2 e outra para Python 3. Entretanto, tome cuidado para não abusar dela; espalhar um `if six.PY3` por todo o seu código fará com que seja difícil para as pessoas o lerem e entenderem.

Quando discutimos os geradores na seção “Geradores” na página 149, vimos que Python 3 tem uma ótima propriedade de acordo com a qual objetos iteráveis são devolvidos no lugar de listas no caso de várias funções embutidas, por exemplo, `map()` ou `filter()`. Desse modo, Python 3 se livrou de métodos como `dict.iteritems()`, que era a versão iterável de `dict.items()` em Python 2, em troca de fazer `dict.items()` devolver um iterador no lugar de uma lista. Essa mudança nos métodos e nos tipos devolvidos pode causar falhas em seu código para Python 2.

O módulo `six` disponibiliza `six.iteritems()` para casos como esses, o qual pode ser usado para substituir um código específico de Python 2, assim:

```
for k, v in mydict.iteritems():  
    print(k, v)
```

Utilizando o `six`, você substituiria o código de `mydict.iteritems()` por um código compatível com Python 2 e com Python 3, da seguinte maneira:

```
import six

for k, v in six.iteritems(mydict):
    print(k, v)
```

E *voilà*, conseguimos ter compatibilidade tanto com Python 2 como com Python 3 em um piscar de olhos! A função `six.iteritems()` usará `dict.iteritems()` ou `dict.items()` para devolver um gerador, conforme a versão de Python que você estiver usando. O módulo `six` disponibiliza muitas funções auxiliares semelhantes, que podem facilitar o suporte para várias versões de Python.

Outro exemplo é a solução do `six` para a palavra reservada `raise`, cuja sintaxe difere entre Python 2 e Python 3. Em Python 2, `raise` aceita vários argumentos; em Python 3, porém, `raise` aceita uma exceção como seu único argumento, e nada mais. Escrever uma instrução `raise` com dois ou três argumentos em Python 3 resultará em um `SyntaxError`.

O módulo `six` oferece uma solução para contornar esse problema, na forma da função `six.reraise()`, que permite relançar uma exceção em qualquer versão de Python que estiver em uso.

Strings e Unicode

A melhor eficácia de Python 3 para lidar com codificações sofisticadas resolveu os problemas de strings e Unicode de Python 2. Em Python 2, o tipo básico para string é `str`, que é capaz de lidar somente com strings ASCII básicas. O tipo `unicode`, acrescentado posteriormente em Python 2.5, lida com verdadeiras strings de texto.

Em Python 3, o tipo string básico continua sendo `str`, mas ele compartilha as propriedades da classe `unicode` de Python 2 e é capaz de lidar com codificações sofisticadas. O tipo `bytes` substituiu o tipo `str` para lidar com streams básicos de caracteres.

Novamente, o módulo `six` disponibiliza funções e constantes, como `six.u` e `six.string_types`, para lidar com a transição. O mesmo tipo de compatibilidade é oferecido para os inteiros, com `six.integer_types` para lidar com o tipo `long` que foi eliminado em Python 3.

Lidando com a mudança nos módulos Python

Na Biblioteca-Padrão de Python, alguns módulos foram movidos ou renomeados quando comparamos Python 2 e Python 3. O módulo `six` disponibiliza um módulo chamado `six.moves` que cuida de várias dessas mudanças de modo transparente.

Por exemplo, o módulo `ConfigParser` de Python 2 foi renomeado para `configparser` em Python 3. A Listagem 13.1 mostra como um código pode ser portado e se tornar compatível com as duas versões principais de Python usando o módulo `six.moves`:

Listagem 13.1 – Usando `six.moves` para utilizar `ConfigParser()` em Python 2 e em Python 3

```
from six.moves.configparser import ConfigParser
```

```
conf = ConfigParser()
```

Você também pode adicionar suas próprias mudanças usando `six.add_move` para lidar com transições de código que não sejam tratadas pelo `six` de modo nativo.

Caso a biblioteca `six` não inclua todos os seus casos de uso, talvez valha a pena criar um módulo de compatibilidade que encapsule o próprio `six`, garantindo, desse modo, que você seja capaz de aperfeiçoar o módulo para acomodar futuras versões de Python ou removê-las (ou parte delas) quando quiser parar de oferecer suporte para uma versão específica da linguagem. Além disso, observe que o `six` tem código aberto, permitindo-lhe fazer contribuições, em vez de manter seus próprios hacks!

Módulo `modernize`

Por fim, há uma ferramenta chamada `modernize`, que utiliza o módulo `six` para “modernizar” o seu código, portando-o para Python 3, em vez de simplesmente converter a sintaxe de Python 2 para a sintaxe de Python 3. Ele oferece suporte tanto para Python 2 como para Python 3. A ferramenta `modernize` ajuda você a ter um ponto de partida robusto para portar o seu código, fazendo a maior parte do trabalho mecânico por você; isso faz dessa ferramenta uma opção melhor em comparação com a ferramenta padrão `2to3`.

Usando Python como Lisp para criar um dispatcher simples

Gosto de dizer que Python é um bom subconjunto da linguagem de programação Lisp e, com o passar do tempo, constato que essa afirmação é cada vez mais verdadeira. A PEP 443 comprova essa questão: ela descreve um modo de despachar funções genéricas, de modo semelhante à maneira proporcionada pelo CLOS (Common Lisp Object System).

Se você tem familiaridade com Lisp, isso não será nenhuma novidade. O sistema de objetos de Lisp, que é um dos componentes básicos do Common Lisp, oferece um modo simples e eficiente de definir e lidar com o dispatching de métodos. Vou mostrar antes como os métodos genéricos funcionam em Lisp.

Criando métodos genéricos em Lisp

Para começar, vamos definir algumas classes muito simples em Lisp, sem nenhuma classe-pai ou atributos:

```
(defclass snare-drum ())  
()
```

```
(defclass cymbal ())  
()
```

```
(defclass stick ())  
()
```

```
(defclass brushes ()  
  ())
```

Esse código define as classes `snare-drum`, `cymbal`, `stick` e `brushes` sem nenhuma classe-pai ou atributos. Essas classes compõem um conjunto de bateria e podem ser combinadas para produzir sons. Para isso, definimos um método `play()` que aceita dois argumentos e devolve um som na forma de uma string:

```
(defgeneric play (instrument accessory)  
  (:documentation "Play sound with instrument and accessory."))
```

Esse código define apenas um método genérico que não está associado a nenhuma classe e, portanto, ainda não pode ser chamado. A essa altura, só informamos ao sistema de objetos que o método é genérico e poderá ser chamado com dois argumentos de nomes `instrument` e `accessory`. Na Listagem 13.2, implementaremos versões desse método que simulam o toque de nossa caixa (`snare drum`).

Listagem 13.2 – Definindo métodos genéricos em Lisp, independentes de classes

```
(defmethod play ((instrument snare-drum) (accessory stick))  
  "POC!")
```

```
(defmethod play ((instrument snare-drum) (accessory brushes))  
  "SHHHH!")
```

```
(defmethod play ((instrument cymbal) (accessory brushes))  
  "FRCCCHHT!")
```

Agora já definimos métodos concretos no código. Cada método aceita dois argumentos: `instrument`, que é uma instância de `snare-drum` ou de `cymbal`, e `accessory`, que é uma instância de `stick` ou de `brushes`.

Neste ponto, você deverá perceber a primeira diferença principal entre esse sistema e os sistemas de objetos de Python (ou sistemas parecidos): o método não está associado a nenhuma classe em particular. Os métodos são *genéricos* e podem ser implementados para qualquer classe.

Vamos testar isso. Podemos chamar o nosso método `play()` com alguns objetos:

```
* (play (make-instance 'snare-drum) (make-instance 'stick))
"POC!"
```

```
* (play (make-instance 'snare-drum) (make-instance 'brushes))
"SHHHH!"
```

Como você pode ver, a função chamada depende da classe dos argumentos – o sistema de objetos *despacha* as chamadas de função para a função correta para nós, com base no tipo dos argumentos que passarmos. Se chamarmos `play()` com um objeto cujas classes não tenham um método definido, um erro será gerado.

Na Listagem 13.3, o método `play()` é chamado com uma instância de `cymbal` e de `stick`; no entanto, o método `play()` não foi definido para esses argumentos, portanto, ele gera um erro.

Listagem 13.3 – Chamando um método com uma assinatura indisponível

```
* (play (make-instance 'cymbal) (make-instance 'stick))
debugger invoked on a SIMPLE-ERROR in thread
#<THREAD "main thread" RUNNING {1002ADAF23}>:
  There is no applicable method for the generic function
    #<STANDARD-GENERIC-FUNCTION PLAY (2)>
  when called with arguments
    (#<CYMBAL {1002B801D3}> #<STICK {1002B82763}>).
```

Type `HELP` for debugger help, or `(SB-EXT:EXIT)` to exit from SBCL.

restarts (invokable by number or by possibly abbreviated name):

0: `[RETRY]` Retry calling the generic function.

1: `[ABORT]` Exit debugger, returning to top level.

```
((:METHOD NO-APPLICABLE-METHOD (T)) #<STANDARD-GENERIC-FUNCTION
PLAY (2)> #<CYMBAL {1002B801D3}> #<STICK {1002B82763}>) [fast-method]
```

O CLOS oferece outros recursos, por exemplo, herança de métodos ou dispatching baseado em objetos, em vez de utilizar classes. Se você estiver realmente curioso quanto aos diversos recursos oferecidos pelo CLOS, sugiro que leia “A Brief Guide to CLOS” (Um

guia conciso para o CLOS) de Jeff Dalton (<http://www.aiai.ed.ac.uk/~jeff/clos-guide.html>) como ponto de partida.

Métodos genéricos com Python

Python implementa uma versão mais simples desse fluxo de trabalho com a função `singledispatch()`, distribuída como parte do módulo `functools` a partir de Python 3.4. Nas versões 2.6 a 3.3, a função `singledispatch()` é disponibilizada por meio do Python Package Index; para aqueles que estejam ávidos para testá-la, basta executar `pip install singledispatch`.

A Listagem 13.4 mostra um código, grosso modo, equivalente ao programa Lisp que criamos na Listagem 13.2.

Listagem 13.4 – Usando singledispatch para despachar chamadas de métodos

```
import functools

class SnareDrum(object): pass
class Cymbal(object): pass
class Stick(object): pass
class Brushes(object): pass

@functools.singledispatch
def play(instrument, accessory):
    raise NotImplementedError("Cannot play these")

❶ @play.register(SnareDrum)
def _(instrument, accessory):
    if isinstance(accessory, Stick):
        return "POC!"
    if isinstance(accessory, Brushes):
        return "SHHHH!"
    raise NotImplementedError("Cannot play these")

@play.register(Cymbal)
def _(instrument, accessory):
    if isinstance(accessory, Brushes):
        return "FRCCCHHT!"
    raise NotImplementedError("Cannot play these")
```

Essa listagem define nossas quatro classes e uma função `play()` básica que gera `NotImplementedError`, informando que, por padrão, não sabemos o que deve ser feito.

Em seguida, escrevemos uma versão especializada da função `play()` para um instrumento específico, o `SnareDrum` ❶. Essa função verifica o tipo de acessório que foi passado e devolve o som apropriado ou gera `NotImplementedError` novamente se o acessório não for reconhecido.

Se executarmos o programa, ele funcionará da seguinte maneira:

```
>>> play(SnareDrum(), Stick())
'POC!'
>>> play(SnareDrum(), Brushes())
'SHHHH!'
>>> play(Cymbal(), Stick())
Traceback (most recent call last):
NotImplementedError: Cannot play these
>>> play(SnareDrum(), Cymbal())
NotImplementedError: Cannot play these
```

O módulo `singledispatch` verifica a classe do primeiro argumento passado e chama a versão apropriada da função `play()`. Para a classe `object`, a primeira versão definida da função será sempre aquela que será executada. Desse modo, se nosso instrumento for uma instância de uma classe que não registramos, essa função básica será chamada.

Conforme vimos na versão Lisp do código, o CLOS disponibiliza um dispatcher múltiplo capaz de despachar com base no tipo de *qualquer um dos argumentos* definidos no protótipo do método, e não apenas no primeiro. O dispatcher de Python se chama `singledispatch` por um bom motivo: ele só sabe despachar com base no primeiro argumento.

Além disso, `singledispatch` não oferece nenhuma maneira de chamar a função-pai diretamente. Não há nenhum equivalente à função `super()` de Python; você terá de usar diversos truques para contornar essa limitação.

Por um lado, Python está aprimorando o seu sistema de objetos e o

mecanismo de dispatch; por outro lado, porém, ainda lhe faltam muitos dos recursos sofisticados que algo como o CLOS oferece prontamente. Isso faz com que seja muito raro deparar com o `singledispatch` por aí. Apesar disso, é interessante saber que ele existe, pois você poderá acabar implementando um sistema desse tipo por conta própria em algum momento.

Gerenciadores de contexto

A instrução `with` introduzida em Python 2.6 provavelmente fará velhos adeptos de Lisp se lembrarem das diversas macros `with-*` usadas com frequência nessa linguagem. Python oferece um sistema de aspecto semelhante, com o uso de objetos que implementam o *protocolo de gerenciamento de contextos*.

Se você nunca usou o protocolo de gerenciamento de contextos, eis o modo como ele funciona. O bloco de código contido na instrução `with` é cercado por duas chamadas de função. O objeto usado na instrução `with` determina as duas chamadas. Dizemos que esses objetos implementam o protocolo de gerenciamento de contexto.

Objetos como aqueles devolvidos por `open()` aceitam esse protocolo; é por isso que você pode escrever um código semelhante ao que vemos a seguir:

```
with open("myfile", "r") as f:
    line = f.readline()
```

O objeto devolvido por `open()` tem dois métodos: um chamado `__enter__` e outro chamado `__exit__`. Esses métodos são chamados no início do bloco `with` e no final dele, respectivamente.

A Listagem 13.5 mostra uma implementação simples de um objeto de contexto.

Listagem 13.5 – Uma implementação simples de um objeto de contexto

```
class MyContext(object):
    def __enter__(self):
        pass
```

```
def __exit__(self, exc_type, exc_value, traceback):  
    pass
```

Essa implementação não faz nada, mas é válida e mostra a assinatura dos métodos que devem ser definidos para oferecer uma classe que obedeça ao protocolo de contexto.

O protocolo de gerenciamento de contexto talvez seja apropriado para ser usado quando você identificar o seguinte padrão em seu código, no qual se espera que uma chamada ao método B deva ser *sempre* feita após uma chamada a A:

1. Chama o método A.
2. Executa um código.
3. Chama o método B.

A função `open()` ilustra bem esse padrão: o construtor que abre o arquivo e aloca um descritor de arquivo internamente é o método A. O método `close()` que libera o descritor de arquivo corresponde ao método B. Obviamente, a função `close()` deverá ser sempre chamada *depois* que você tiver instanciado o objeto arquivo.

Implementar esse protocolo manualmente pode ser tedioso, de modo que a biblioteca-padrão `contextlib` disponibiliza o decorador `contextmanager` para facilitar a implementação. O decorador `contextmanager` deve ser usado em uma função geradora. Os métodos `__enter__` e `__exit__` serão implementados dinamicamente para você com base no código ao redor da instrução `yield` do gerador.

Na Listagem 13.6, `MyContext` é definido como um gerenciador de contexto.

Listagem 13. 6– Usando `contextlib.contextmanager`

```
import contextlib  
  
@contextlib.contextmanager  
def MyContext():  
    print("do something first")  
    yield  
    print("do something else")
```

```
with MyContext():  
    print("hello world")
```

O código antes da instrução `yield` será executado antes do corpo da instrução `with` ser executado; o código depois da instrução `yield` será executado assim que o corpo da instrução `with` tiver sido executado. Ao ser executado, a saída a seguir será exibida por esse programa:

```
do something first  
hello world  
do something else
```

Contudo, há alguns pontos que devem ser tratados nesse caso. Em primeiro lugar, é possível fazer `yield` de algo em nosso gerador, que possa ser utilizado como parte do bloco `with`.

A Listagem 13.7 mostra como fazer o `yield` de um valor para quem fez a chamada. A palavra reservada `as` é utilizada para armazenar esse valor em uma variável.

Listagem 13.7 – Definindo um gerenciador de contexto que faz `yield` de um valor.

```
import contextlib  
  
@contextlib.contextmanager  
def MyContext():  
    print("do something first")  
    yield 42  
    print("do something else")
```

```
with MyContext() as value:  
    print(value)
```

A Listagem 13.7 mostra como fazer o `yield` de um valor para quem fez a chamada. A palavra reservada `as` é utilizada para armazenar esse valor em uma variável. Ao ser executado, o código gera a saída a seguir:

```
do something first  
42
```



```
do something else
```

Ao usar um gerenciador de contexto, talvez seja necessário lidar com exceções que possam ser lançadas de dentro do bloco de código `with`. Isso pode ser feito colocando um bloco `try...except` ao redor da instrução `yield`, como mostra a Listagem 13.8.

Listagem 13.8 – Lidando com exceções em um gerenciador de contexto

```
import contextlib

@contextlib.contextmanager
def MyContext():
    print("do something first")
    try:
        yield 42
    finally:
        print("do something else")
```

```
with MyContext() as value:
    print("about to raise")
❶ raise ValueError("let's try it")
    print(value)
```

Nesse caso, um `ValueError` é gerado no início do bloco de código de `with` ❶; Python propagará esse erro de volta para o gerenciador de contexto, e parecerá que a própria instrução `yield` lançou a exceção. Cercamos a instrução `yield` com `try` e `finally` a fim de garantir que o `print()` final seja executado.

Ao ser executada, a Listagem 13.8 gera o seguinte:

```
do something first
about to raise
do something else
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
ValueError: let's try it
```

Como podemos ver, o erro é enviado para o gerenciador de contexto, e o programa retoma e finaliza a execução, pois a exceção foi ignorada usando um bloco `try...finally`.

Em alguns contextos, pode ser conveniente utilizar vários gerenciadores de contexto ao mesmo tempo, por exemplo, ao abrir dois arquivos simultaneamente para copiar o conteúdo, conforme mostra a Listagem 13.9.

Listagem 13.9 – Abrindo dois arquivos ao mesmo tempo para copiar o conteúdo

```
with open("file1", "r") as source:
    with open("file2", "w") as destination:
        destination.write(source.read())
```

Apesar do que dissemos, como a instrução `with` aceita vários argumentos, na verdade, será mais eficiente escrever uma versão com um único `with`, como mostra a Listagem 13.10.

Listagem 13.10 – Abrindo dois arquivos ao mesmo tempo usando apenas uma instrução with

```
with open("file1", "r") as source, open("file2", "w") as destination:
    destination.write(source.read())
```

Os gerenciadores de contexto são padrões de projeto extremamente eficazes, que ajudam a garantir que o fluxo de seu código esteja sempre correto, independentemente da exceção que possa ocorrer. Eles podem ajudar a oferecer uma interface de programação consistente e organizada em várias situações nas quais o código deva estar encapsulado por outro código e por `contextlib.contextmanager`.

Menos boilerplate com `attr`

Escrever classes Python pode ser uma tarefa desajeitada. Com frequência, você se verá repetindo apenas alguns padrões porque não haverá outras opções. Um dos exemplos mais comuns, conforme mostra a Listagem 13.11, é quando inicializamos um objeto com alguns atributos passados para o construtor.

Listagem 13.11 – Boilerplate comum para inicialização de classes

```
class Car(object):
    def __init__(self, color, speed=0):
        self.color = color
```

```
self.speed = speed
```

O processo é sempre o mesmo: copiamos o valor dos argumentos passados para a função `__init__` para alguns atributos armazenados no objeto. Às vezes, também teremos de conferir o valor passado, calcular um default e assim por diante.

Obviamente, também vamos querer que o objeto seja representado corretamente ao ser exibido, portanto, teremos de implementar um método `__repr__`. Há uma chance de que algumas de suas classes sejam suficientemente simples, a ponto de poderem ser convertidas em dicionários para serialização. A situação começará a se tornar mais complicada se falarmos de comparação e da possibilidade de hashing (a capacidade de usar `hash` em um objeto e armazená-lo em um `set`).

Na verdade, a maioria dos programadores Python não faz nada disso porque o fardo de escrever todas essas verificações e métodos é muito pesado, sobretudo se você nem sempre tiver certeza de que vai precisar deles. Por exemplo, talvez você ache que `__repr__` seja útil em seu programa somente naquela única ocasião em que tenta depurá-lo ou gerar um `trace` e decide exibir objetos na saída-padrão – e em nenhuma outra ocasião adicional.

A biblioteca `attr` visa oferecer uma solução simples, disponibilizando um boilerplate genérico para todas as suas classes e gerando boa parte do código para você. Você pode instalar o `attr` com o `pip`, utilizando o comando `pip install attr`. Prepare-se para apreciá-lo!

Assim que estiver instalado, o decorador `attr.s` será seu ponto de entrada para o mundo maravilhoso do `attr`. Utilize-o antes da declaração de uma classe e, em seguida, use a função `attr.ib()` para declarar atributos em suas classes. A Listagem 13.12 mostra uma forma de reescrever o código da Listagem 13.11 usando `attr`.

Listagem 13.12 – Usando `attr.ib()` para declarar atributos

```
import attr
```

```
@attr.s
```

```
class Car(object):
    color = attr.ib()
    speed = attr.ib(default=0)
```

Ao ser declarada dessa forma, a classe adquire gratuitamente alguns métodos convenientes de modo automático, por exemplo, o método `__repr__`, que é chamado para representar objetos quando são exibidos no `stdout` no interpretador Python:

```
>>> Car("blue")
Car(color='blue', speed=0)
```

Essa saída é mais elegante do que a saída default que `__repr__` teria exibido:

```
<__main__.Car object at 0x104ba4cf8>.
```

Você também pode acrescentar outras validações em seus atributos usando os argumentos nomeados `validator` e `converter`.

A Listagem 13.13 mostra como a função `attr.ib()` pode ser usada para declarar um atributo com algumas restrições.

Listagem 13.13 – Usando `attr.ib()` com seu argumento `converter`

```
import attr

@attr.s
class Car(object):
    color = attr.ib(converter=str)
    speed = attr.ib(default=0)

    @speed.validator
    def speed_validator(self, attribute, value):
        if value < 0:
            raise ValueError("Value cannot be negative")
```

O argumento `converter` cuida da conversão de qualquer dado passado para o construtor. A função `validator()` pode ser passada como argumento para `attr.ib()` ou pode ser usada como um decorador, como vemos na Listagem 13.13.

O módulo `attr` inclui alguns validadores próprios (por exemplo, `attr.validators.instance_of()` para verificar o tipo do atributo), portanto, não se esqueça de consultá-los antes de desperdiçar seu tempo criando

a sua própria função.

O módulo `attr` também oferece ajustes para fazer com que seu objeto seja hashable, de modo que possa ser usado em um conjunto ou como chave de um dicionário: basta passar `frozen=True` para `attr.s()` para tornar as instâncias da classe imutáveis.

A Listagem 13.14 mostra como o uso do parâmetro `frozen` modifica o comportamento da classe.

Listagem 13.14 – Utilizando `frozen=True`

```
>>> import attr
>>> @attr.s(frozen=True)
... class Car(object):
...     color = attr.ib()
...
>>> {Car("blue"), Car("blue"), Car("red")}
{Car(color='red'), Car(color='blue')}
>>> Car("blue").color = "red"
attr.exceptions.FrozenInstanceError
```

A Listagem 13.14 mostra como o uso do parâmetro `frozen` modifica o comportamento da classe `Car`: ela se torna passível de hashing e, desse modo, poderá ser armazenada em um conjunto, mas os objetos não poderão mais ser modificados.

Em suma, `attr` possibilita a implementação de uma série de métodos úteis, evitando, desse modo, que você tenha de escrevê-los por conta própria. Em virtude de sua eficiência, recomendo enfaticamente que tire proveito de `attr` quando estiver criando suas classes e modelando o seu software.

Resumo

Parabéns! Você chegou ao final do livro. Você simplesmente levou seu conhecimento de Python para o próximo patamar e tem uma ideia melhor de como escrever um código Python mais eficiente e produtivo. Espero que tenha apreciado a leitura deste livro, na mesma medida em que gostei de escrevê-lo.

Python é uma linguagem maravilhosa, e pode ser usado em muitas áreas diferentes, mas há várias partes diferentes de Python que não foram exploradas neste livro. Mas todo livro precisa ter um fim, certo?

Recomendo que você tire proveito dos projetos de código aberto, lendo o código-fonte disponível por aí e contribuindo com esses projetos. Ter seu código revisado e discutido por outros desenvolvedores em geral é uma ótima forma de aprendizagem.

Um ótimo hacking para você!

O'REILLY

Python para Análise de Dados

TRATAMENTO DE DADOS COM
PANDAS, NUMPY E IPYTHON



novatec

Wes McKinney

Python para análise de dados

McKinney, Wes

9788575227510

616 páginas

[Compre agora e leia](#)

Obtenha instruções completas para manipular, processar, limpar e extrair informações de conjuntos de dados em Python. Atualizada para Python 3.6, este guia prático está repleto de casos de estudo práticos que mostram como resolver um amplo conjunto de problemas de análise de dados de forma eficiente. Você conhecerá as versões mais recentes do pandas, da NumPy, do IPython e do Jupyter no processo. Escrito por Wes McKinney, criador do projeto Python pandas, este livro contém uma introdução prática e moderna às ferramentas de ciência de dados em Python. É ideal para analistas, para quem Python é uma novidade, e para programadores Python iniciantes nas áreas de ciência de dados e processamento científico. Os arquivos de dados e os materiais relacionados ao livro estão disponíveis no GitHub.

- utilize o shell IPython e o Jupyter Notebook para processamentos exploratórios;
- conheça os recursos básicos e avançados da NumPy (Numerical Python);
- comece a trabalhar com ferramentas de análise de dados da biblioteca pandas;
- utilize ferramentas flexíveis para carregar, limpar, transformar, combinar e reformatar dados;
- crie visualizações informativas com a matplotlib;
- aplique o recurso

groupby do pandas para processar e sintetizar conjuntos de dados; • analise e manipule dados de séries temporais regulares e irregulares; • aprenda a resolver problemas de análise de dados do mundo real com exemplos completos e detalhados.

[Compre agora e leia](#)

CANDLESTICK

Um método para ampliar lucros na Bolsa de Valores



novatec

Carlos Alberto Debastiani

Candlestick

Debastiani, Carlos Alberto

9788575225943

200 páginas

[Compre agora e leia](#)

A análise dos gráficos de Candlestick é uma técnica amplamente utilizada pelos operadores de bolsas de valores no mundo inteiro. De origem japonesa, este refinado método avalia o comportamento do mercado, sendo muito eficaz na previsão de mudanças em tendências, o que permite desvendar fatores psicológicos por trás dos gráficos, incrementando a lucratividade dos investimentos.

Candlestick – Um método para ampliar lucros na Bolsa de Valores é uma obra bem estruturada e totalmente ilustrada. A preocupação do autor em utilizar uma linguagem clara e acessível a torna leve e de fácil assimilação, mesmo para leigos. Cada padrão de análise abordado possui um modelo com sua figura clássica, facilitando a identificação. Depois das características, das peculiaridades e dos fatores psicológicos do padrão, é apresentado o gráfico de um caso real aplicado a uma ação negociada na Bovespa. Este livro possui, ainda, um índice resumido dos padrões para pesquisa rápida na utilização cotidiana.

[Compre agora e leia](#)



AVALIANDO
EMPRESAS

INVESTINDO EM AÇÕES

A APLICAÇÃO PRÁTICA DA
ANÁLISE FUNDAMENTALISTA NA
AVALIAÇÃO DE EMPRESAS

novatec

CARLOS ALBERTO DEBASTIANI
FELIPE AUGUSTO RUSSO

Avaliando Empresas, Investindo em Ações

Debastiani, Carlos Alberto

9788575225974

224 páginas

[Compre agora e leia](#)

Avaliando Empresas, Investindo em Ações é um livro destinado a investidores que desejam conhecer, em detalhes, os métodos de análise que integram a linha de trabalho da escola fundamentalista, trazendo ao leitor, em linguagem clara e acessível, o conhecimento profundo dos elementos necessários a uma análise criteriosa da saúde financeira das empresas, envolvendo indicadores de balanço e de mercado, análise de liquidez e dos riscos pertinentes a fatores setoriais e conjunturas econômicas nacional e internacional. Por meio de exemplos práticos e ilustrações, os autores exercitam os conceitos teóricos abordados, desde os fundamentos básicos da economia até a formulação de estratégias para investimentos de longo prazo.

[Compre agora e leia](#)

Marcos Abe

MANUAL DE ANÁLISE TÉCNICA

ESSÊNCIA E ESTRATÉGIAS AVANÇADAS

TUDO O QUE UM INVESTIDOR PRECISA SABER PARA
PROSPERAR NA BOLSA DE VALORES ATÉ EM TEMPOS DE CRISE

novatec

Manual de Análise Técnica

Abe, Marcos

9788575227022

256 páginas

[Compre agora e leia](#)

Este livro aborda o tema Investimento em Ações de maneira inédita e tem o objetivo de ensinar os investidores a lucrarem nas mais diversas condições do mercado, inclusive em tempos de crise. Ensinará ao leitor que, para ganhar dinheiro, não importa se o mercado está em alta ou em baixa, mas sim saber como operar em cada situação. Com o Manual de Análise Técnica o leitor aprenderá:

- os conceitos clássicos da Análise Técnica de forma diferenciada, de maneira que assimile não só os princípios, mas que desenvolva o raciocínio necessário para utilizar os gráficos como meio de interpretar os movimentos da massa de investidores do mercado;
- identificar oportunidades para lucrar na bolsa de valores, a longo e curto prazo, até mesmo em mercados baixistas; um sistema de investimentos completo com estratégias para abrir, conduzir e fechar operações, de forma que seja possível maximizar lucros e minimizar prejuízos;
- estruturar e proteger operações por meio do gerenciamento de capital.

Destina-se a iniciantes na bolsa de valores e investidores que ainda não desenvolveram uma metodologia própria para operar lucrativamente.

[Compre agora e leia](#)

O'REILLY



Microserviços prontos para a produção

CONSTRUINDO SISTEMAS PADRONIZADOS EM UMA
ORGANIZAÇÃO DE ENGENHARIA DE SOFTWARE



novatec

Susan J. Fowler

Microserviços prontos para a produção

Fowler, Susan J.

9788575227473

224 páginas

[Compre agora e leia](#)

Um dos maiores desafios para as empresas que adotaram a arquitetura de microserviços é a falta de padronização de arquitetura – operacional e organizacional. Depois de dividir uma aplicação monolítica ou construir um ecossistema de microserviços a partir do zero, muitos engenheiros se perguntam o que vem a seguir. Neste livro prático, a autora Susan Fowler apresenta com profundidade um conjunto de padrões de microserviço, aproveitando sua experiência de padronização de mais de mil microserviços do Uber. Você aprenderá a projetar microserviços que são estáveis, confiáveis, escaláveis, tolerantes a falhas, de alto desempenho, monitorados, documentados e preparados para qualquer catástrofe. Explore os padrões de disponibilidade de produção, incluindo: Estabilidade e confiabilidade – desenvolva, implante, introduza e descontinue microserviços; proteja-se contra falhas de dependência. Escalabilidade e desempenho – conheça os componentes essenciais para alcançar mais eficiência do microserviço. Tolerância a falhas e prontidão para catástrofes – garanta a disponibilidade forçando ativamente os microserviços a

falhar em tempo real. Monitoramento – aprenda como monitorar, gravar logs e exibir as principais métricas; estabeleça procedimentos de alerta e de prontidão. Documentação e compreensão – atenua os efeitos negativos das contrapartidas que acompanham a adoção dos microsserviços, incluindo a dispersão organizacional e a defasagem técnica.

[Compre agora e leia](#)